control-systems-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `control_systems_ai999x.about()` for the same answer.

Every capability control-systems-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import control_systems_ai999x

result = control_systems_ai999x.solve('ctrl_first_order_step', K=2, tau=5, t=5)
result.show()
# y = 1.26424, fraction = 0.632121, y_final = 2
```

By topic

topic	capabilities
Compensators / lag	Lag compensator placement
Compensators / lead	Maximum phase lead of a lead compensator, Lead compensator design for a required phase lead
Design / specifications	Desired closed-loop poles from overshoot and settling time
Discretization / digital PID	Discrete PID in velocity (incremental) form
Discretization / discrete systems	DC gain and final value of a discrete transfer function, Step response of a discrete transfer function at sample k
Discretization / Euler	Forward Euler (forward difference) discretization, Backward Euler (backward difference) discretization
Discretization / FOH	First-order-hold (triangle-hold) discretization
Discretization / pole mapping	Mapping an s-plane pole to the z-plane (z = e^{sT}), Continuous equivalent of a discrete pole (s = ln z / T)
Discretization / Tustin	Tustin (bilinear) discretization, Tustin discretization with frequency prewarping
Discretization / ZOH	Zero-order-hold discretization of a first-order system, Zero-order-hold discretization of a transfer function
Estimation / Kalman filter	One step of a scalar Kalman filter, Steady-state gain of a scalar Kalman filter, Steady-state Kalman-Bucy filter gain, Steady-state discrete Kalman filter gain
Frequency response /	Bandwidth of a standard second-order system, -3 dB bandwidth of a transfer function

topic	capabilities
bandwidth	
Frequency response / Bode	Frequency response of a first-order lag, Bode magnitude and phase at one frequency, Frequency response of a standard second-order system
Frequency response / norms	H2 norm of a stable transfer function, H-infinity norm (peak gain) of a stable transfer function
Frequency response / resonance	Resonant peak and resonant frequency
Identification / FOPDT	First-order-plus-dead-time fit by Smith's two-point method
Identification / second order	Damping ratio from the logarithmic decrement, Second-order model from a step response
Modelling / DC motor	Speed transfer function of a DC motor
Optimal control / LQR	LQR for a first-order system, Continuous-time LQR gain (Riccati equation), Discrete-time LQR gain (discrete Riccati equation)
PID tuning / closed loop	Closed-loop poles of a PID loop
PID tuning / Cohen-Coon	Cohen-Coon tuning of a first-order-plus-dead-time process
PID tuning / forms	PID: standard form to parallel gains, PID: parallel gains to standard form
PID tuning / pole placement	PI gains placing the closed-loop poles of a first-order plant
PID tuning / SIMC	SIMC (Skogestad) PI tuning
PID tuning / Ziegler-Nichols	Ziegler-Nichols tuning from the ultimate gain and period, Ziegler-Nichols tuning from the process reaction curve, Ziegler-Nichols tuning from a plant model
Root locus / asymptotes	Root-locus asymptotes (centroid and angles)
Root locus / breakaway points	Root-locus breakaway and break-in points
Root locus / departure and arrival angles	Root-locus angle of departure from a complex pole, Root-locus angle of arrival at a complex zero
Root locus / magnitude and angle criteria	Root-locus gain and angle at a test point
Stability / discrete	Jury (Schur-Cohn) stability test for a discrete polynomial
Stability / Lyapunov	Continuous Lyapunov equation, Discrete Lyapunov equation
Stability / margins	Gain and phase margins
Stability / Nyquist	Nyquist criterion: encirclements of -1
Stability / poles	Poles and stability of a characteristic polynomial
Stability / robustness	Peak sensitivity Ms and modulus margin
Stability / Routh-Hurwitz	Routh-Hurwitz stability test, Range of gain for closed-loop stability (Routh, j-omega crossing)

topic	capabilities
State space / controllability	Controllability matrix and rank
State space / conversions	Transfer function of a state-space model, Controllable canonical form of a transfer function
State space / DC gain	DC gain of a state-space model
State space / Gramians	Controllability Gramian, Observability Gramian
State space / observability	Observability matrix and rank
State space / observers	Observer (estimator) gain by pole placement
State space / pole placement	Pole placement by Ackermann's formula, Deadbeat state feedback (discrete time)
State space / stability	Eigenvalues (poles) of a state matrix
State space / state feedback	Closed-loop poles under state feedback u = -K x
State space / time response	State-transition matrix e^{At}, State response to an initial state and a constant input
Steady-state error	System type, static error constant and steady-state error
Time delay / Pade	Pade approximation of a time delay
Time response / closed loop	Proportional control of a first-order plant
Time response / dead time	Step response of a first-order-plus-dead-time process
Time response / final and initial values	Final value theorem, Initial value theorem (value and slope at t = 0+)
Time response / first order	Step response of a first-order system, Time and frequency specifications of a first-order system
Time response / general	Step response of a transfer function at a given time
Time response / second order	Mass-spring-damper natural frequency and damping ratio, Natural frequency and damping ratio of a second-order denominator, Percent overshoot from the damping ratio, Damping ratio from percent overshoot, Peak time of a second-order step response, Settling time of a second-order system (rule and envelope), Second-order specifications from the pole location, Step response of a standard second-order system, Rise time of an underdamped second-order system, Exact settling time of a second-order step response
Transfer functions / block diagrams	Series (cascade) connection of two transfer functions, Parallel connection of two transfer functions, Closed-loop transfer function of a feedback loop
Transfer functions / DC gain	DC gain and final value of a step response
Transfer functions / evaluation	Value of a transfer function at a complex point
Transfer functions / poles and zeros	Poles, zeros and gain of a transfer function

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
ctrl_first_order_step	Step response of a first-order system	K, tau, t, A	y, fraction, y_final
ctrl_first_order_specs	Time and frequency specifications of a first-order system	K, tau	time_constant, rise_time, settling_time_2, settling_time_5, bandwidth, dc_gain
ctrl_first_order_freq	Frequency response of a first-order lag	K, tau, w	magnitude, magnitude_db, phase_deg
ctrl_mass_spring_damper	Mass-spring-damper natural frequency and damping ratio	m, c, k	wn, zeta, wd, dc_gain, damping
ctrl_second_order_params	Natural frequency and damping ratio of a second-order denominator	den, K	wn, zeta, sigma, wd, damping, dc_gain
ctrl_percent_overshoot	Percent overshoot from the damping ratio	zeta	PO, Mp, pole_angle_deg
ctrl_damping_from_overshoot	Damping ratio from percent overshoot	OS	zeta, pole_angle_deg
ctrl_peak_time	Peak time of a second-order step response	zeta, wn	Tp, wd
ctrl_settling_time	Settling time of a second-order system (rule and envelope)	zeta, wn, band	Ts, Ts_envelope, time_constant
ctrl_dc_gain	DC gain and final value of a step response	num, den, A	dc_gain, final_value
ctrl_pole_stability	Poles and stability of a characteristic polynomial	den	poles, max_real_part, stability
ctrl_closed_loop_first_order_p	Proportional control of a first-order plant	K, tau, Kp	dc_gain_cl, tau_cl, ess, pole
ctrl_second_order_from_poles	Second-order specifications from the pole location	sigma, wd	wn, zeta, Tp, Ts, PO
ctrl_s_to_z_pole	Mapping an s-plane pole to the z-plane (z = e^{sT})	s_re, s_im, T	z_re, z_im, z_mag, z_angle
ctrl_zn_ultimate	Ziegler-Nichols tuning from the ultimate gain and period	Ku, Pu, controller	Kp, Ti, Td, Ki, Kd
ctrl_zn_reaction	Ziegler-Nichols tuning from the process reaction curve	K, L, T, controller	Kp, Ti, Td, Ki, Kd
ctrl_pid_standard_to_parallel	PID: standard form to parallel gains	Kp, Ti, Td	Ki, Kd
ctrl_pid_parallel_to_standard	PID: parallel gains to standard form	Kp, Ki, Kd	Ti, Td
ctrl_series_connection	Series (cascade) connection of two transfer functions	num1, den1, num2, den2	num, den
ctrl_parallel_connection	Parallel connection of two transfer functions	num1, den1, num2, den2	num, den
ctrl_discrete_dc_gain	DC gain and final value of a discrete transfer function	num, den, A	dc_gain, final_value
ctrl_poles_zeros	Poles, zeros and gain of a transfer function	num, den	zeros, poles, gain

id	operation	inputs	outputs
ctrl_tf_evaluate	Value of a transfer function at a complex point	num, den, s_re, s_im	G_re, G_im, magnitude, angle_deg
ctrl_feedback_connection	Closed-loop transfer function of a feedback loop	num_g, den_g, num_h, den_h, sign	num, den
ctrl_second_order_step	Step response of a standard second-order system	zeta, wn, t, K	y, error
ctrl_rise_time_second	Rise time of an underdamped second-order system	zeta, wn	Tr_0_100, Tr_10_90, t10, t90
ctrl_final_value_theorem	Final value theorem	num, den	final_value
ctrl_initial_value_theorem	Initial value theorem (value and slope at t = 0+)	num, den	y0, dy0
ctrl_error_constants	System type, static error constant and steady-state error	num, den, input, A	system_type, K_static, ess
ctrl_bode_point	Bode magnitude and phase at one frequency	num, den, w	magnitude, magnitude_db, phase_deg
ctrl_second_order_freq	Frequency response of a standard second-order system	zeta, wn, w	magnitude, magnitude_db, phase_deg
ctrl_bandwidth_second	Bandwidth of a standard second-order system	zeta, wn	wb
ctrl_resonant_peak	Resonant peak and resonant frequency	zeta, wn	Mr, Mr_db, wr
ctrl_routh_hurwitz	Routh-Hurwitz stability test	coefficients	first_column, rhp_roots, stable
ctrl_locus_asymptotes	Root-locus asymptotes (centroid and angles)	num, den	n_asymptotes, centroid, angles_deg
ctrl_locus_gain_at_point	Root-locus gain and angle at a test point	num, den, s_re, s_im	K, angle_deg, angle_error_deg
ctrl_ss_to_tf	Transfer function of a state-space model	A, B, C, D	num, den
ctrl_tf_to_ss	Controllable canonical form of a transfer function	num, den	A, B, C, D
ctrl_ss_eigenvalues	Eigenvalues (poles) of a state matrix	A	eigenvalues, char_poly, max_real_part, stable
ctrl_controllability	Controllability matrix and rank	A, B	Wc, rank, controllable, det_Wc
ctrl_observability	Observability matrix and rank	A, C	Wo, rank, observable, det_Wo
ctrl_ss_dc_gain	DC gain of a state-space model	A, B, C, D	dc_gain
ctrl_zoh_first_order	Zero-order-hold discretization of a first-order system	K, tau, T	b, a
ctrl_tustin	Tustin (bilinear) discretization	num, den, T	numz, denz
ctrl_forward_euler	Forward Euler (forward difference) discretization	num, den, T	numz, denz
ctrl_backward_euler	Backward Euler (backward difference) discretization	num, den, T	numz, denz
ctrl_discrete_step	Step response of a discrete transfer function at sample k	numz, denz, k, A	y

id	operation	inputs	outputs
ctrl_discrete_pid	Discrete PID in velocity (incremental) form	Kp, Ki, Kd, T	q0, q1, q2
ctrl_z_to_s_pole	Continuous equivalent of a discrete pole (s = ln z / T)	z_re, z_im, T	s_re, s_im, wn, zeta
ctrl_cohen_coon	Cohen-Coon tuning of a first-order-plus-dead-time process	K, tau, theta, controller	Kp, Ti, Td, Ki, Kd
ctrl_simc_pi	SIMC (Skogestad) PI tuning	K, tau, theta, tau_c	Kp, Ti, Ki
ctrl_pid_closed_loop	Closed-loop poles of a PID loop	num, den, Kp, Ki, Kd	char_poly, poles, max_real_part, stable
ctrl_lead_max_phase	Maximum phase lead of a lead compensator	z, p	phi_max_deg, w_max, alpha, magnitude_db
ctrl_specs_to_poles	Desired closed-loop poles from overshoot and settling time	OS, Ts	zeta, wn, sigma, wd, char_poly
ctrl_log_decrement	Damping ratio from the logarithmic decrement	x1, xn, n	delta, zeta
ctrl_identify_second_order	Second-order model from a step response	y_peak, y_final, t_peak, A	K, PO, zeta, wn
ctrl_fopdt_step	Step response of a first-order-plus-dead-time process	K, tau, theta, t, A	y
ctrl_kalman_scalar_step	One step of a scalar Kalman filter	x, P, a, q, r, z	x_pred, P_pred, gain, x_post, P_post
ctrl_dc_motor	Speed transfer function of a DC motor	J, b, Kt, R, L	num, den, dc_gain, poles
ctrl_stability_margins	Gain and phase margins	num, den	gm, gm_db, w_pc, pm_deg, w_gc
ctrl_bandwidth	-3 dB bandwidth of a transfer function	num, den	wb, dc_gain
ctrl_stable_gain_range	Range of gain for closed-loop stability (Routh, j-omega crossing)	num, den	K_min, K_max, w_cross
ctrl_breakaway_points	Root-locus breakaway and break-in points	num, den	points, gains, count
ctrl_departure_angle	Root-locus angle of departure from a complex pole	num, den	pole, departure_deg
ctrl_arrival_angle	Root-locus angle of arrival at a complex zero	num, den	zero, arrival_deg
ctrl_state_feedback_poles	Closed-loop poles under state feedback u = -K x	A, B, K	char_poly, poles, stable
ctrl_ackermann	Pole placement by Ackermann's formula	A, B, alpha	K
ctrl_observer_gain	Observer (estimator) gain by pole placement	A, C, alpha	L
ctrl_deadbeat_gain	Deadbeat state feedback (discrete time)	A, B	K
ctrl_state_transition	State-transition matrix e^{At}	A, t	Phi

id	operation	inputs	outputs
ctrl_ss_response	State response to an initial state and a constant input	A, B, x0, u, t	x
ctrl_c2d_zoh	Zero-order-hold discretization of a transfer function	num, den, T	numz, denz
ctrl_c2d_foh	First-order-hold (triangle-hold) discretization	num, den, T	numz, denz
ctrl_tustin_prewarp	Tustin discretization with frequency prewarping	num, den, T, w	numz, denz
ctrl_jury_test	Jury (Schur-Cohn) stability test for a discrete polynomial	coefficients	stable, P1, Pm1
ctrl_step_response	Step response of a transfer function at a given time	num, den, t, A	y
ctrl_true_settling_time	Exact settling time of a second-order step response	zeta, wn, band	Ts, last_extremum
ctrl_zn_from_plant	Ziegler-Nichols tuning from a plant model	num, den, controller	Kp, Ti, Td, Ki, Kd, Ku, Pu
ctrl_lead_design	Lead compensator design for a required phase lead	phi_max_deg, w_m	alpha, T, zero, pole, gain_db
ctrl_lag_design	Lag compensator placement	beta, w_c, ratio	zero, pole, dc_gain, phase_deg, magnitude_db
ctrl_pade	Pade approximation of a time delay	theta, n	num, den
ctrl_scalar_lqr	LQR for a first-order system	a, b, q, r	P, K, pole
ctrl_pi_pole_placement	PI gains placing the closed-loop poles of a first-order plant	K, tau, zeta, wn	Kp, Ki
ctrl_kalman_scalar_steady	Steady-state gain of a scalar Kalman filter	a, c, q, r	P_prior, gain, P_post
ctrl_fopdt_smith	First-order-plus-dead-time fit by Smith's two-point method	t28, t63, dy, du	K, tau, theta
ctrl_lqr	Continuous-time LQR gain (Riccati equation)	A, B, Q, R	K, P, poles
ctrl_dlqr	Discrete-time LQR gain (discrete Riccati equation)	A, B, Q, R	K, P, poles
ctrl_kalman_bucy	Steady-state Kalman-Bucy filter gain	A, C, W, V	L, P, poles
ctrl_discrete_kalman_gain	Steady-state discrete Kalman filter gain	A, C, Q, R	K, P_prior
ctrl_lyapunov	Continuous Lyapunov equation	A, Q	P, positive_definite
ctrl_discrete_lyapunov	Discrete Lyapunov equation	A, Q	P, positive_definite
ctrl_ctrb_gramian	Controllability Gramian	A, B	Wc, controllable
ctrl_obsv_gramian	Observability Gramian	A, C	Wo, observable
ctrl_h2_norm	H2 norm of a stable transfer function	num, den	h2, h2_squared
ctrl_hinf_norm	H-infinity norm (peak gain) of a stable transfer function	num, den	hinf, hinf_db, w_peak

id	operation	inputs	outputs
ctrl_nyquist	Nyquist criterion: encirclements of -1	num, den, K	P, Z, N, stable
ctrl_sensitivity_peak	Peak sensitivity Ms and modulus margin	num, den	Ms, Ms_db, w_ms, sm

Step response of a first-order system, `ctrl_first_order_step`

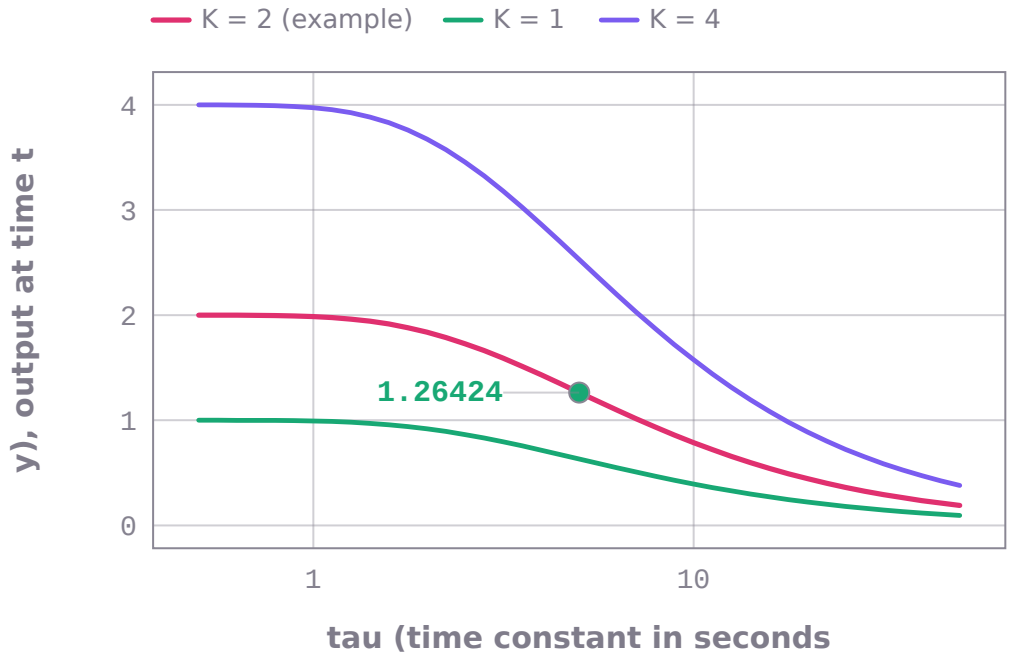
The output y(t) = K A (1 - e^{-t/tau}) of K/(tau s + 1) at time t after a step of size A, and the fraction of the final value reached.

$$y(t) = KA\left(1 - e^{-t/\tau}\right)$$

input	meaning	condition
`K`	steady-state (DC) gain of the system	
`tau`	time constant in seconds	
`t`	time after the step in seconds	
`A`	step size (default 1)	

output	meaning
`y`	output at time t
`fraction`	fraction of the final value reached, $1 - e$^{-t/tau}
`y_final`	final value K A

- **Method:** $y = K$ A (1 - exp(-t/tau))
- **Assumes:** $\tau > 0$; $t \geq 0$; K and A not zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y (output at time t) against tau (time constant in seconds) from 0.5 to 50 on a logarithmic axis. One curve for each value of K (steady-state (DC) gain of the system): 2, 1, 4. Other inputs as in the worked example: t = 5.0. 41 points per curve computed by control-systems-ai999x; marked: the worked example, tau = 5, y = 1.26424.

```
control_systems_ai999x.solve('ctrl_first_order_step', K=2, tau=5, t=5)
# -> y = 1.26424, fraction = 0.632121, y_final = 2   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_first_order_step', K=1, tau=2, t=4, A=10)
# -> y = 8.64665, fraction = 0.864665, y_final = 10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
first-order step response: K = 2, tau = 5, t = 5	SOLVED	y = 1.26424, fraction = 0.632121, y_final = 2
first-order step response: K = 1, tau = 2, t = 4, A = 10	SOLVED	y = 8.64665, fraction = 0.864665, y_final = 10
first-order step response: tau = 5, t = 5	NEEDS_INPUT	ctrl_first_order_step needs K (steady-state (DC) gain of the system). Example: solve('ctrl_first_order_step', K=2, tau=5
first-order step response: K = 2, tau = 5, t = 5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
first-order step response: K = 2, tau = -5, t = 5	INVALID_INPUT	Step response of a first-order system: tau must be positive.

Time and frequency specifications of a first-order system, `ctrl_first_order_specs`

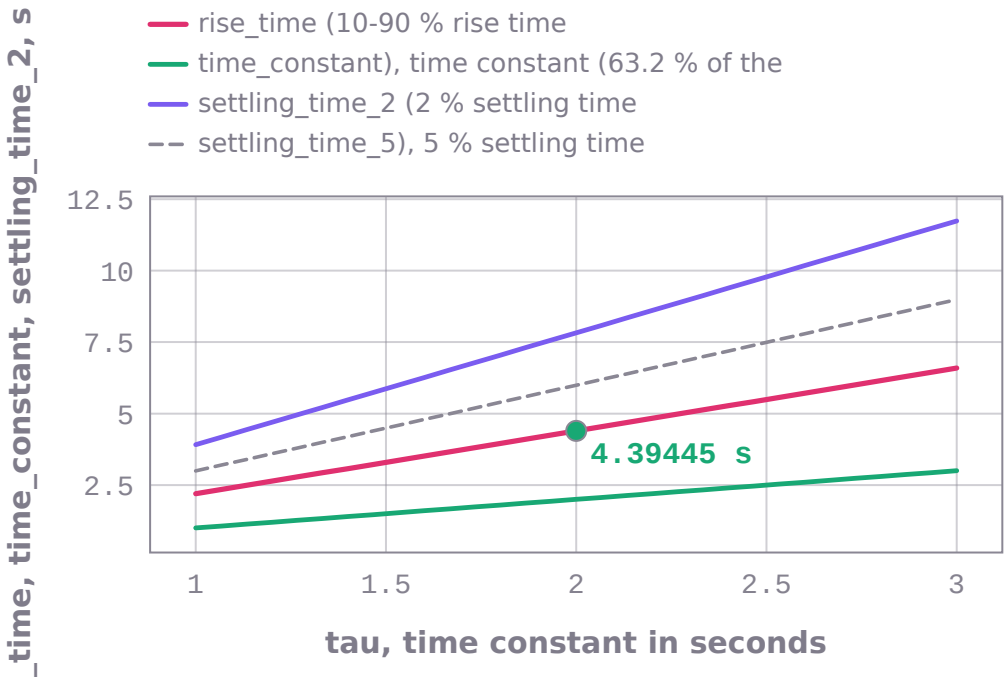
Time constant, $10 - 90$ % rise time tau ln 9, 2 % and 5 % settling times tau ln 50 and tau ln 20, bandwidth $1/\tau$ and DC gain of K/(tau s + 1).

$$t_r = \tau \ln 9, \quad t_{s,2\%} = \tau \ln 50, \quad \omega_B = 1/\tau$$

input	meaning	condition
`K`	steady-state (DC) gain	
`tau`	time constant in seconds	

output	meaning
`time_constant`	time constant (63.2 % of the final value)
`rise_time`	$10-90$ % rise time
`settling_time_2`	2 % settling time
`settling_time_5`	5 % settling time
`bandwidth`	-3 dB bandwidth
`dc_gain`	DC gain

- **Method:** exact crossing times of $1 - exp$(-t/tau)
- **Assumes:** $\tau > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rise_time (10-90 % rise time) against tau (time constant in seconds) from 1 to 3. Outputs with the same unit drawn together: time_constant (time constant (63.2 % of the final value)), settling_time_2 (2 % settling time), settling_time_5 (5 % settling time). Other inputs as in the worked example: K = 3.0. 31 points computed by control-systems-ai999x; marked: the worked example, tau = 2, rise_time = 4.39445 s.

```
control_systems_ai999x.solve('ctrl_first_order_specs', K=3, tau=2)
# -> time_constant = 2 s, rise_time = 4.39445 s, settling_time_2 = 7.82405 s,
settling_time_5 = 5.99146 s, bandwidth = 0.5 rad/s, dc_gain = 3   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_first_order_specs', K=1, tau=0.1)
# -> time_constant = 0.1 s, rise_time = 0.219722 s, settling_time_2 = 0.391202 s,
settling_time_5 = 0.299573 s, bandwidth = 10 rad/s, dc_gain = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
first-order specifications: K = 3, tau = 2	SOLVED	time_constant = 2 s, rise_time = 4.39445 s, settling_time_2 = 7.82405 s, settling_time_5 = 5.99146 s, bandwidth = 0.5 rad/s, dc_gain = 3
first-order specifications: K = 1, tau = 0.1	SOLVED	time_constant = 0.1 s, rise_time = 0.219722 s, settling_time_2 = 0.391202 s, settling_time_5 = 0.299573 s, bandwidth = 10 rad/s, dc_gain = 1
first-order specifications: tau = 2	NEEDS_INPUT	ctrl_first_order_specs needs K (steady-state (DC) gain). Example: solve('ctrl_first_order_specs', K=3, tau=2).
first-order specifications: K = 3, tau = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
first-order specifications: K = 3, tau = 0	INVALID_INPUT	Time and frequency specifications of a first-order system: tau must be positive.

Frequency response of a first-order lag, `ctrl_first_order_freq`

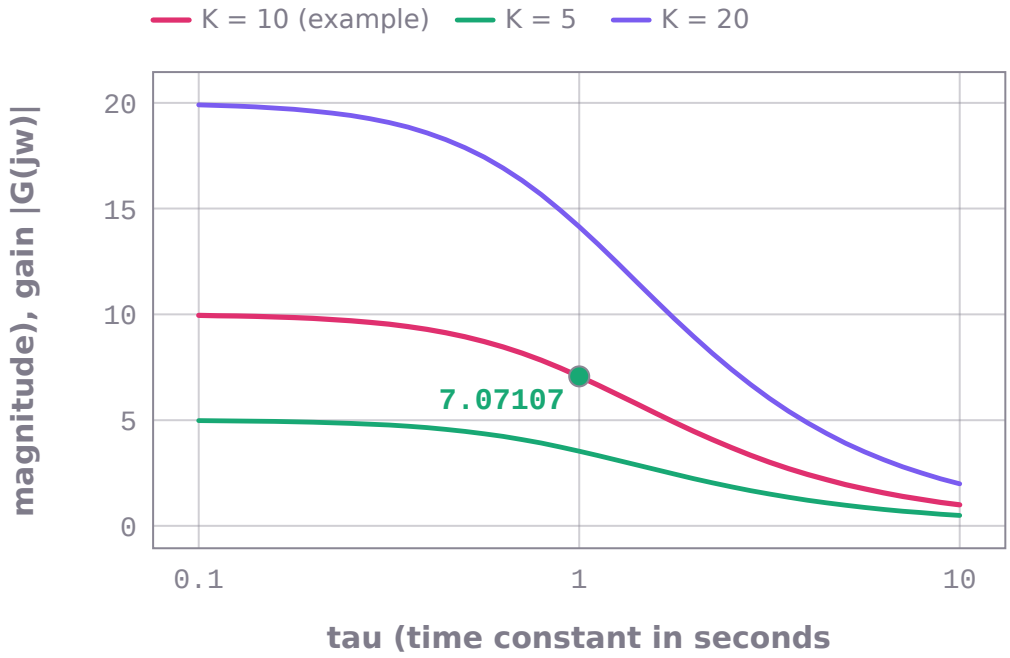
Magnitude K/sqrt(1 + (w tau)^2), its value in dB and the phase -atan(w tau) of K/(tau s + 1) at the angular frequency w.

$$|G(j\omega)| = \frac{K}{\sqrt{1+\omega^2\tau^2}}, \quad \angle G = -\arctan(\omega\tau)$$

input	meaning	condition
`K`	DC gain (positive)	
`tau`	time constant in seconds	
`w`	angular frequency in rad/s	

output	meaning
`magnitude`	gain |G(jw)|
`magnitude_db`	gain in dB
`phase_deg`	phase in degrees

- **Method:** G(jw) = $K/(1 + j$ w tau)
- **Assumes:** $K > 0$; $\tau > 0$; $w \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



magnitude (gain |G(jw)|) against tau (time constant in seconds) from 0.1 to 10 on a logarithmic axis. One curve for each value of K (DC gain (positive)): 10, 5, 20. Other inputs as in the worked example: w = 1.0. 41 points per curve computed by control-systems-ai999x; marked: the worked example, tau = 1, magnitude = 7.07107.

```
control_systems_ai999x.solve('ctrl_first_order_freq', K=10, tau=1, w=1)
# -> magnitude = 7.07107, magnitude_db = 16.9897, phase_deg = -45   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_first_order_freq', K=1, tau=0.5, w=4)
# -> magnitude = 0.447214, magnitude_db = -6.9897, phase_deg = -63.4349   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
first-order frequency response: K = 10, tau = 1, w = 1	SOLVED	magnitude = 7.07107, magnitude_db = 16.9897, phase_deg = -45
first-order frequency response: K = 1, tau = 0.5, w = 4	SOLVED	magnitude = 0.447214, magnitude_db = -6.9897, phase_deg = -63.4349
first-order frequency response: tau = 1, w = 1	NEEDS_INPUT	ctrl_first_order_freq needs K (DC gain (positive)). Example: solve('ctrl_first_order_freq', K=10, tau=1, w=1).
first-order frequency response: K = 10, tau = 1, w = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
first-order frequency response: K = 10, tau = 1, w = -1	INVALID_INPUT	Frequency response of a first-order lag: w must be zero or positive.

Mass-spring-damper natural frequency and damping ratio, `ctrl_mass_spring_damper`

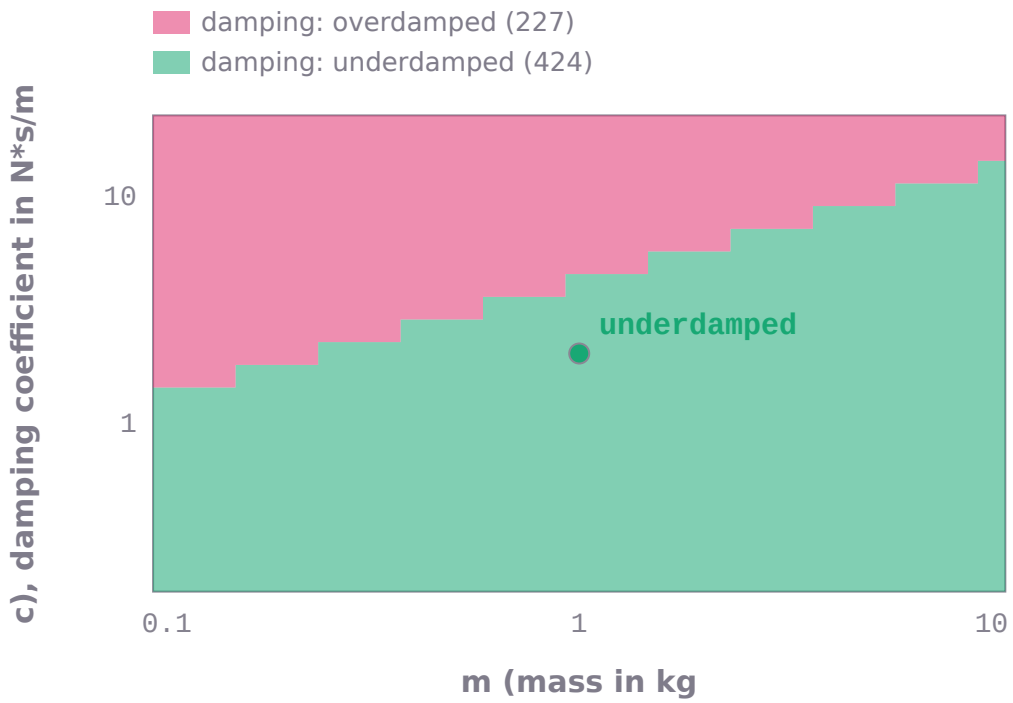
Natural frequency sqrt(k/m), damping ratio c/(2 sqrt(k m)), damped frequency and static compliance $1/k$ of m x'' + c x' + k $x = F$.

$$\omega_n = \sqrt{k/m}, \quad \zeta = \frac{c}{2\sqrt{km}}$$

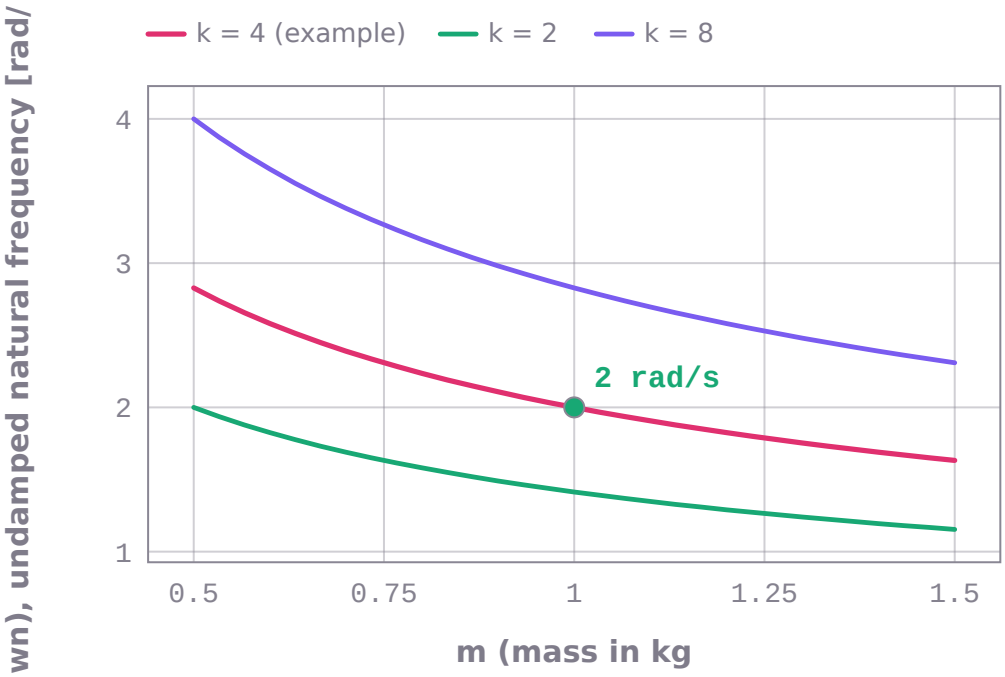
input	meaning	condition
`m`	mass in kg	
`c`	damping coefficient in Ns/m	
`k`	spring stiffness in N/m	

output	meaning
`wn`	undamped natural frequency
`zeta`	damping ratio
`wd`	damped natural frequency (0 when not underdamped)
`dc_gain`	static gain $x/F = 1/k$ in m/N
`damping`	damping class

- **Method:** G(s) $= 1/(m\,s^2 + c$ s + k)
- **Assumes:** $m > 0$; $c \geq 0$; $k > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



damping class (damping) over m (mass in kg) from 0.1 to 10 and c (damping coefficient in N*s/m) from 0.2 to 20 (logarithmic axes): each of the 31 × 21 cells is one run of control-systems-ai999x, coloured by the verdict it returned, overdamped: 227 cells, underdamped: 424 cells. Other inputs as in the worked example: k = 4.0. Marked: the worked example, m = 1, c = 2 → underdamped.



wn (undamped natural frequency) against m (mass in kg) from 0.5 to 1.5. One curve for each value of k (spring stiffness in N/m): 4, 2, 8. Other inputs as in the worked example: c = 2.0. 31 points per curve computed by control-systems-ai999x; marked: the worked example, m = 1, wn = 2 rad/s.

```
control_systems_ai999x.solve('ctrl_mass_spring_damper', m=1, c=2, k=4)
# -> wn = 2 rad/s, zeta = 0.5, wd = 1.73205 rad/s, dc_gain = 0.25, damping = underdamped
(SOLVED)
```

```
control_systems_ai999x.solve('ctrl_mass_spring_damper', m=2, c=8, k=8)
# -> wn = 2 rad/s, zeta = 1, wd = 0 rad/s, dc_gain = 0.125, damping = critically damped
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
mass-spring-damper: m = 1, c = 2, k = 4	SOLVED	wn = 2 rad/s, zeta = 0.5, wd = 1.73205 rad/s, dc_gain = 0.25, damping = underdamped
mass-spring-damper: m = 2, c = 8, k = 8	SOLVED	wn = 2 rad/s, zeta = 1, wd = 0 rad/s, dc_gain = 0.125, damping = critically damped
mass-spring-damper: c = 2, k = 4	NEEDS_INPUT	ctrl_mass_spring_damper needs m (mass in kg). Example: solve('ctrl_mass_spring_damper', m=1, c=2, k=4).
mass-spring-damper: m = 1, c = 2, k = 4, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
mass-spring-damper: m = 0, c = 2, k = 4	INVALID_INPUT	Mass-spring-damper natural frequency and damping ratio: m must be positive.

Natural frequency and damping ratio of a second-order denominator, `ctrl_second_order_params`

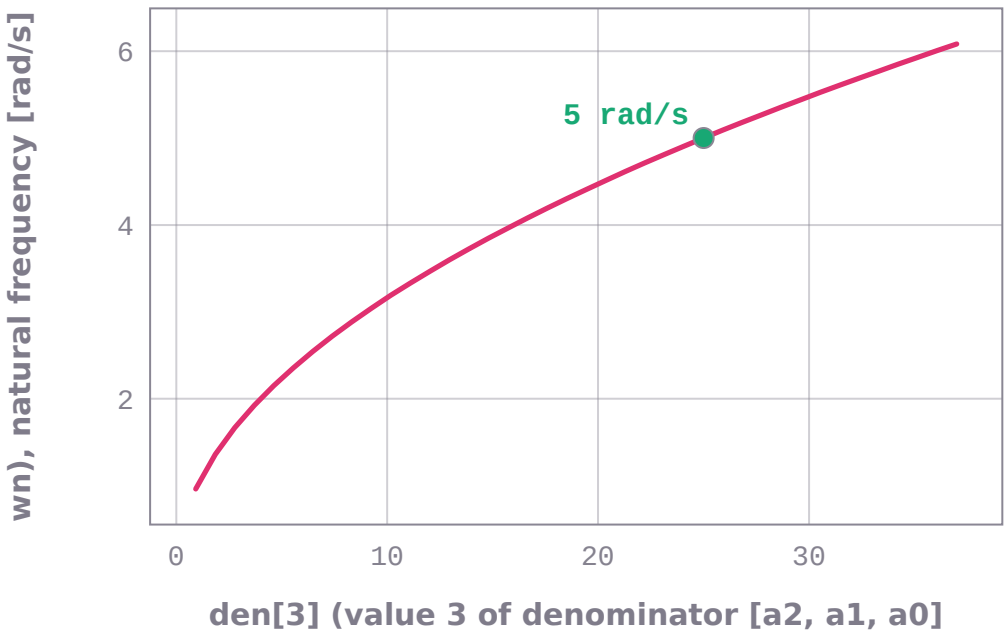
wn, zeta, the decay rate $\sigma = \zeta$ wn, the damped frequency wd and the DC gain of K/(a2 $s^2 + a_1$ s + a0).

$$s^2 + 2\zeta\omega_n s + \omega_n^2$$

input	meaning	condition
`den`	denominator [a2, a1, a0]	
`K`	numerator constant (default: unit DC gain)	

output	meaning
`wn`	natural frequency
`zeta`	damping ratio
`sigma`	decay rate zeta wn $(1/s)$
`wd`	damped frequency (0 unless underdamped)
`damping`	damping class
`dc_gain`	DC gain K/a_0

- **Method:** compare with $s^2 + 2$ zeta wn $s + wn^2$
- **Assumes:** $a_0/a_2 > 0$; $a_1/a_2 \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



wn (natural frequency) as value 3 of den (denominator [a2, a1, a0]) moves from 0 to 37, the other values as given ([1, 4, 25]); 40 runs of control-systems-ai999x. Marked: the worked example, den[3] = 25, wn = 5 rad/s.

```
control_systems_ai999x.solve('ctrl_second_order_params', den=[1, 4, 25])
# -> wn = 5 rad/s, zeta = 0.4, sigma = 2, wd = 4.58258 rad/s, damping = underdamped,
dc_gain = 1   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_second_order_params', den=[2, 12, 50], K=50)
# -> wn = 5 rad/s, zeta = 0.6, sigma = 3, wd = 4 rad/s, damping = underdamped, dc_gain = 1
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
second-order parameters: den = [1, 4, 25]	SOLVED	wn = 5 rad/s, zeta = 0.4, sigma = 2, wd = 4.58258 rad/s, damping = underdamped, dc_gain = 1
second-order parameters: den = [2, 12, 50], K = 50	SOLVED	wn = 5 rad/s, zeta = 0.6, sigma = 3, wd = 4 rad/s, damping = underdamped, dc_gain = 1
second-order parameters:	NEEDS_INPUT	ctrl_second_order_params needs den (denominator [a2, a1, a0]). Example: solve('ctrl_second_order_params', den=[1, 4, 25]
second-order parameters: den = [1, 4, 25], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
second-order parameters: den = [1, 4, -25]	INVALID_INPUT	Natural frequency and damping ratio of a second-order denominator: the polynomial must be s^2 + 2 zeta wn s + wn^2 with

Percent overshoot from the damping ratio, `ctrl_percent_overshoot`

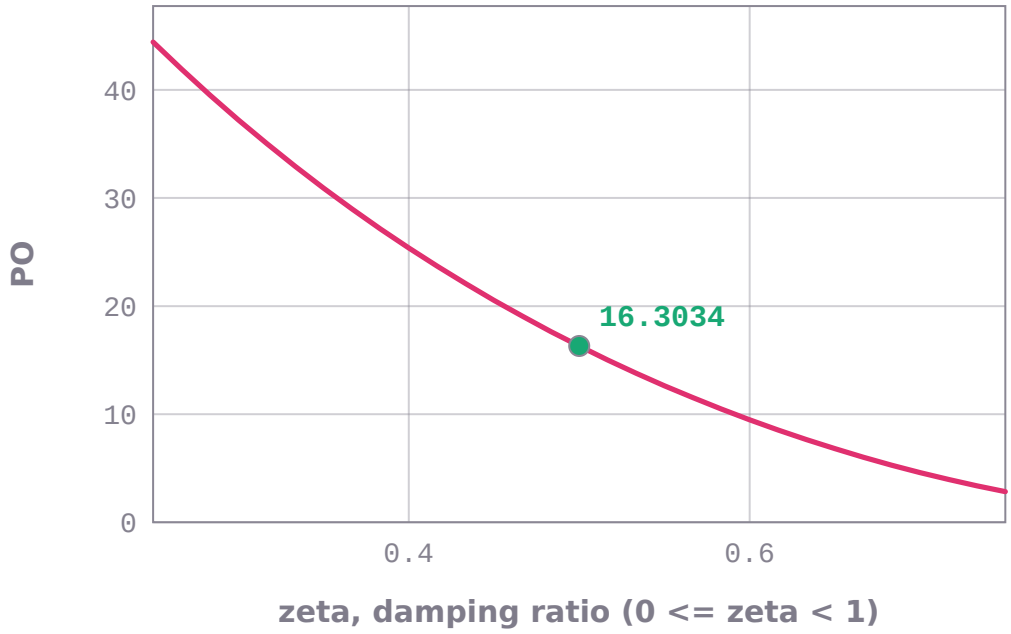
Percent overshoot 100 e^{-zeta pi/sqrt(1 - zeta^2)} of the step response of a standard second-order system, and the pole angle acos(zeta).

$$\%OS = 100\, e^{-\zeta\pi/\sqrt{1-\zeta^2}}$$

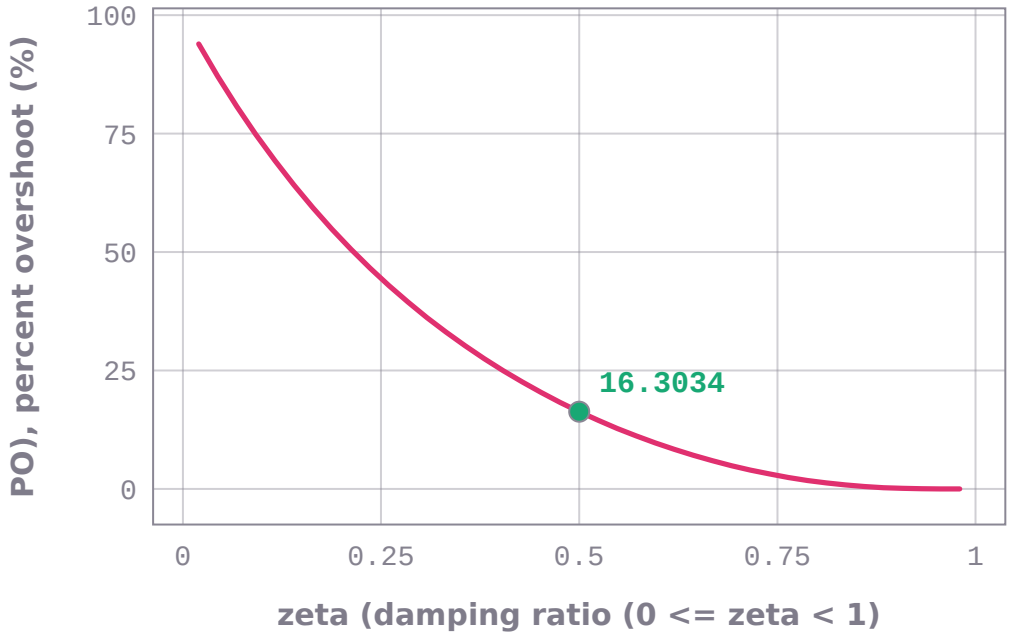
input	meaning	condition
`zeta`	damping ratio $(0 \le \zeta < 1)$	

output	meaning
`PO`	percent overshoot (%)
`Mp`	overshoot as a fraction of the final value
`pole_angle_deg`	angle of the poles from the negative real axis, degrees

- **Method:** peak of the step response at $t = pi/wd$
- **Assumes:** $0 \le \zeta < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



PO (percent overshoot (%)) against zeta (damping ratio (0 <= zeta < 1)). 31 points computed by control-systems-ai999x; marked: zeta = 0.5, PO = 16.3034.



PO (percent overshoot (%)) against zeta (damping ratio (0 <= zeta < 1)) from 0.02 to 0.98. 41 points computed by control-systems-ai999x; marked: the worked example, zeta = 0.5, PO = 16.3034.

```
control_systems_ai999x.solve('ctrl_percent_overshoot', zeta=0.5)
# -> PO = 16.3034, Mp = 0.163034, pole_angle_deg = 60   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_percent_overshoot', zeta=0.7)
# -> PO = 4.59879, Mp = 0.0459879, pole_angle_deg = 45.573   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
percent overshoot: zeta = 0.5	SOLVED	PO = 16.3034, Mp = 0.163034, pole_angle_deg = 60
percent overshoot: zeta = 0.7	SOLVED	PO = 4.59879, Mp = 0.0459879, pole_angle_deg = 45.573
percent overshoot:	NEEDS_INPUT	ctrl_percent_overshoot needs zeta (damping ratio (0 <= zeta < 1)). Example: solve('ctrl_percent_overshoot', zeta=0.5).
percent overshoot: zeta = 0.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
percent overshoot: zeta = 1.2	INVALID_INPUT	Percent overshoot from the damping ratio: overshoot needs an underdamped system: 0 <= zeta < 1.

Damping ratio from percent overshoot, `ctrl_damping_from_overshoot`

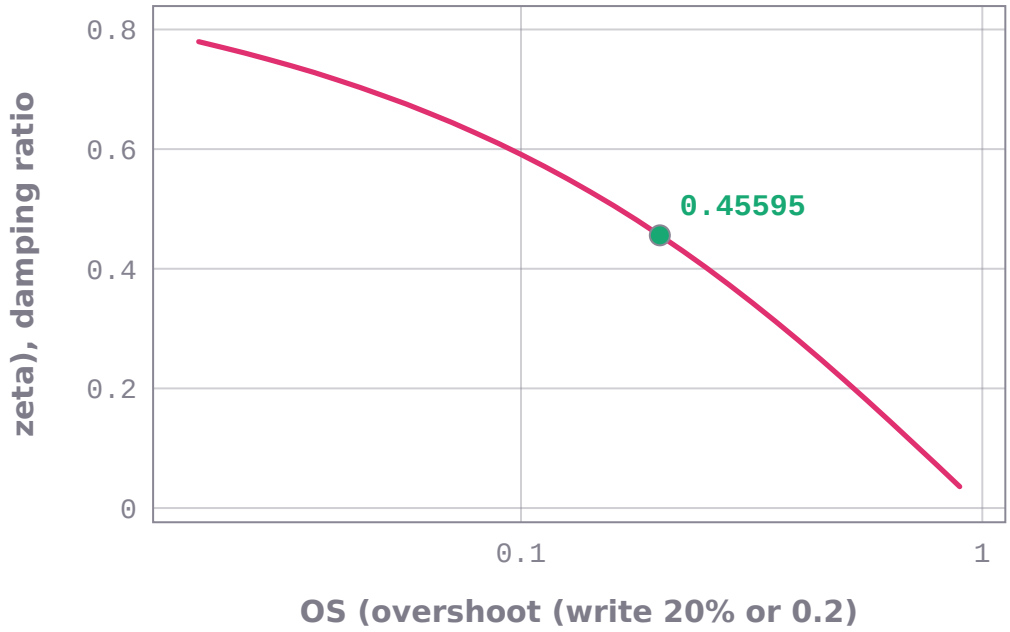
The damping ratio $\zeta = -ln$(OS)/sqrt($pi^2 + ln^2$ OS) that gives a stated overshoot.

$$\zeta = \frac{-\ln(\%OS/100)}{\sqrt{\pi^2 + \ln^2(\%OS/100)}}$$

input	meaning	condition
`OS`	overshoot (write 20% or 0.2)	

output	meaning
`zeta`	damping ratio
`pole_angle_deg`	pole angle from the negative real axis, degrees

- **Method:** invert $OS = exp$(-zeta pi / sqrt(1 - zeta^2))
- **Assumes:** $0 < OS < 100$ %
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



zeta (damping ratio) against OS (overshoot (write 20% or 0.2)) from 0.02 to 2 on a logarithmic axis. 34 points computed by control-systems-ai999x; marked: the worked example, OS = 0.2, zeta = 0.45595.

```
control_systems_ai999x.solve('ctrl_damping_from_overshoot', OS=0.2)
# -> zeta = 0.45595, pole_angle_deg = 62.8739   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_damping_from_overshoot', OS=0.05)
# -> zeta = 0.690107, pole_angle_deg = 46.3614   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
damping ratio from overshoot: OS = 0.2	SOLVED	zeta = 0.45595, pole_angle_deg = 62.8739
damping ratio from overshoot: OS = 0.05	SOLVED	zeta = 0.690107, pole_angle_deg = 46.3614
damping ratio from overshoot:	NEEDS_INPUT	ctrl_damping_from_overshoot needs OS (overshoot (write 20% or 0.2)). Example: solve('ctrl_damping_from_overshoot', OS=0.
damping ratio from overshoot: OS = 0.2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
damping ratio from overshoot: OS = 20	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
damping ratio from overshoot: OS = 20%	SOLVED	zeta = 0.45595, pole_angle_deg = 62.8739
damping ratio from overshoot: OS = 0	INVALID_INPUT	Damping ratio from percent overshoot: the overshoot must lie strictly between 0 and 100 %.

Peak time of a second-order step response, `ctrl_peak_time`

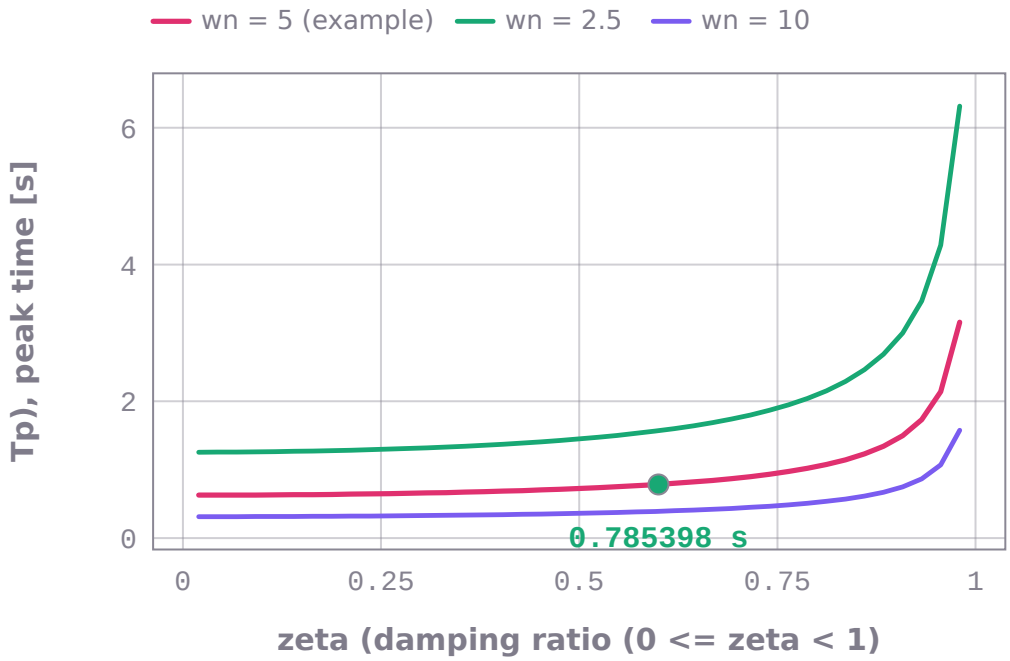
Peak time $Tp = pi/wd$ and damped frequency $wd = wn$ sqrt(1 - zeta^2) of wn^2/(s^2 + 2 zeta wn s + wn^2).

$$T_p = \frac{\pi}{\omega_n \sqrt{1 - \zeta^2}}$$

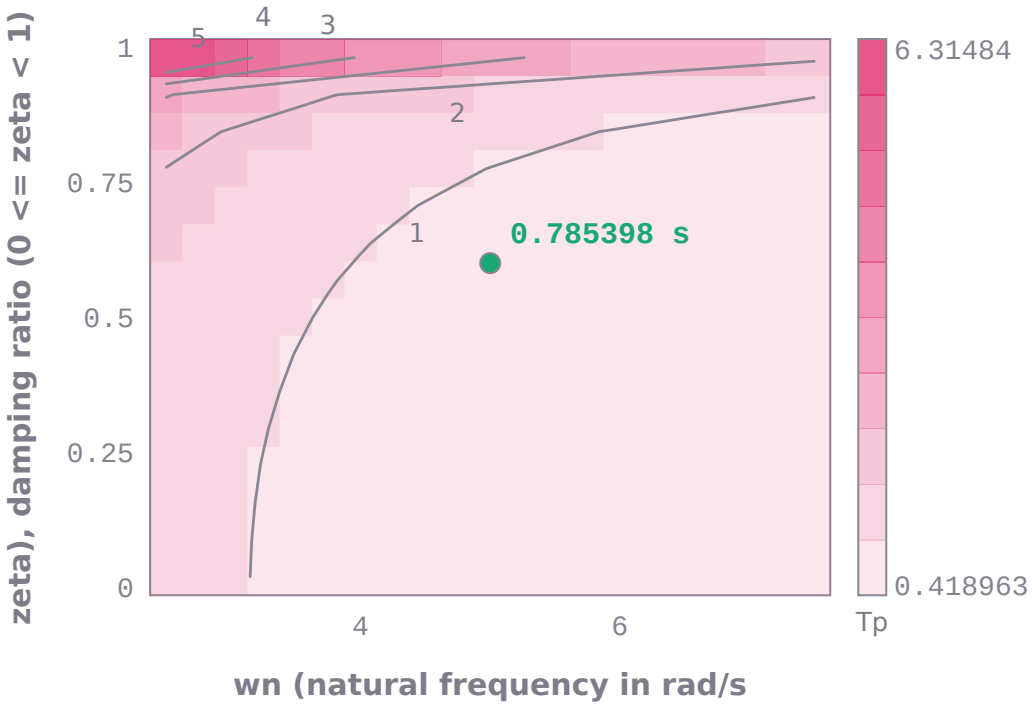
input	meaning	condition
`zeta`	damping ratio $(0 \leq \zeta < 1)$	
`wn`	natural frequency in rad/s	

output	meaning
`Tp`	peak time
`wd`	damped frequency

- **Method:** first zero of the impulse response
- **Assumes:** $0 \leq \zeta < 1$; $wn > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Tp (peak time) against zeta (damping ratio (0 <= zeta < 1)) from 0.02 to 0.98. One curve for each value of wn (natural frequency in rad/s): 5, 2.5, 10. 41 points per curve computed by control-systems-ai999x; marked: the worked example, zeta = 0.6, Tp = 0.785398 s.



Tp (peak time) over wn (natural frequency in rad/s) from 2.5 to 7.5 and zeta (damping ratio (0 <= zeta < 1)) from 0.02 to 0.98: each of the 21 × 15 cells is one run of control-systems-ai999x, shaded by its value (light 0.418963 s to dark 6.31484 s); iso-lines at Tp = 1, 2, 3, 4, 5 s. Marked: the worked example, wn = 5, zeta = 0.6, Tp = 0.785398 s.

```
control_systems_ai999x.solve('ctrl_peak_time', zeta=0.6, wn=5)
# -> Tp = 0.785398 s, wd = 4 rad/s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_peak_time', zeta=0, wn=2)
# -> Tp = 1.5708 s, wd = 2 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
peak time: zeta = 0.6, wn = 5	SOLVED	Tp = 0.785398 s, wd = 4 rad/s
peak time: zeta = 0, wn = 2	SOLVED	Tp = 1.5708 s, wd = 2 rad/s
peak time: wn = 5	NEEDS_INPUT	ctrl_peak_time needs zeta (damping ratio (0 <= zeta < 1)). Example: solve('ctrl_peak_time', zeta=0.6, wn=5).
peak time: zeta = 0.6, wn = 5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
peak time: zeta = 1, wn = 5	INVALID_INPUT	Peak time of a second-order step response: a peak needs an underdamped system: 0 <= zeta < 1.

Settling time of a second-order system (rule and envelope), `ctrl_settling_time`

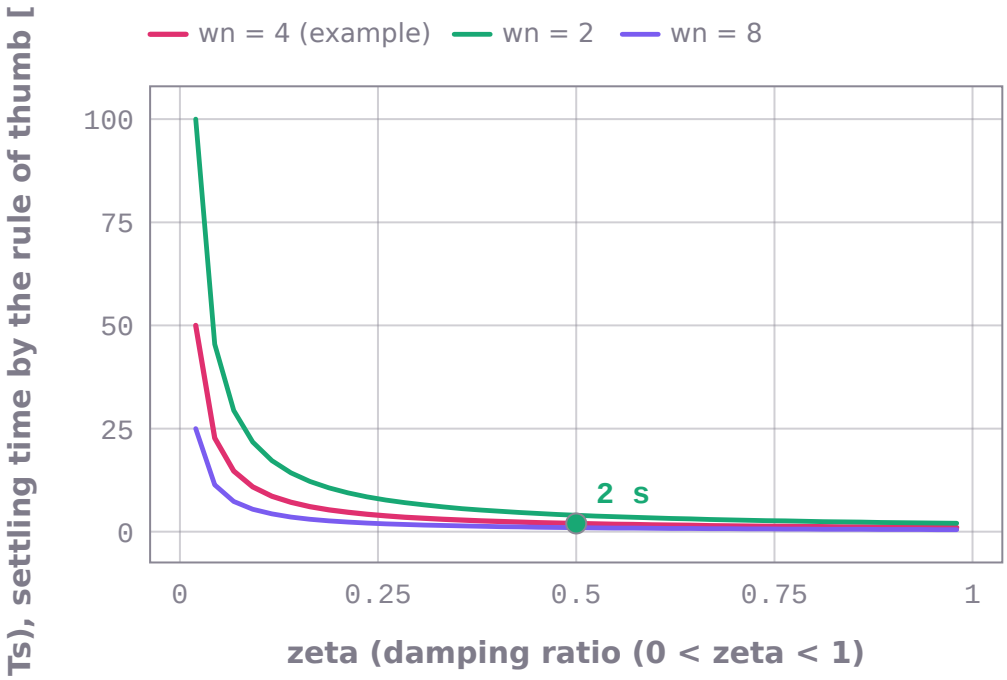
Settling time by the rule 4/(zeta wn) (2 % band) or 3/(zeta wn) (5 %), and the time at which the decay envelope enters the band.

$$T_s \approx \frac{4}{\zeta \omega_n}\ (2\%), \qquad T_s \approx \frac{3}{\zeta \omega_n}\ (5\%)$$

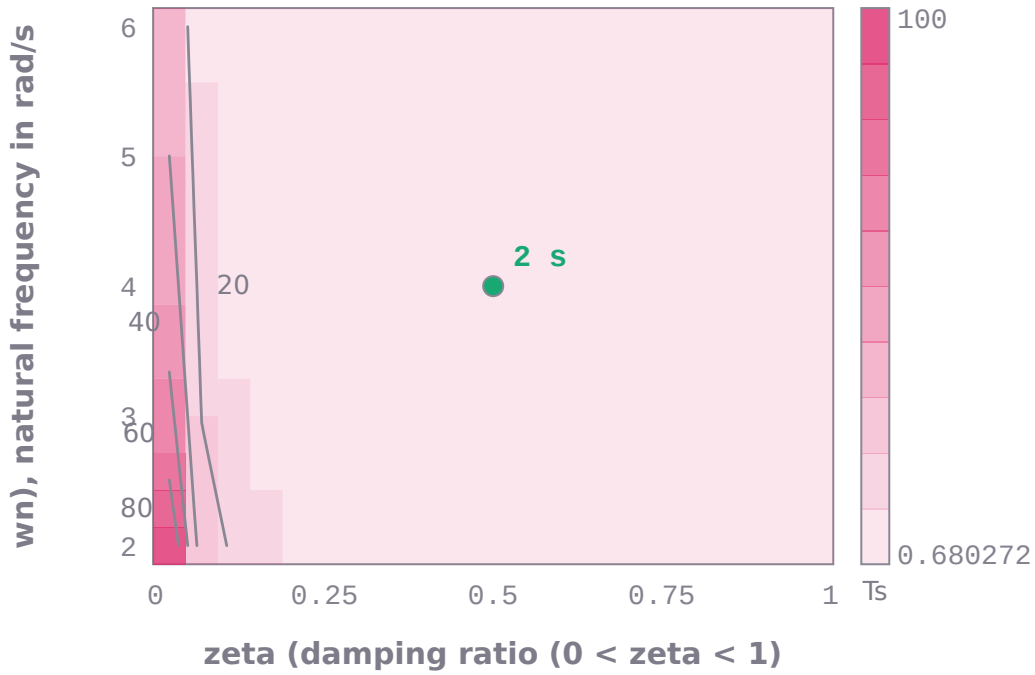
input	meaning	condition
`zeta`	damping ratio $(0 < \zeta < 1)$	
`wn`	natural frequency in rad/s	
`band`	settling band: 2% (default) or 5%	

output	meaning
`Ts`	settling time by the rule of thumb
`Ts_envelope`	time at which e^{-zeta wn t}/sqrt(1-zeta^2) equals the band
`time_constant`	envelope time constant 1/(zeta wn)

- **Method:** Ts = 4/(zeta wn) or 3/(zeta wn)
- **Assumes:** $0 < \zeta < 1$; band 2 % or 5 %
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ts (settling time by the rule of thumb) against zeta (damping ratio (0 < zeta < 1)) from 0.02 to 0.98. One curve for each value of wn (natural frequency in rad/s): 4, 2, 8. 41 points per curve computed by control-systems-ai999x; marked: the worked example, zeta = 0.5, Ts = 2 s.



Ts (settling time by the rule of thumb) over zeta (damping ratio (0 < zeta < 1)) from 0.02 to 0.98 and wn (natural frequency in rad/s) from 2 to 6: each of the 21 × 15 cells is one run of control-systems-ai999x, shaded by its value (light 0.680272 s to dark 100 s); iso-lines at Ts = 20, 40, 60, 80 s. Marked: the worked example, zeta = 0.5, wn = 4, Ts = 2 s.

```
control_systems_ai999x.solve('ctrl_settling_time', zeta=0.5, wn=4)
# -> Ts = 2 s, Ts_envelope = 2.02793 s, time_constant = 0.5 s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_settling_time', zeta=0.5, wn=4, band=0.05)
# -> Ts = 1.5 s, Ts_envelope = 1.56979 s, time_constant = 0.5 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
settling time: zeta = 0.5, wn = 4	SOLVED	Ts = 2 s, Ts_envelope = 2.02793 s, time_constant = 0.5 s
settling time: zeta = 0.5, wn = 4, band = 0.05	SOLVED	Ts = 1.5 s, Ts_envelope = 1.56979 s, time_constant = 0.5 s
settling time: wn = 4	NEEDS_INPUT	ctrl_settling_time needs zeta (damping ratio (0 < zeta < 1)). Example: solve('ctrl_settling_time', zeta=0.5, wn=4).
settling time: zeta = 0.5, wn = 4, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
settling time: zeta = 0.5, wn = 4, band = 0.1	INVALID_INPUT	Settling time of a second-order system (rule and envelope): the rule of thumb is stated for a 2 % or a 5 % band.

DC gain and final value of a step response, `ctrl_dc_gain`

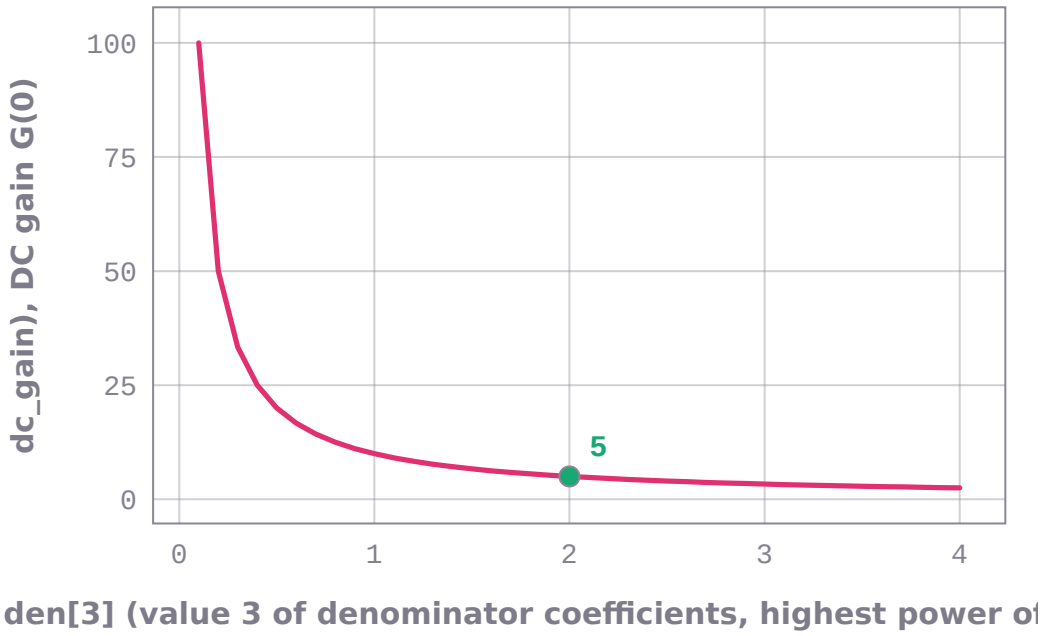
The DC gain G(0) of a stable transfer function and the final value G(0) A of its response to a step of size A.

$$y(\infty) = A\,G(0)$$

input	meaning	condition
`num`	numerator coefficients, highest power of s first	
`den`	denominator coefficients, highest power of s first	
`A`	step size (default 1)	

output	meaning
`dc_gain`	DC gain G(0)
`final_value`	final value of the step response

- **Method:** G(0) = num(0)/den(0) after a stability check of the poles
- **Assumes:** proper G(s); every pole in the left half plane
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dc_gain (DC gain G(0)) as value 3 of den (denominator coefficients, highest power of s first) moves from 0 to 4, the other values as given ([1, 3, 2]); 40 runs of control-systems-ai999x. Other inputs: num = [10]. Marked: the worked example, den[3] = 2, dc_gain = 5.

```
control_systems_ai999x.solve('ctrl_dc_gain', num=[10], den=[1, 3, 2])
# -> dc_gain = 5, final_value = 5   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_dc_gain', num=[2, 6], den=[1, 5, 4], A=3)
# -> dc_gain = 1.5, final_value = 4.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
DC gain: num = [10], den = [1, 3, 2]	SOLVED	dc_gain = 5, final_value = 5
DC gain: num = [2, 6], den = [1, 5, 4], A = 3	SOLVED	dc_gain = 1.5, final_value = 4.5

question	outcome	result
DC gain: den = [1, 3, 2]	NEEDS_INPUT	ctrl_dc_gain needs num (numerator coefficients, highest power of s first). Example: solve('ctrl_dc_gain', num=[10], den=
DC gain: num = [10], den = [1, 3, 2], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
DC gain: num = [1], den = [1, -1]	INVALID_INPUT	DC gain and final value of a step response: the system is not stable, so its step response has no final value.
DC gain: num = [1], den = [1, 2, 0]	INVALID_INPUT	DC gain and final value of a step response: G(s) has a pole at s = 0: the DC gain is infinite.

Poles and stability of a characteristic polynomial, `ctrl_pole_stability`

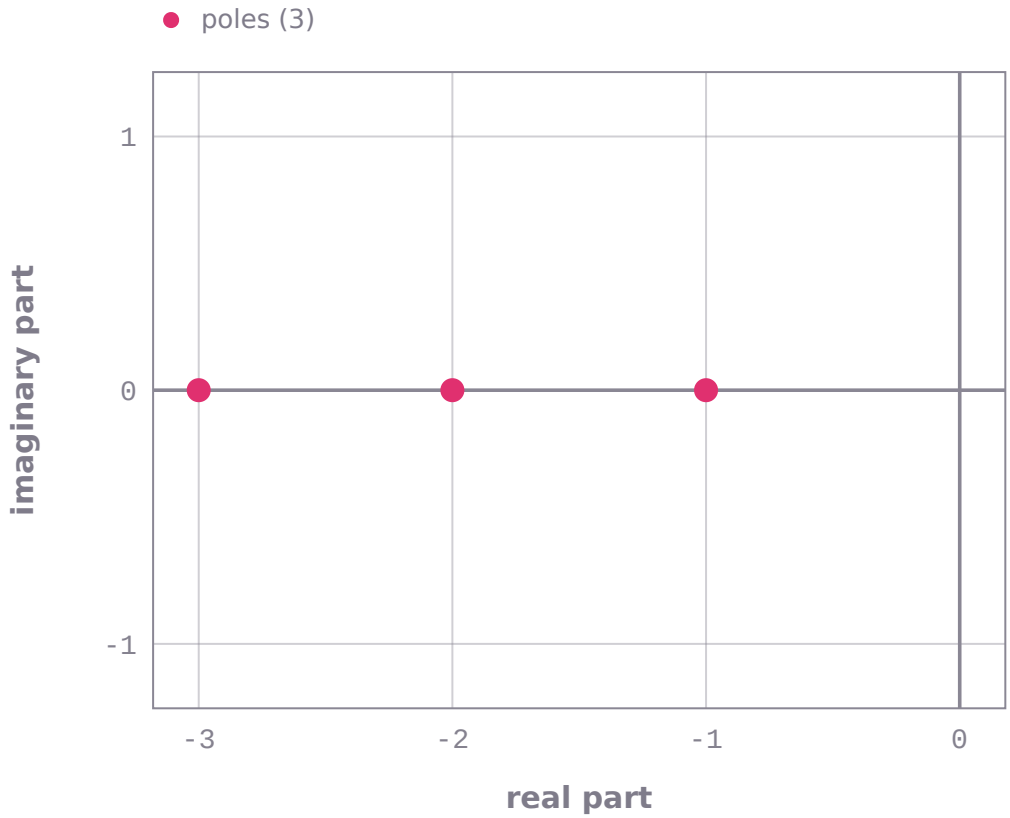
The roots (poles) of a characteristic polynomial and whether the system is asymptotically stable, marginally stable or unstable.

$$\text{stable} \iff \operatorname{Re} p_i < 0 \ \forall i$$

input	meaning	condition
`den`	characteristic polynomial (denominator) coefficients, highest power first	

output	meaning
`poles`	the poles
`max_real_part`	largest real part of the poles
`stability`	asymptotically stable, marginally stable or unstable

- **Method:** roots by the Aberth-Ehrlich iteration; simple poles on the imaginary axis are marginal, repeated ones unstable
- **Assumes:** degree 1 or more
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



the poles (poles) returned by control-systems-ai999x for the worked example (den = [1, 6, 11, 6]), drawn in the complex plane: -3 + 0i, -2 + 0i, -1 + 0i.

```
control_systems_ai999x.solve('ctrl_pole_stability', den=[1, 6, 11, 6])
# -> poles = [-3.0, -2.0, -1.0], max_real_part = -1, stability = asymptotically stable
(SOLVED)
```

```
control_systems_ai999x.solve('ctrl_pole_stability', den=[1, 1, 4, 4])
# -> poles = [-1.0, -2j, 2j], max_real_part = 0, stability = marginally stable   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_pole_stability', den=[1, -1, 2])
# -> poles = [(0.5-1.32287565553j), (0.5+1.32287565553j)], max_real_part = 0.5, stability =
unstable   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
pole stability: den = [1, 6, 11, 6]	SOLVED	poles = [-3.0, -2.0, -1.0], max_real_part = -1, stability = asymptotically stable
pole stability: den = [1, 1, 4, 4]	SOLVED	poles = [-1.0, -2j, 2j], max_real_part = 0, stability = marginally stable
pole stability: den = [1, -1, 2]	SOLVED	poles = [(0.5-1.32287565553j), (0.5+1.32287565553j)], max_real_part = 0.5, stability = unstable
pole stability:	NEEDS_INPUT	ctrl_pole_stability needs den (characteristic polynomial (denominator) coefficients, highest power first). Example: solv
pole stability: den = [1, 6, 11, 6], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

Proportional control of a first-order plant, `ctrl_closed_loop_first_order_p`

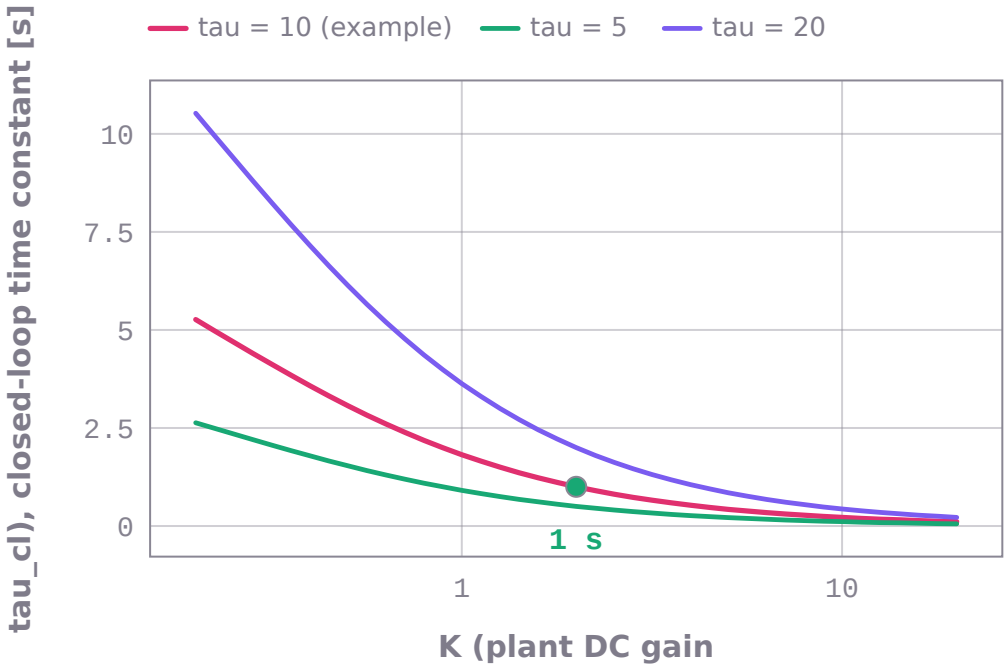
Closed-loop DC gain K Kp/(1 + K Kp), time constant tau/(1 + K Kp), steady-state error 1/(1 + K Kp) to a unit step and the closed-loop pole, for K/(tau s + 1) under unity feedback with gain Kp.

$$T(s) = \frac{K K_p}{\tau s + 1 + K K_p}$$

input	meaning	condition
`K`	plant DC gain	
`tau`	plant time constant in seconds	
`Kp`	proportional gain	

output	meaning
`dc_gain_cl`	closed-loop DC gain
`tau_cl`	closed-loop time constant
`ess`	steady-state error to a unit step
`pole`	closed-loop pole $(1/s)$

- **Method:** T(s) = K Kp/(tau $s + 1 + K$ Kp)
- **Assumes:** $\tau > 0$; $1 + K\ Kp > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



tau_cl (closed-loop time constant) against K (plant DC gain) from 0.2 to 20 on a logarithmic axis. One curve for each value of tau (plant time constant in seconds): 10, 5, 20. Other inputs as in the worked example: Kp = 4.5. 41 points per curve computed by control-systems-ai999x; marked: the worked example, K = 2, tau_cl = 1 s.

```
control_systems_ai999x.solve('ctrl_closed_loop_first_order_p', K=2, tau=10, Kp=4.5)
# -> dc_gain_cl = 0.9, tau_cl = 1 s, ess = 0.1, pole = -1   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_closed_loop_first_order_p', K=1, tau=3, Kp=2)
# -> dc_gain_cl = 0.666667, tau_cl = 1 s, ess = 0.333333, pole = -1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
proportional control of a first-order plant: K = 2, tau = 10, Kp = 4.5	SOLVED	dc_gain_cl = 0.9, tau_cl = 1 s, ess = 0.1, pole = -1
proportional control of a first-order plant: K = 1, tau = 3, Kp = 2	SOLVED	dc_gain_cl = 0.666667, tau_cl = 1 s, ess = 0.333333, pole = -1
proportional control of a first-order plant: tau = 10, Kp = 4.5	NEEDS_INPUT	ctrl_closed_loop_first_order_p needs K (plant DC gain). Example: solve('ctrl_closed_loop_first_order_p', K=2, tau=10, Kp
proportional control of a first-order plant: K = 2, tau = 10, Kp = 4.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
proportional control of a first-order plant: K = 1, tau = 3, Kp = -2	INVALID_INPUT	Proportional control of a first-order plant: the closed loop is unstable (1 + K Kp must be positive).

Second-order specifications from the pole location, `ctrl_second_order_from_poles`

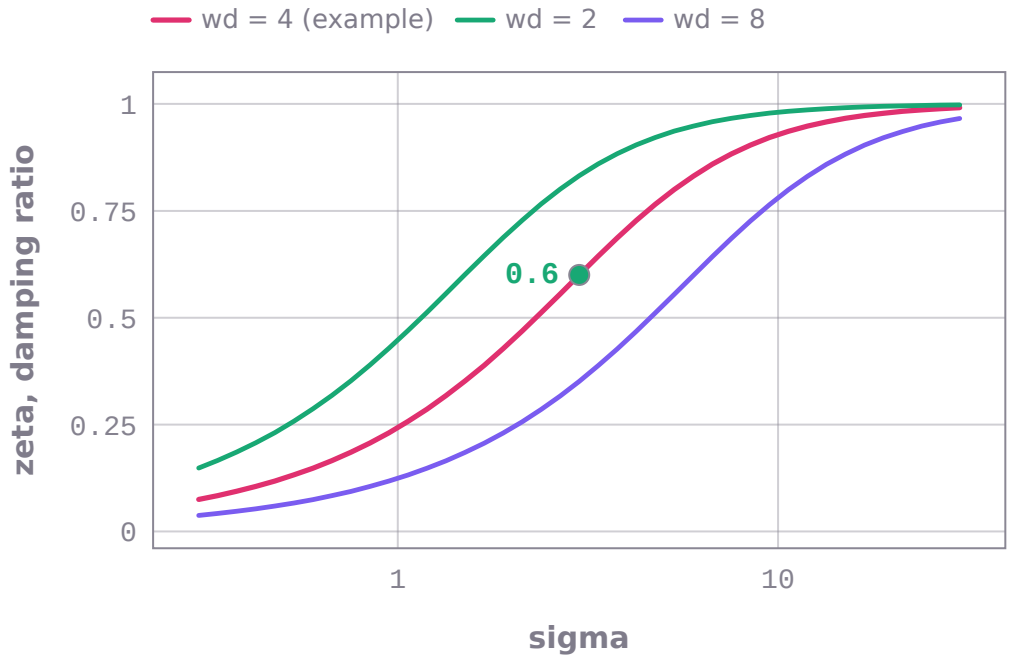
From poles at -sigma +/- j wd: wn, zeta, peak time pi/wd, 2 % settling time $4/\sigma$ and percent overshoot 100 e^{-sigma pi/wd}.

$$\omega_n = \sqrt{\sigma^2 + \omega_d^2}, \quad \zeta = \sigma/\omega_n, \quad \%OS = 100e^{-\sigma\pi/\omega_d}$$

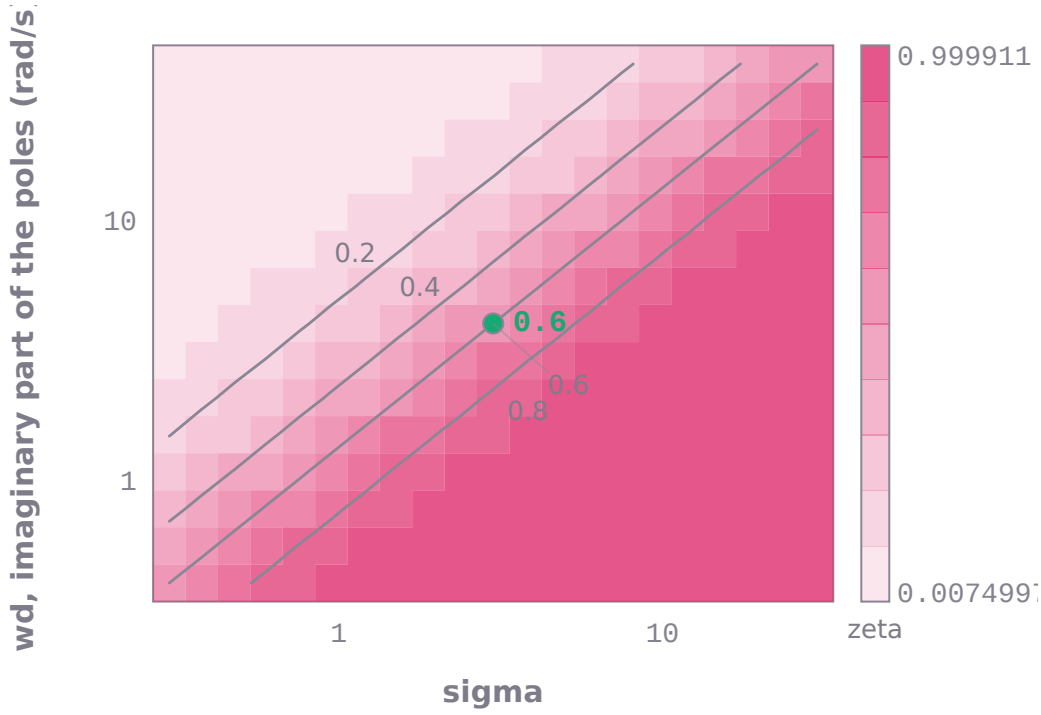
input	meaning	condition
`sigma`	decay rate: the poles' real part is -sigma $(\sigma > 0)$	
`wd`	imaginary part of the poles (rad/s)	

output	meaning
`wn`	natural frequency
`zeta`	damping ratio
`Tp`	peak time
`Ts`	2 % settling time $4/\sigma$
`PO`	percent overshoot (%)

- **Method:** wn = |p|, $\zeta = \sigma/wn$
- **Assumes:** $\sigma > 0$; $wd > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



zeta (damping ratio) against sigma (decay rate: the poles' real part is -sigma (sigma > 0)) from 0.3 to 30 on a logarithmic axis. One curve for each value of wd (imaginary part of the poles (rad/s)): 4, 2, 8. 41 points per curve computed by control-systems-ai999x; marked: the worked example, sigma = 3, zeta = 0.6.



zeta (damping ratio) over sigma (decay rate: the poles' real part is -sigma (sigma > 0)) from 0.3 to 30 and wd (imaginary part of the poles (rad/s)) from 0.4 to 40 (logarithmic axes): each of the 21 × 15 cells is one run of control-systems-ai999x, shaded by its value (light 0.00749979 to dark 0.999911); iso-lines at zeta = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, sigma = 3, wd = 4, zeta = 0.6.

```
control_systems_ai999x.solve('ctrl_second_order_from_poles', sigma=3, wd=4)
# -> wn = 5 rad/s, zeta = 0.6, Tp = 0.785398 s, Ts = 1.33333 s, PO = 9.47802   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_second_order_from_poles', sigma=2, wd=2)
# -> wn = 2.82843 rad/s, zeta = 0.707107, Tp = 1.5708 s, Ts = 2 s, PO = 4.32139   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
second-order specifications from poles: sigma = 3, wd = 4	SOLVED	wn = 5 rad/s, zeta = 0.6, Tp = 0.785398 s, Ts = 1.33333 s, PO = 9.47802
second-order specifications from poles: sigma = 2, wd = 2	SOLVED	wn = 2.82843 rad/s, zeta = 0.707107, Tp = 1.5708 s, Ts = 2 s, PO = 4.32139
second-order specifications from poles: wd = 4	NEEDS_INPUT	ctrl_second_order_from_poles needs sigma (decay rate: the poles' real part is -sigma (sigma > 0)). Example: solve('ctrl_
second-order specifications from poles: sigma = 3, wd = 4, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
second-order specifications from poles: sigma = -3, wd = 4	INVALID_INPUT	Second-order specifications from the pole location: sigma must be positive.

Mapping an s-plane pole to the z-plane (z = e^{sT}), `ctrl_s_to_z_pole`

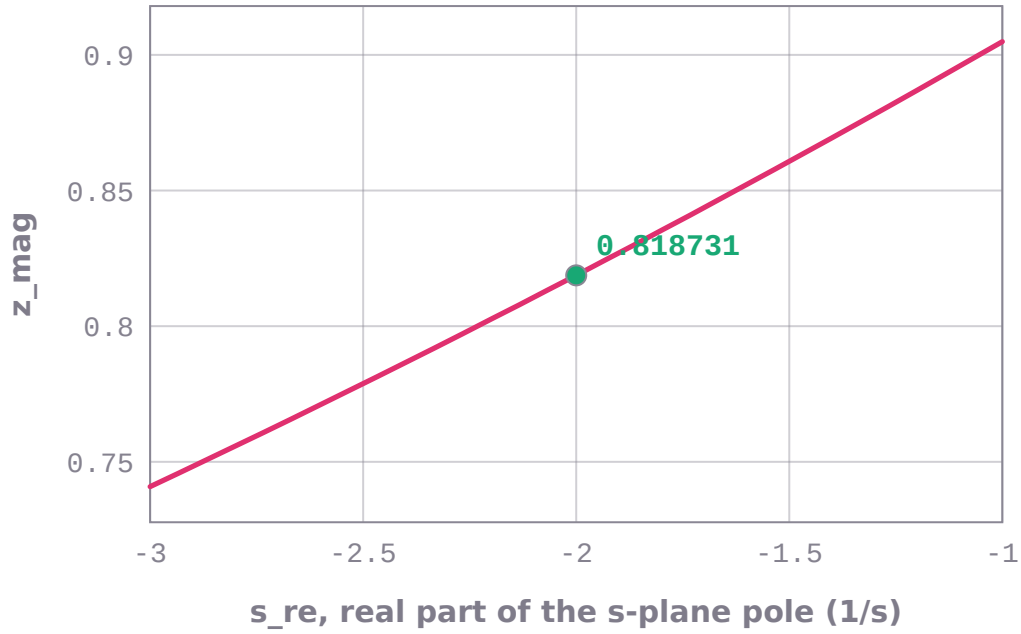
The discrete pole $z = e$^{sT} of a continuous pole $s = s_re + j$ s_im sampled with period T: real and imaginary parts, magnitude e^{s_re T} and angle s_im T.

$$z = e^{sT} = e^{\sigma T} \angle \omega T$$

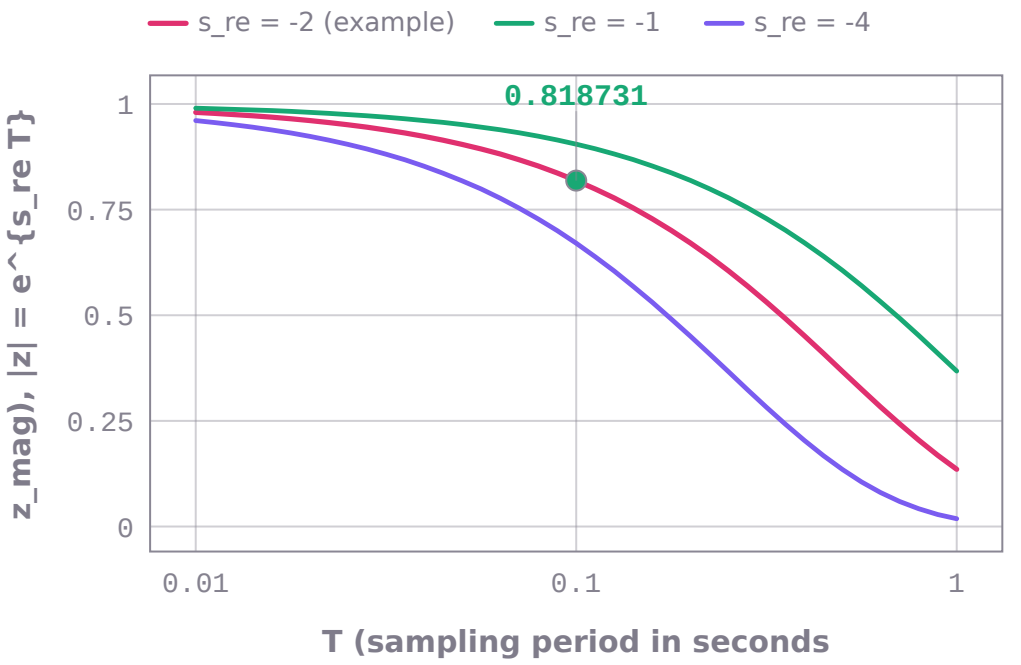
input	meaning	condition
`s_re`	real part of the s-plane pole $(1/s)$	
`s_im`	imaginary part of the pole, >= 0 (rad/s)	
`T`	sampling period in seconds	

output	meaning
`z_re`	real part of z
`z_im`	imaginary part of z
`z_mag`	|z| = e^{s_re T}
`z_angle`	angle of z in radians, s_im T

- **Method:** $z = e$^{s_re T}(cos s_im T + j sin s_im T)
- **Assumes:** $T > 0$; $0 \le s_im\ T < pi$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



z_mag (|z| = e^{s_re T}) against s_re (real part of the s-plane pole (1/s)), with T = 0.1, s_im = 0. 31 points computed by control-systems-ai999x; marked: s_re = -2, z_mag = 0.818731.



z_mag (|z| = e^{s_re T}) against T (sampling period in seconds) from 0.01 to 1 on a logarithmic axis. One curve for each value of s_re (real part of the s-plane pole (1/s)): -2, -1, -4. Other inputs as in the worked example: s_im = 0.0. 41 points per curve computed by control-systems-ai999x; marked: the worked example, T = 0.1, z_mag = 0.818731.

```
control_systems_ai999x.solve('ctrl_s_to_z_pole', s_re=-2, s_im=0, T=0.1)
# -> z_re = 0.818731, z_im = 0, z_mag = 0.818731, z_angle = 0   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_s_to_z_pole', s_re=-1, s_im=3.14159265359, T=0.25)
# -> z_re = 0.550695, z_im = 0.550695, z_mag = 0.778801, z_angle = 0.785398   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
s-plane to z-plane pole: s_re = -2, s_im = 0, T = 0.1	SOLVED	z_re = 0.818731, z_im = 0, z_mag = 0.818731, z_angle = 0
s-plane to z-plane pole: s_re = -1, s_im = 3.14159265359, T = 0.25	SOLVED	z_re = 0.550695, z_im = 0.550695, z_mag = 0.778801, z_angle = 0.785398
s-plane to z-plane pole: s_im = 0, T = 0.1	NEEDS_INPUT	ctrl_s_to_z_pole needs s_re (real part of the s-plane pole (1/s)). Example: solve('ctrl_s_to_z_pole', s_re=-2, s_im=0, T
s-plane to z-plane pole: s_re = -2, s_im = 0, T = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
s-plane to z-plane pole: s_re = -1, s_im = 40, T = 0.1	INVALID_INPUT	Mapping an s-plane pole to the z-plane (z = e^{sT}): the pole's frequency is above the Nyquist limit (s_im T must be bel

Ziegler-Nichols tuning from the ultimate gain and period, `ctrl_zn_ultimate`

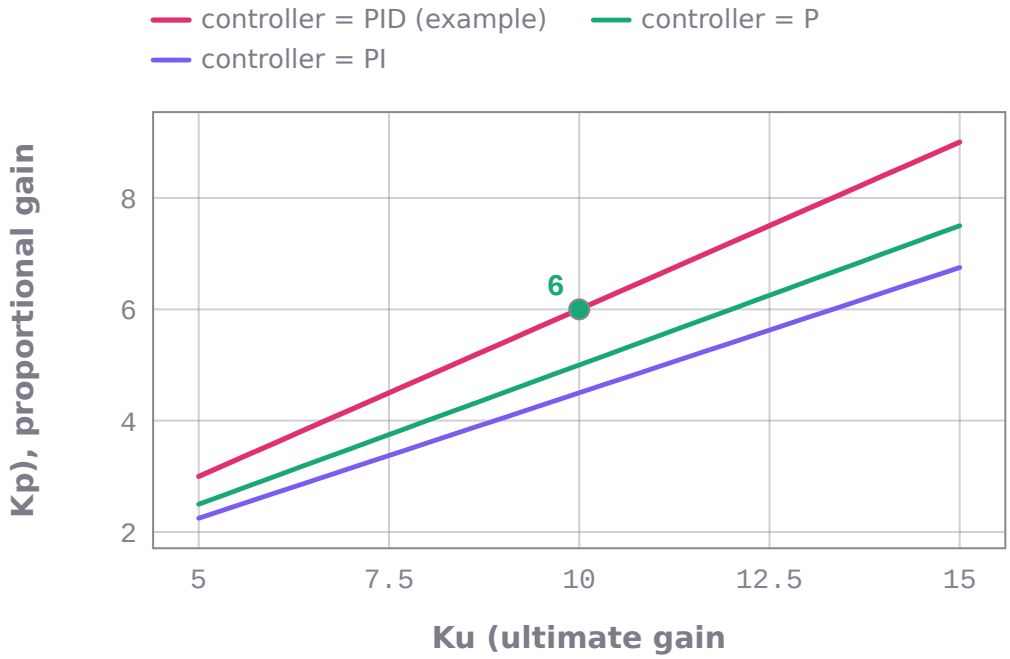
P, PI or PID settings from the closed-loop (ultimate-cycle) Ziegler-Nichols table: P 0.5 Ku; PI 0.45 Ku, $Ti = Pu/1.2$; PID 0.6 Ku, $Ti = Pu/2$, $Td = Pu/8$.

$$\text{PID: } K_p = 0.6K_u,\ T_i = P_u/2,\ T_d = P_u/8$$

input	meaning	condition
`Ku`	ultimate gain (gain at sustained oscillation)	
`Pu`	ultimate period in seconds	
`controller`	controller type	

output	meaning
`Kp`	proportional gain
`Ti`	integral time (0 when there is no integral action)
`Td`	derivative time
`Ki`	integral gain Kp/Ti
`Kd`	derivative gain Kp Td

- **Method:** Ziegler-Nichols ultimate-cycle table
- **Assumes:** $Ku > 0$; $Pu > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Kp (proportional gain) against Ku (ultimate gain (gain at sustained oscillation)) from 5 to 15. One curve for each value of controller (controller type): PID, P, PI. Other inputs as in the worked example: Pu = 4.0. 31 points per curve computed by control-systems-ai999x; marked: the worked example, Ku = 10, Kp = 6.

```
control_systems_ai999x.solve('ctrl_zn_ultimate', Ku=10, Pu=4, controller='PID')
# -> Kp = 6, Ti = 2 s, Td = 0.5 s, Ki = 3, Kd = 3   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_zn_ultimate', Ku=8, Pu=3, controller='PI')
# -> Kp = 3.6, Ti = 2.5 s, Td = 0 s, Ki = 1.44, Kd = 0   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_zn_ultimate', Ku=8, Pu=3, controller='P')
# -> Kp = 4, Ti = 0 s, Td = 0 s, Ki = 0, Kd = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ziegler-Nichols ultimate tuning: Ku = 10, Pu = 4, controller = PID	SOLVED	Kp = 6, Ti = 2 s, Td = 0.5 s, Ki = 3, Kd = 3
Ziegler-Nichols ultimate tuning: Ku = 8, Pu = 3, controller = PI	SOLVED	Kp = 3.6, Ti = 2.5 s, Td = 0 s, Ki = 1.44, Kd = 0
Ziegler-Nichols ultimate tuning: Ku = 8, Pu = 3, controller = P	SOLVED	Kp = 4, Ti = 0 s, Td = 0 s, Ki = 0, Kd = 0
Ziegler-Nichols ultimate tuning: Pu = 4, controller = PID	NEEDS_INPUT	ctrl_zn_ultimate needs Ku (ultimate gain (gain at sustained oscillation)). Example: solve('ctrl_zn_ultimate', Ku=10, Pu=
Ziegler-Nichols ultimate tuning: Ku = 10, Pu = 4, controller = PID, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

Ziegler-Nichols tuning from the process reaction curve, `ctrl_zn_reaction`

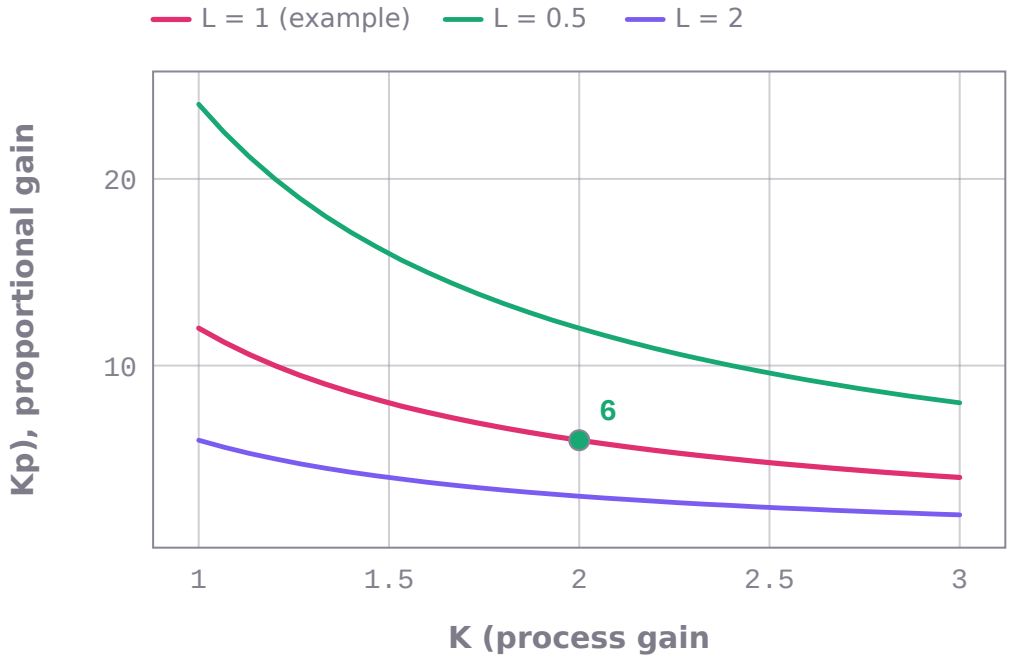
P, PI or PID settings from the open-loop (reaction-curve) Ziegler-Nichols table for a process with gain K, dead time L and time constant T.

$$\text{PID: } K_p = \frac{1.2\,T}{KL},\ T_i = 2L,\ T_d = 0.5L$$

input	meaning	condition
`K`	process gain	
`L`	apparent dead time in seconds	
`T`	apparent time constant in seconds	
`controller`	controller type	

output	meaning
`Kp`	proportional gain
`Ti`	integral time (0 when there is no integral action)
`Td`	derivative time
`Ki`	integral gain Kp/Ti
`Kd`	derivative gain Kp Td

- **Method:** Ziegler-Nichols reaction-curve table
- **Assumes:** K, L, $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Kp (proportional gain) against K (process gain) from 1 to 3. One curve for each value of L (apparent dead time in seconds): 1, 0.5, 2. Other inputs as in the worked example: T = 10.0, controller = PID. 31 points per curve computed by control-systems-ai999x; marked: the worked example, K = 2, Kp = 6.

```
control_systems_ai999x.solve('ctrl_zn_reaction', K=2, L=1, T=10, controller='PID')
# -> Kp = 6, Ti = 2 s, Td = 0.5 s, Ki = 3, Kd = 3   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_zn_reaction', K=2, L=1.5, T=9, controller='PI')
# -> Kp = 2.7, Ti = 5 s, Td = 0 s, Ki = 0.54, Kd = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ziegler-Nichols reaction curve tuning: K = 2, L = 1, T = 10, controller = PID	SOLVED	Kp = 6, Ti = 2 s, Td = 0.5 s, Ki = 3, Kd = 3
Ziegler-Nichols reaction curve tuning: K = 2, L = 1.5, T = 9, controller = PI	SOLVED	Kp = 2.7, Ti = 5 s, Td = 0 s, Ki = 0.54, Kd = 0
Ziegler-Nichols reaction curve tuning: L = 1, T = 10, controller = PID	NEEDS_INPUT	ctrl_zn_reaction needs K (process gain). Example: solve('ctrl_zn_reaction', K=2, L=1, T=10, controller='PID').
Ziegler-Nichols reaction curve tuning: K = 2, L = 1, T = 10, controller = PID, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

PID: standard form to parallel gains, `ctrl_pid_standard_to_parallel`

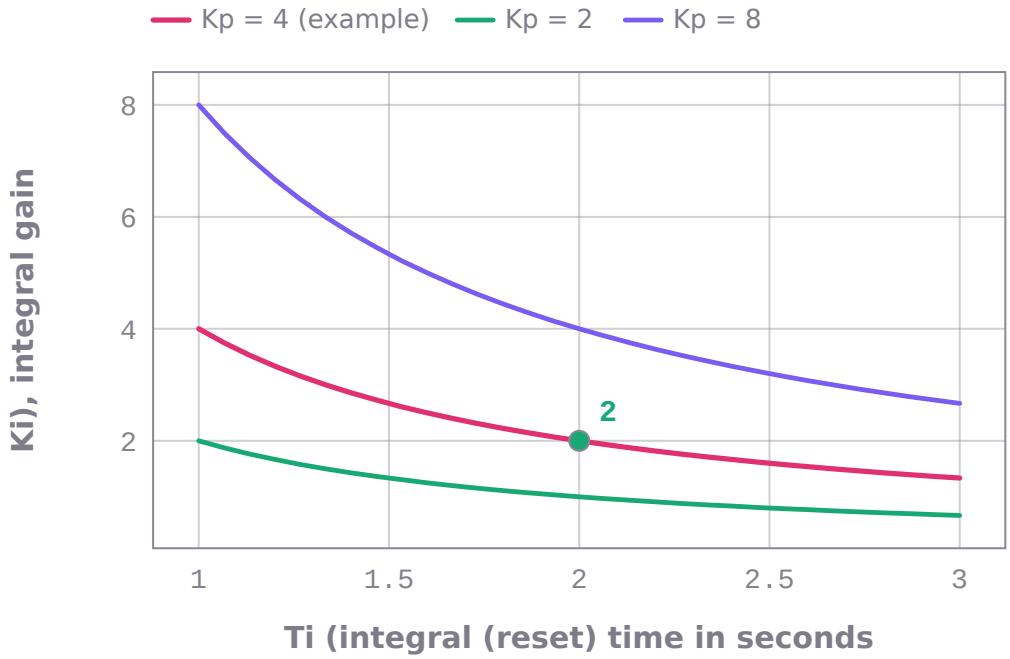
$Ki = Kp/Ti$ and $Kd = Kp$ Td from the standard (ideal ISA) form Kp (1 + 1/(Ti s) + Td s).

$$K_i = K_p/T_i, \quad K_d = K_p T_d$$

input	meaning	condition
`Kp`	proportional gain	
`Ti`	integral (reset) time in seconds	
`Td`	derivative time in seconds	

output	meaning
`Ki`	integral gain
`Kd`	derivative gain

- **Method:** $Ki = Kp/Ti$, $Kd = Kp$ Td
- **Assumes:** $Ti > 0$; $Td \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ki (integral gain) against Ti (integral (reset) time in seconds) from 1 to 3. One curve for each value of Kp (proportional gain): 4, 2, 8. Other inputs as in the worked example: Td = 0.5. 31 points per curve computed by control-systems-ai999x; marked: the worked example, Ti = 2, Ki = 2.

```
control_systems_ai999x.solve('ctrl_pid_standard_to_parallel', Kp=4, Ti=2, Td=0.5)
# -> Ki = 2, Kd = 2   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_pid_standard_to_parallel', Kp=1.2, Ti=3, Td=0)
# -> Ki = 0.4, Kd = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PID standard to parallel: Kp = 4, Ti = 2, Td = 0.5	SOLVED	Ki = 2, Kd = 2
PID standard to parallel: Kp = 1.2, Ti = 3, Td = 0	SOLVED	Ki = 0.4, Kd = 0
PID standard to parallel: Ti = 2, Td = 0.5	NEEDS_INPUT	ctrl_pid_standard_to_parallel needs Kp (proportional gain). Example: solve('ctrl_pid_standard_to_parallel', Kp=4, Ti=2,
PID standard to parallel: Kp = 4, Ti = 2, Td = 0.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
PID standard to parallel: Kp = 4, Ti = 0, Td = 0.5	INVALID_INPUT	PID: standard form to parallel gains: Ti must be positive.

PID: parallel gains to standard form, `ctrl_pid_parallel_to_standard`

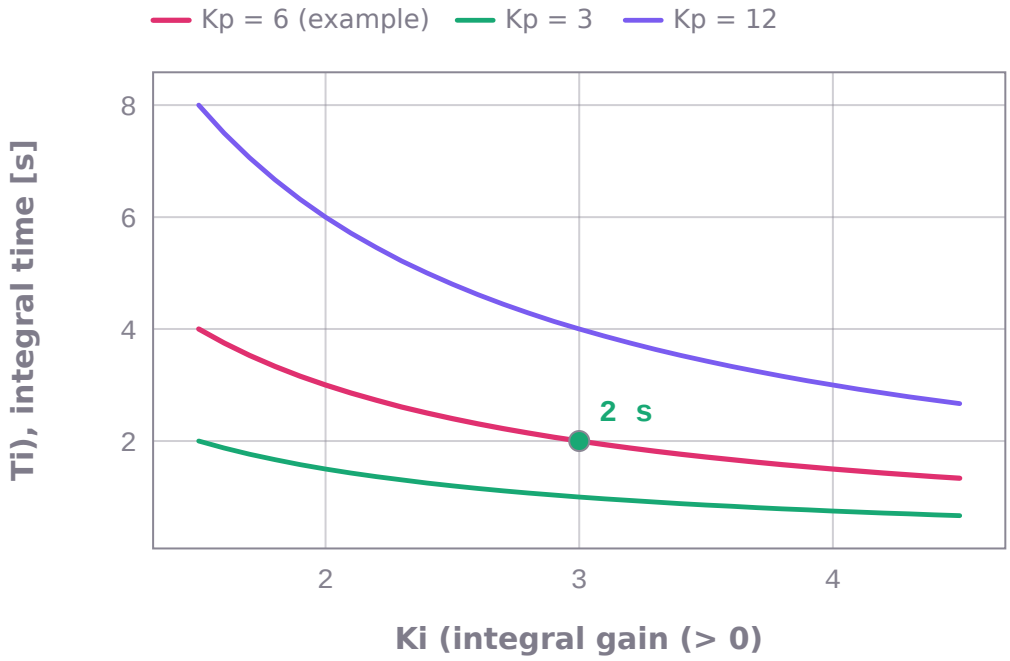
The integral time $Ti = Kp/Ki$ and derivative time $Td = Kd/Kp$ of a parallel PID Kp + Ki/s + Kd s.

$$T_i = K_p/K_i, \quad T_d = K_d/K_p$$

input	meaning	condition
`Kp`	proportional gain (> 0)	
`Ki`	integral gain (> 0)	
`Kd`	derivative gain	

output	meaning
`Ti`	integral time
`Td`	derivative time

- **Method:** $Ti = Kp/Ki, Td = Kd/Kp$
- **Assumes:** $Kp > 0; Ki > 0; Kd \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ti (integral time) against Ki (integral gain (> 0)) from 1.5 to 4.5. One curve for each value of Kp (proportional gain (> 0)): 6, 3, 12. Other inputs as in the worked example: Kd = 3.0. 31 points per curve computed by control-systems-ai999x; marked: the worked example, Ki = 3, Ti = 2 s.

```
control_systems_ai999x.solve('ctrl_pid_parallel_to_standard', Kp=6, Ki=3, Kd=3)
# -> Ti = 2 s, Td = 0.5 s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_pid_parallel_to_standard', Kp=2, Ki=0.5, Kd=0)
# -> Ti = 4 s, Td = 0 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PID parallel to standard: Kp = 6, Ki = 3, Kd = 3	SOLVED	Ti = 2 s, Td = 0.5 s
PID parallel to standard: Kp = 2, Ki = 0.5, Kd = 0	SOLVED	Ti = 4 s, Td = 0 s

question	outcome	result
PID parallel to standard: Ki = 3, Kd = 3	NEEDS_INPUT	ctrl_pid_parallel_to_standard needs Kp (proportional gain (> 0)). Example: solve('ctrl_pid_parallel_to_standard', Kp=6,
PID parallel to standard: Kp = 6, Ki = 3, Kd = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
PID parallel to standard: Kp = 6, Ki = 0, Kd = 3	INVALID_INPUT	PID: parallel gains to standard form: Ki must be positive.

Series (cascade) connection of two transfer functions, `ctrl_series_connection`

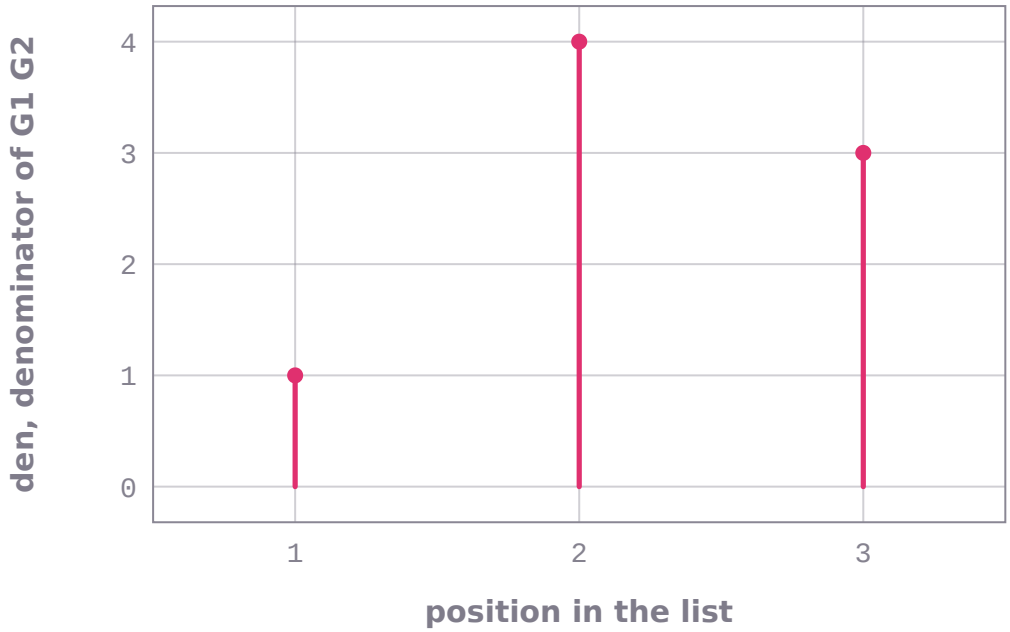
The transfer function G1 G2 of two blocks in series: numerators multiplied, denominators multiplied (no cancellation).

$$G(s) = G_1(s)G_2(s) = \frac{N_1 N_2}{D_1 D_2}$$

input	meaning	condition
`num1`	numerator of G1	
`den1`	denominator of G1	
`num2`	numerator of G2	
`den2`	denominator of G2	

output	meaning
`num`	numerator of G1 G2
`den`	denominator of G1 G2

- **Method:** polynomial multiplication
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



denominator of G1 G2 (den) returned by control-systems-ai999x for the worked example, one value for each position in the list (3 values, first 1, last 3, largest 4). Inputs: num1 = [1], den1 = [1, 1], num2 = [2], den2 = [1, 3].

```
control_systems_ai999x.solve('ctrl_series_connection', num1=[1], den1=[1, 1], num2=[2],
den2=[1, 3])
# -> num = [2.0], den = [1.0, 4.0, 3.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_series_connection', num1=[1, 2], den1=[1, 0], num2=[5],
den2=[1, 5])
# -> num = [5.0, 10.0], den = [1.0, 5.0, 0.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
series connection: num1 = [1], den1 = [1, 1], num2 = [2], den2 = [1, 3]	SOLVED	num = [2.0], den = [1.0, 4.0, 3.0]
series connection: num1 = [1, 2], den1 = [1, 0], num2 = [5], den2 = [1, 5]	SOLVED	num = [5.0, 10.0], den = [1.0, 5.0, 0.0]
series connection: den1 = [1, 1], num2 = [2], den2 = [1, 3]	NEEDS_INPUT	ctrl_series_connection needs num1 (numerator of G1). Example: solve('ctrl_series_connection', num1=[1], den1=[1, 1], num
series connection: num1 = [1], den1 = [1, 1], num2 = [2], den2 = [1, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

Parallel connection of two transfer functions, `ctrl_parallel_connection`

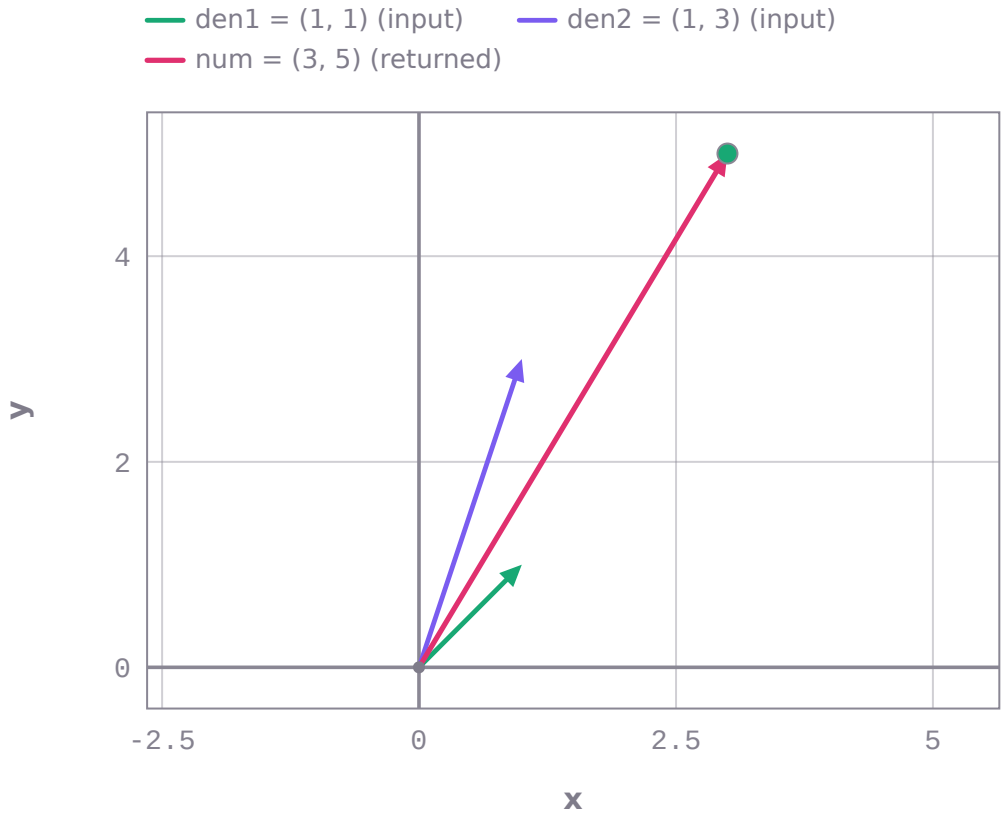
The transfer function $G_1 + G_2$ of two blocks in parallel: (N1 $D_2 + N_2\ D_1$)/(D_1 D2).

$$G_1 + G_2 = \frac{N_1 D_2 + N_2 D_1}{D_1 D_2}$$

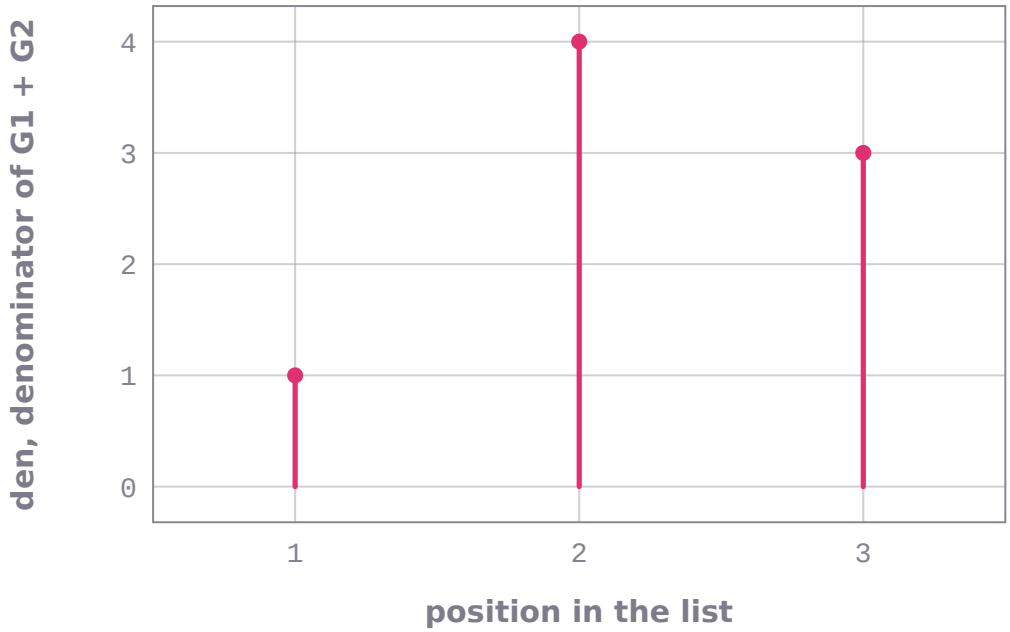
input	meaning	condition
`num1`	numerator of G1	
`den1`	denominator of G1	
`num2`	numerator of G2	
`den2`	denominator of G2	

output	meaning
`num`	numerator of $G_1 + G_2$
`den`	denominator of $G_1 + G_2$

- **Method:** $N = N_1\ D_2 + N_2$ D1, $D = D_1$ D2
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input den1 = (1, 1), den2 = (1, 3); returned by control-systems-ai999x: num = (3, 5). Other inputs: num1 = [1], num2 = [2]. Marked: the tip of each returned vector.



denominator of G1 + G2 (den) returned by control-systems-ai999x for the worked example, one value for each position in the list (3 values, first 1, last 3, largest 4). Inputs: num1 = [1], den1 = [1, 1], num2 = [2], den2 = [1, 3].

```
control_systems_ai999x.solve('ctrl_parallel_connection', num1=[1], den1=[1, 1], num2=[2],
den2=[1, 3])
# -> num = [3.0, 5.0], den = [1.0, 4.0, 3.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_parallel_connection', num1=[1], den1=[1, 0], num2=[1],
den2=[1, 2])
# -> num = [2.0, 2.0], den = [1.0, 2.0, 0.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
parallel connection: num1 = [1], den1 = [1, 1], num2 = [2], den2 = [1, 3]	SOLVED	num = [3.0, 5.0], den = [1.0, 4.0, 3.0]
parallel connection: num1 = [1], den1 = [1, 0], num2 = [1], den2 = [1, 2]	SOLVED	num = [2.0, 2.0], den = [1.0, 2.0, 0.0]
parallel connection: den1 = [1, 1], num2 = [2], den2 = [1, 3]	NEEDS_INPUT	ctrl_parallel_connection needs num1 (numerator of G1). Example: solve('ctrl_parallel_connection', num1=[1], den1= [1, 1],
parallel connection: num1 = [1], den1 = [1, 1], num2 = [2], den2 = [1, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

DC gain and final value of a discrete transfer function, `ctrl_discrete_dc_gain`

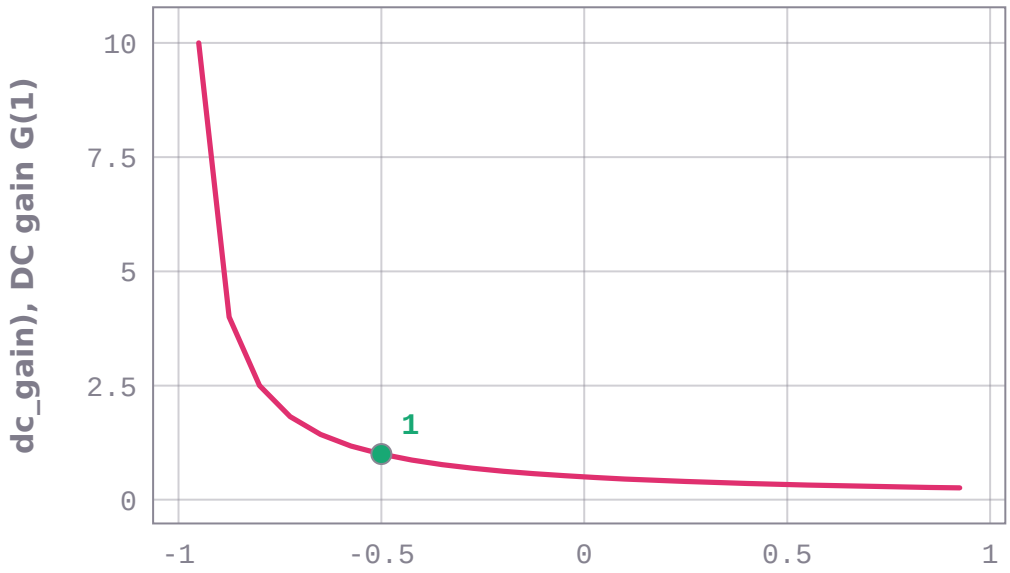
G(1) = sum(num)/sum(den) of a stable discrete transfer function G(z), and the final value of its response to a step of size A.

$$y(\infty) = A\,G(1)$$

input	meaning	condition
`num`	numerator coefficients in z, highest power first	
`den`	denominator coefficients in z, highest power first	
`A`	step size (default 1)	

output	meaning
`dc_gain`	DC gain G(1)
`final_value`	final value of the step response

- **Method:** G(1) after a stability check
- **Assumes:** proper G(z); every pole inside the unit circle
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dc_gain (DC gain G(1)) as value 2 of den (denominator coefficients in z, highest power first) moves from -1.25 to 1.75, the other values as given ([1, -0.5]); 26 runs of control-systems-ai999x. Other inputs: num = [0.5]. Marked: the worked example, den[2] = -0.5, dc_gain = 1.

```
control_systems_ai999x.solve('ctrl_discrete_dc_gain', num=[0.5], den=[1, -0.5])
# -> dc_gain = 1, final_value = 1   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_discrete_dc_gain', num=[1, 0.5], den=[1, -1.2, 0.35],
A=2)
# -> dc_gain = 10, final_value = 20   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
discrete DC gain: num = [0.5], den = [1, -0.5]	SOLVED	dc_gain = 1, final_value = 1
discrete DC gain: num = [1, 0.5], den = [1, -1.2,	SOLVED	dc_gain = 10, final_value = 20

question	outcome	result
0.35], A = 2		
discrete DC gain: den = [1, -0.5]	NEEDS_INPUT	ctrl_discrete_dc_gain needs num (numerator coefficients in z, highest power first). Example: solve('ctrl_discrete_dc_gai
discrete DC gain: num = [0.5], den = [1, -0.5], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
discrete DC gain: num = [1], den = [1, -1.5]	INVALID_INPUT	DC gain and final value of a discrete transfer function: the system is not stable (a pole on or outside the unit circle)

Poles, zeros and gain of a transfer function, `ctrl_poles_zeros`

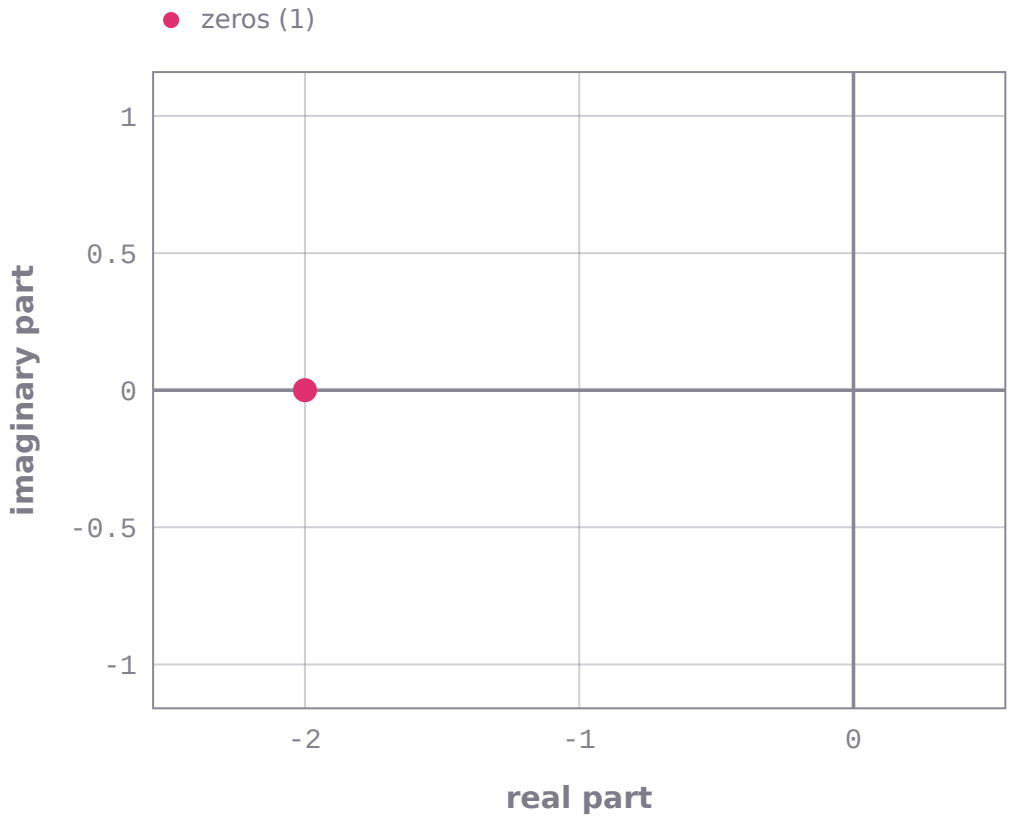
The zeros (roots of the numerator), the poles (roots of the denominator) and the zero-pole-gain constant k of G(s) = num/den.

$$G(s) = k\,\frac{\prod(s - z_i)}{\prod(s - p_j)}$$

input	meaning	condition
`num`	numerator coefficients, highest power of s first	
`den`	denominator coefficients, highest power of s first	

output	meaning
`zeros`	the zeros
`poles`	the poles
`gain`	gain k of the zero-pole-gain form (num[0]/den[0])

- **Method:** roots of both polynomials (Aberth-Ehrlich iteration)
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



the zeros (zeros) returned by control-systems-ai999x for the worked example (num = [1, 2], den = [1, 4, 3]), drawn in the complex plane: -2 + 0i.

```
control_systems_ai999x.solve('ctrl_poles_zeros', num=[1, 2], den=[1, 4, 3])
# -> zeros = [-2.0], poles = [-3.0, -1.0], gain = 1   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_poles_zeros', num=[5], den=[1, 2, 5])
# -> zeros = [], poles = [(-1-2j), (-1+2j)], gain = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
poles and zeros: num = [1, 2], den = [1, 4, 3]	SOLVED	zeros = [-2.0], poles = [-3.0, -1.0], gain = 1
poles and zeros: num = [5], den = [1, 2, 5]	SOLVED	zeros = [], poles = [(-1-2j), (-1+2j)], gain = 5
poles and zeros: den = [1, 4, 3]	NEEDS_INPUT	ctrl_poles_zeros needs num (numerator coefficients, highest power of s first). Example: solve('ctrl_poles_zeros', num=[1
poles and zeros: num = [1, 2], den = [1, 4, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

Value of a transfer function at a complex point, `ctrl_tf_evaluate`

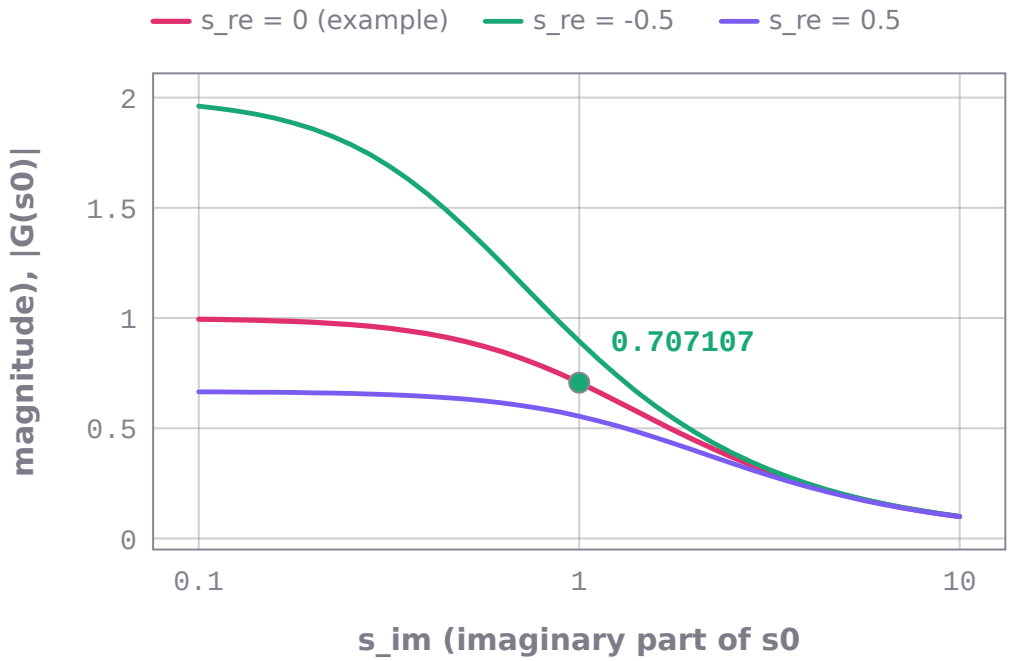
G(s0) = num(s0)/den(s0) at $s_0 = s_r e + j$ s_im: real and imaginary parts, magnitude and angle.

$$G(s_0) = \frac{N(s_0)}{D(s_0)}$$

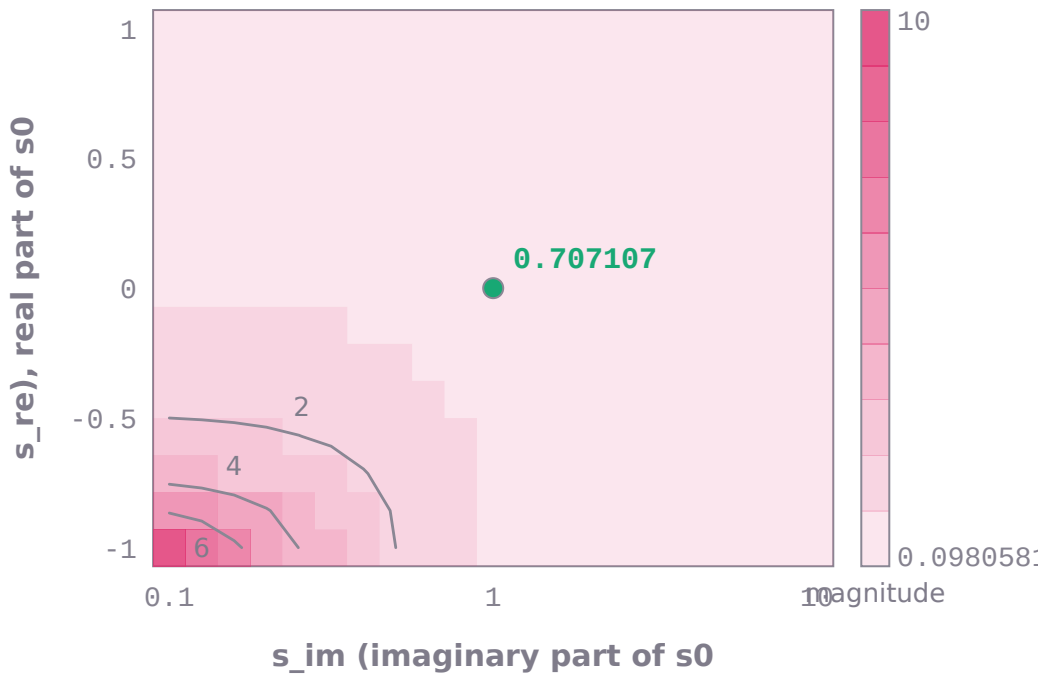
input	meaning	condition
`num`	numerator coefficients	
`den`	denominator coefficients	
`s_re`	real part of s0	
`s_im`	imaginary part of s0	

output	meaning
`G_re`	real part of G(s0)
`G_im`	imaginary part of G(s0)
`magnitude`	|G(s0)|
`angle_deg`	angle of G(s0) in degrees, (-180, 180]

- **Method:** Horner evaluation in complex arithmetic
- **Assumes:** s0 is not a pole
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



magnitude (|G(s0)|) against s_im (imaginary part of s0) from 0.1 to 10 on a logarithmic axis. One curve for each value of s_re (real part of s0): 0, -0.5, 0.5. Other inputs as in the worked example: num = [1], den = [1, 1]. 41 points per curve computed by control-systems-ai999x; marked: the worked example, s_im = 1, magnitude = 0.707107.



magnitude (|G(s0)|) over s_im (imaginary part of s0) from 0.1 to 10 and s_re (real part of s0) from -1 to 1 (logarithmic horizontal axis): each of the 21 × 15 cells is one run of control-systems-ai999x, shaded by its value (light 0.0980581 to dark 10); iso-lines at magnitude = 2, 4, 6. Other inputs as in the worked example: num = [1], den = [1, 1]. Marked: the worked example, s_im = 1, s_re = 0, magnitude = 0.707107.

```
control_systems_ai999x.solve('ctrl_tf_evaluate', num=[1], den=[1, 1], s_re=0, s_im=1)
# -> G_re = 0.5, G_im = -0.5, magnitude = 0.707107, angle_deg = -45   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_tf_evaluate', num=[2, 1], den=[1, 3, 2], s_re=-3,
s_im=1)
# -> G_re = -1.1, G_im = -1.3, magnitude = 1.70294, angle_deg = -130.236   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
evaluate the transfer function: num = [1], den = [1, 1], s_re = 0, s_im = 1	SOLVED	G_re = 0.5, G_im = -0.5, magnitude = 0.707107, angle_deg = -45
evaluate the transfer function: num = [2, 1], den = [1, 3, 2], s_re = -3, s_im = 1	SOLVED	G_re = -1.1, G_im = -1.3, magnitude = 1.70294, angle_deg = -130.236
evaluate the transfer function: den = [1, 1], s_re = 0, s_im = 1	NEEDS_INPUT	ctrl_tf_evaluate needs num (numerator coefficients). Example: solve('ctrl_tf_evaluate', num=[1], den=[1, 1], s_re=0, s_i
evaluate the transfer function: num = [1], den = [1, 1], s_re = 0, s_im = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
evaluate the transfer function: num = [1], den = [1, 1], s_re = -1, s_im = 0	INVALID_INPUT	Value of a transfer function at a complex point: the point is a pole of G(s).

Closed-loop transfer function of a feedback loop, `ctrl_feedback_connection`

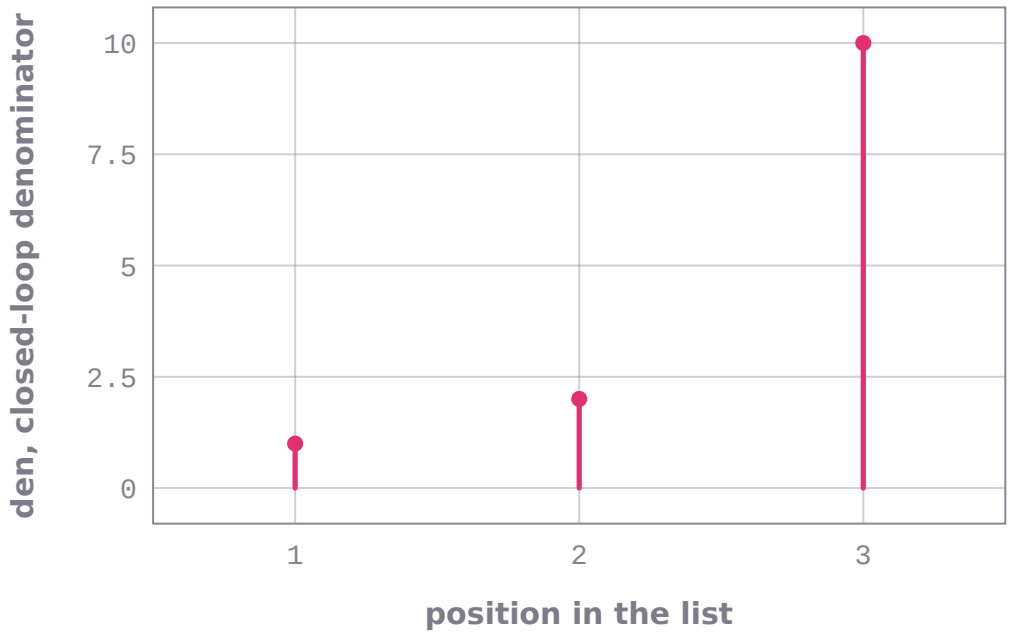
G/(1 + G H) for negative feedback (or G/(1 - G H) for positive feedback): numerator N_G D_H and denominator D_G D_H +/- N_G N_H.

$$T(s) = \frac{G}{1 + GH} = \frac{N_G D_H}{D_G D_H + N_G N_H}$$

input	meaning	condition
`num_g`	numerator of the forward path G	
`den_g`	denominator of G	
`num_h`	numerator of the feedback path H (default 1)	
`den_h`	denominator of H (default 1)	
`sign`	feedback sign (default negative)	

output	meaning
`num`	closed-loop numerator
`den`	closed-loop denominator

- **Method:** $T = N_G$ D_H / (D_G D_H + N_G N_H)
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



closed-loop denominator (den) returned by control-systems-ai999x for the worked example, one value for each position in the list (3 values, first 1, last 10, largest 10). Inputs: num_g = [10], den_g = [1, 2, 0].

```
control_systems_ai999x.solve('ctrl_feedback_connection', num_g=[10], den_g=[1, 2, 0])
# -> num = [10.0], den = [1.0, 2.0, 10.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_feedback_connection', num_g=[1], den_g=[1, 1], num_h=
[2], den_h=[1, 3])
# -> num = [1.0, 3.0], den = [1.0, 4.0, 5.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_feedback_connection', num_g=[1], den_g=[1, 3],
sign='positive')
# -> num = [1.0], den = [1.0, 2.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
feedback connection: num_g = [10], den_g = [1, 2, 0]	SOLVED	num = [10.0], den = [1.0, 2.0, 10.0]
feedback connection: num_g = [1], den_g = [1, 1], num_h = [2], den_h = [1, 3]	SOLVED	num = [1.0, 3.0], den = [1.0, 4.0, 5.0]
feedback connection: num_g = [1], den_g = [1, 3], sign = positive	SOLVED	num = [1.0], den = [1.0, 2.0]
feedback connection: den_g = [1, 2, 0]	NEEDS_INPUT	ctrl_feedback_connection needs num_g (numerator of the forward path G). Example: solve('ctrl_feedback_connection', num_g
feedback connection: num_g = [10], den_g = [1, 2, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

Step response of a standard second-order system, `ctrl_second_order_step`

y(t) of K wn^2/(s^2 + 2 zeta wn s + wn^2) after a unit step, for underdamped, critically damped and overdamped systems, and the remaining error K - y.

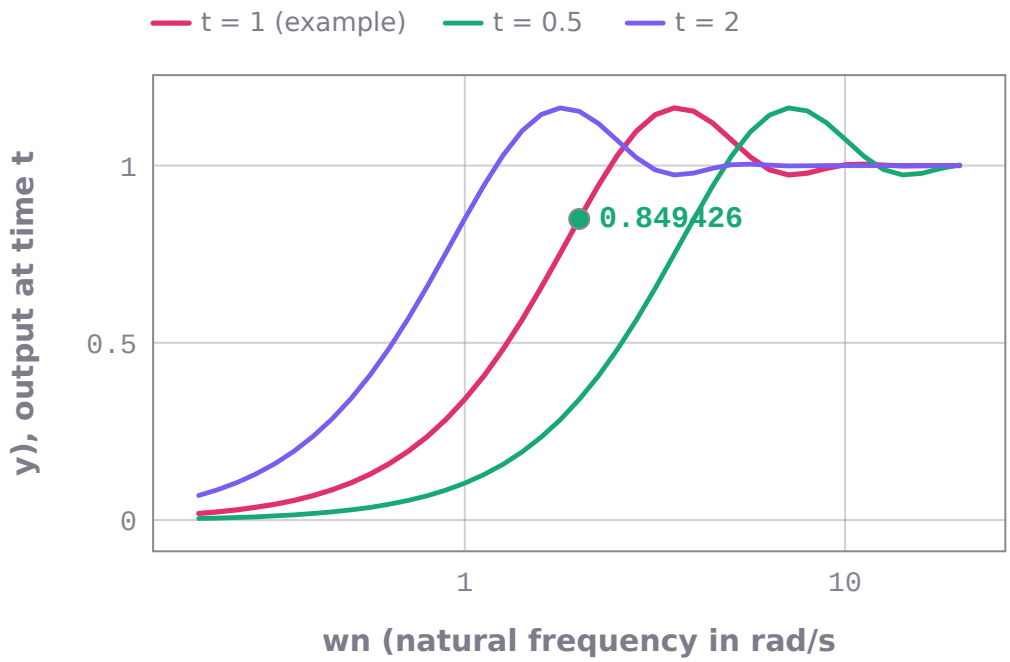
$$y(t) = 1 - e^{-\zeta\omega_n t}\left(\cos\omega_d t + \frac{\zeta}{\sqrt{1-\zeta^2}}\sin\omega_d t\right)$$

input	meaning	condition
`zeta`	damping ratio (>= 0)	
`wn`	natural frequency in rad/s	
`t`	time in seconds	
`K`	DC gain (default 1)	

output	meaning
`y`	output at time t
`error`	K - y(t)

- **Method:** closed-form step response for each damping case
- **Assumes:** $\zeta \geq 0$; $wn > 0$; $t \geq 0$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y (output at time t) against wn (natural frequency in rad/s) from 0.2 to 20 on a logarithmic axis. One curve for each value of t (time in seconds): 1, 0.5, 2. Other inputs as in the worked example: zeta = 0.5. 41 points per curve computed by control-systems-ai999x; marked: the worked example, wn = 2, y = 0.849426.

```
control_systems_ai999x.solve('ctrl_second_order_step', zeta=0.5, wn=2, t=1)
# -> y = 0.849426, error = 0.150574   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_second_order_step', zeta=1, wn=1, t=2)
# -> y = 0.593994, error = 0.406006   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_second_order_step', zeta=2, wn=1, t=1, K=3)
# -> y = 0.53321, error = 2.46679   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
second-order step response: zeta = 0.5, wn = 2, t = 1	SOLVED	y = 0.849426, error = 0.150574
second-order step response: zeta = 1, wn = 1, t = 2	SOLVED	y = 0.593994, error = 0.406006
second-order step response: zeta = 2, wn = 1, t = 1, K = 3	SOLVED	y = 0.53321, error = 2.46679
second-order step response: wn = 2, t = 1	NEEDS_INPUT	ctrl_second_order_step needs zeta (damping ratio (>= 0)). Example: solve('ctrl_second_order_step', zeta=0.5, wn=2, t=1).
second-order step response: zeta = 0.5, wn = 2, t = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

question	outcome	result
second-order step response: zeta = -0.5, wn = 2, t = 1	INVALID_INPUT	Step response of a standard second-order system: zeta must be zero or positive.

Rise time of an underdamped second-order system, `ctrl_rise_time_second`

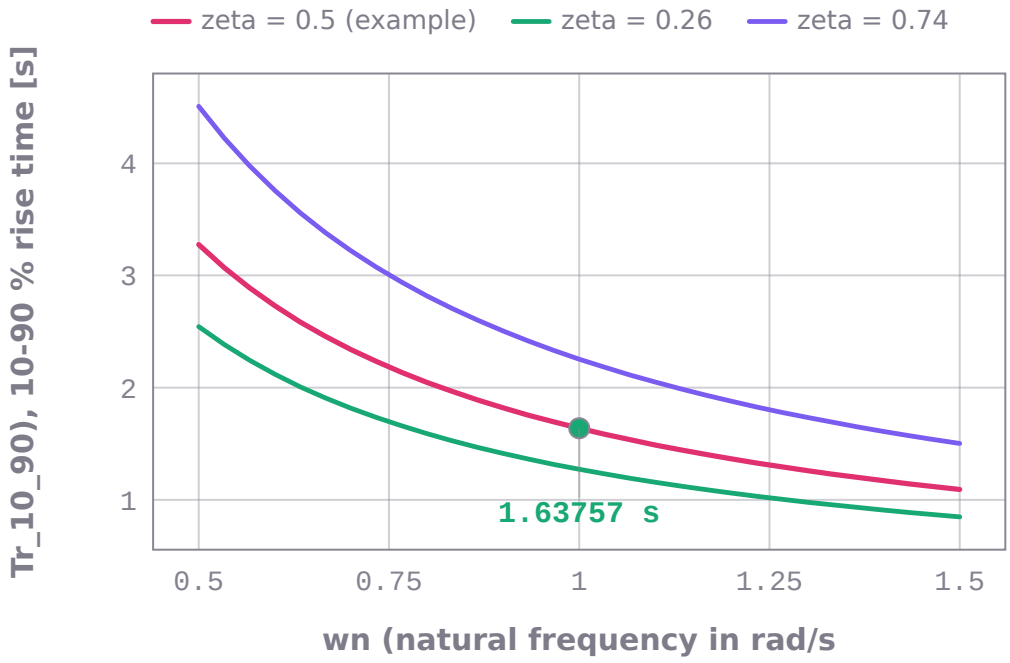
The $0-100$ % rise time (pi - acos zeta)/wd and the $10-90$ % rise time (found exactly by bisection on the step response).

$$T_{r,0-100} = \frac{\pi - \cos^{-1}\zeta}{\omega_d}$$

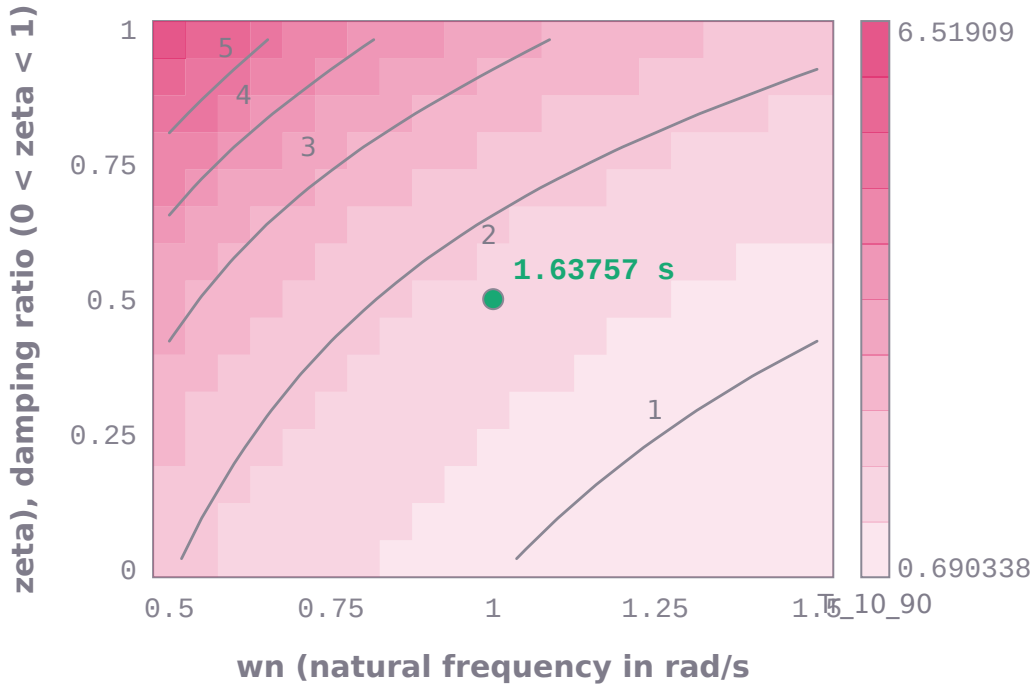
input	meaning	condition
`zeta`	damping ratio $(0 < \zeta < 1)$	
`wn`	natural frequency in rad/s	

output	meaning
`Tr_0_100`	$0-100$ % rise time
`Tr_10_90`	$10-90$ % rise time
`t10`	time to 10 %
`t90`	time to 90 %

- **Method:** closed form (0-100 %); bisection on y(t) (10-90 %)
- **Assumes:** $0 < \zeta < 1$; $wn > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Tr_10_90 (10-90 % rise time) against wn (natural frequency in rad/s) from 0.5 to 1.5. One curve for each value of zeta (damping ratio (0 < zeta < 1)): 0.5, 0.26, 0.74. 31 points per curve computed by control-systems-ai999x; marked: the worked example, wn = 1, Tr_10_90 = 1.63757 s.



Tr_10_90 (10-90 % rise time) over wn (natural frequency in rad/s) from 0.5 to 1.5 and zeta (damping ratio (0 < zeta < 1)) from 0.02 to 0.98: each of the 21 × 15 cells is one run of control-systems-ai999x, shaded by its value (light 0.690338 s to dark 6.51909 s); iso-lines at Tr_10_90 = 1, 2, 3, 4, 5 s. Marked: the worked example, wn = 1, zeta = 0.5, Tr_10_90 = 1.63757 s.

```
control_systems_ai999x.solve('ctrl_rise_time_second', zeta=0.5, wn=1)
# -> Tr_0_100 = 2.4184 s, Tr_10_90 = 1.63757 s, t10 = 0.488229 s, t90 = 2.1258 s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_rise_time_second', zeta=0.7, wn=10)
# -> Tr_0_100 = 0.328533 s, Tr_10_90 = 0.21262 s, t10 = 0.0504835 s, t90 = 0.263104 s
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
rise time: zeta = 0.5, wn = 1	SOLVED	Tr_0_100 = 2.4184 s, Tr_10_90 = 1.63757 s, t10 = 0.488229 s, t90 = 2.1258 s
rise time: zeta = 0.7, wn = 10	SOLVED	Tr_0_100 = 0.328533 s, Tr_10_90 = 0.21262 s, t10 = 0.0504835 s, t90 = 0.263104 s
rise time: wn = 1	NEEDS_INPUT	ctrl_rise_time_second needs zeta (damping ratio (0 < zeta < 1)). Example: solve('ctrl_rise_time_second', zeta=0.5, wn=1)
rise time: zeta = 0.5, wn = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
rise time: zeta = 1, wn = 1	INVALID_INPUT	Rise time of an underdamped second-order system: the rise-time formulas need an underdamped system: 0 < zeta < 1.

Final value theorem, `ctrl_final_value_theorem`

lim_{t->inf} y(t) = lim_{s->0} s Y(s) for Y(s) = num/den, after checking that s Y(s) has every pole in the open left half plane.

$$y(\infty) = \lim_{s \to 0} sY(s)$$

input	meaning	condition
`num`	numerator of Y(s)	
`den`	denominator of Y(s)	

output	meaning
`final_value`	final value of y(t)

- **Method:** cancel factors s, then evaluate s Y(s) at $s = 0$
- **Assumes:** at most one pole at the origin; all other poles in the open left half plane
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Final value theorem as inputs and results: stated num = [10], den = [1, 3, 2, 0]; returned by control-systems-ai999x: final_value = 5.

```
control_systems_ai999x.solve('ctrl_final_value_theorem', num=[10], den=[1, 3, 2, 0])
# -> final_value = 5   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_final_value_theorem', num=[1, 2], den=[1, 4, 3])
# -> final_value = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
final value theorem: num = [10], den = [1, 3, 2, 0]	SOLVED	final_value = 5
final value theorem: num = [1, 2], den = [1, 4, 3]	SOLVED	final_value = 0
final value theorem: den = [1, 3, 2, 0]	NEEDS_INPUT	ctrl_final_value_theorem needs num (numerator of Y(s)). Example: solve('ctrl_final_value_theorem', num=[10], den=[1, 3,
final value theorem: num = [10], den = [1, 3, 2, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
final value theorem: num = [1], den = [1, -1, 0]	INVALID_INPUT	Final value theorem: s Y(s) has a pole in the right half plane or on the imaginary axis: the final value theorem does no
final value theorem: num = [1], den = [1, 0, 0]	INVALID_INPUT	Final value theorem: Y(s) has a repeated pole at the origin: y(t) grows without bound.

Initial value theorem (value and slope at t = 0+), `ctrl_initial_value_theorem`

y(0+) = lim_{s->inf} s Y(s) and y'(0+) = lim_{s->inf} s (s Y(s) - y(0+)) for a strictly proper Y(s).

$$y(0^+) = \lim_{s \to \infty} sY(s)$$

input	meaning	condition
`num`	numerator of Y(s)	
`den`	denominator of Y(s)	

output	meaning
`y0`	initial value y(0+)
`dy0`	initial slope y'(0+)

- **Method:** long division of num by den in powers of $1/s$
- **Assumes:** Y(s) strictly proper
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Initial value theorem (value and slope at t = 0+) as inputs and results: stated num = [2, 3], den = [1, 5, 6]; returned by control-systems-ai999x: y0 = 2, dy0 = -7.

```
control_systems_ai999x.solve('ctrl_initial_value_theorem', num=[2, 3], den=[1, 5, 6])
# -> y0 = 2, dy0 = -7   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_initial_value_theorem', num=[4], den=[1, 2, 4])
# -> y0 = 0, dy0 = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
initial value theorem: num = [2, 3], den = [1, 5, 6]	SOLVED	y0 = 2, dy0 = -7
initial value theorem: num = [4], den = [1, 2, 4]	SOLVED	y0 = 0, dy0 = 4
initial value theorem: den = [1, 5, 6]	NEEDS_INPUT	ctrl_initial_value_theorem needs num (numerator of Y(s)). Example: solve('ctrl_initial_value_theorem', num=[2, 3], den=[
initial value theorem: num = [2, 3], den = [1, 5, 6], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
initial value theorem: num = [1, 2], den = [1, 3]	INVALID_INPUT	Initial value theorem (value and slope at t = 0+): the transfer function must be strictly proper (numerator degree below

System type, static error constant and steady-state error, `ctrl_error_constants`

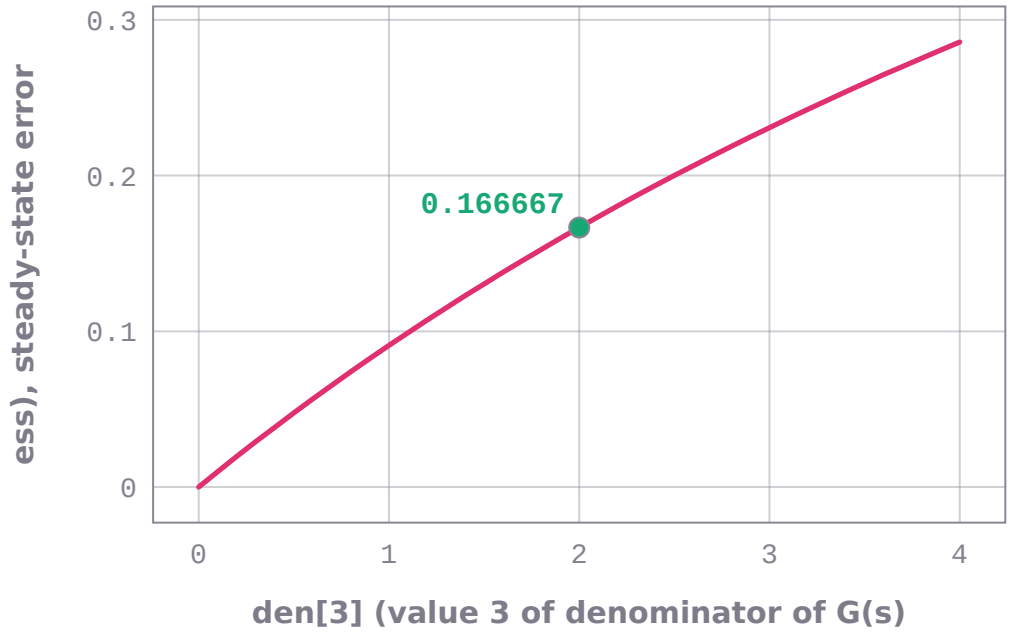
For unity feedback around G(s): the system type N (poles at the origin), the static error constant lim s^N G(s) (Kp, Kv or Ka) and the steady-state error to a step, ramp or parabola of size A.

$$e_{ss} = \frac{A}{1+K_p},\ \frac{A}{K_v},\ \frac{A}{K_a}$$

input	meaning	condition
`num`	numerator of the open-loop G(s)	
`den`	denominator of G(s)	
`input`	test input	
`A`	input size (default 1)	

output	meaning
`system_type`	system type N
`K_static`	static error constant of type N (Kp if 0, Kv if 1, Ka if 2)
`ess`	steady-state error

- **Method:** ess = A/(1 + Kp) (step, type 0), A/Kv (ramp, type 1), A/Ka (parabola, type 2), 0 for a higher type
- **Assumes:** closed loop stable; proper G(s); type at least the input's order
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ess (steady-state error) as value 3 of den (denominator of G(s)) moves from 0 to 4, the other values as given ([1, 3, 2]); 41 runs of control-systems-ai999x. Other inputs: num = [10], input = step. Marked: the worked example, den[3] = 2, ess = 0.166667.

```
control_systems_ai999x.solve('ctrl_error_constants', num=[10], den=[1, 3, 2], input='step')
# -> system_type = 0, K_static = 5, ess = 0.166667   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_error_constants', num=[10], den=[1, 5, 4, 0],
input='ramp', A=2)
# -> system_type = 1, K_static = 2.5, ess = 0.8   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_error_constants', num=[10], den=[1, 5, 4, 0],
input='step')
# -> system_type = 1, K_static = 2.5, ess = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
steady-state error: num = [10], den = [1, 3, 2], input = step	SOLVED	system_type = 0, K_static = 5, ess = 0.166667
steady-state error: num = [10], den = [1, 5, 4, 0], input = ramp, A = 2	SOLVED	system_type = 1, K_static = 2.5, ess = 0.8
steady-state error: num = [10], den = [1, 5, 4, 0], input = step	SOLVED	system_type = 1, K_static = 2.5, ess = 0
steady-state error: den = [1, 3, 2], input = step	NEEDS_INPUT	ctrl_error_constants needs num (numerator of the open-loop G(s)). Example: solve('ctrl_error_constants', num=[10], den=[
steady-state error: num = [10], den = [1, 3, 2], input = step, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
steady-state error: num = [10], den = [1, 3, 2], input = ramp	INVALID_INPUT	System type, static error constant and steady-state error: the steady-state error grows without bound (the system type i

Bode magnitude and phase at one frequency, `ctrl_bode_point`

|G(jw)|, 20 log10 |G(jw)| and the phase of G(jw) (principal value) of a transfer function at the angular frequency w.

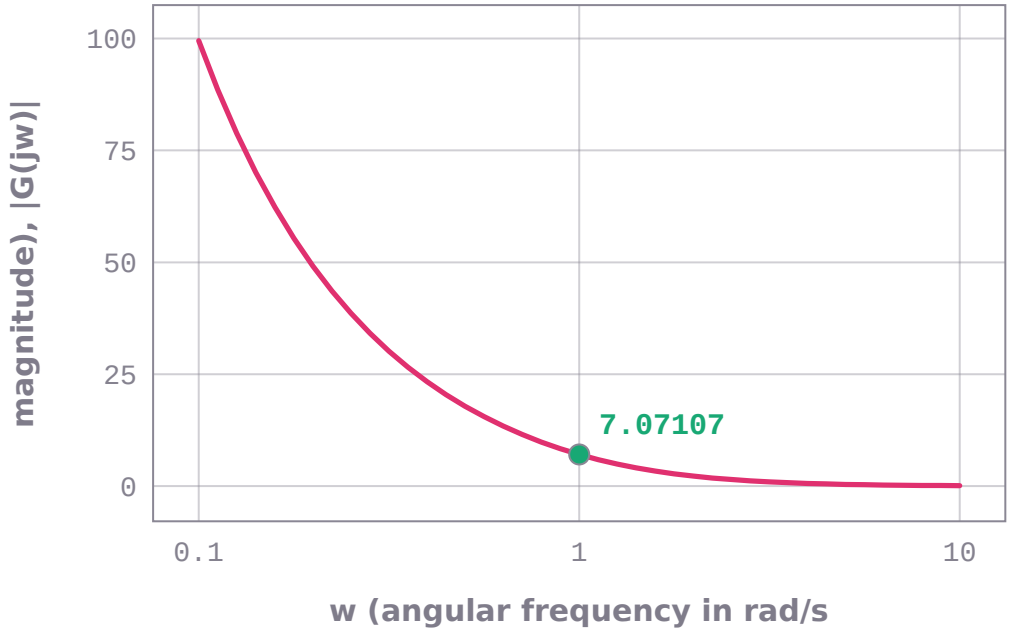
$$|G(j\omega)|_{dB} = 20\log_{10}|G(j\omega)|$$

input	meaning	condition
`num`	numerator coefficients	
`den`	denominator coefficients	
`w`	angular frequency in rad/s	

output	meaning
`magnitude`	|G(jw)|
`magnitude_db`	magnitude in dB
`phase_deg`	phase in degrees, (-180, 180]

- **Method:** G(jw) by complex evaluation
- **Assumes:** proper G(s); $w > 0$; not a pole on the imaginary axis

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



magnitude (|G(jw)|) against w (angular frequency in rad/s) from 0.1 to 10 on a logarithmic axis. Other inputs as in the worked example: num = [10], den = [1, 1, 0]. 41 points computed by control-systems-ai999x; marked: the worked example, w = 1, magnitude = 7.07107.

```
control_systems_ai999x.solve('ctrl_bode_point', num=[10], den=[1, 1, 0], w=1)
# -> magnitude = 7.07107, magnitude_db = 16.9897, phase_deg = -135   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_bode_point', num=[100], den=[1, 10, 100], w=10)
# -> magnitude = 1, magnitude_db = 0, phase_deg = -90   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bode point: num = [10], den = [1, 1, 0], w = 1	SOLVED	magnitude = 7.07107, magnitude_db = 16.9897, phase_deg = -135
Bode point: num = [100], den = [1, 10, 100], w = 10	SOLVED	magnitude = 1, magnitude_db = 0, phase_deg = -90
Bode point: den = [1, 1, 0], w = 1	NEEDS_INPUT	ctrl_bode_point needs num (numerator coefficients). Example: solve('ctrl_bode_point', num=[10], den=[1, 1, 0], w=1).
Bode point: num = [10], den = [1, 1, 0], w = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Bode point: num = [1], den = [1, 0, 4], w = 2	INVALID_INPUT	Bode magnitude and phase at one frequency: the transfer function has a pole at this point.

Frequency response of a standard second-order system, `ctrl_second_order_freq`

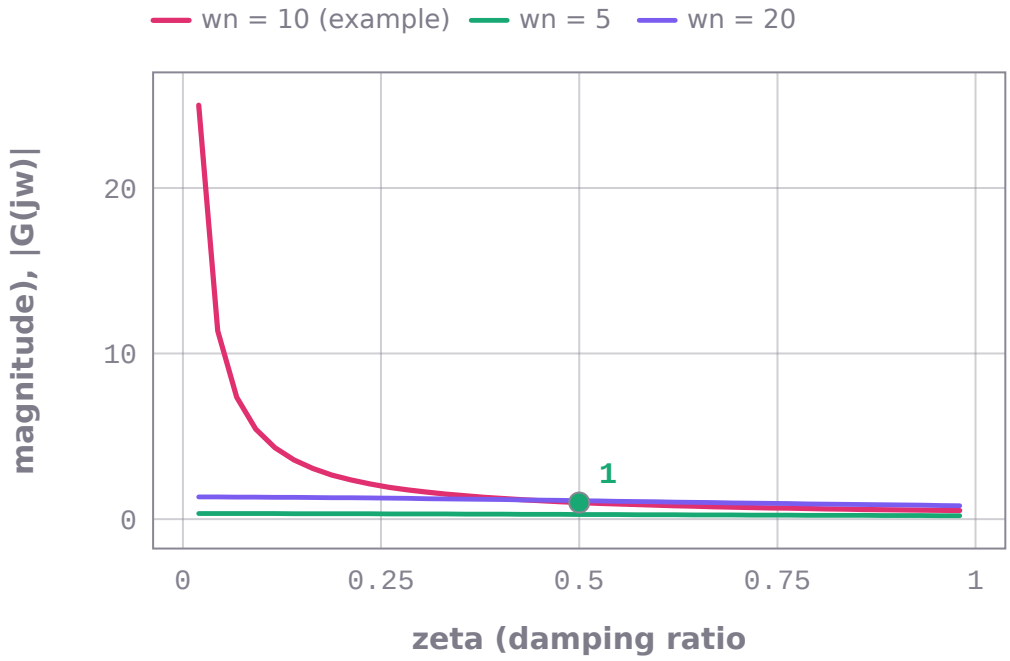
Magnitude, dB gain and phase (between 0 and -180 degrees) of wn^2/(s^2 + 2 zeta wn s + wn^2) at the angular frequency w.

$$|G| = \frac{\omega_n^2}{\sqrt{(\omega_n^2 - \omega^2)^2 + (2\zeta\omega_n\omega)^2}}$$

input	meaning	condition
`zeta`	damping ratio	
`wn`	natural frequency in rad/s	
`w`	angular frequency in rad/s	

output	meaning
`magnitude`	|G(jw)|
`magnitude_db`	magnitude in dB
`phase_deg`	phase in degrees, [-180, 0]

- **Method:** G(jw) = wn^2/(wn^2 - w^2 + 2 j zeta wn w)
- **Assumes:** $\zeta \geq 0$; $wn > 0$; $w > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



magnitude (|G(jw)|) against zeta (damping ratio) from 0.02 to 0.98. One curve for each value of wn (natural frequency in rad/s): 10, 5, 20. Other inputs as in the worked example: w = 10.0. 41 points per curve computed by control-systems-ai999x; marked: the worked example, zeta = 0.5, magnitude = 1.

```
control_systems_ai999x.solve('ctrl_second_order_freq', zeta=0.5, wn=10, w=10)
# -> magnitude = 1, magnitude_db = 0, phase_deg = -90   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_second_order_freq', zeta=0.1, wn=1, w=1)
# -> magnitude = 5, magnitude_db = 13.9794, phase_deg = -90   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
second-order frequency response: zeta = 0.5, wn = 10, w = 10	SOLVED	magnitude = 1, magnitude_db = 0, phase_deg = -90
second-order frequency response: zeta = 0.1, wn = 1, w = 1	SOLVED	magnitude = 5, magnitude_db = 13.9794, phase_deg = -90
second-order frequency response: wn = 10, w = 10	NEEDS_INPUT	ctrl_second_order_freq needs zeta (damping ratio). Example: solve('ctrl_second_order_freq', zeta=0.5, wn=10, w=10).
second-order frequency response: zeta = 0.5, wn = 10, w = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
second-order frequency response: zeta = 0, wn = 1, w = 1	INVALID_INPUT	Frequency response of a standard second-order system: the frequency is at an undamped resonance.

Bandwidth of a standard second-order system, `ctrl_bandwidth_second`

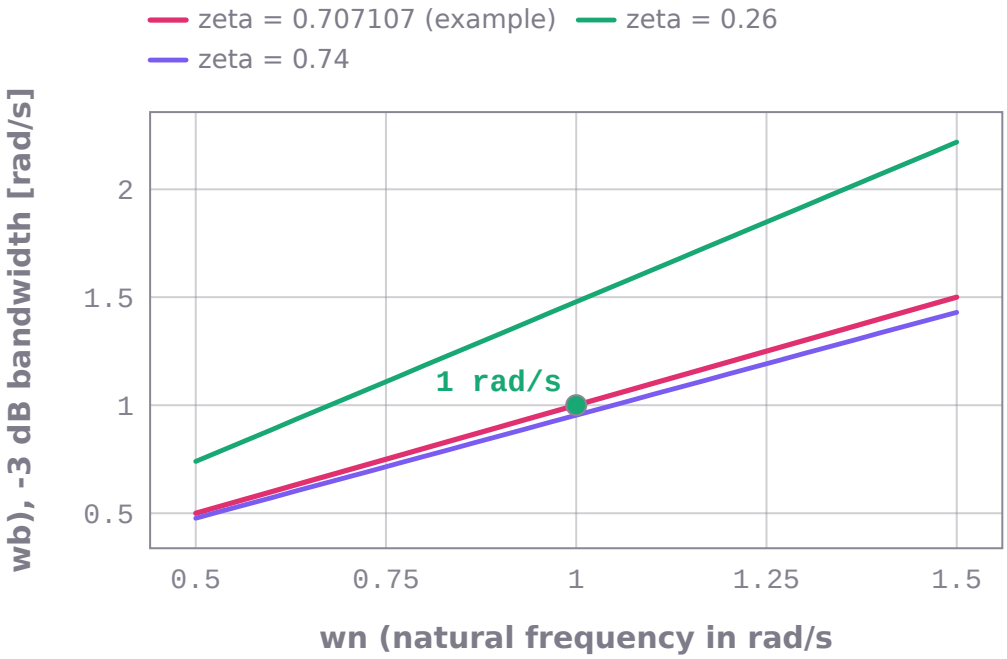
The -3 dB bandwidth wn sqrt($1 - 2\,\zeta^2 + sqrt$(4 $\zeta^4 - 4$ zeta^2 + 2)) of wn^2/(s^2 + 2 zeta wn s + wn^2).

$$\omega_B = \omega_n \sqrt{1 - 2\zeta^2 + \sqrt{4\zeta^4 - 4\zeta^2 + 2}}$$

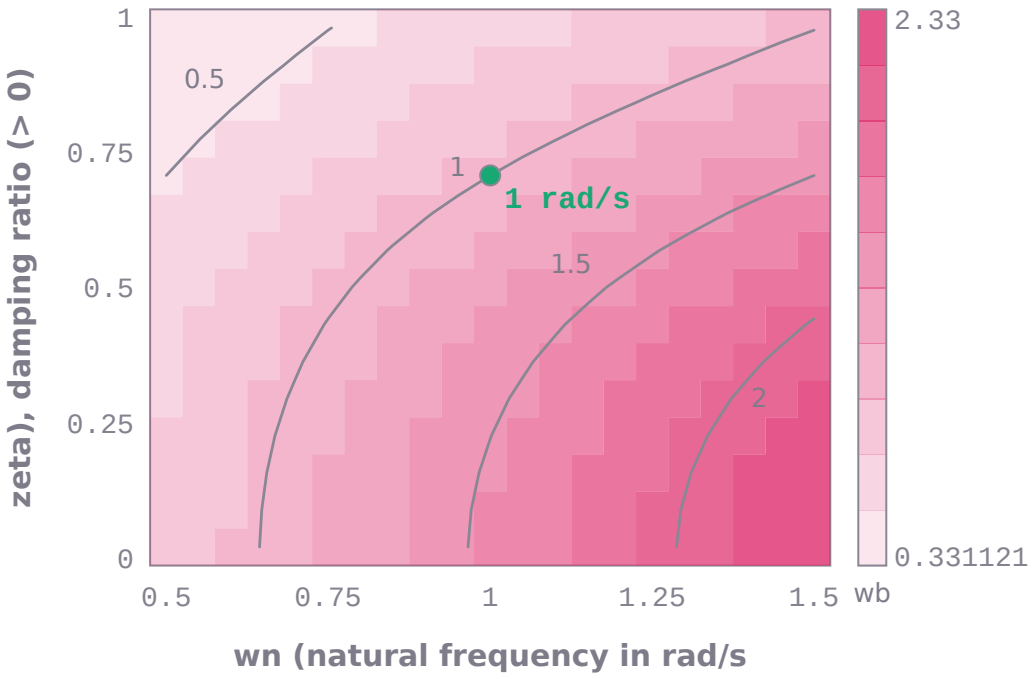
input	meaning	condition
`zeta`	damping ratio (> 0)	
`wn`	natural frequency in rad/s	

output	meaning
`wb`	-3 dB bandwidth

- **Method:** solve |G(jw)| = $1/sqrt$(2)
- **Assumes:** $\zeta > 0$; $wn > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



wb (-3 dB bandwidth) against wn (natural frequency in rad/s) from 0.5 to 1.5. One curve for each value of zeta (damping ratio (> 0)): 0.707107, 0.26, 0.74. 31 points per curve computed by control-systems-ai999x; marked: the worked example, wn = 1, wb = 1 rad/s.



wb (-3 dB bandwidth) over wn (natural frequency in rad/s) from 0.5 to 1.5 and zeta (damping ratio (> 0)) from 0.02 to 0.98: each of the 21 × 15 cells is one run of control-systems-ai999x, shaded by its value (light 0.331121 rad/s to dark 2.33 rad/s); iso-lines at wb = 0.5, 1, 1.5, 2 rad/s. Marked: the worked example, wn = 1, zeta = 0.707107, wb = 1 rad/s.

```
control_systems_ai999x.solve('ctrl_bandwidth_second', zeta=0.7071067811865476, wn=1)
# -> wb = 1 rad/s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_bandwidth_second', zeta=0.5, wn=10)
# -> wb = 12.7202 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
second-order bandwidth: zeta = 0.7071067811865476, wn = 1	SOLVED	wb = 1 rad/s
second-order bandwidth: zeta = 0.5, wn = 10	SOLVED	wb = 12.7202 rad/s
second-order bandwidth: wn = 1	NEEDS_INPUT	ctrl_bandwidth_second needs zeta (damping ratio (> 0)). Example: solve('ctrl_bandwidth_second', zeta=0.707107, wn=1).
second-order bandwidth: zeta = 0.7071067811865476, wn = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
second-order bandwidth: zeta = 0, wn = 10	INVALID_INPUT	Bandwidth of a standard second-order system: zeta must be positive.

Resonant peak and resonant frequency, `ctrl_resonant_peak`

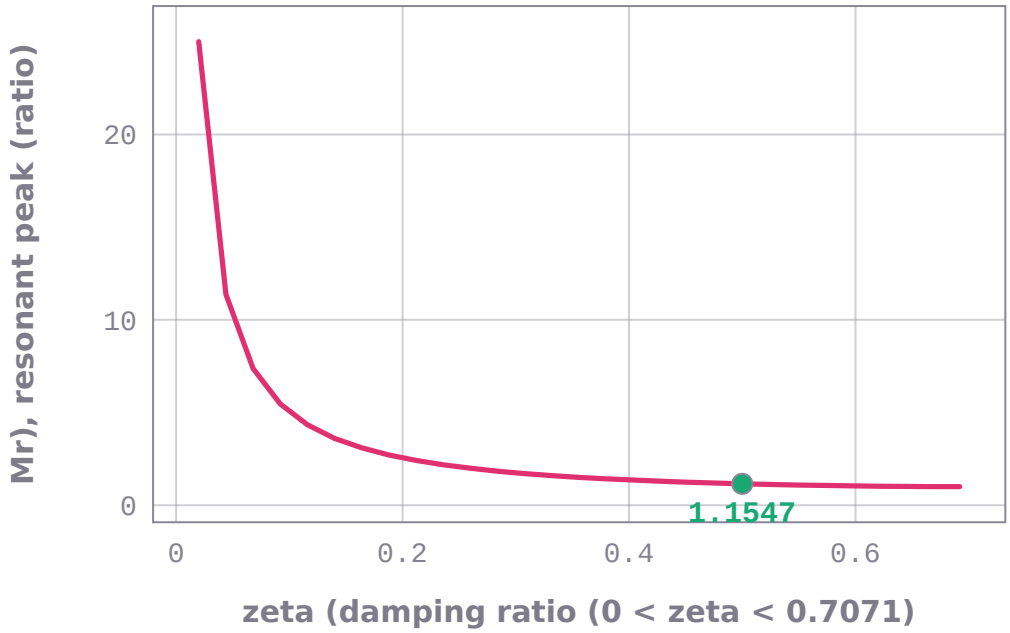
Peak magnitude Mr = 1/(2 zeta sqrt(1 - zeta^2)) and resonant frequency $wr = wn$ sqrt($1 - 2$ zeta^2) of a standard second-order system.

$$M_r = \frac{1}{2\zeta\sqrt{1-\zeta^2}}, \quad \omega_r = \omega_n\sqrt{1-2\zeta^2}$$

input	meaning	condition
`zeta`	damping ratio $(0 < \zeta < 0.7071)$	
`wn`	natural frequency in rad/s	

output	meaning
`Mr`	resonant peak (ratio)
`Mr_db`	resonant peak in dB
`wr`	resonant frequency

- **Method:** maximum of |G(jw)|
- **Assumes:** $0 < \zeta < 1/sqrt(2)$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Mr (resonant peak (ratio)) against zeta (damping ratio (0 < zeta < 0.7071)) from 0.02 to 0.98. Other inputs as in the worked example: wn = 10.0. 29 points computed by control-systems-ai999x; marked: the worked example, zeta = 0.5, Mr = 1.1547.

```
control_systems_ai999x.solve('ctrl_resonant_peak', zeta=0.5, wn=10)
# -> Mr = 1.1547, Mr_db = 1.24939, wr = 7.07107 rad/s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_resonant_peak', zeta=0.1, wn=1)
# -> Mr = 5.02519, Mr_db = 14.023, wr = 0.989949 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
resonant peak: zeta = 0.5, wn = 10	SOLVED	Mr = 1.1547, Mr_db = 1.24939, wr = 7.07107 rad/s
resonant peak: zeta = 0.1, wn = 1	SOLVED	Mr = 5.02519, Mr_db = 14.023, wr = 0.989949 rad/s
resonant peak: wn = 10	NEEDS_INPUT	ctrl_resonant_peak needs zeta (damping ratio (0 < zeta < 0.7071)). Example: solve('ctrl_resonant_peak', zeta=0.5, wn=10)
resonant peak: zeta = 0.5, wn = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
resonant peak: zeta = 0.8, wn = 10	INVALID_INPUT	Resonant peak and resonant frequency: there is no resonant peak unless zeta < 0.7071.

Routh-Hurwitz stability test, `ctrl_routh_hurwitz`

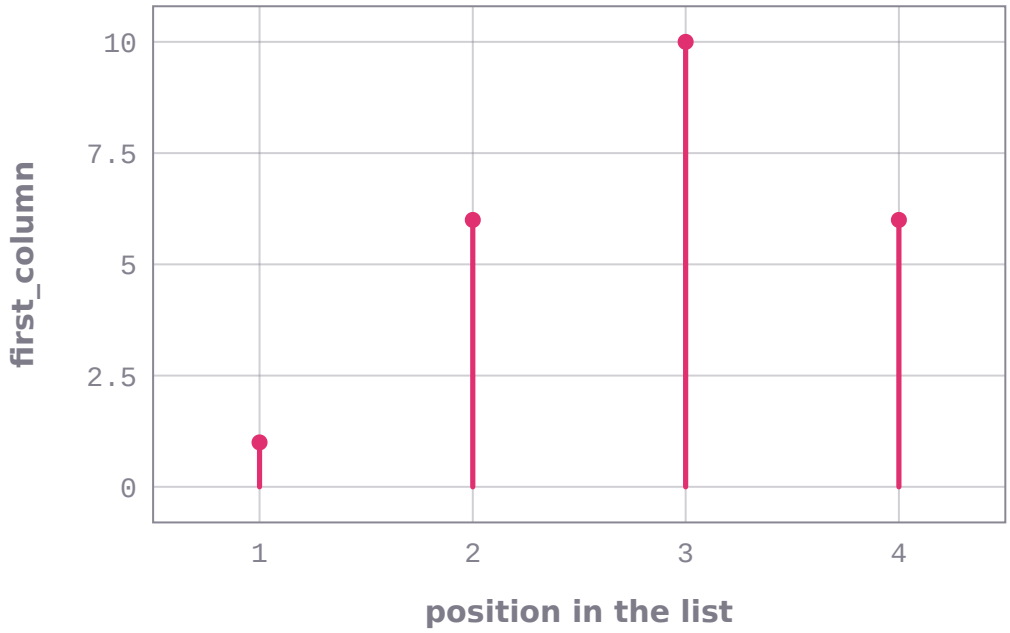
The first column of the Routh array (exact arithmetic), the number of right-half-plane roots (its sign changes) and whether the characteristic polynomial is stable.

$$b_i = \frac{a_1 a_{0,i+1} - a_0 a_{1,i+1}}{a_1}$$

input	meaning	condition
`coefficients`	characteristic polynomial coefficients, highest power of s first	

output	meaning
`first_column`	first column of the Routh array
`rhp_roots`	number of roots in the right half plane
`stable`	true when every root has a negative real part

- **Method:** Routh array in exact fractions; sign changes of the first column
- **Assumes:** no zero in the first column (special cases are refused)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



first column of the Routh array (first_column) returned by control-systems-ai999x for the worked example, one value for each position in the list (4 values, first 1, last 6, largest 10). Inputs: coefficients = [1, 6, 11, 6].

```
control_systems_ai999x.solve('ctrl_routh_hurwitz', coefficients=[1, 6, 11, 6])
# -> first_column = [1.0, 6.0, 10.0, 6.0], rhp_roots = 0, stable = True   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_routh_hurwitz', coefficients=[1, 2, 3, 4, 5])
# -> first_column = [1.0, 2.0, 1.0, -6.0, 5.0], rhp_roots = 2, stable = False   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_routh_hurwitz', coefficients=[1, 1, 2, 24])
# -> first_column = [1.0, 1.0, -22.0, 24.0], rhp_roots = 2, stable = False   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Routh-Hurwitz: coefficients = [1, 6, 11, 6]	SOLVED	first_column = [1.0, 6.0, 10.0, 6.0], rhp_roots = 0, stable = True

question	outcome	result
Routh-Hurwitz: coefficients = [1, 2, 3, 4, 5]	SOLVED	first_column = [1.0, 2.0, 1.0, -6.0, 5.0], rhp_roots = 2, stable = False
Routh-Hurwitz: coefficients = [1, 1, 2, 24]	SOLVED	first_column = [1.0, 1.0, -22.0, 24.0], rhp_roots = 2, stable = False
Routh-Hurwitz:	NEEDS_INPUT	ctrl_routh_hurwitz needs coefficients (characteristic polynomial coefficients, highest power of s first). Example: solve
Routh-Hurwitz: coefficients = [1, 6, 11, 6], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Routh-Hurwitz: coefficients = [1, 2, 1, 2]	INVALID_INPUT	Routh-Hurwitz stability test: a zero appears in the first column of the Routh array (a special case: roots on or symmetr

Root-locus asymptotes (centroid and angles), `ctrl_locus_asymptotes`

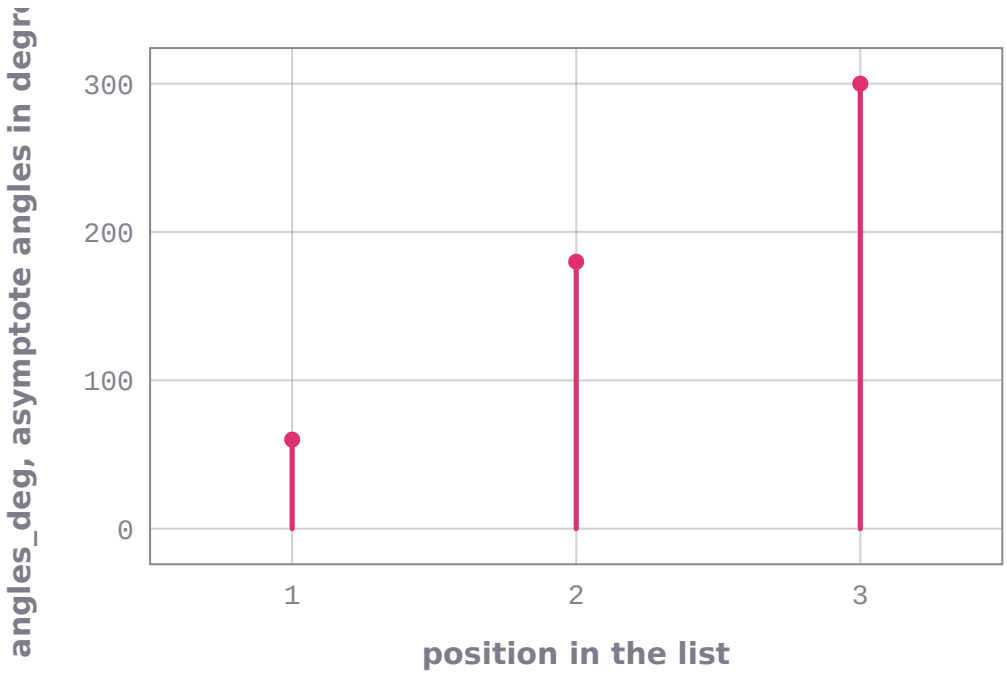
Number of asymptotes n - m, their real-axis intercept (sum of poles - sum of zeros)/(n - m) and their angles $(2k+1)\ 180/(n-m)$ degrees.

$$\sigma_a = \frac{\sum p_i - \sum z_j}{n-m}, \quad \theta_k = \frac{(2k+1)180^\circ}{n-m}$$

input	meaning	condition
`num`	numerator of the open-loop G(s)	
`den`	denominator of G(s)	

output	meaning
`n_asymptotes`	number of asymptotes
`centroid`	real-axis intercept
`angles_deg`	asymptote angles in degrees

- **Method:** sums of poles and zeros from the coefficients
- **Assumes:** more poles than zeros
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



asymptote angles in degrees (angles_deg) returned by control-systems-ai999x for the worked example, one value for each position in the list (3 values, first 60, last 300, largest 300). Inputs: num = [1], den = [1, 3, 2, 0].

```
control_systems_ai999x.solve('ctrl_locus_asymptotes', num=[1], den=[1, 3, 2, 0])
# -> n_asymptotes = 3, centroid = -1, angles_deg = [60.0, 180.0, 300.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_locus_asymptotes', num=[1, 3], den=[1, 6, 8, 0])
# -> n_asymptotes = 2, centroid = -1.5, angles_deg = [90.0, 270.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
root locus asymptotes: num = [1], den = [1, 3, 2, 0]	SOLVED	n_asymptotes = 3, centroid = -1, angles_deg = [60.0, 180.0, 300.0]
root locus asymptotes: num = [1, 3], den = [1, 6, 8, 0]	SOLVED	n_asymptotes = 2, centroid = -1.5, angles_deg = [90.0, 270.0]
root locus asymptotes: den = [1, 3, 2, 0]	NEEDS_INPUT	ctrl_locus_asymptotes needs num (numerator of the open-loop G(s)). Example: solve('ctrl_locus_asymptotes', num=[1], den=
root locus asymptotes: num = [1], den = [1, 3, 2, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
root locus asymptotes: num = [1, 2], den = [1, 3]	INVALID_INPUT	Root-locus asymptotes (centroid and angles): asymptotes need more poles than zeros.

Root-locus gain and angle at a test point, `ctrl_locus_gain_at_point`

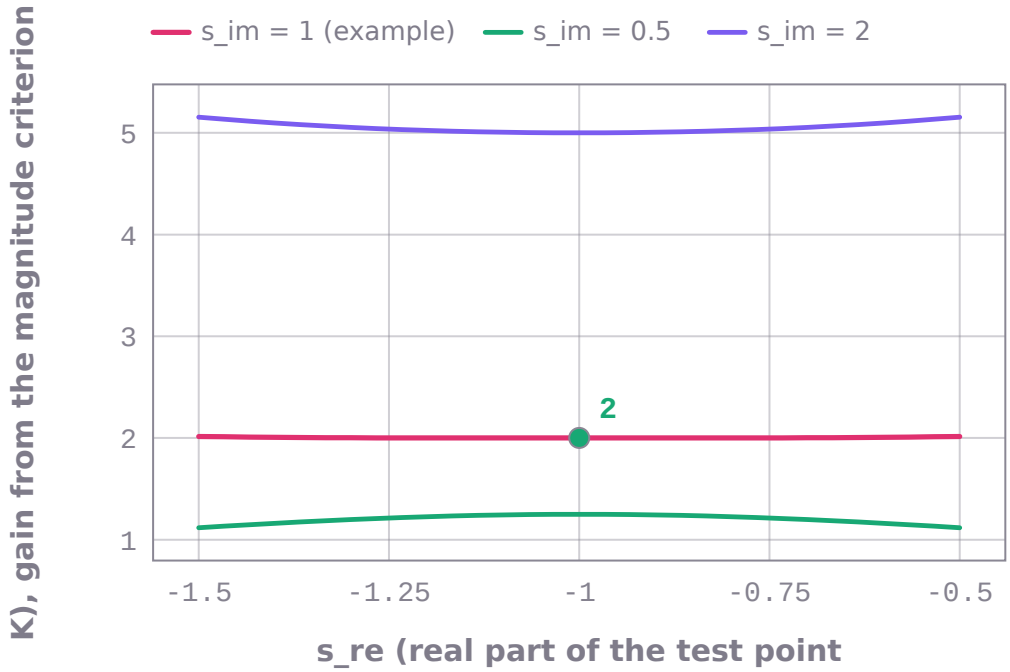
The gain $K = 1/|G(s0)|$ that would place a closed-loop pole at s0 (magnitude criterion) and the angle of G(s0); s0 lies on the root locus for $K > 0$ when the angle is 180 degrees (angle error 0).

$$K = \frac{1}{|G(s_0)|}, \quad \angle G(s_0) = \pm 180^\circ$$

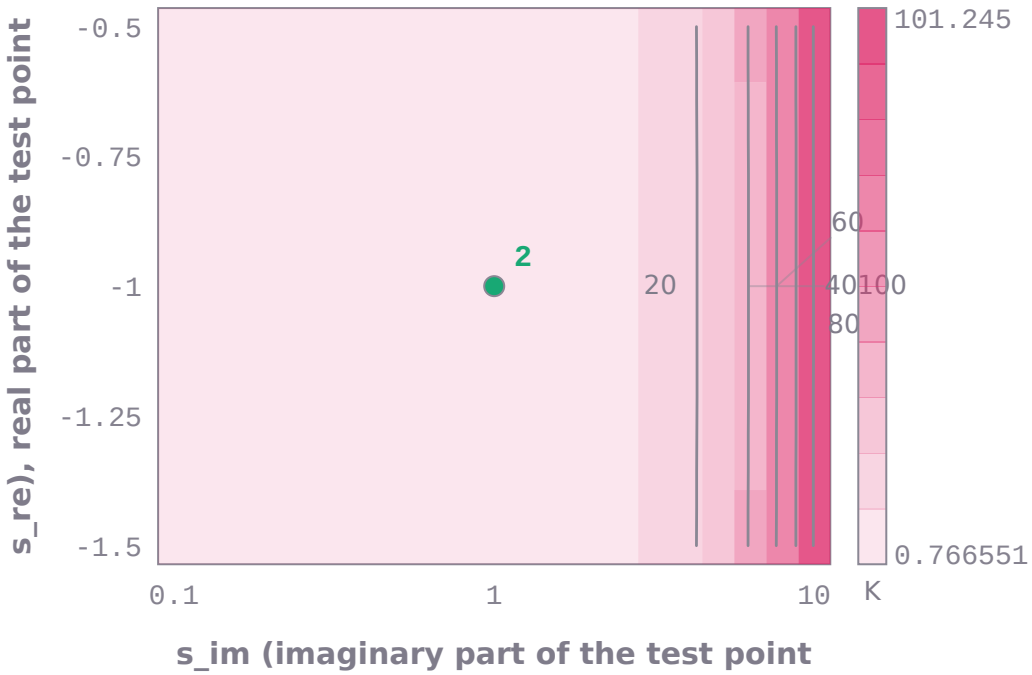
input	meaning	condition
`num`	numerator of G(s)	
`den`	denominator of G(s)	
`s_re`	real part of the test point	
`s_im`	imaginary part of the test point	

output	meaning
`K`	gain from the magnitude criterion
`angle_deg`	angle of G(s0) in degrees
`angle_error_deg`	180 - |angle|: 0 on the locus

- **Method:** K = |D(s0)|/|N(s0)|
- **Assumes:** s0 neither a pole nor a zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



K (gain from the magnitude criterion) against s_re (real part of the test point) from -1.5 to -0.5. One curve for each value of s_im (imaginary part of the test point): 1, 0.5, 2. Other inputs as in the worked example: num = [1], den = [1, 2, 0]. 31 points per curve computed by control-systems-ai999x; marked: the worked example, s_re = -1, K = 2.



K (gain from the magnitude criterion) over s_im (imaginary part of the test point) from 0.1 to 10 and s_re (real part of the test point) from -1.5 to -0.5 (logarithmic horizontal axis): each of the 21 × 15 cells is one run of control-systems-ai999x, shaded by its value (light 0.766551 to dark 101.245); iso-lines at K = 20, 40, 60, 80, 100. Other inputs as in the worked example: num = [1], den = [1, 2, 0]. Marked: the worked example, s_im = 1, s_re = -1, K = 2.

```
control_systems_ai999x.solve('ctrl_locus_gain_at_point', num=[1], den=[1, 2, 0], s_re=-1,
s_im=1)
# -> K = 2, angle_deg = 180, angle_error_deg = 0   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_locus_gain_at_point', num=[1], den=[1, 3, 2, 0], s_re=0,
s_im=1.4142135623730951)
# -> K = 6, angle_deg = 180, angle_error_deg = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
root locus gain at a point: num = [1], den = [1, 2, 0], s_re = -1, s_im = 1	SOLVED	K = 2, angle_deg = 180, angle_error_deg = 0
root locus gain at a point: num = [1], den = [1, 3, 2, 0], s_re = 0, s_im = 1.4142135623730951	SOLVED	K = 6, angle_deg = 180, angle_error_deg = 0
root locus gain at a point: den = [1, 2, 0], s_re = -1, s_im = 1	NEEDS_INPUT	ctrl_locus_gain_at_point needs num (numerator of G(s)). Example: solve('ctrl_locus_gain_at_point', num=[1], den=[1, 2, 0
root locus gain at a point: num = [1], den = [1, 2, 0], s_re = -1, s_im = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

Transfer function of a state-space model, `ctrl_ss_to_tf`

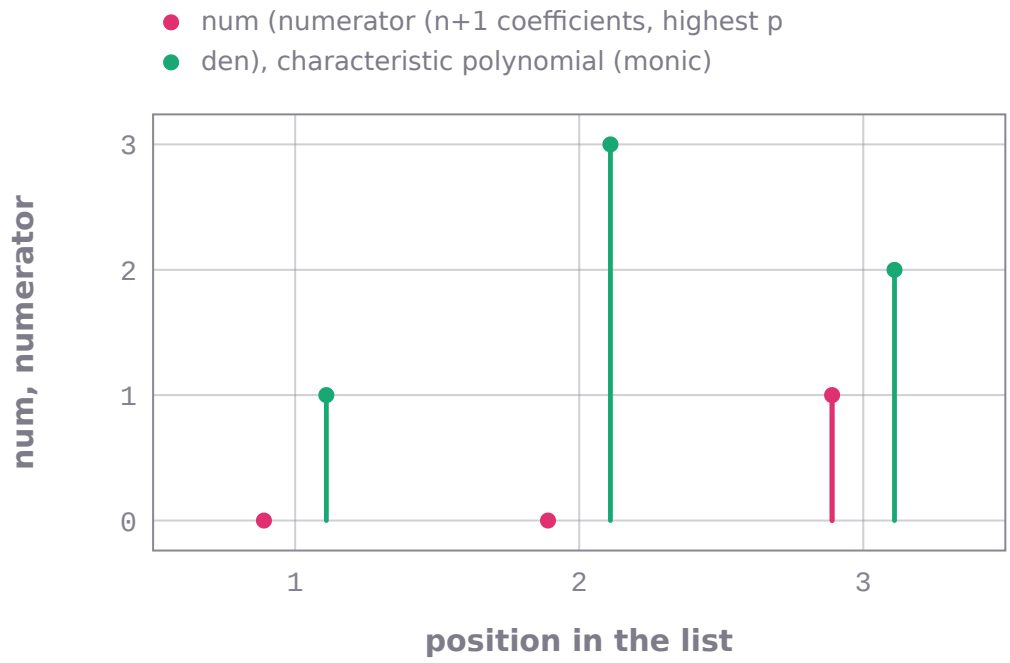
G(s) = C (sI - A)^{-1} B + D of a single-input single-output state-space model, computed exactly (Faddeev-LeVerrier).

$$G(s) = C(sI - A)^{-1}B + D = \frac{C\,\mathrm{adj}(sI - A)\,B}{\det(sI - A)} + D$$

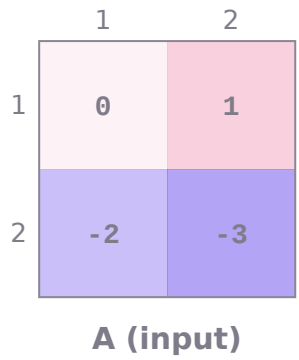
input	meaning	condition
`A`	state matrix (n x n)	
`B`	input vector (column, n entries)	
`C`	output vector (row, n entries)	
`D`	feedthrough (default 0)	

output	meaning
`num`	numerator (n+1 coefficients, highest power first)
`den`	characteristic polynomial (monic)

- **Method:** Faddeev-LeVerrier in exact fractions
- **Assumes:** A square, at most 6 x 6
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



numerator (n+1 coefficients, highest power first) (num) returned by control-systems-ai999x for the worked example, one value for each position in the list (3 values, first 0, last 1, largest 1); with den (characteristic polynomial (monic)). Inputs: A = [[0, 1], [-2, -3]], B = [0, 1], C = [1, 0].



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Other inputs: B = [0, 1], C = [1, 0].

```
control_systems_ai999x.solve('ctrl_ss_to_tf', A=[[0, 1], [-2, -3]], B=[0, 1], C=[1, 0])
# -> num = [0.0, 0.0, 1.0], den = [1.0, 3.0, 2.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_ss_to_tf', A=[[0, 1], [-6, -5]], B=[0, 1], C=[3, 1],
D=1)
# -> num = [1.0, 6.0, 9.0], den = [1.0, 5.0, 6.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
state space to transfer function: A = [[0, 1], [-2, -3]], B = [0, 1], C = [1, 0]	SOLVED	num = [0.0, 0.0, 1.0], den = [1.0, 3.0, 2.0]
state space to transfer function: A = [[0, 1], [-6, -5]], B = [0, 1], C = [3, 1], D = 1	SOLVED	num = [1.0, 6.0, 9.0], den = [1.0, 5.0, 6.0]
state space to transfer function: B = [0, 1], C = [1, 0]	NEEDS_INPUT	ctrl_ss_to_tf needs A (state matrix (n x n)). Example: solve('ctrl_ss_to_tf', A=[[0, 1], [-2, -3]], B=[0, 1], C=[1, 0]).
state space to transfer function: A = [[0, 1], [-2, -3]], B = [0, 1], C = [1, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
state space to transfer function: A = [[0, 1], [-2, -3]], B = [0, 1, 2], C = [1, 0]	INVALID_INPUT	Transfer function of a state-space model: B needs between 2 and 2 entries.

Controllable canonical form of a transfer function, `ctrl_tf_to_ss`

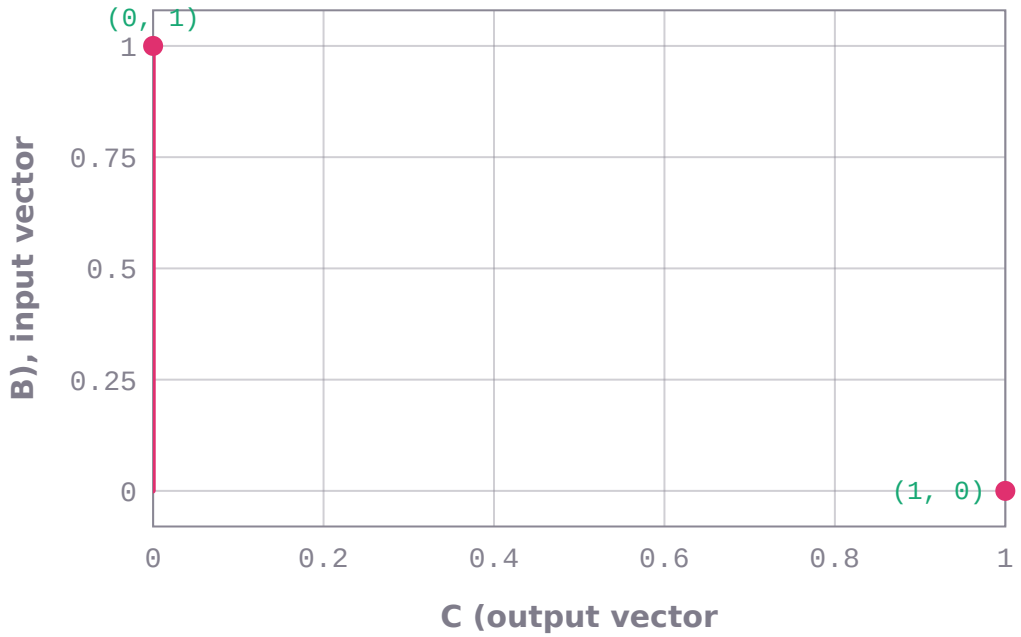
A, B, C, D in the controllable canonical form (scipy's convention) of a proper transfer function.

$$A = \begin{bmatrix} -a_1 & \cdots & -a_n \\ 1 & & 0 \\ & \ddots & \end{bmatrix},\ B = e_1$$

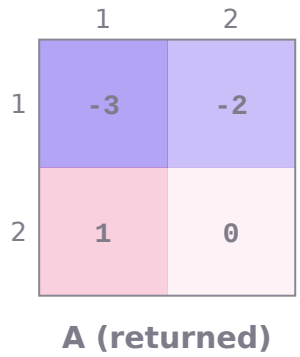
input	meaning	condition
`num`	numerator coefficients	
`den`	denominator coefficients	

output	meaning
`A`	state matrix
`B`	input vector
`C`	output vector
`D`	feedthrough

- **Method:** normalise den; first row of $A = -a_1 .. -an$; C from num - D den
- **Assumes:** proper G(s)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



input vector (B) at each output vector (C), both returned by control-systems-ai999x for the worked example (num = [1], den = [1, 3, 2]): 0 → 1; 1 → 0.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (returned). Returned with them by control-systems-ai999x: D = 0. Other inputs: num = [1], den = [1, 3, 2].

```
control_systems_ai999x.solve('ctrl_tf_to_ss', num=[1], den=[1, 3, 2])
# -> A = [[-3.0, -2.0], [1.0, 0.0]], B = [1.0, 0.0], C = [0.0, 1.0], D = 0   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_tf_to_ss', num=[2, 1, 4], den=[2, 6, 4])
# -> A = [[-3.0, -2.0], [1.0, 0.0]], B = [1.0, 0.0], C = [-2.5, 0.0], D = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
controllable canonical form: num = [1], den = [1, 3, 2]	SOLVED	A = [[-3.0, -2.0], [1.0, 0.0]], B = [1.0, 0.0], C = [0.0, 1.0], D = 0
controllable canonical form: num = [2, 1, 4], den = [2, 6, 4]	SOLVED	A = [[-3.0, -2.0], [1.0, 0.0]], B = [1.0, 0.0], C = [-2.5, 0.0], D = 1
controllable canonical form: den = [1, 3, 2]	NEEDS_INPUT	ctrl_tf_to_ss needs num (numerator coefficients). Example: solve('ctrl_tf_to_ss', num=[1], den=[1, 3, 2]).
controllable canonical form: num = [1], den = [1, 3, 2], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
controllable canonical form: num = [1, 2, 3], den = [1, 1]	INVALID_INPUT	Controllable canonical form of a transfer function: the transfer function must be proper (numerator degree at most the d

Eigenvalues (poles) of a state matrix, `ctrl_ss_eigenvalues`

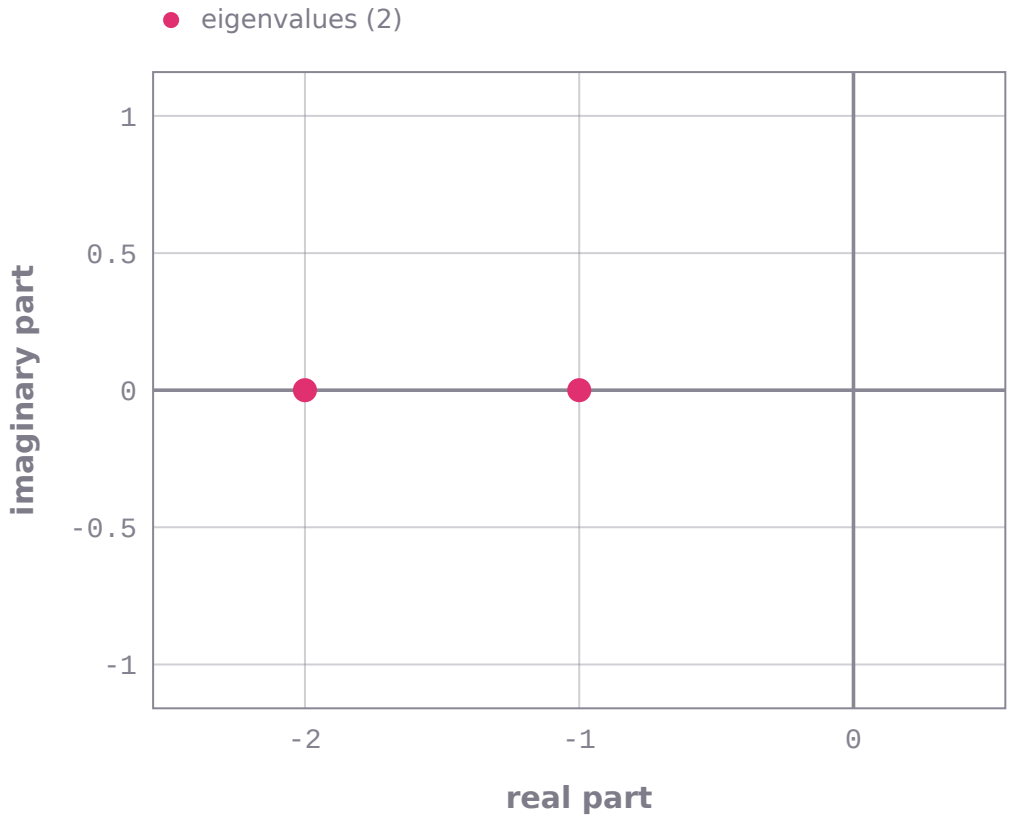
The characteristic polynomial det(sI - A) (exact), the eigenvalues of A and whether the system is asymptotically stable.

$$\det(sI - A) = 0$$

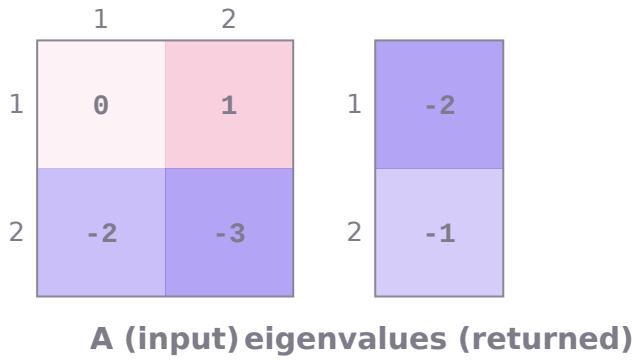
input	meaning	condition
`A`	state matrix (n x n)	

output	meaning
`eigenvalues`	eigenvalues of A
`char_poly`	characteristic polynomial (monic)
`max_real_part`	largest real part
`stable`	true when every eigenvalue has a negative real part

- **Method:** Faddeev-LeVerrier, then polynomial roots
- **Assumes:** A square, at most 6 x 6
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



eigenvalues of A (eigenvalues) returned by control-systems-ai999x for the worked example (A = [[0, 1], [-2, -3]]), drawn in the complex plane: -2 + 0i, -1 + 0i.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); eigenvalues (returned). Returned with them by control-systems-ai999x: max_real_part = -1.

```
control_systems_ai999x.solve('ctrl_ss_eigenvalues', A=[[0, 1], [-2, -3]])
# -> eigenvalues = [-2.0, -1.0], char_poly = [1.0, 3.0, 2.0], max_real_part = -1, stable =
True   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_ss_eigenvalues', A=[[0, 1], [-5, -2]])
# -> eigenvalues = [(-1-2j), (-1+2j)], char_poly = [1.0, 2.0, 5.0], max_real_part = -1,
stable = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
eigenvalues of the state matrix: A = [[0, 1], [-2, -3]]	SOLVED	eigenvalues = [-2.0, -1.0], char_poly = [1.0, 3.0, 2.0], max_real_part = -1, stable = True
eigenvalues of the state matrix: A = [[0, 1], [-5, -2]]	SOLVED	eigenvalues = [(-1-2j), (-1+2j)], char_poly = [1.0, 2.0, 5.0], max_real_part = -1, stable = True
eigenvalues of the state matrix:	NEEDS_INPUT	ctrl_ss_eigenvalues needs A (state matrix (n x n)). Example: solve('ctrl_ss_eigenvalues', A=[[0, 1], [-2, -3]]).
eigenvalues of the state matrix: A = [[0, 1], [-2, -3]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
eigenvalues of the state matrix: A = [[0, 1, 2], [-2, -3, 1]]	INVALID_INPUT	Eigenvalues (poles) of a state matrix: A must be square.

Controllability matrix and rank, `ctrl_controllability`

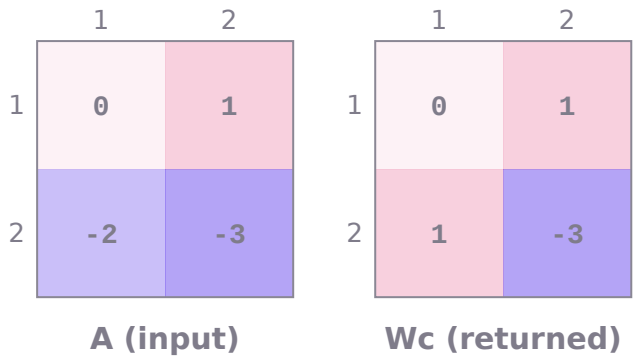
The controllability matrix [B, AB, ..., A^{$n-1$}B], its exact rank and determinant, and whether (A, B) is controllable.

$$\mathcal{C} = [B\ AB\ \cdots\ A^{n-1}B]$$

input	meaning	condition
`A`	state matrix (n x n)	
`B`	input vector (n entries)	

output	meaning
`Wc`	controllability matrix
`rank`	rank of the controllability matrix
`controllable`	true when the rank is n
`det_Wc`	determinant of the controllability matrix

- **Method:** exact rank by elimination in fractions
- **Assumes:** single input
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); Wc (returned). Returned with them by control-systems-ai999x: rank = 2, det_Wc = -1. Other inputs: B = [0, 1].

```
control_systems_ai999x.solve('ctrl_controllability', A=[[0, 1], [-2, -3]], B=[0, 1])
# -> Wc = [[0.0, 1.0], [1.0, -3.0]], rank = 2, controllable = True, det_Wc = -1   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_controllability', A=[[-1, 0], [0, -2]], B=[1, 0])
# -> Wc = [[1.0, -1.0], [0.0, 0.0]], rank = 1, controllable = False, det_Wc = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
controllability: A = [[0, 1], [-2, -3]], B = [0, 1]	SOLVED	Wc = [[0.0, 1.0], [1.0, -3.0]], rank = 2, controllable = True, det_Wc = -1
controllability: A = [[-1, 0], [0, -2]], B = [1, 0]	SOLVED	Wc = [[1.0, -1.0], [0.0, 0.0]], rank = 1, controllable = False, det_Wc = 0
controllability: B = [0, 1]	NEEDS_INPUT	ctrl_controllability needs A (state matrix (n x n)). Example: solve('ctrl_controllability', A=[[0, 1], [-2, -3]], B=[0,
controllability: A = [[0, 1], [-2, -3]], B = [0, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

Observability matrix and rank, `ctrl_observability`

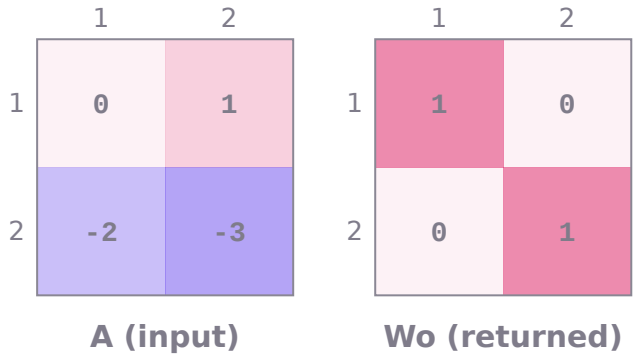
The observability matrix [C; CA; ...; CA^{$n-1$}], its exact rank and determinant, and whether (A, C) is observable.

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

input	meaning	condition
`A`	state matrix (n x n)	
`C`	output vector (n entries)	

output	meaning
`Wo`	observability matrix
`rank`	rank of the observability matrix
`observable`	true when the rank is n
`det_Wo`	determinant of the observability matrix

- **Method:** exact rank by elimination in fractions
- **Assumes:** single output
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); Wo (returned). Returned with them by control-systems-ai999x: rank = 2, det_Wo = 1. Other inputs: C = [1, 0].

```
control_systems_ai999x.solve('ctrl_observability', A=[[0, 1], [-2, -3]], C=[1, 0])
# -> Wo = [[1.0, 0.0], [0.0, 1.0]], rank = 2, observable = True, det_Wo = 1   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_observability', A=[[-1, 0], [0, -2]], C=[1, 0])
# -> Wo = [[1.0, 0.0], [-1.0, 0.0]], rank = 1, observable = False, det_Wo = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
observability: A = [[0, 1], [-2, -3]], C = [1, 0]	SOLVED	Wo = [[1.0, 0.0], [0.0, 1.0]], rank = 2, observable = True, det_Wo = 1
observability: A = [[-1, 0], [0, -2]], C = [1, 0]	SOLVED	Wo = [[1.0, 0.0], [-1.0, 0.0]], rank = 1, observable = False, det_Wo = 0
observability: C = [1, 0]	NEEDS_INPUT	ctrl_observability needs A (state matrix (n x n)). Example: solve('ctrl_observability', A=[[0, 1], [-2, -3]], C=[1, 0]).
observability: A = [[0, 1], [-2, -3]], C = [1, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

DC gain of a state-space model, `ctrl_ss_dc_gain`

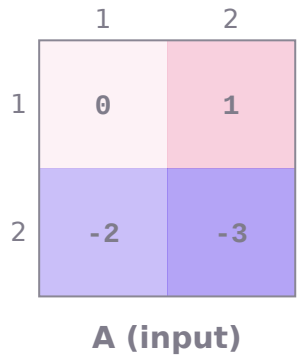
The steady-state gain G(0) = D - C A^{-1} B of a state-space model (exact).

$$G(0) = D - CA^{-1}B$$

input	meaning	condition
`A`	state matrix (n x n)	
`B`	input vector	
`C`	output vector	
`D`	feedthrough (default 0)	

output	meaning
`dc_gain`	DC gain G(0)

- **Method:** solve A $x = B$ exactly, then D - C x
- **Assumes:** A nonsingular
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input). Returned with them by control-systems-ai999x: dc_gain = 0.5. Other inputs: B = [0, 1], C = [1, 0].

```
control_systems_ai999x.solve('ctrl_ss_dc_gain', A=[[0, 1], [-2, -3]], B=[0, 1], C=[1, 0])
# -> dc_gain = 0.5   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_ss_dc_gain', A=[[-1, 0], [0, -4]], B=[1, 2], C=[3, 1],
D=1)
# -> dc_gain = 4.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
state-space DC gain: A = [[0, 1], [-2, -3]], B = [0, 1], C = [1, 0]	SOLVED	dc_gain = 0.5
state-space DC gain: A = [[-1, 0], [0, -4]], B = [1, 2], C = [3, 1], D = 1	SOLVED	dc_gain = 4.5

question	outcome	result
state-space DC gain: B = [0, 1], C = [1, 0]	NEEDS_INPUT	ctrl_ss_dc_gain needs A (state matrix (n x n)). Example: solve('ctrl_ss_dc_gain', A=[[0, 1], [-2, -3]], B=[0, 1], C=[1,
state-space DC gain: A = [[0, 1], [-2, -3]], B = [0, 1], C = [1, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
state-space DC gain: A = [[0, 1], [0, -3]], B = [0, 1], C = [1, 0]	INVALID_INPUT	DC gain of a state-space model: A is singular (an eigenvalue at 0): the DC gain is infinite.

Zero-order-hold discretization of a first-order system, `ctrl_zoh_first_order`

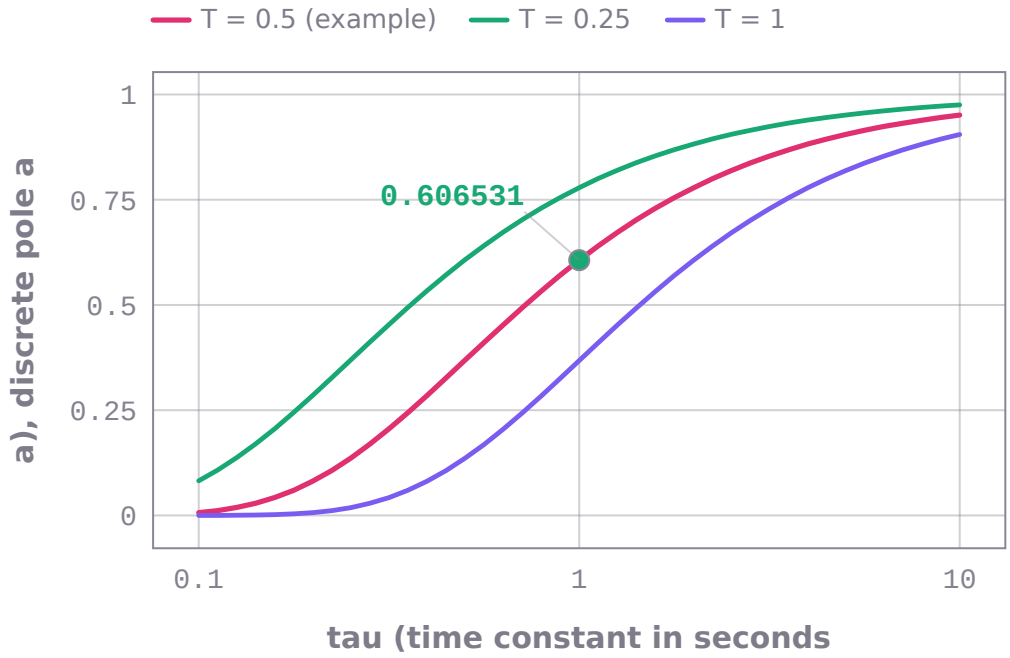
G(z) = b/(z - a) with $a = e$^{-T/tau} and $b = K\,(1 - a)$: the exact step-invariant (ZOH) equivalent of K/(tau s + 1) sampled every T seconds.

$$G(z) = \frac{K(1 - e^{-T/\tau})}{z - e^{-T/\tau}}$$

input	meaning	condition
`K`	DC gain	
`tau`	time constant in seconds	
`T`	sampling period in seconds	

output	meaning
`b`	numerator coefficient b
`a`	discrete pole a

- **Method:** $a = e^{\frac{-T}{\tau}}$, $b = K\,(1 - a)$
- **Assumes:** $\tau > 0$; $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



a (discrete pole a) against tau (time constant in seconds) from 0.1 to 10 on a logarithmic axis. One curve for each value of T (sampling period in seconds): 0.5, 0.25, 1. Other inputs as in the worked example: K = 2.0. 41 points per curve computed by control-systems-ai999x; marked: the worked example, tau = 1, a = 0.606531.

```
control_systems_ai999x.solve('ctrl_zoh_first_order', K=2, tau=1, T=0.5)
# -> b = 0.786939, a = 0.606531   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_zoh_first_order', K=1, tau=10, T=1)
# -> b = 0.0951626, a = 0.904837   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ZOH discretization of a first-order system: K = 2, tau = 1, T = 0.5	SOLVED	b = 0.786939, a = 0.606531
ZOH discretization of a first-order system: K = 1, tau = 10, T = 1	SOLVED	b = 0.0951626, a = 0.904837
ZOH discretization of a first-order system: tau = 1, T = 0.5	NEEDS_INPUT	ctrl_zoh_first_order needs K (DC gain). Example: solve('ctrl_zoh_first_order', K=2, tau=1, T=0.5).
ZOH discretization of a first-order system: K = 2, tau = 1, T = 0.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
ZOH discretization of a first-order system: K = 2, tau = 1, T = 0	INVALID_INPUT	Zero-order-hold discretization of a first-order system: T must be positive.

Tustin (bilinear) discretization, `ctrl_tustin`

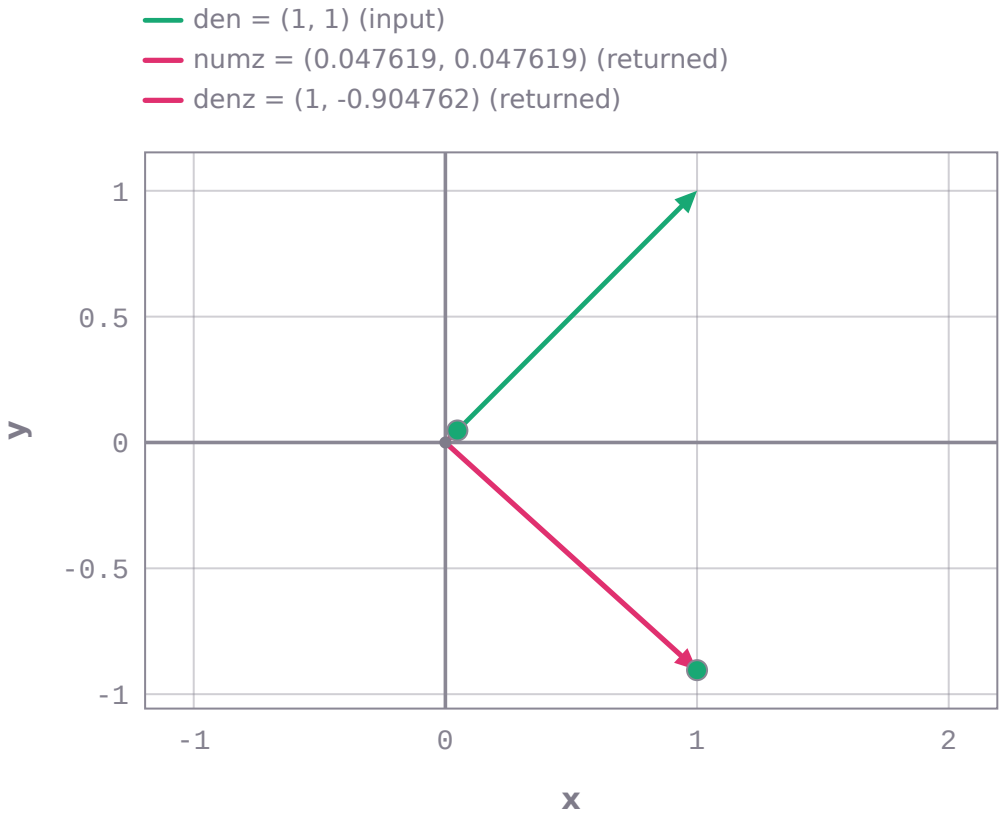
The discrete transfer function obtained by substituting $s = (2/T)(z-1)/(z+1)$ into G(s).

$$s \leftarrow \frac{2}{T}\,\frac{z-1}{z+1}$$

input	meaning	condition
`num`	numerator in s	
`den`	denominator in s	
`T`	sampling period in seconds	

output	meaning
`numz`	numerator in z (padded, normalised)
`denz`	denominator in z (leading coefficient 1)

- **Method:** polynomial substitution, multiplied through by $(z+1)^n$
- **Assumes:** proper G(s); $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input den = (1, 1); returned by control-systems-ai999x: numz = (0.047619, 0.047619), denz = (1, -0.904762). Other inputs: num = [1], T = 0.1. Marked: the tip of each returned vector.

```
control_systems_ai999x.solve('ctrl_tustin', num=[1], den=[1, 1], T=0.1)
# -> numz = [0.047619047619, 0.047619047619], denz = [1.0, -0.904761904762]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_tustin', num=[1], den=[1, 0], T=2)
# -> numz = [1.0, 1.0], denz = [1.0, -1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Tustin discretization: num = [1], den = [1, 1], T = 0.1	SOLVED	numz = [0.047619047619, 0.047619047619], denz = [1.0, -0.904761904762]
Tustin discretization: num = [1], den = [1, 0], T = 2	SOLVED	numz = [1.0, 1.0], denz = [1.0, -1.0]
Tustin discretization: den = [1, 1], T = 0.1	NEEDS_INPUT	ctrl_tustin needs num (numerator in s). Example: solve('ctrl_tustin', num=[1], den=[1, 1], T=0.1).
Tustin discretization: num = [1], den = [1, 1], T = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Tustin discretization: num = [1], den = [1, 1], T = -0.1	INVALID_INPUT	Tustin (bilinear) discretization: T must be positive.

Forward Euler (forward difference) discretization, `ctrl_forward_euler`

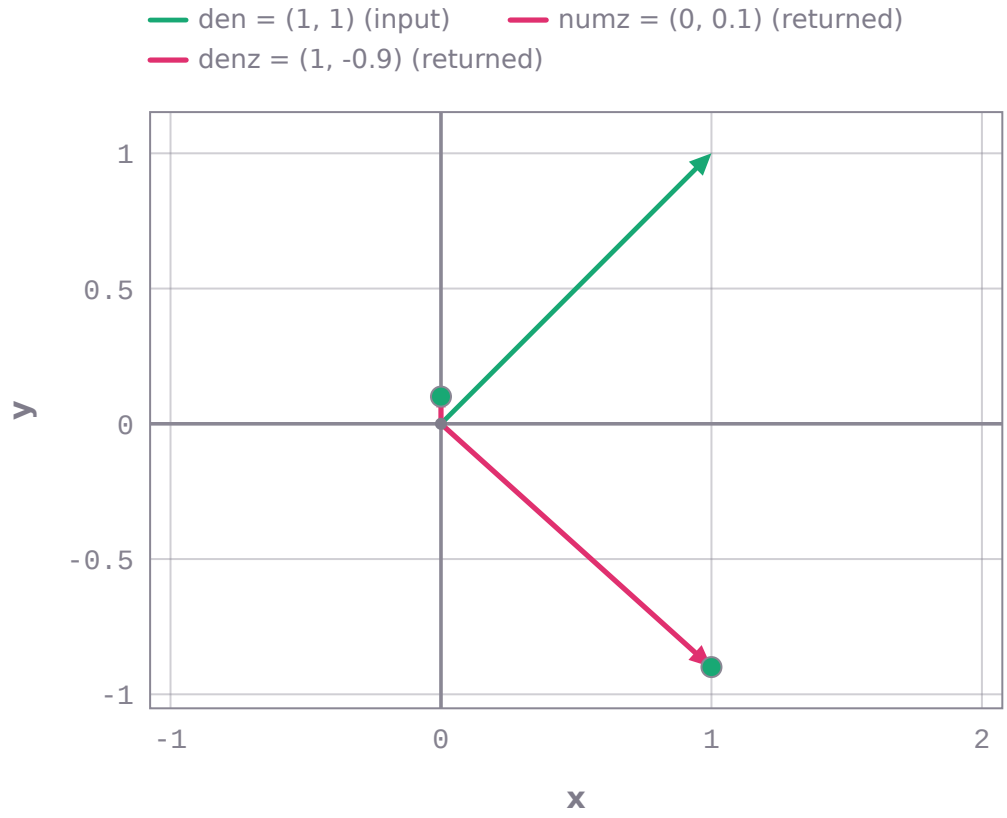
The discrete transfer function obtained by substituting $s = (z-1)/T$ into G(s).

$$s \leftarrow \frac{z-1}{T}$$

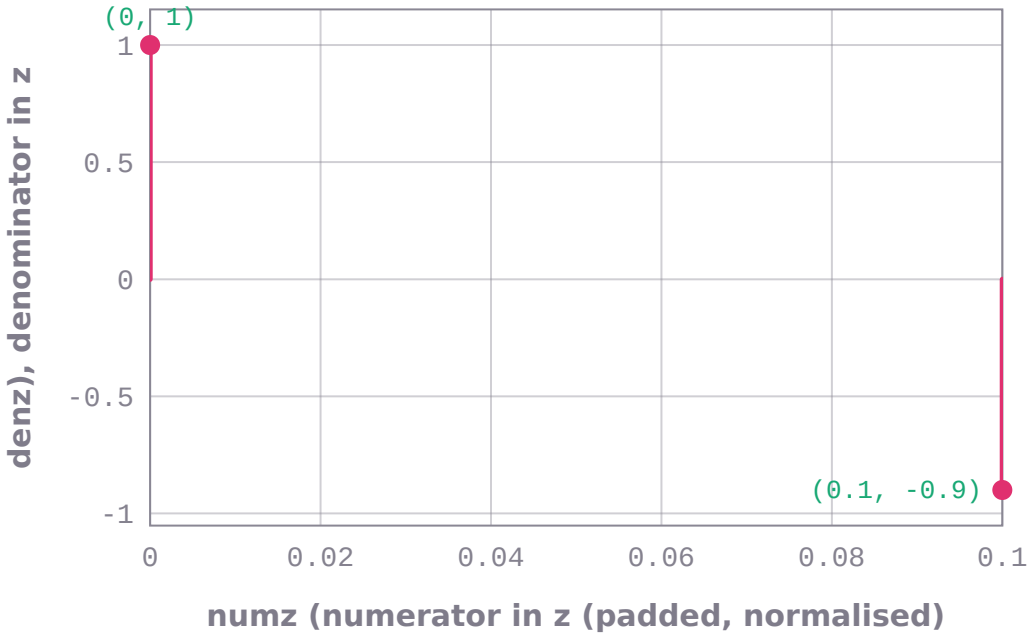
input	meaning	condition
`num`	numerator in s	
`den`	denominator in s	
`T`	sampling period in seconds	

output	meaning
`numz`	numerator in z (padded, normalised)
`denz`	denominator in z (leading coefficient 1)

- **Method:** polynomial substitution $s = (z-1)/T$
- **Assumes:** proper G(s); $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input den = (1, 1); returned by control-systems-ai999x: numz = (0, 0.1), denz = (1, -0.9). Other inputs: num = [1], T = 0.1. Marked: the tip of each returned vector.



denominator in z (leading coefficient 1) (denz) at each numerator in z (padded, normalised) (numz), both returned by control-systems-ai999x for the worked example (num = [1], den = [1, 1], T = 0.1): 0 → 1; 0.1 → -0.9.

```
control_systems_ai999x.solve('ctrl_forward_euler', num=[1], den=[1, 1], T=0.1)
# -> numz = [0.0, 0.1], denz = [1.0, -0.9]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_forward_euler', num=[2], den=[1, 3, 2], T=0.5)
# -> numz = [0.0, 0.0, 0.5], denz = [1.0, -0.5, 0.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
forward Euler discretization: num = [1], den = [1, 1], T = 0.1	SOLVED	numz = [0.0, 0.1], denz = [1.0, -0.9]
forward Euler discretization: num = [2], den = [1, 3, 2], T = 0.5	SOLVED	numz = [0.0, 0.0, 0.5], denz = [1.0, -0.5, 0.0]
forward Euler discretization: den = [1, 1], T = 0.1	NEEDS_INPUT	ctrl_forward_euler needs num (numerator in s). Example: solve('ctrl_forward_euler', num=[1], den=[1, 1], T=0.1).
forward Euler discretization: num = [1], den = [1, 1], T = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
forward Euler discretization: num = [1, 1, 1], den = [1, 1], T = 0.1	INVALID_INPUT	Forward Euler (forward difference) discretization: the transfer function must be proper (numerator degree at most the de

Backward Euler (backward difference) discretization, `ctrl_backward_euler`

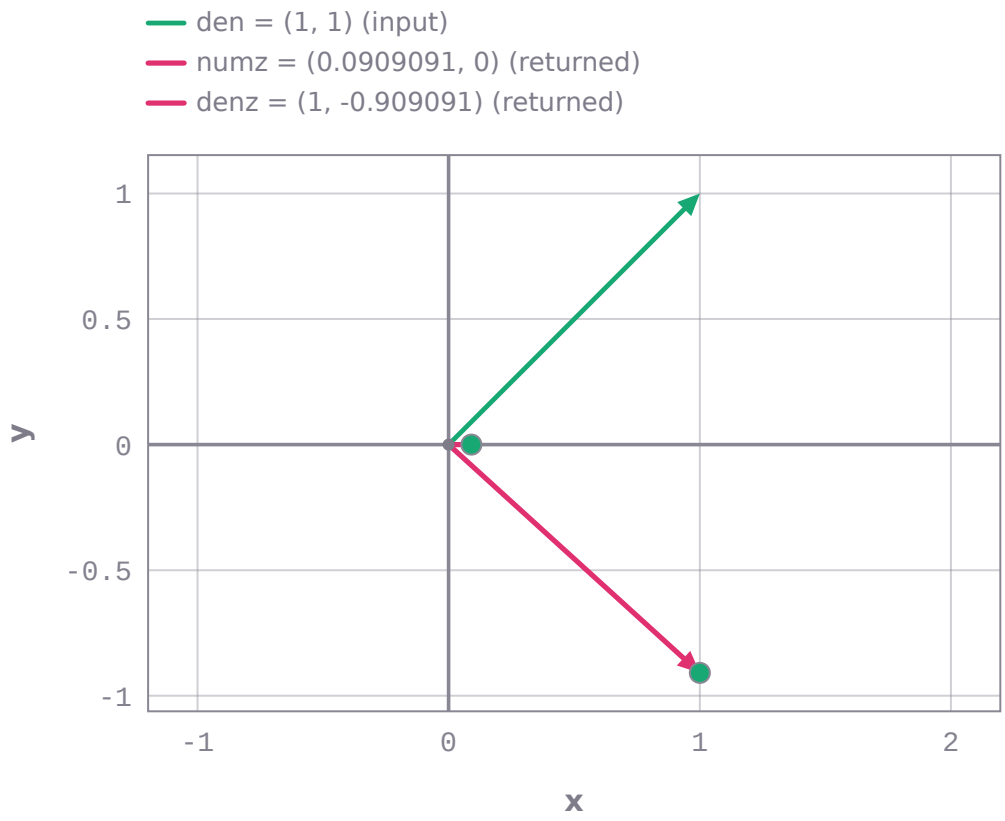
The discrete transfer function obtained by substituting s = (z - 1)/(T z) into G(s).

$$s \leftarrow \frac{z-1}{Tz}$$

input	meaning	condition
`num`	numerator in s	
`den`	denominator in s	
`T`	sampling period in seconds	

output	meaning
`numz`	numerator in z (padded, normalised)
`denz`	denominator in z (leading coefficient 1)

- **Method:** polynomial substitution s = (z - 1)/(T z)
- **Assumes:** proper G(s); $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input den = (1, 1); returned by control-systems-ai999x: numz = (0.0909091, 0), denz = (1, -0.909091). Other inputs: num = [1], T = 0.1. Marked: the tip of each returned vector.

```
control_systems_ai999x.solve('ctrl_backward_euler', num=[1], den=[1, 1], T=0.1)
# -> numz = [0.0909090909091, 0.0], denz = [1.0, -0.909090909091]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_backward_euler', num=[1], den=[1, 0], T=0.5)
# -> numz = [0.5, 0.0], denz = [1.0, -1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
backward Euler discretization: num = [1], den = [1, 1], T = 0.1	SOLVED	numz = [0.0909090909091, 0.0], denz = [1.0, -0.909090909091]
backward Euler discretization: num = [1], den = [1, 0], T = 0.5	SOLVED	numz = [0.5, 0.0], denz = [1.0, -1.0]
backward Euler discretization: den = [1, 1], T = 0.1	NEEDS_INPUT	ctrl_backward_euler needs num (numerator in s). Example: solve('ctrl_backward_euler', num=[1], den=[1, 1], T=0.1).
backward Euler discretization: num = [1], den = [1, 1], T = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
backward Euler discretization: num = [1], den = [1, 1], T = 0	INVALID_INPUT	Backward Euler (backward difference) discretization: T must be positive.

Step response of a discrete transfer function at sample k, `ctrl_discrete_step`

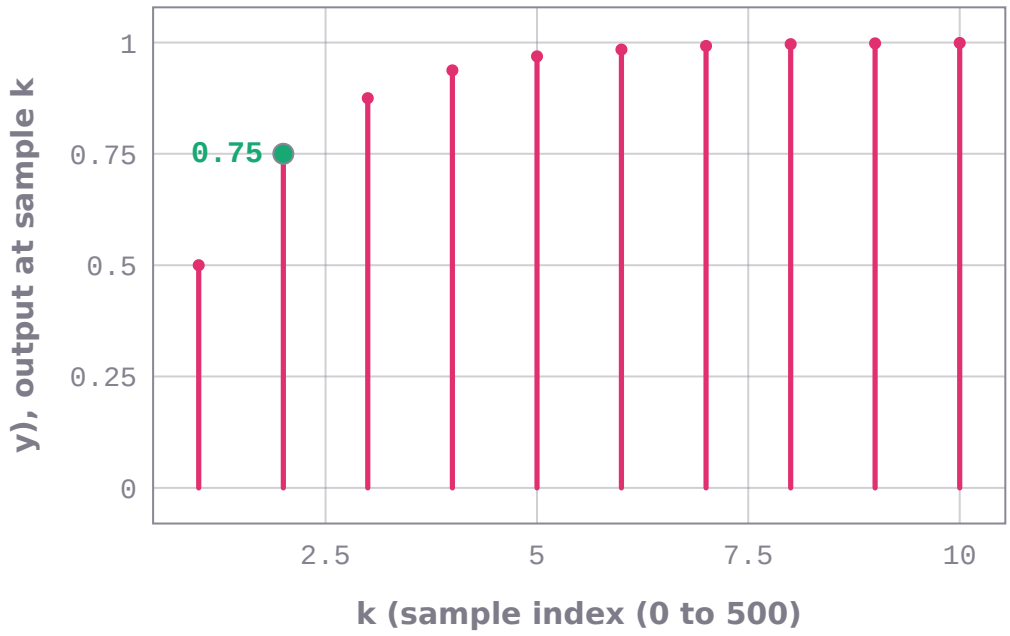
y[k] of G(z) = numz/denz after a step of size A applied at $k = 0$, by the difference equation.

$$y[k] = \sum_i b_i u[k-i] - \sum_{i \geq 1} a_i y[k-i]$$

input	meaning	condition
`numz`	numerator in z, highest power first	
`denz`	denominator in z, highest power first	
`k`	sample index (0 to 500)	
`A`	step size (default 1)	

output	meaning
`y`	output at sample k

- **Method:** recursion y[j] = sum b_i u[j-i] - sum a_i y[j-i]
- **Assumes:** proper (causal) G(z); $0 \leq k \leq 500$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y (output at sample k) against k (sample index (0 to 500)) from 1 to 10. Other inputs as in the worked example: numz = [0.5], denz = [1, -0.5]. 10 points computed by control-systems-ai999x; marked: the worked example, k = 2, y = 0.75.

```
control_systems_ai999x.solve('ctrl_discrete_step', numz=[0.5], denz=[1, -0.5], k=2)
# -> y = 0.75   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_discrete_step', numz=[1, 0], denz=[1, -0.5], k=3, A=2)
# -> y = 3.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
discrete step response: numz = [0.5], denz = [1,	SOLVED	y = 0.75

question	outcome	result
-0.5], k = 2		
discrete step response: numz = [1, 0], denz = [1, -0.5], k = 3, A = 2	SOLVED	y = 3.75
discrete step response: denz = [1, -0.5], k = 2	NEEDS_INPUT	ctrl_discrete_step needs numz (numerator in z, highest power first). Example: solve('ctrl_discrete_step', numz=[0.5], de
discrete step response: numz = [0.5], denz = [1, -0.5], k = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
discrete step response: numz = [0.5], denz = [1, -0.5], k = 900	INVALID_INPUT	k: must be at most 500, got 900.

Discrete PID in velocity (incremental) form, `ctrl_discrete_pid`

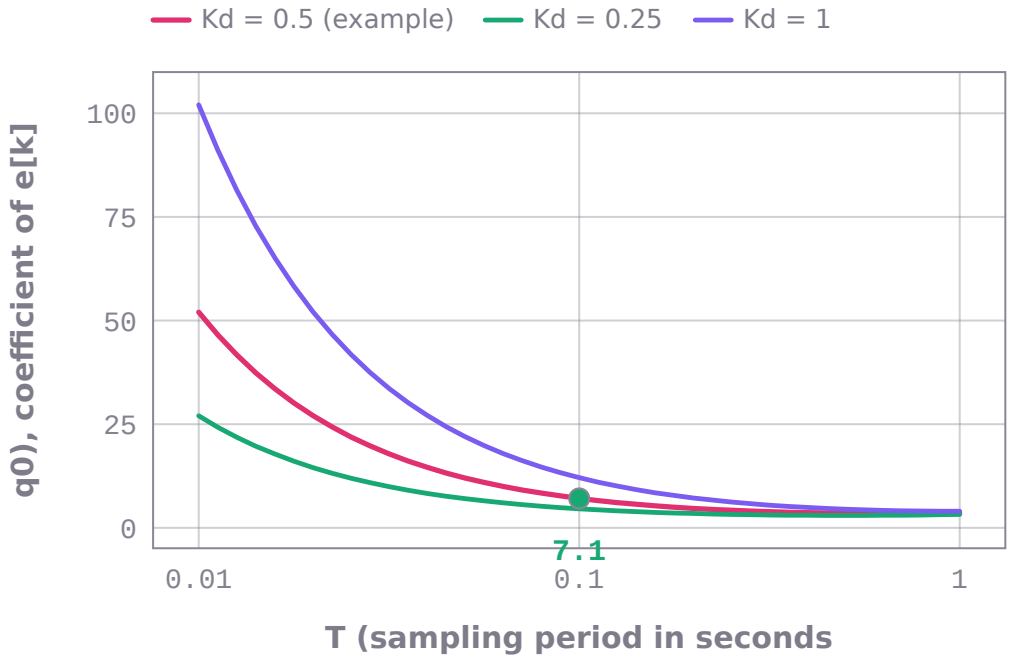
Coefficients of u[k] = u[$k-1$] + q0 e[k] + q1 e[$k-1$] + q2 e[$k-2$] for a PID with backward-difference integral and derivative: $q_0 = Kp + Ki$ T + Kd/T, $q_1 = -Kp - 2$ Kd/T, $q_2 = Kd/T$.

$$\Delta u_k = (K_p + K_i T + \frac{K_d}{T})e_k - (K_p + \frac{2K_d}{T})e_{k-1} + \frac{K_d}{T}e_{k-2}$$

input	meaning	condition
`Kp`	proportional gain	
`Ki`	integral gain	
`Kd`	derivative gain	
`T`	sampling period in seconds	

output	meaning
`q0`	coefficient of e[k]
`q1`	coefficient of e[$k-1$]
`q2`	coefficient of e[$k-2$]

- **Method:** difference of the positional backward-difference PID
- **Assumes:** $T > 0$; Ki, $Kd \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



q0 (coefficient of e[k]) against T (sampling period in seconds) from 0.01 to 1 on a logarithmic axis. One curve for each value of Kd (derivative gain): 0.5, 0.25, 1. Other inputs as in the worked example: Kp = 2.0, Ki = 1.0. 41 points per curve computed by control-systems-ai999x; marked: the worked example, T = 0.1, q0 = 7.1.

```
control_systems_ai999x.solve('ctrl_discrete_pid', Kp=2, Ki=1, Kd=0.5, T=0.1)
# -> q0 = 7.1, q1 = -12, q2 = 5   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_discrete_pid', Kp=1, Ki=4, Kd=0, T=0.25)
# -> q0 = 2, q1 = -1, q2 = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
discrete PID: Kp = 2, Ki = 1, Kd = 0.5, T = 0.1	SOLVED	q0 = 7.1, q1 = -12, q2 = 5
discrete PID: Kp = 1, Ki = 4, Kd = 0, T = 0.25	SOLVED	q0 = 2, q1 = -1, q2 = 0
discrete PID: Ki = 1, Kd = 0.5, T = 0.1	NEEDS_INPUT	ctrl_discrete_pid needs Kp (proportional gain). Example: solve('ctrl_discrete_pid', Kp=2, Ki=1, Kd=0.5, T=0.1).
discrete PID: Kp = 2, Ki = 1, Kd = 0.5, T = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
discrete PID: Kp = 2, Ki = 1, Kd = 0.5, T = 0	INVALID_INPUT	Discrete PID in velocity (incremental) form: T must be positive.

Continuous equivalent of a discrete pole (s = ln z / T), `ctrl_z_to_s_pole`

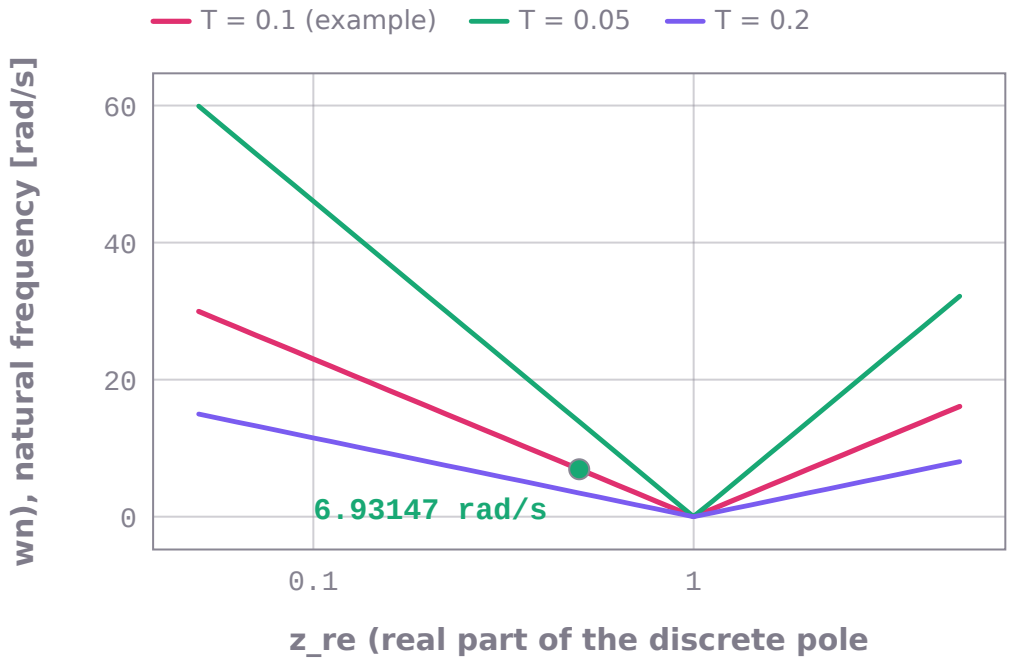
The s-plane pole $s = ln$(z)/T (principal branch) of a discrete pole $z = z_r e + j$ z_im, with its natural frequency and damping ratio.

$$s = \frac{\ln z}{T}$$

input	meaning	condition
`z_re`	real part of the discrete pole	
`z_im`	imaginary part of the discrete pole (>= 0)	
`T`	sampling period in seconds	

output	meaning
`s_re`	real part of s $(1/s)$
`s_im`	imaginary part of s (rad/s)
`wn`	natural frequency
`zeta`	damping ratio

- **Method:** s = (ln|z| + j arg z)/T
- **Assumes:** $T > 0$; z not on the negative real axis
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



wn (natural frequency) against z_re (real part of the discrete pole) from 0.05 to 5 on a logarithmic axis. One curve for each value of T (sampling period in seconds): 0.1, 0.05, 0.2. Other inputs as in the worked example: z_im = 0.0. 41 points per curve computed by control-systems-ai999x; marked: the worked example, z_re = 0.5, wn = 6.93147 rad/s.

```
control_systems_ai999x.solve('ctrl_z_to_s_pole', z_re=0.5, z_im=0, T=0.1)
# -> s_re = -6.93147, s_im = 0, wn = 6.93147 rad/s, zeta = 1   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_z_to_s_pole', z_re=0, z_im=0.5, T=1)
# -> s_re = -0.693147, s_im = 1.5708, wn = 1.71693 rad/s, zeta = 0.403713   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
z-plane to s-plane pole: z_re = 0.5, z_im = 0, T = 0.1	SOLVED	s_re = -6.93147, s_im = 0, wn = 6.93147 rad/s, zeta = 1
z-plane to s-plane pole: z_re = 0, z_im = 0.5, T = 1	SOLVED	s_re = -0.693147, s_im = 1.5708, wn = 1.71693 rad/s, zeta = 0.403713
z-plane to s-plane pole: z_im = 0, T = 0.1	NEEDS_INPUT	ctrl_z_to_s_pole needs z_re (real part of the discrete pole). Example: solve('ctrl_z_to_s_pole', z_re=0.5, z_im=0, T=0.1
z-plane to s-plane pole: z_re = 0.5, z_im = 0, T = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
z-plane to s-plane pole: z_re = -0.5, z_im = 0, T = 0.1	INVALID_INPUT	Continuous equivalent of a discrete pole (s = ln z / T): a pole on the negative real axis (or at 0) has no unique contin

Cohen-Coon tuning of a first-order-plus-dead-time process, `ctrl_cohen_coon`

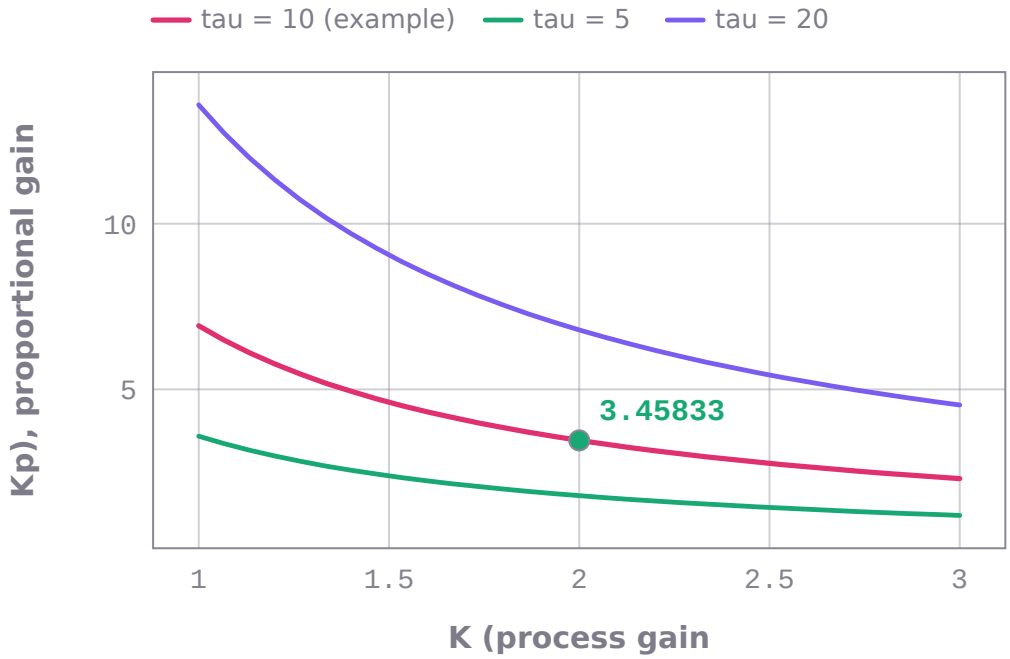
P, PI or PID settings from the Cohen-Coon formulas for K e^{-theta s}/(tau s + 1).

$$\text{PID: } K_p = \frac{\tau}{K\theta}\left(\frac{4}{3} + \frac{\theta}{4\tau}\right),\ T_i = \theta\frac{32 + 6\theta/\tau}{13 + 8\theta/\tau},\ T_d = \frac{4\theta}{11 + 2\theta/\tau}$$

input	meaning	condition
`K`	process gain	
`tau`	time constant in seconds	
`theta`	dead time in seconds	
`controller`	controller type	

output	meaning
`Kp`	proportional gain
`Ti`	integral time (0 when there is no integral action)
`Td`	derivative time
`Ki`	integral gain Kp/Ti
`Kd`	derivative gain Kp Td

- **Method:** Cohen-Coon formulas $(r = \theta/\tau)$
- **Assumes:** K, tau, $\theta > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Kp (proportional gain) against K (process gain) from 1 to 3. One curve for each value of tau (time constant in seconds): 10, 5, 20. Other inputs as in the worked example: theta = 2.0, controller = PID. 31 points per curve computed by control-systems-ai999x; marked: the worked example, K = 2, Kp = 3.45833.

```
control_systems_ai999x.solve('ctrl_cohen_coon', K=2, tau=10, theta=2, controller='PID')
# -> Kp = 3.45833, Ti = 4.54795 s, Td = 0.701754 s, Ki = 0.760417, Kd = 2.4269   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_cohen_coon', K=1, tau=5, theta=1, controller='PI')
# -> Kp = 4.58333, Ti = 2.35385 s, Td = 0 s, Ki = 1.94717, Kd = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cohen-Coon tuning: K = 2, tau = 10, theta = 2, controller = PID	SOLVED	Kp = 3.45833, Ti = 4.54795 s, Td = 0.701754 s, Ki = 0.760417, Kd = 2.4269
Cohen-Coon tuning: K = 1, tau = 5, theta = 1, controller = PI	SOLVED	Kp = 4.58333, Ti = 2.35385 s, Td = 0 s, Ki = 1.94717, Kd = 0
Cohen-Coon tuning: tau = 10, theta = 2, controller = PID	NEEDS_INPUT	ctrl_cohen_coon needs K (process gain). Example: solve('ctrl_cohen_coon', K=2, tau=10, theta=2, controller='PID').
Cohen-Coon tuning: K = 2, tau = 10, theta = 2, controller = PID, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Cohen-Coon tuning: K = 2, tau = 10, theta = 0, controller = PID	INVALID_INPUT	Cohen-Coon tuning of a first-order-plus-dead-time process: theta must be positive.

SIMC (Skogestad) PI tuning, `ctrl_simc_pi`

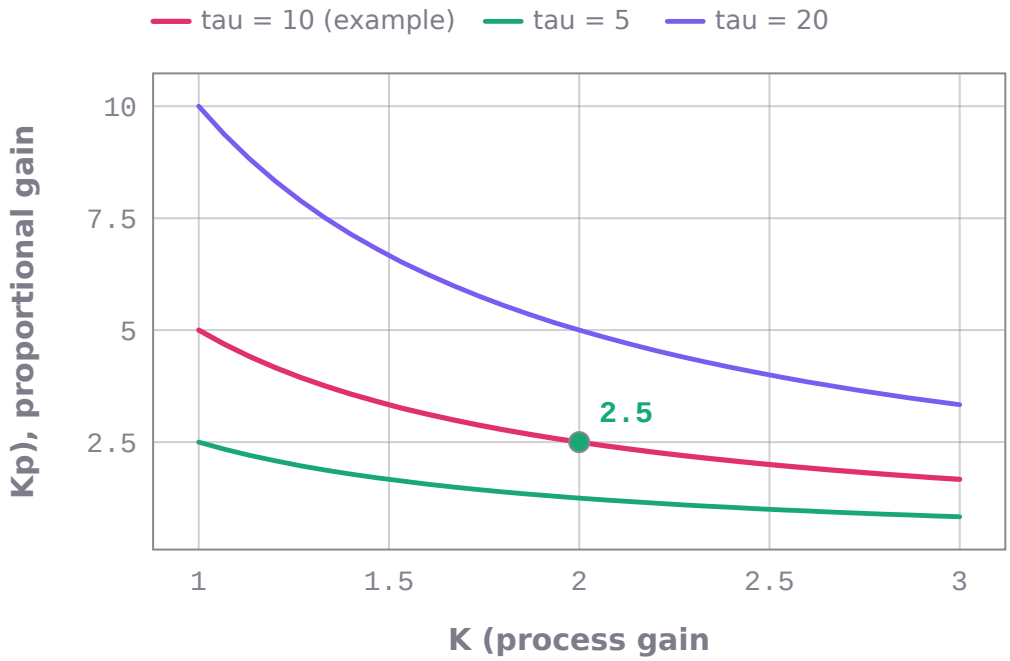
PI settings Kp = tau/(K (tau_c + theta)), Ti = min(tau, 4 (tau_c + theta)) for K e^{-theta s}/(tau s + 1); the default tau_c equals theta.

$$K_p = \frac{\tau}{K(\tau_c + \theta)},\ T_i = \min(\tau, 4(\tau_c + \theta))$$

input	meaning	condition
`K`	process gain	
`tau`	time constant in seconds	
`theta`	dead time in seconds	
`tau_c`	desired closed-loop time constant (default theta)	

output	meaning
`Kp`	proportional gain
`Ti`	integral time
`Ki`	integral gain

- **Method:** Skogestad's SIMC rule
- **Assumes:** K, tau, $\theta > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Kp (proportional gain) against K (process gain) from 1 to 3. One curve for each value of tau (time constant in seconds): 10, 5, 20. Other inputs as in the worked example: theta = 1.0. 31 points per curve computed by control-systems-ai999x; marked: the worked example, K = 2, Kp = 2.5.

```
control_systems_ai999x.solve('ctrl_simc_pi', K=2, tau=10, theta=1)
# -> Kp = 2.5, Ti = 8 s, Ki = 0.3125   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_simc_pi', K=1, tau=4, theta=0.5, tau_c=2)
# -> Kp = 1.6, Ti = 4 s, Ki = 0.4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
SIMC PI tuning: K = 2, tau = 10, theta = 1	SOLVED	Kp = 2.5, Ti = 8 s, Ki = 0.3125
SIMC PI tuning: K = 1, tau = 4, theta = 0.5, tau_c = 2	SOLVED	Kp = 1.6, Ti = 4 s, Ki = 0.4
SIMC PI tuning: tau = 10, theta = 1	NEEDS_INPUT	ctrl_simc_pi needs K (process gain). Example: solve('ctrl_simc_pi', K=2, tau=10, theta=1).
SIMC PI tuning: K = 2, tau = 10, theta = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
SIMC PI tuning: K = 2, tau = 10, theta = -1	INVALID_INPUT	SIMC (Skogestad) PI tuning: theta must be positive.

Closed-loop poles of a PID loop, `ctrl_pid_closed_loop`

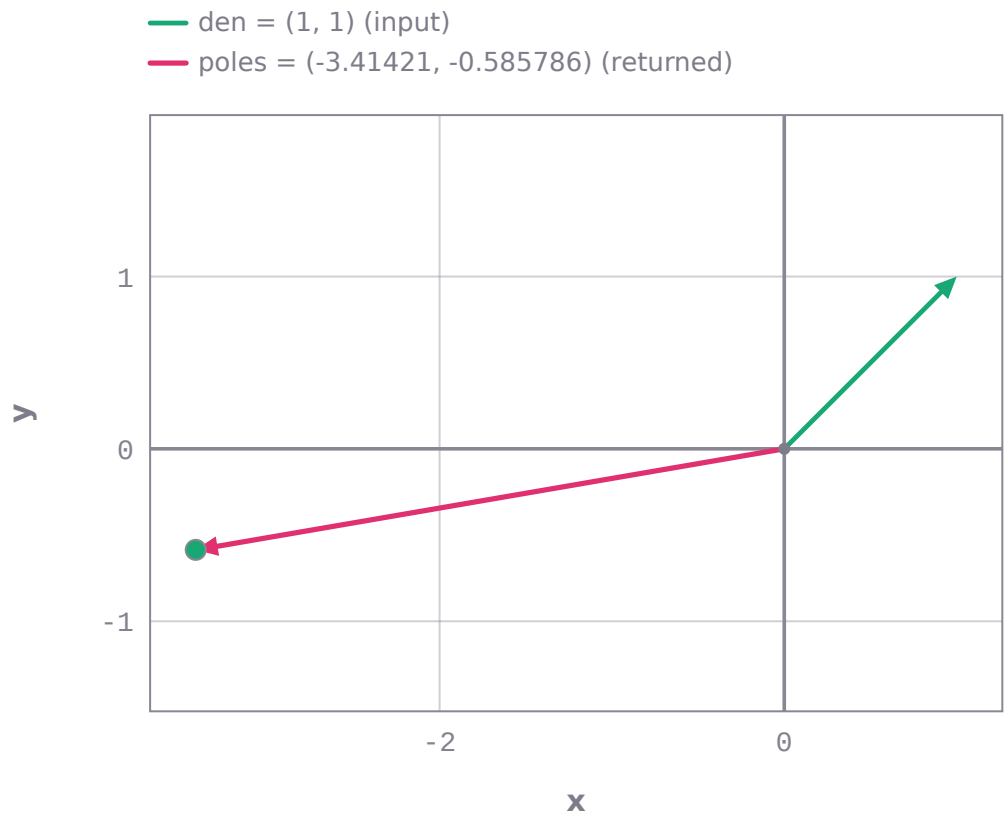
The characteristic polynomial s D(s) + (Kd $s^2 + Kp$ s + Ki) N(s), the closed-loop poles and stability of a plant N/D under a PID controller with unity feedback.

$$sD(s) + (K_d s^2 + K_p s + K_i)N(s) = 0$$

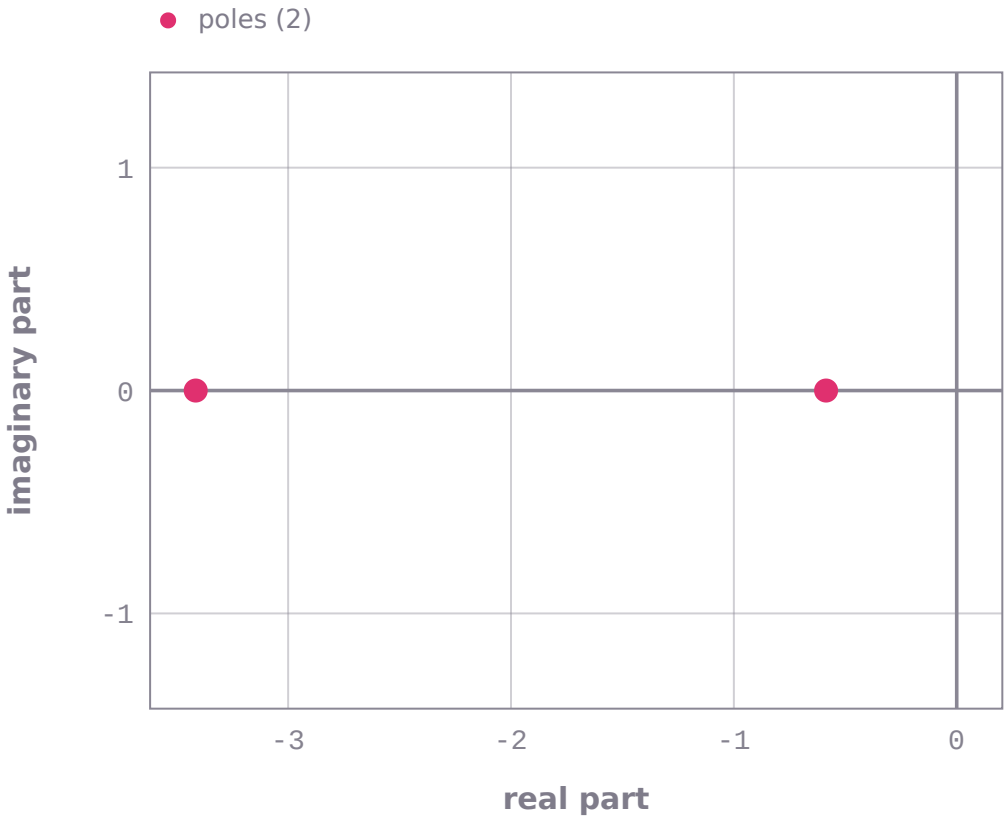
input	meaning	condition
`num`	plant numerator	
`den`	plant denominator	
`Kp`	proportional gain	
`Ki`	integral gain (default 0)	
`Kd`	derivative gain (default 0)	

output	meaning
`char_poly`	closed-loop characteristic polynomial
`poles`	closed-loop poles
`max_real_part`	largest real part
`stable`	true when every pole is in the left half plane

- **Method:** $1 + C$(s) G(s) = 0, roots by the Aberth-Ehrlich iteration
- **Assumes:** strictly proper plant
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input den = (1, 1); returned by control-systems-ai999x: poles = (-3.41421, -0.585786); also returned max_real_part = -0.585786. Other inputs: num = [1], Kp = 3.0, Ki = 2.0. Marked: the tip of each returned vector.



closed-loop poles (poles) returned by control-systems-ai999x for the worked example (num = [1], den = [1, 1], Kp = 3.0, Ki = 2.0), drawn in the complex plane: -3.41421 + 0i, -0.585786 + 0i.

```
control_systems_ai999x.solve('ctrl_pid_closed_loop', num=[1], den=[1, 1], Kp=3, Ki=2)
# -> char_poly = [1.0, 4.0, 2.0], poles = [-3.41421356237, -0.585786437627], max_real_part
= -0.585786, stable = True   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_pid_closed_loop', num=[1], den=[1, 3, 2], Kp=10)
# -> char_poly = [1.0, 3.0, 12.0], poles = [(-1.5-3.1224989992j), (-1.5+3.1224989992j)],
max_real_part = -1.5, stable = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PID closed-loop poles: num = [1], den = [1, 1], Kp = 3, Ki = 2	SOLVED	char_poly = [1.0, 4.0, 2.0], poles = [-3.41421356237, -0.585786437627], max_real_part = -0.585786, stable = True
PID closed-loop poles: num = [1], den = [1, 3, 2], Kp = 10	SOLVED	char_poly = [1.0, 3.0, 12.0], poles = [(-1.5-3.1224989992j), (-1.5+3.1224989992j)], max_real_part = -1.5, stable = True
PID closed-loop poles: den = [1, 1], Kp = 3, Ki = 2	NEEDS_INPUT	ctrl_pid_closed_loop needs num (plant numerator). Example: solve('ctrl_pid_closed_loop', num=[1], den=[1, 1], Kp=3, Ki=2
PID closed-loop poles: num = [1], den = [1, 1], Kp = 3, Ki = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
PID closed-loop poles: num = [1, 1], den = [1, 1], Kp = 3	INVALID_INPUT	Closed-loop poles of a PID loop: the transfer function must be strictly proper (numerator degree below the denominator's

Maximum phase lead of a lead compensator, `ctrl_lead_max_phase`

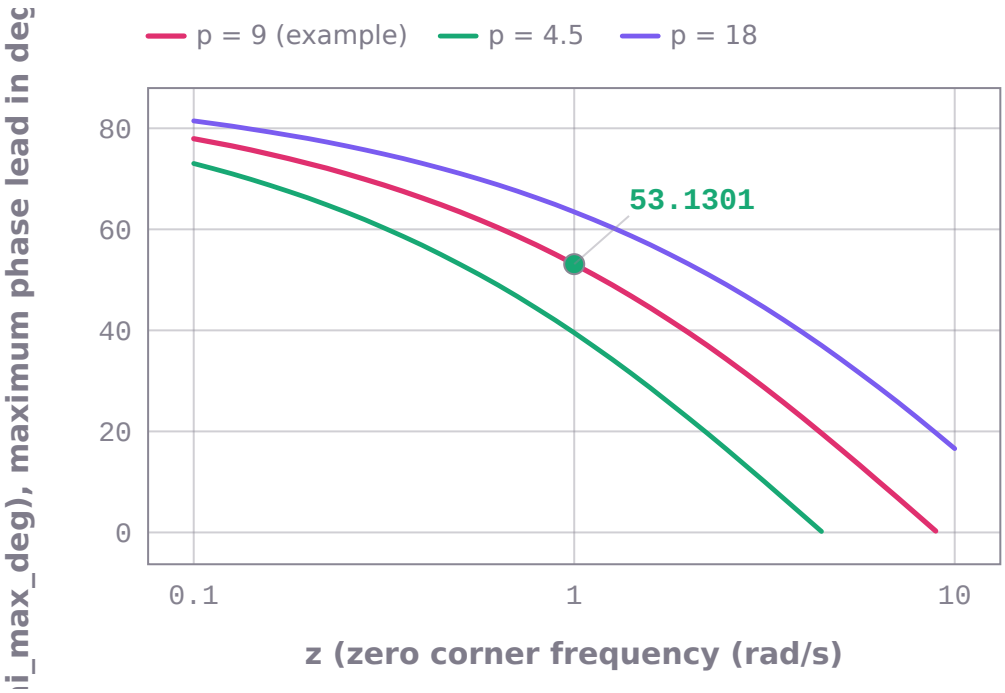
For C(s) = (s + z)/(s + p) with $z < p$: the maximum phase lead asin((p - z)/(p + z)), the frequency sqrt(z p) where it occurs, $\alpha = z/p$ and the gain there, 10 log10(z/p) dB.

$$\sin\phi_{max} = \frac{p-z}{p+z} = \frac{1-\alpha}{1+\alpha}, \quad \omega_m = \sqrt{zp}$$

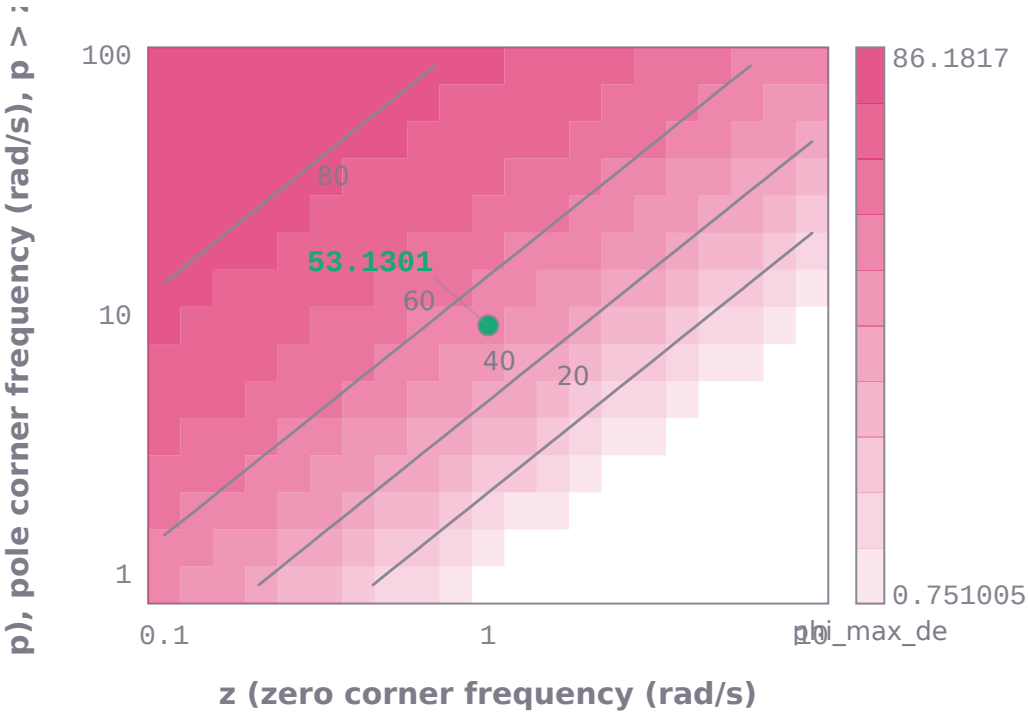
input	meaning	condition
`z`	zero corner frequency (rad/s)	
`p`	pole corner frequency (rad/s), $p > z$	

output	meaning
`phi_max_deg`	maximum phase lead in degrees
`w_max`	frequency of the maximum
`alpha`	$\alpha = z/p$
`magnitude_db`	|C(j w_max)| in dB

- **Method:** stationary point of the phase
- **Assumes:** $0 < z < p$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



phi_max_deg (maximum phase lead in degrees) against z (zero corner frequency (rad/s)) from 0.1 to 10 on a logarithmic axis. One curve for each value of p (pole corner frequency (rad/s), p > z): 9, 4.5, 18. 40 points per curve computed by control-systems-ai999x; marked: the worked example, z = 1, phi_max_deg = 53.1301.



phi_max_deg (maximum phase lead in degrees) over z (zero corner frequency (rad/s)) from 0.1 to 10 and p (pole corner frequency (rad/s), p > z) from 0.9 to 90 (logarithmic axes): each of the 21 × 15 cells is one run of control-systems-ai999x, shaded by its value (light 0.751005 to dark 86.1817); iso-lines at phi_max_deg = 20, 40, 60, 80; blank cells: inputs the capability refuses. Marked: the worked example, z = 1, p = 9, phi_max_deg = 53.1301.

```
control_systems_ai999x.solve('ctrl_lead_max_phase', z=1, p=9)
# -> phi_max_deg = 53.1301, w_max = 3 rad/s, alpha = 0.111111, magnitude_db = -9.54243
(SOLVED)
```

```
control_systems_ai999x.solve('ctrl_lead_max_phase', z=2, p=8)
# -> phi_max_deg = 36.8699, w_max = 4 rad/s, alpha = 0.25, magnitude_db = -6.0206
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
lead compensator maximum phase: z = 1, p = 9	SOLVED	phi_max_deg = 53.1301, w_max = 3 rad/s, alpha = 0.111111, magnitude_db = -9.54243
lead compensator maximum phase: z = 2, p = 8	SOLVED	phi_max_deg = 36.8699, w_max = 4 rad/s, alpha = 0.25, magnitude_db = -6.0206
lead compensator maximum phase: p = 9	NEEDS_INPUT	ctrl_lead_max_phase needs z (zero corner frequency (rad/s)). Example: solve('ctrl_lead_max_phase', z=1, p=9).
lead compensator maximum phase: z = 1, p = 9, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
lead compensator maximum phase: z = 9, p = 1	INVALID_INPUT	Maximum phase lead of a lead compensator: a lead compensator has its zero below its pole (z < p).

Desired closed-loop poles from overshoot and settling time, `ctrl_specs_to_poles`

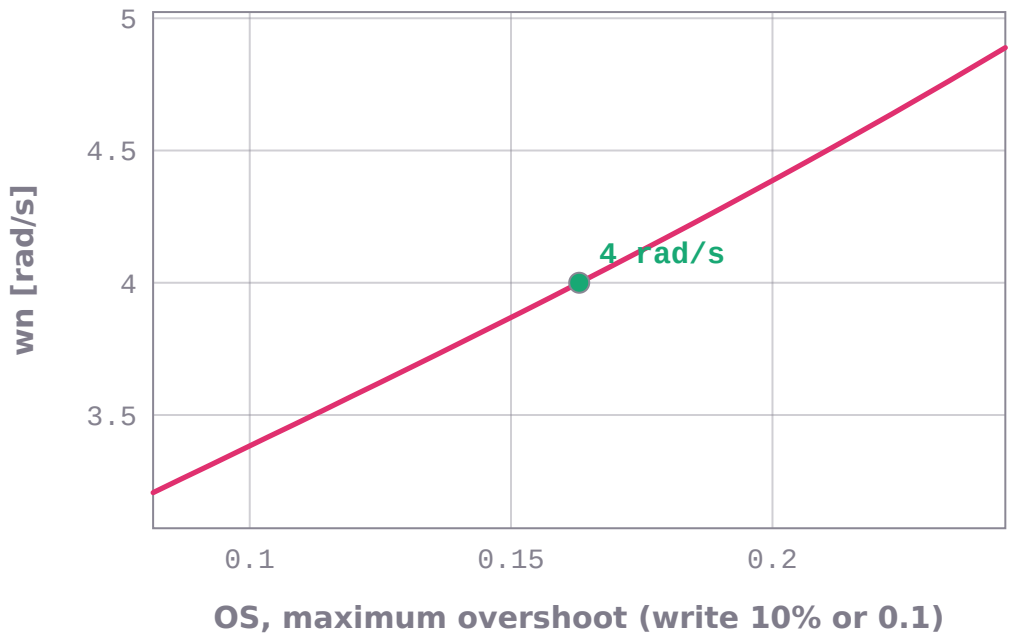
zeta from the overshoot, $\sigma = 4/Ts$ from the 2 % settling time, $wn = \sigma/\zeta$, the poles -sigma +/- j wd and the desired characteristic polynomial.

$$\zeta = \frac{-\ln OS}{\sqrt{\pi^2 + \ln^2 OS}}, \quad \sigma = \frac{4}{T_s}, \quad \omega_n = \frac{\sigma}{\zeta}$$

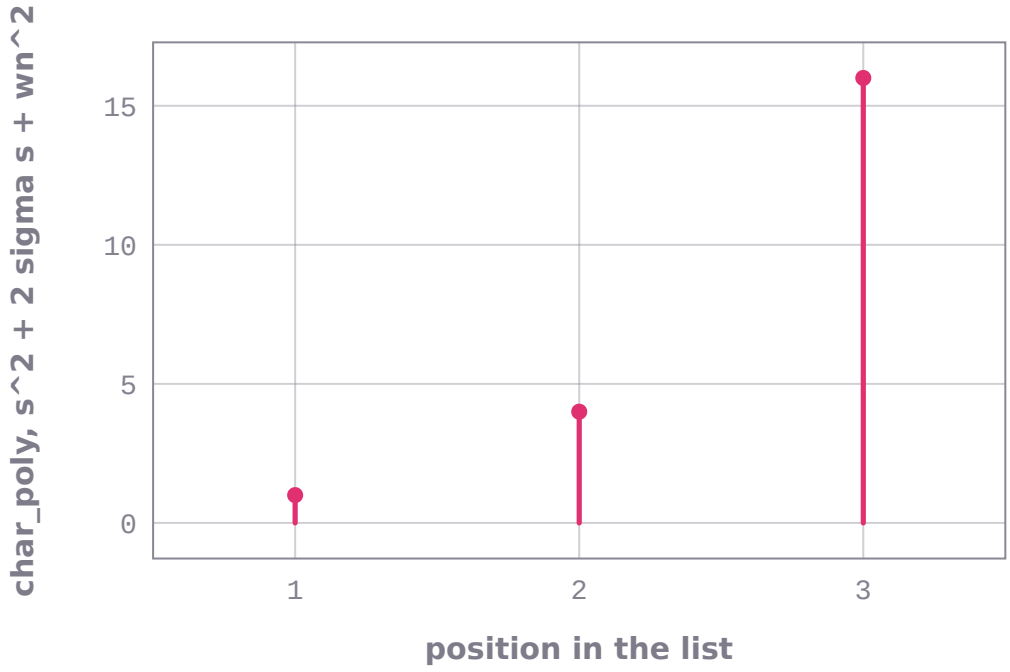
input	meaning	condition
`OS`	maximum overshoot (write 10% or 0.1)	
`Ts`	2 % settling time in seconds	

output	meaning
`zeta`	damping ratio
`wn`	natural frequency
`sigma`	real part magnitude $4/Ts$
`wd`	imaginary part of the poles
`char_poly`	$s^2 + 2$ sigma $s + wn^2$

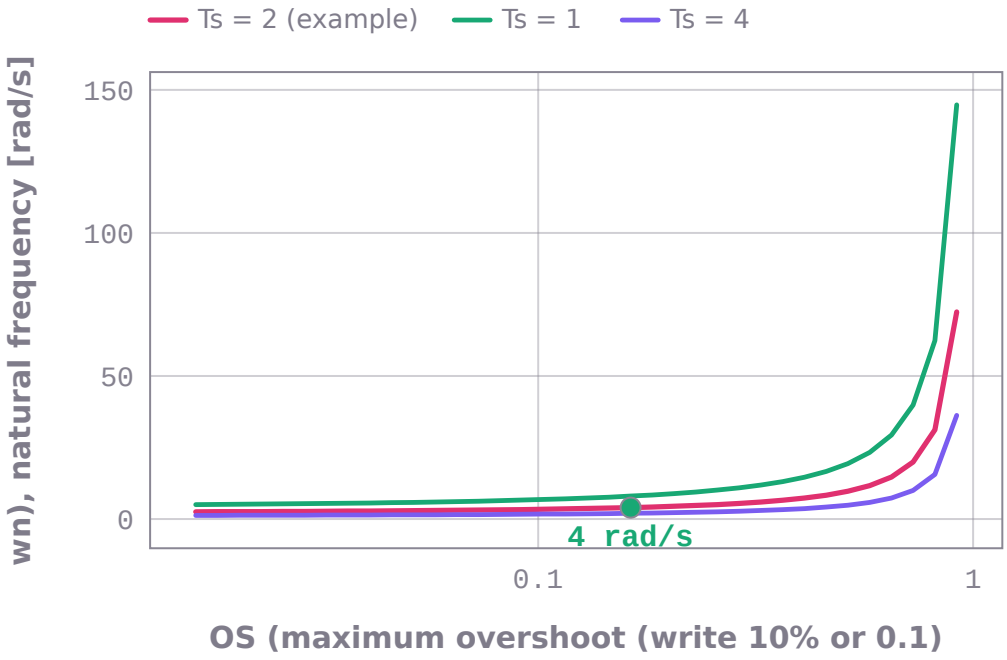
- **Method:** zeta from the overshoot formula; $\sigma = 4/Ts$
- **Assumes:** $0 < OS < 100$ %; $Ts > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



wn (natural frequency) against OS (maximum overshoot (write 10% or 0.1)), with Ts = 2. 31 points computed by control-systems-ai999x; marked: OS = 0.163034, wn = 4 rad/s.



s^2 + 2 sigma s + wn^2 (char_poly) returned by control-systems-ai999x for the worked example, one value for each position in the list (3 values, first 1, last 16, largest 16). Inputs: OS = 0.163033534821, Ts = 2.0.



wn (natural frequency) against OS (maximum overshoot (write 10% or 0.1)) from 0.0163034 to 1.63034 on a logarithmic axis. One curve for each value of Ts (2 % settling time in seconds): 2, 1, 4. 36 points per curve computed by control-systems-ai999x; marked: the worked example, OS = 0.163034, wn = 4 rad/s.

```
control_systems_ai999x.solve('ctrl_specs_to_poles', OS=0.163033534821, Ts=2)
# -> zeta = 0.5, wn = 4 rad/s, sigma = 2, wd = 3.4641 rad/s, char_poly = [1.0, 4.0, 16.0]
(SOLVED)
```

```
control_systems_ai999x.solve('ctrl_specs_to_poles', OS=0.1, Ts=1)
# -> zeta = 0.591155, wn = 6.76641 rad/s, sigma = 4, wd = 5.45751 rad/s, char_poly = [1.0,
8.0, 45.7843653588]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
poles from overshoot and settling time: OS = 0.163033534821, Ts = 2	SOLVED	zeta = 0.5, wn = 4 rad/s, sigma = 2, wd = 3.4641 rad/s, char_poly = [1.0, 4.0, 16.0]
poles from overshoot and settling time: OS = 0.1, Ts = 1	SOLVED	zeta = 0.591155, wn = 6.76641 rad/s, sigma = 4, wd = 5.45751 rad/s, char_poly = [1.0, 8.0, 45.7843653588]
poles from overshoot and settling time: Ts = 2	NEEDS_INPUT	ctrl_specs_to_poles needs OS (maximum overshoot (write 10% or 0.1)). Example: solve('ctrl_specs_to_poles', OS=0.163034,
poles from overshoot and settling time: OS = 0.163033534821, Ts = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
poles from overshoot and settling time: OS = 0.1, Ts = -1	INVALID_INPUT	Desired closed-loop poles from overshoot and settling time: Ts must be positive.

Damping ratio from the logarithmic decrement, `ctrl_log_decrement`

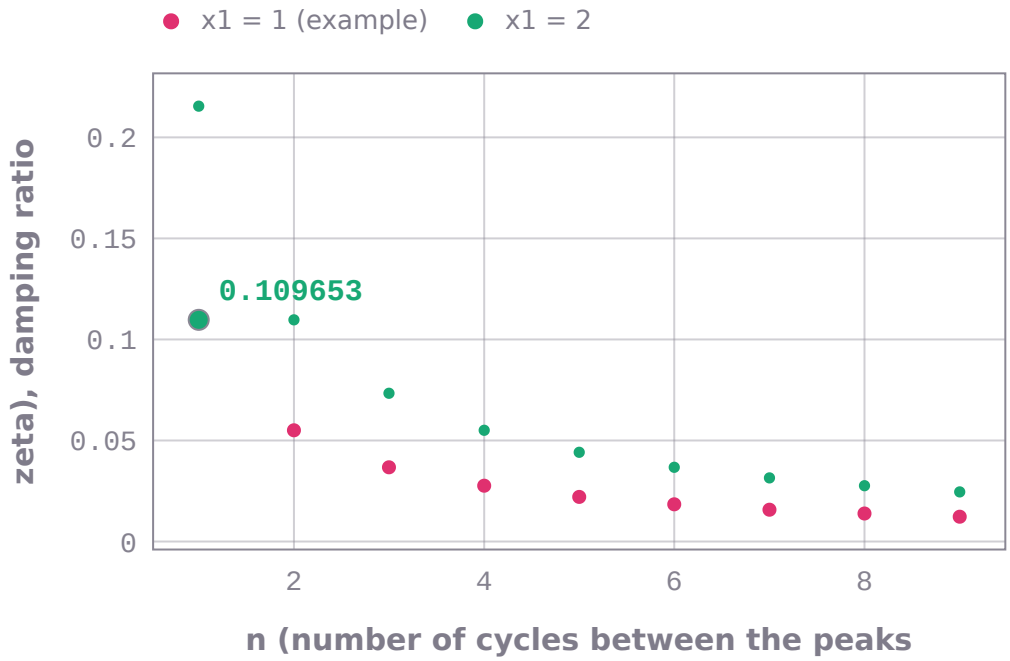
$\delta = ln$(x1/xn)/n from two peak amplitudes n cycles apart, and $\zeta = \delta / sqrt$(4 pi^2 + delta^2).

$$\delta = \frac{1}{n} \ln \frac{x_1}{x_{n+1}}, \quad \zeta = \frac{\delta}{\sqrt{4\pi^2 + \delta^2}}$$

input	meaning	condition
`x1`	first peak amplitude (measured from the final value)	
`xn`	peak amplitude n cycles later	
`n`	number of cycles between the peaks	

output	meaning
`delta`	logarithmic decrement per cycle
`zeta`	damping ratio

- **Method:** ratio of successive peaks
- **Assumes:** $0 < xn < x_1; n \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



zeta (damping ratio) against n (number of cycles between the peaks) from 1 to 9. One curve for each value of x1 (first peak amplitude (measured from the final value)): 1, 2. Other inputs as in the worked example: xn = 0.5. 9 points per curve computed by control-systems-ai999x; marked: the worked example, n = 1, zeta = 0.109653.

```
control_systems_ai999x.solve('ctrl_log_decrement', x1=1, xn=0.5, n=1)
# -> delta = 0.693147, zeta = 0.109653   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_log_decrement', x1=4, xn=1, n=3)
# -> delta = 0.462098, zeta = 0.0733471   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
logarithmic decrement: x1 = 1, xn = 0.5, n = 1	SOLVED	delta = 0.693147, zeta = 0.109653
logarithmic decrement: x1 = 4, xn = 1, n = 3	SOLVED	delta = 0.462098, zeta = 0.0733471
logarithmic decrement: xn = 0.5, n = 1	NEEDS_INPUT	ctrl_log_decrement needs x1 (first peak amplitude (measured from the final value)). Example: solve('ctrl_log_decrement',
logarithmic decrement: x1 = 1, xn = 0.5, n = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
logarithmic decrement: x1 = 1, xn = 2, n = 1	INVALID_INPUT	Damping ratio from the logarithmic decrement: the later peak must be smaller.

Second-order model from a step response, `ctrl_identify_second_order`

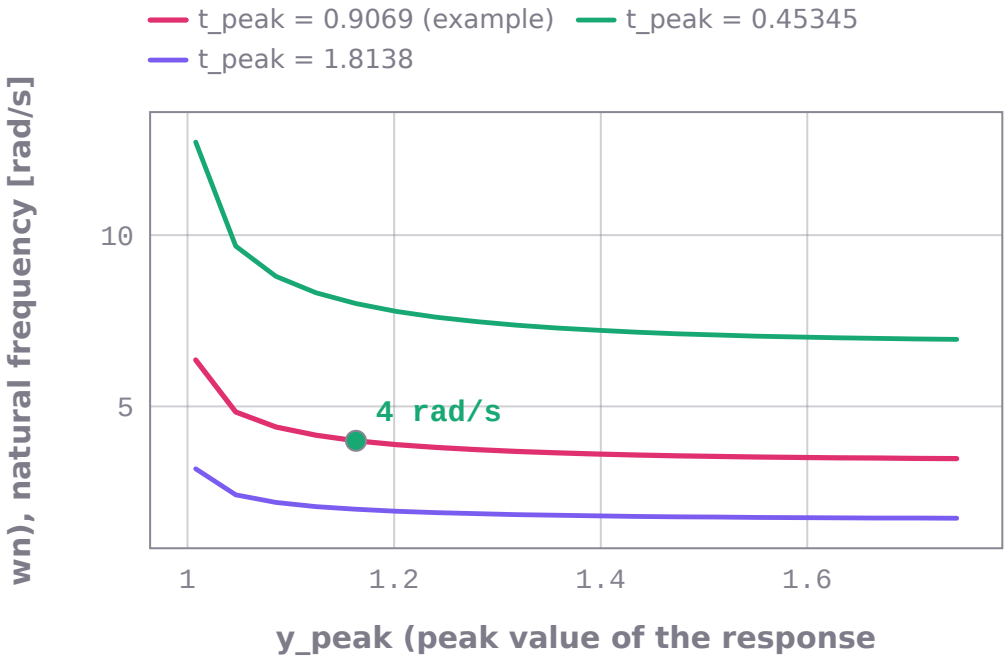
Gain K = y_final/A, percent overshoot, zeta and wn of a standard second-order model from the peak value, the final value and the peak time of a measured step response.

$$\omega_n = \frac{\pi}{T_p\sqrt{1-\zeta^2}}$$

input	meaning	condition
`y_peak`	peak value of the response	
`y_final`	final (steady) value	
`t_peak`	time of the peak in seconds	
`A`	step size (default 1)	

output	meaning
`K`	DC gain
`PO`	percent overshoot (%)
`zeta`	damping ratio
`wn`	natural frequency

- **Method:** zeta from the overshoot, wn = pi/(tp sqrt(1 - zeta^2))
- **Assumes:** 0 < overshoot < 100 %; t_peak > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



wn (natural frequency) against y_peak (peak value of the response) from 0.581517 to 1.74455. One curve for each value of t_peak (time of the peak in seconds): 0.9069, 0.45345, 1.8138. Other inputs as in the worked example: y_final = 1.0. 20 points per curve computed by control-systems-ai999x; marked: the worked example, y_peak = 1.16303, wn = 4 rad/s.

```
control_systems_ai999x.solve('ctrl_identify_second_order', y_peak=1.163033534821,
y_final=1, t_peak=0.906899682118)
# -> K = 1, PO = 16.3034, zeta = 0.5, wn = 4 rad/s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_identify_second_order', y_peak=12.5, y_final=10,
t_peak=1.2, A=2)
# -> K = 5, PO = 25, zeta = 0.403713, wn = 2.86155 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
second-order model from step response: y_peak = 1.163033534821, y_final = 1, t_peak = 0.906899682118	SOLVED	K = 1, PO = 16.3034, zeta = 0.5, wn = 4 rad/s
second-order model from step response: y_peak = 12.5, y_final = 10, t_peak = 1.2, A = 2	SOLVED	K = 5, PO = 25, zeta = 0.403713, wn = 2.86155 rad/s
second-order model from step response: y_final = 1, t_peak = 0.906899682118	NEEDS_INPUT	ctrl_identify_second_order needs y_peak (peak value of the response). Example: solve('ctrl_identify_second_order', y_pea
second-order model from step response: y_peak = 1.163033534821, y_final = 1, t_peak = 0.906899682118, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
second-order model from step response: y_peak = 0.9, y_final = 1, t_peak = 1	INVALID_INPUT	Second-order model from a step response: the peak must exceed the final value by less than 100 %.

Step response of a first-order-plus-dead-time process, `ctrl_fopdt_step`

y(t) of K e^{-theta s}/(tau s + 1) after a step of size A: 0 before the dead time, K A (1 - e^{-(t - theta)/tau}) after it.

$$y(t) = KA\left(1 - e^{-(t-\theta)/\tau}\right) u(t-\theta)$$

input	meaning	condition
`K`	process gain	
`tau`	time constant in seconds	
`theta`	dead time in seconds	
`t`	time in seconds	
`A`	step size (default 1)	

output	meaning
`y`	output at time t

- **Method:** delayed first-order response
- **Assumes:** $\tau > 0; \theta \geq 0; t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y (output at time t) against theta (dead time in seconds) from 0.1 to 10 on a logarithmic axis. One curve for each value of K (process gain): 2, 1, 4. Other inputs as in the worked example: tau = 5.0, t = 6.0. 41 points per curve computed by control-systems-ai999x; marked: the worked example, theta = 1, y = 1.26424.

```
control_systems_ai999x.solve('ctrl_fopdt_step', K=2, tau=5, theta=1, t=6)
# -> y = 1.26424   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_fopdt_step', K=2, tau=5, theta=1, t=0.5)
# -> y = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
dead-time step response: K = 2, tau = 5, theta = 1, t = 6	SOLVED	y = 1.26424
dead-time step response: K = 2, tau = 5, theta = 1, t = 0.5	SOLVED	y = 0
dead-time step response: tau = 5, theta = 1, t = 6	NEEDS_INPUT	ctrl_fopdt_step needs K (process gain). Example: solve('ctrl_fopdt_step', K=2, tau=5, theta=1, t=6).
dead-time step response: K = 2, tau = 5, theta = 1, t = 6, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
dead-time step response: K = 2, tau = 0, theta = 1, t = 6	INVALID_INPUT	Step response of a first-order-plus-dead-time process: tau must be positive.

One step of a scalar Kalman filter, `ctrl_kalman_scalar_step`

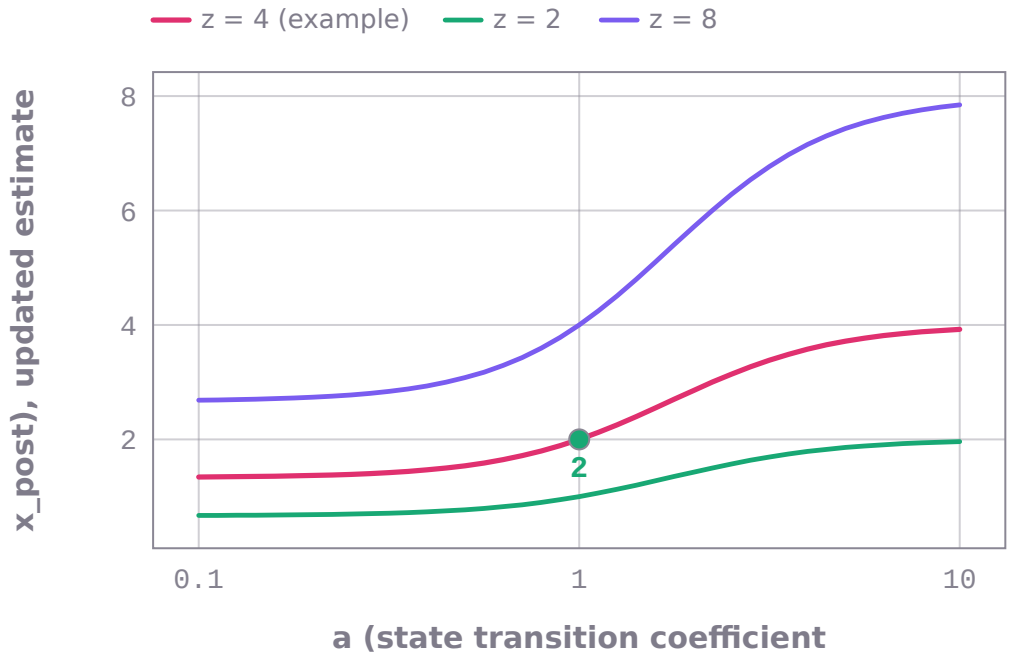
Predict (x- = a x, P- = a^2 P + q) and update with a measurement z of variance r (gain P-/(P- + r)) for $x_k = a$ x_{$k-1$} + w, $z_k = x_k + v$.

$$K = \frac{P^-}{P^- + r}, \quad \hat{x} = \hat{x}^- + K(z - \hat{x}^-), \quad P = (1 - K)P^-$$

input	meaning	condition
`x`	previous estimate	
`P`	previous error variance	
`a`	state transition coefficient	
`q`	process noise variance	
`r`	measurement noise variance	
`z`	new measurement	

output	meaning
`x_pred`	predicted estimate
`P_pred`	predicted variance
`gain`	Kalman gain
`x_post`	updated estimate
`P_post`	updated variance

- **Method:** Kalman predict and update equations
- **Assumes:** P, $q \geq 0$; $r > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x_post (updated estimate) against a (state transition coefficient) from 0.1 to 10 on a logarithmic axis. One curve for each value of z (new measurement): 4, 2, 8. Other inputs as in the worked example: x = 0.0, P = 1.0, q = 1.0, r = 2.0. 41 points per curve computed by control-systems-ai999x; marked: the worked example, a = 1, x_post = 2.

```
control_systems_ai999x.solve('ctrl_kalman_scalar_step', x=0, P=1, a=1, q=1, r=2, z=4)
# -> x_pred = 0, P_pred = 2, gain = 0.5, x_post = 2, P_post = 1   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_kalman_scalar_step', x=1, P=0.5, a=2, q=0, r=2, z=5)
# -> x_pred = 2, P_pred = 2, gain = 0.5, x_post = 3.5, P_post = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
scalar Kalman filter step: x = 0, P = 1, a = 1, q = 1, r = 2, z = 4	SOLVED	x_pred = 0, P_pred = 2, gain = 0.5, x_post = 2, P_post = 1
scalar Kalman filter step: x = 1, P = 0.5, a = 2, q = 0, r = 2, z = 5	SOLVED	x_pred = 2, P_pred = 2, gain = 0.5, x_post = 3.5, P_post = 1
scalar Kalman filter step: P = 1, a = 1, q = 1, r = 2, z = 4	NEEDS_INPUT	ctrl_kalman_scalar_step needs x (previous estimate). Example: solve('ctrl_kalman_scalar_step', x=0, P=1, a=1, q=1, r=2,
scalar Kalman filter step: x = 0, P = 1, a = 1, q = 1, r = 2, z = 4, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
scalar Kalman filter step: x = 0, P = 1, a = 1, q = 1, r = 0, z = 4	INVALID_INPUT	One step of a scalar Kalman filter: r must be positive.

Speed transfer function of a DC motor, `ctrl_dc_motor`

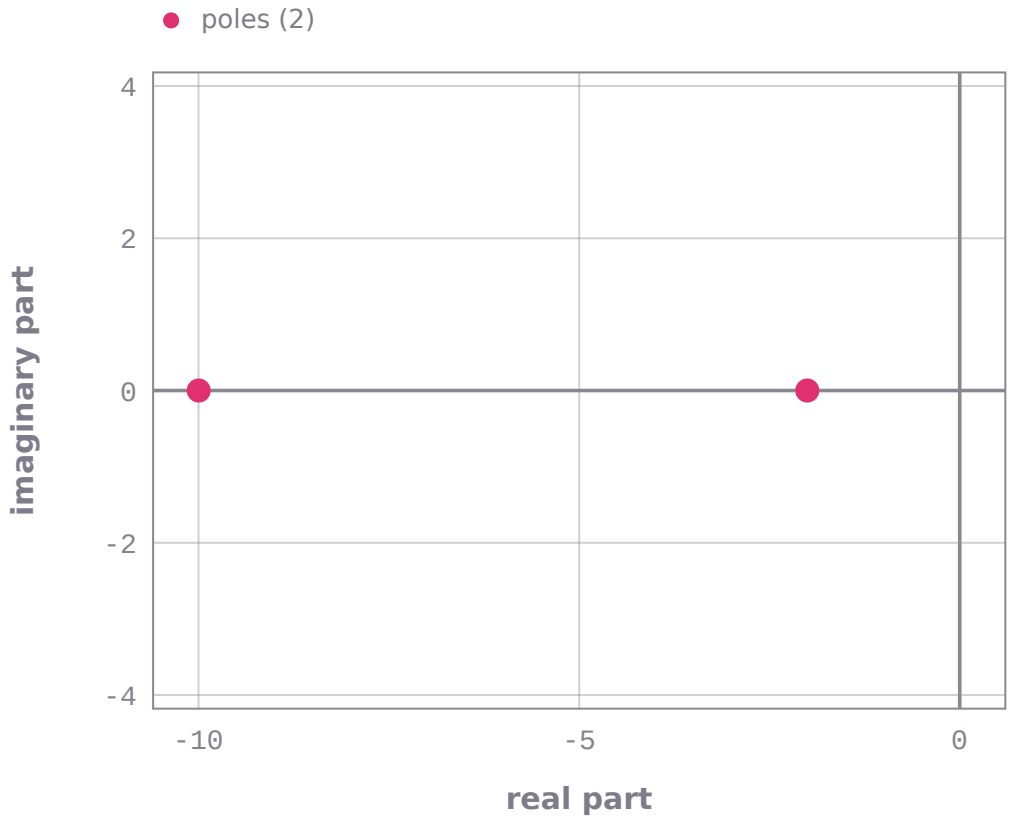
omega(s)/V(s) = Kt/((J s + b)(L s + R) + Kt^2) of an armature-controlled DC motor (Ke = Kt in SI units): coefficients, DC gain and poles.

$$\frac{\Omega(s)}{V(s)} = \frac{K_t}{(Js + b)(Ls + R) + K_t^2}$$

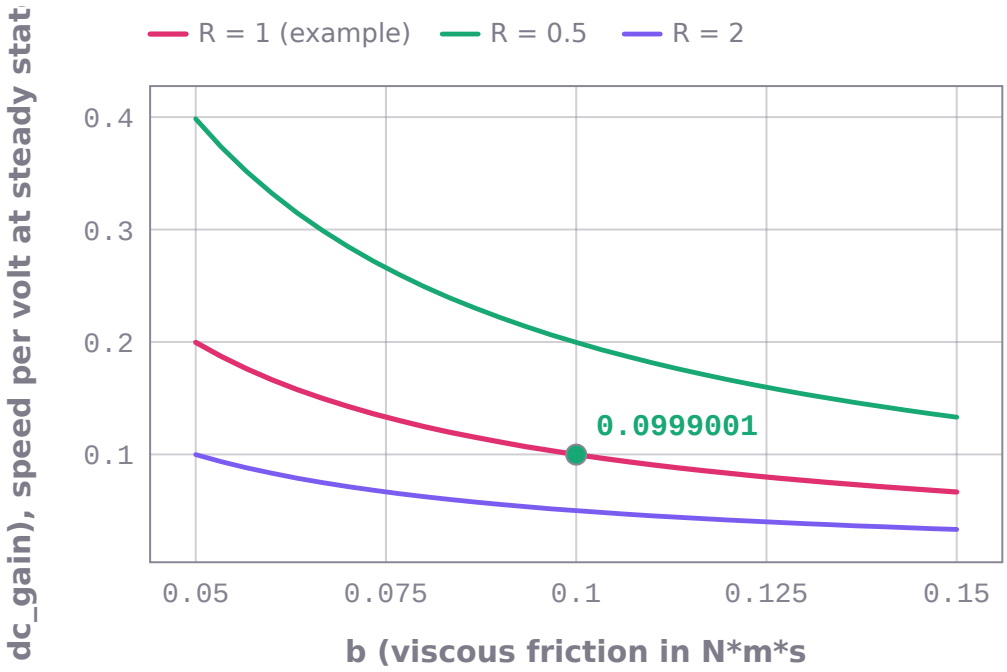
input	meaning	condition
`J`	rotor inertia in kgm^2	
`b`	viscous friction in Nms	
`Kt`	torque (= back-emf) constant in Nm/A	
`R`	armature resistance in ohm	
`L`	armature inductance in H	

output	meaning
`num`	numerator
`den`	denominator [J L, b L + J R, b $R + Kt^2$]
`dc_gain`	speed per volt at steady state (rad/s per V)
`poles`	poles

- **Method:** electrical and mechanical equations combined
- **Assumes:** J, Kt, R, $L > 0$; $b \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



poles (poles) returned by control-systems-ai999x for the worked example (J = 0.01, b = 0.1, Kt = 0.01, R = 1.0, L = 0.5), drawn in the complex plane: -9.9975 + 0i, -2.0025 + 0i.



dc_gain (speed per volt at steady state (rad/s per V)) against b (viscous friction in N*m*s) from 0.05 to 0.15. One curve for each value of R (armature resistance in ohm): 1, 0.5, 2. Other inputs as in the worked example: J = 0.01, Kt = 0.01, L = 0.5. 31 points per curve computed by control-systems-ai999x; marked: the worked example, b = 0.1, dc_gain = 0.0999001.

```
control_systems_ai999x.solve('ctrl_dc_motor', J=0.01, b=0.1, Kt=0.01, R=1, L=0.5)
# -> num = [0.01], den = [0.005, 0.06, 0.1001], dc_gain = 0.0999001, poles =
[-9.99749921826, -2.00250078174]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_dc_motor', J=1, b=0, Kt=1, R=2, L=1)
# -> num = [1.0], den = [1.0, 2.0, 1.0], dc_gain = 1, poles = [-1.0, -1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
DC motor transfer function: J = 0.01, b = 0.1, Kt = 0.01, R = 1, L = 0.5	SOLVED	num = [0.01], den = [0.005, 0.06, 0.1001], dc_gain = 0.0999001, poles = [-9.99749921826, -2.00250078174]
DC motor transfer function: J = 1, b = 0, Kt = 1, R = 2, L = 1	SOLVED	num = [1.0], den = [1.0, 2.0, 1.0], dc_gain = 1, poles = [-1.0, -1.0]
DC motor transfer function: b = 0.1, Kt = 0.01, R = 1, L = 0.5	NEEDS_INPUT	ctrl_dc_motor needs J (rotor inertia in kg*m^2). Example: solve('ctrl_dc_motor', J=0.01, b=0.1, Kt=0.01, R=1, L=0.5).
DC motor transfer function: J = 0.01, b = 0.1, Kt = 0.01, R = 1, L = 0.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
DC motor transfer function: J = 0, b = 0.1, Kt = 0.01, R = 1, L = 0.5	INVALID_INPUT	Speed transfer function of a DC motor: J must be positive.

Gain and phase margins, `ctrl_stability_margins`

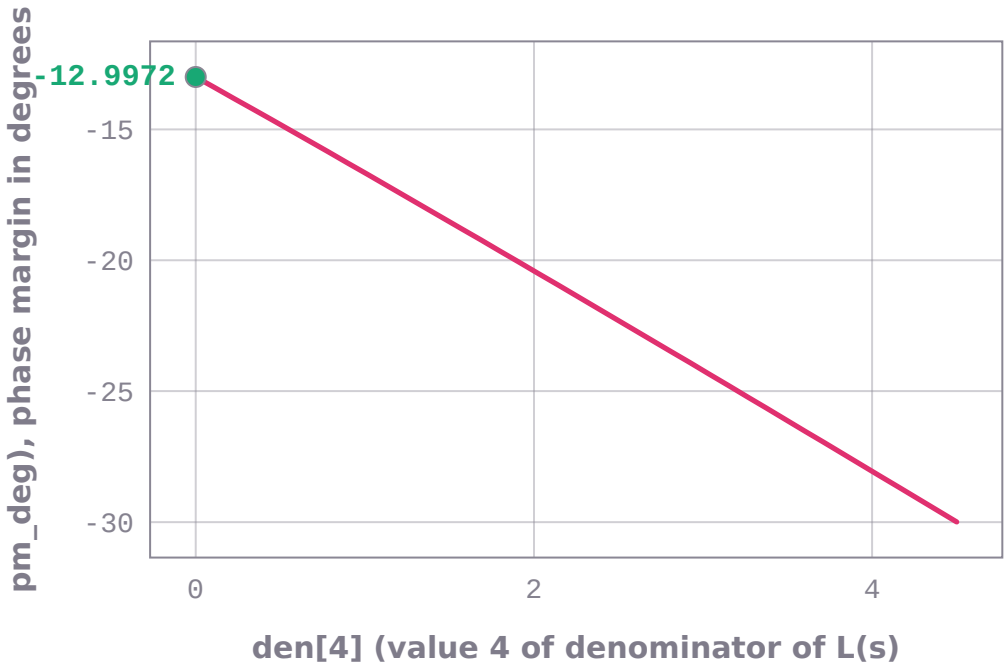
Gain margin 1/|L(j w_pc)| (ratio and dB) at the phase crossover and phase margin 180 + arg L(j w_gc) at the gain crossover of an open loop L(s).

$$GM = \frac{1}{|L(j\omega_{pc})|}, \quad PM = 180^\circ + \angle L(j\omega_{gc})$$

input	meaning	condition
`num`	numerator of the open loop L(s)	
`den`	denominator of L(s)	

output	meaning
`gm`	gain margin (ratio)
`gm_db`	gain margin in dB
`w_pc`	phase-crossover frequency
`pm_deg`	phase margin in degrees
`w_gc`	gain-crossover frequency

- **Method:** crossovers as positive real roots of |N|^2 - |D|^2 and Im(N(jw) conj D(jw)), Newton-polished
- **Assumes:** exactly one gain crossover and one phase crossover
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



pm_deg (phase margin in degrees) as value 4 of den (denominator of L(s)) moves from 0 to 4.5, the other values as given ([1, 3, 2, 0]); 41 runs of control-systems-ai999x. Other inputs: num = [10]. Marked: the worked example, den[4] = 0, pm_deg = -12.9972.

```
control_systems_ai999x.solve('ctrl_stability_margins', num=[10], den=[1, 3, 2, 0])
# -> gm = 0.6, gm_db = -4.43697, w_pc = 1.41421 rad/s, pm_deg = -12.9972, w_gc = 1.8022
rad/s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_stability_margins', num=[2], den=[1, 3, 2, 0])
# -> gm = 3, gm_db = 9.54243, w_pc = 1.41421 rad/s, pm_deg = 32.6131, w_gc = 0.749368 rad/s
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
stability margins: num = [10], den = [1, 3, 2, 0]	SOLVED	gm = 0.6, gm_db = -4.43697, w_pc = 1.41421 rad/s, pm_deg = -12.9972, w_gc = 1.8022 rad/s
stability margins: num = [2], den = [1, 3, 2, 0]	SOLVED	gm = 3, gm_db = 9.54243, w_pc = 1.41421 rad/s, pm_deg = 32.6131, w_gc = 0.749368 rad/s
stability margins: den = [1, 3, 2, 0]	NEEDS_INPUT	ctrl_stability_margins needs num (numerator of the open loop L(s)). Example: solve('ctrl_stability_margins', num=[10], d
stability margins: num = [10], den = [1, 3, 2, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
stability margins: num = [1], den = [1, 1]	INVALID_INPUT	Gain and phase margins: |L(jw)| never crosses 1: the phase margin is not defined.

-3 dB bandwidth of a transfer function, `ctrl_bandwidth`

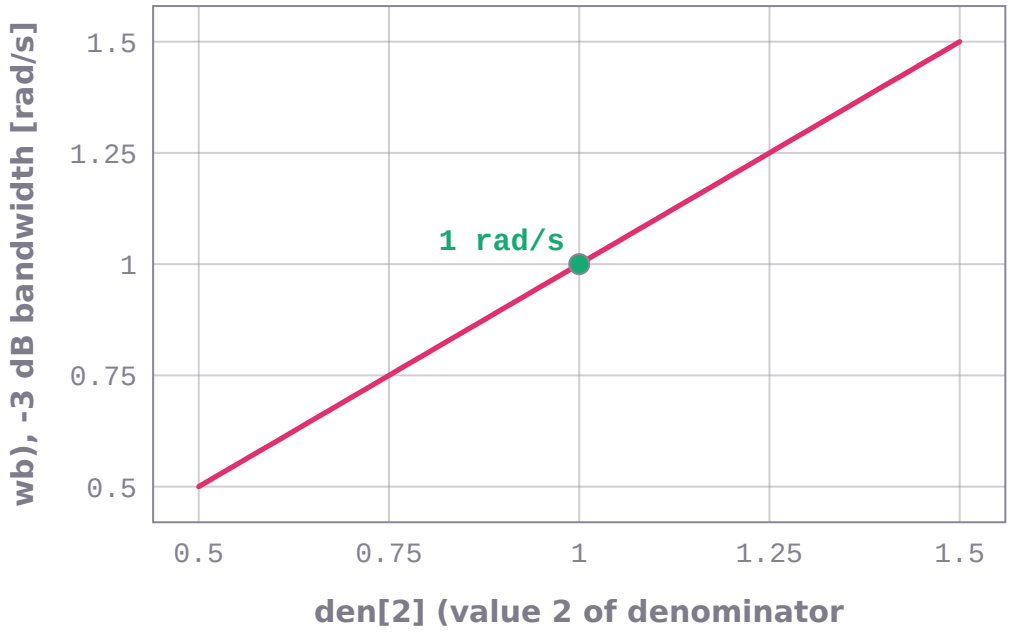
The lowest frequency at which |G(jw)| falls to |G(0)|/sqrt(2) for a stable, strictly proper G(s), and the DC gain.

$$|G(j\omega_B)| = \frac{|G(0)|}{\sqrt{2}}$$

input	meaning	condition
`num`	numerator	
`den`	denominator	

output	meaning
`wb`	-3 dB bandwidth
`dc_gain`	DC gain

- **Method:** smallest positive root of 2|N(jw)|^$2 - G$(0)^2 |D(jw)|^2
- **Assumes:** stable; strictly proper; nonzero DC gain
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



wb (-3 dB bandwidth) as value 2 of den (denominator) moves from 0.5 to 1.5, the other values as given ([1, 1]); 41 runs of control-systems-ai999x. Other inputs: num = [1]. Marked: the worked example, den[2] = 1, wb = 1 rad/s.

```
control_systems_ai999x.solve('ctrl_bandwidth', num=[1], den=[1, 1])
# -> wb = 1 rad/s, dc_gain = 1   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_bandwidth', num=[100], den=[1, 10, 100])
# -> wb = 12.7202 rad/s, dc_gain = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
bandwidth: num = [1], den = [1, 1]	SOLVED	wb = 1 rad/s, dc_gain = 1
bandwidth: num = [100], den = [1, 10, 100]	SOLVED	wb = 12.7202 rad/s, dc_gain = 1
bandwidth: den = [1, 1]	NEEDS_INPUT	ctrl_bandwidth needs num (numerator). Example: solve('ctrl_bandwidth', num=[1], den=[1, 1]).
bandwidth: num = [1], den = [1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
bandwidth: num = [1], den = [1, -1]	INVALID_INPUT	-3 dB bandwidth of a transfer function: the system must be stable.

Range of gain for closed-loop stability (Routh, j-omega crossing), `ctrl_stable_gain_range`

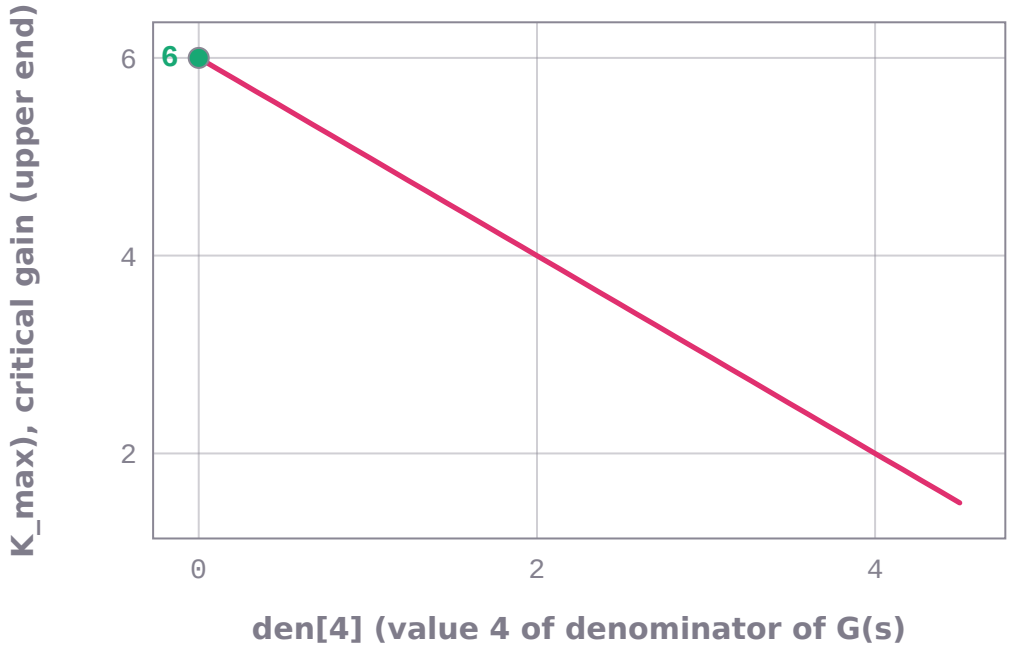
The interval K_min < K < K_max of positive gains for which $1 + K$ G(s) = 0 has every root in the left half plane, and the frequency at which the root locus crosses the imaginary axis at K_max.

$$D(j\omega) + KN(j\omega) = 0$$

input	meaning	condition
`num`	numerator of G(s)	
`den`	denominator of G(s)	

output	meaning
`K_min`	lower end of the stable range (0 when stable from small gains)
`K_max`	critical gain (upper end)
`w_cross`	imaginary-axis crossing frequency at K_max (0 for a crossing at s = 0)

- **Method:** critical gains from D(jw) + K N(jw) = 0 and D(0) + K N(0) = 0; each interval tested by an exact Routh array
- **Assumes:** one bounded interval of stable positive gains
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



K_max (critical gain (upper end)) as value 4 of den (denominator of G(s)) moves from 0 to 4.5, the other values as given ([1, 3, 2, 0]); 41 runs of control-systems-ai999x. Other inputs: num = [1]. Marked: the worked example, den[4] = 0, K_max = 6.

```
control_systems_ai999x.solve('ctrl_stable_gain_range', num=[1], den=[1, 3, 2, 0])
# -> K_min = 0, K_max = 6, w_cross = 1.41421 rad/s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_stable_gain_range', num=[1], den=[1, 6, 11, 6])
# -> K_min = 0, K_max = 60, w_cross = 3.31662 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
stable gain range: num = [1], den = [1, 3, 2, 0]	SOLVED	K_min = 0, K_max = 6, w_cross = 1.41421 rad/s
stable gain range: num = [1], den = [1, 6, 11, 6]	SOLVED	K_min = 0, K_max = 60, w_cross = 3.31662 rad/s

question	outcome	result
stable gain range: den = [1, 3, 2, 0]	NEEDS_INPUT	ctrl_stable_gain_range needs num (numerator of G(s)). Example: solve('ctrl_stable_gain_range', num=[1], den=[1, 3, 2, 0]
stable gain range: num = [1], den = [1, 3, 2, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
stable gain range: num = [1], den = [1, 1]	INVALID_INPUT	Range of gain for closed-loop stability (Routh, j-omega crossing): the closed loop is stable for every large gain (no up

Root-locus breakaway and break-in points, `ctrl_breakaway_points`

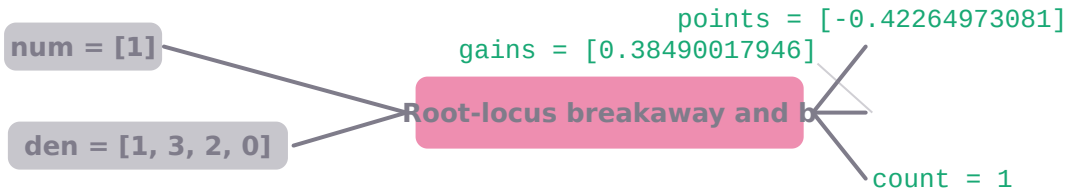
Real points where root-locus branches meet and leave (or enter) the real axis: real roots of N D' - N' $D = 0$ with a positive gain $K = -D/N$ there.

$$\frac{dK}{ds} = 0 \iff ND' - N'D = 0$$

input	meaning	condition
`num`	numerator of G(s)	
`den`	denominator of G(s)	

output	meaning
`points`	breakaway/break-in points, ascending
`gains`	gain K at each point
`count`	number of points

- **Method:** $dK/ds = 0$ with $K = -D$(s)/N(s)
- **Assumes:** proper G(s)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Root-locus breakaway and break-in points as inputs and results: stated num = [1], den = [1, 3, 2, 0]; returned by control-systems-ai999x: points = [-0.42264973081], gains = [0.38490017946], count = 1.

```
control_systems_ai999x.solve('ctrl_breakaway_points', num=[1], den=[1, 3, 2, 0])
# -> points = [-0.42264973081], gains = [0.38490017946], count = 1   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_breakaway_points', num=[1, 3], den=[1, 3, 2])
# -> points = [-4.41421356237, -1.58578643763], gains = [5.82842712475, 0.171572875254],
count = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
breakaway points: num = [1], den = [1, 3, 2, 0]	SOLVED	points = [-0.42264973081], gains = [0.38490017946], count = 1
breakaway points: num = [1, 3], den = [1, 3, 2]	SOLVED	points = [-4.41421356237, -1.58578643763], gains = [5.82842712475, 0.171572875254], count = 2
breakaway points: den = [1, 3, 2, 0]	NEEDS_INPUT	ctrl_breakaway_points needs num (numerator of G(s)). Example: solve('ctrl_breakaway_points', num=[1], den=[1, 3, 2, 0]).
breakaway points: num = [1], den = [1, 3, 2, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

Root-locus angle of departure from a complex pole, `ctrl_departure_angle`

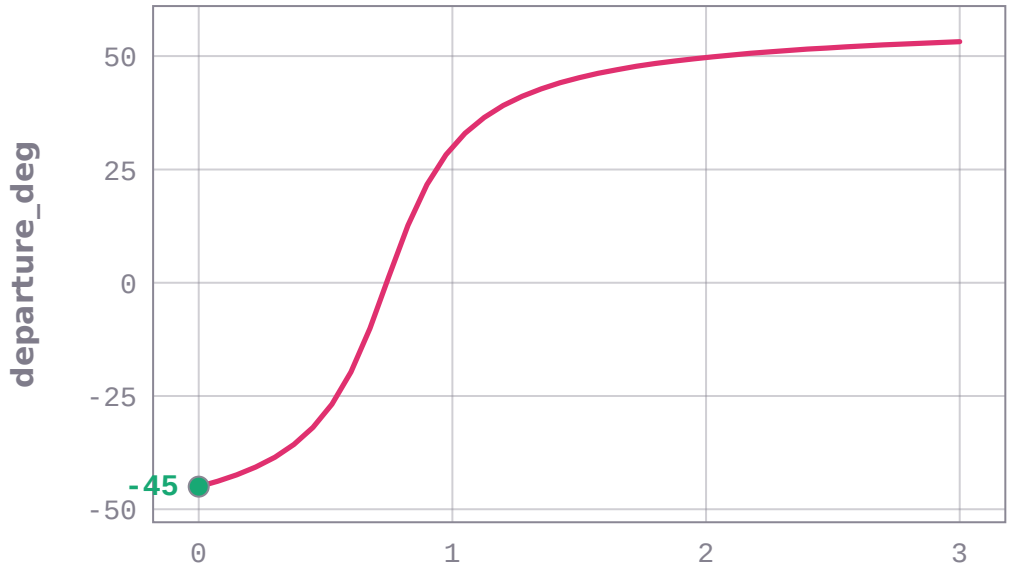
The angle at which the root locus leaves the complex pole p (upper half plane): 180 + sum of angles from the zeros - sum of angles from the other poles.

$$\theta_d = 180^\circ + \sum \angle(p - z_i) - \sum_{j \neq} \angle(p - p_j)$$

input	meaning	condition
`num`	numerator of G(s)	
`den`	denominator of G(s) (exactly one complex pole pair)	

output	meaning
`pole`	the complex pole (positive imaginary part)
`departure_deg`	angle of departure in degrees, (-180, 180]

- **Method:** angle criterion at the pole
- **Assumes:** exactly one complex pole pair; same-sign leading coefficients
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



den[4], value 4 of denominator of G(s) (exactly one complex p

departure_deg (angle of departure in degrees, (-180, 180]) as value 4 of den (denominator of G(s) (exactly one complex pole pair)) moves from 0 to 3, the other values as given ([1, 2, 2, 0]); 41 runs of control-systems-ai999x. Other inputs: num = [1]. Marked: the worked example, den[4] = 0, departure_deg = -45.

```
control_systems_ai999x.solve('ctrl_departure_angle', num=[1], den=[1, 2, 2, 0])
# -> pole = -1 + 1i, departure_deg = -45   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_departure_angle', num=[1, 2], den=[1, 2, 5])
# -> pole = -1 + 2i, departure_deg = 153.435   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
angle of departure: num = [1], den = [1, 2, 2, 0]	SOLVED	pole = -1 + 1i, departure_deg = -45
angle of departure: num = [1, 2], den = [1, 2, 5]	SOLVED	pole = -1 + 2i, departure_deg = 153.435
angle of departure: den = [1, 2, 2, 0]	NEEDS_INPUT	ctrl_departure_angle needs num (numerator of G(s)). Example: solve('ctrl_departure_angle', num=[1], den=[1, 2, 2, 0]).
angle of departure: num = [1], den = [1, 2, 2, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
angle of departure: num = [1], den = [1, 3, 2]	INVALID_INPUT	Root-locus angle of departure from a complex pole: this capability needs exactly one complex-conjugate pair of poles.

Root-locus angle of arrival at a complex zero, `ctrl_arrival_angle`

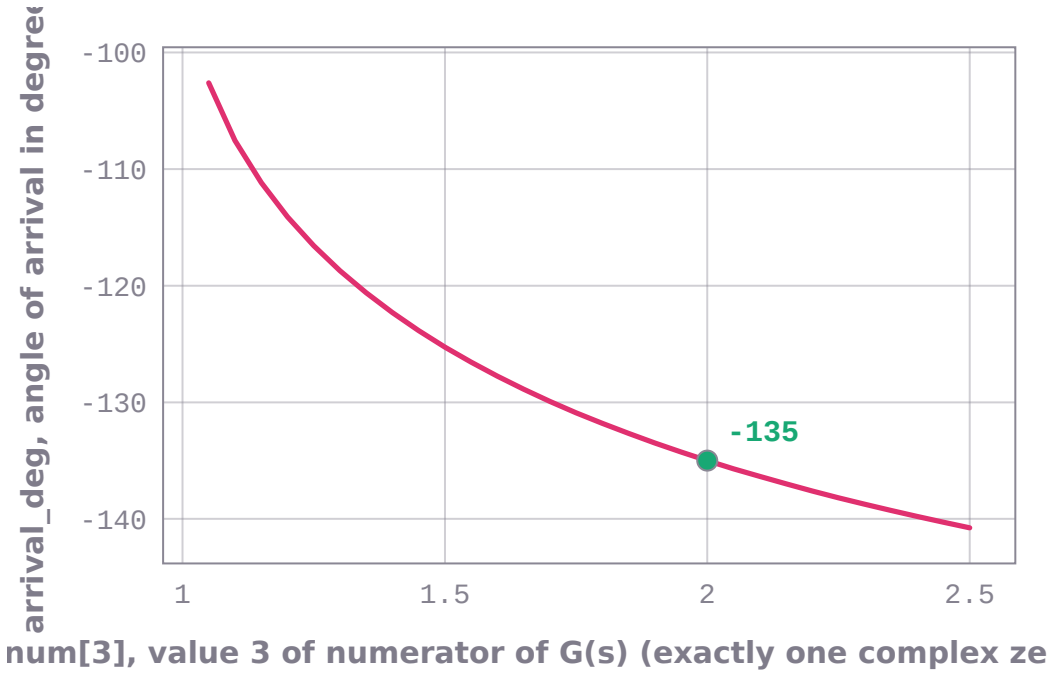
The angle at which the root locus arrives at the complex zero z (upper half plane): 180 + sum of angles from the poles - sum of angles from the other zeros.

$$\theta_a = 180^\circ + \sum \angle(z - p_j) - \sum_{i \neq} \angle(z - z_i)$$

input	meaning	condition
`num`	numerator of G(s) (exactly one complex zero pair)	
`den`	denominator of G(s)	

output	meaning
`zero`	the complex zero (positive imaginary part)
`arrival_deg`	angle of arrival in degrees, (-180, 180]

- **Method:** angle criterion at the zero
- **Assumes:** exactly one complex zero pair; same-sign leading coefficients
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



arrival_deg (angle of arrival in degrees, (-180, 180]) as value 3 of num (numerator of G(s) (exactly one complex zero pair)) moves from 0.5 to 2.5, the other values as given ([1, 2, 2]); 30 runs of control-systems-ai999x. Other inputs: den = [1, 0]. Marked: the worked example, num[3] = 2, arrival_deg = -135.

```
control_systems_ai999x.solve('ctrl_arrival_angle', num=[1, 2, 2], den=[1, 0])
# -> zero = -1 + 1i, arrival_deg = -135   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_arrival_angle', num=[1, 2, 5], den=[1, 3, 0])
# -> zero = -1 + 2i, arrival_deg = -108.435   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
angle of arrival: num = [1, 2, 2], den = [1, 0]	SOLVED	zero = -1 + 1i, arrival_deg = -135
angle of arrival: num = [1, 2, 5], den = [1, 3, 0]	SOLVED	zero = -1 + 2i, arrival_deg = -108.435
angle of arrival: den = [1, 0]	NEEDS_INPUT	ctrl_arrival_angle needs num (numerator of G(s) (exactly one complex zero pair)). Example: solve('ctrl_arrival_angle', n

question	outcome	result
angle of arrival: num = [1, 2, 2], den = [1, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
angle of arrival: num = [1, 3, 2], den = [1, 2, 0]	INVALID_INPUT	Root-locus angle of arrival at a complex zero: this capability needs exactly one complex-conjugate pair of zeros.

Closed-loop poles under state feedback u = -K x, `ctrl_state_feedback_poles`

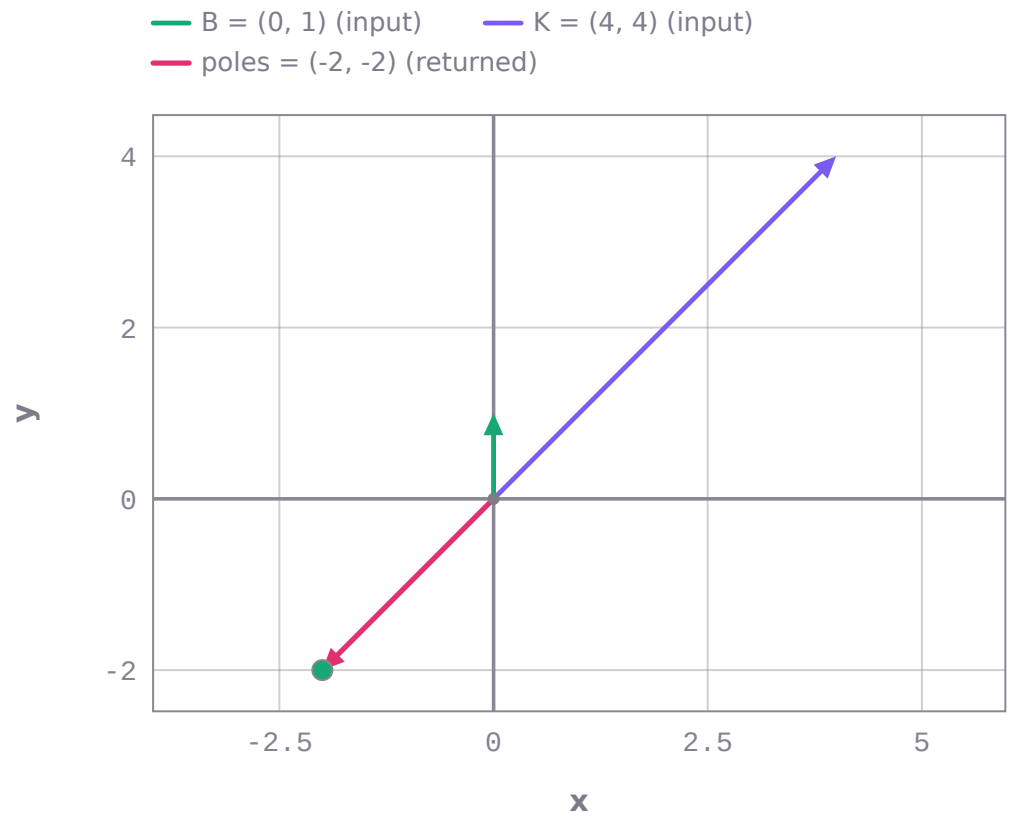
The characteristic polynomial det(sI - (A - B K)) (exact), the closed-loop poles and stability for a state-feedback gain K.

$$\det(sI - A + BK) = 0$$

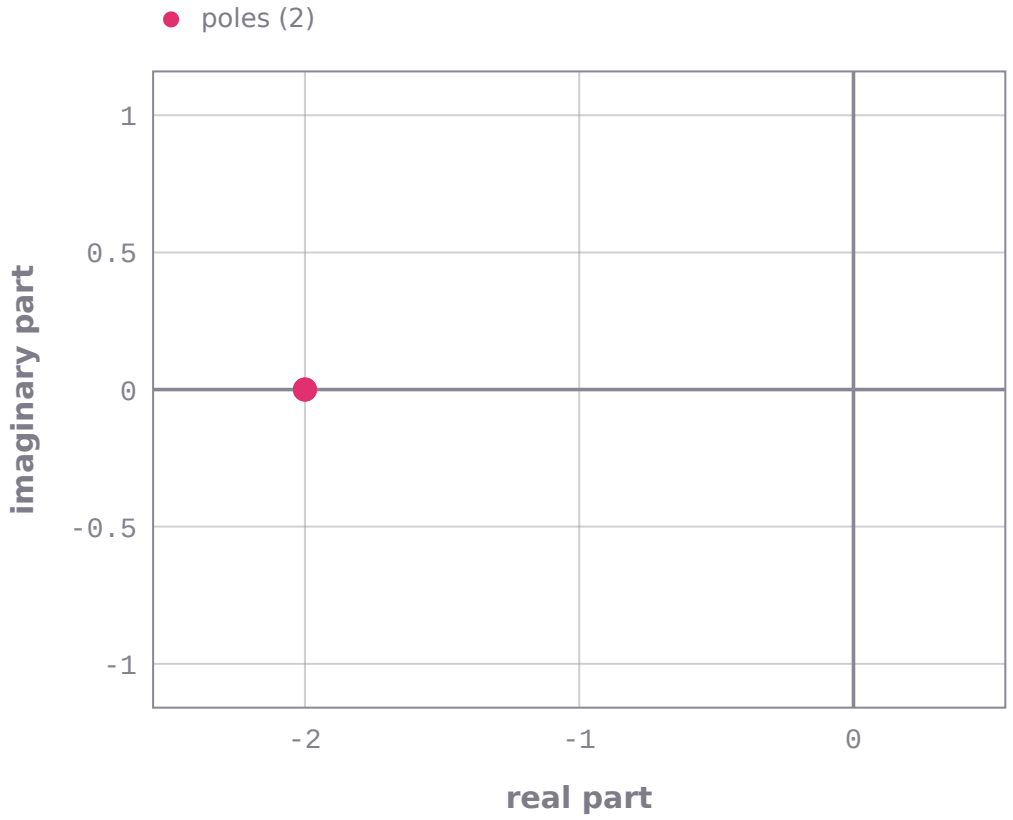
input	meaning	condition
`A`	state matrix (n x n)	
`B`	input vector	
`K`	state-feedback gain (row, n entries)	

output	meaning
`char_poly`	closed-loop characteristic polynomial
`poles`	closed-loop poles
`stable`	true when every pole is in the left half plane

- **Method:** Faddeev-LeVerrier on A - B K
- **Assumes:** single input
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input B = (0, 1), K = (4, 4); returned by control-systems-ai999x: poles = (-2, -2). Other inputs: A = [[0, 1], [0, 0]]. Marked: the tip of each returned vector.



closed-loop poles (poles) returned by control-systems-ai999x for the worked example (A = [[0, 1], [0, 0]], B = [0, 1], K = [4, 4]), drawn in the complex plane: -2 + 0i, -2 + 0i.

```
control_systems_ai999x.solve('ctrl_state_feedback_poles', A=[[0, 1], [0, 0]], B=[0, 1], K=
[4, 4])
# -> char_poly = [1.0, 4.0, 4.0], poles = [-2.0, -2.0], stable = True   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_state_feedback_poles', A=[[0, 1], [-2, -3]], B=[0, 1],
K=[3, 1])
# -> char_poly = [1.0, 4.0, 5.0], poles = [(-2-1j), (-2+1j)], stable = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
state feedback poles: A = [[0, 1], [0, 0]], B = [0, 1], K = [4, 4]	SOLVED	char_poly = [1.0, 4.0, 4.0], poles = [-2.0, -2.0], stable = True
state feedback poles: A = [[0, 1], [-2, -3]], B = [0, 1], K = [3, 1]	SOLVED	char_poly = [1.0, 4.0, 5.0], poles = [(-2-1j), (-2+1j)], stable = True
state feedback poles: B = [0, 1], K = [4, 4]	NEEDS_INPUT	ctrl_state_feedback_poles needs A (state matrix (n x n)). Example: solve('ctrl_state_feedback_poles', A=[[0, 1], [0, 0]]
state feedback poles: A = [[0, 1], [0, 0]], B = [0, 1], K = [4, 4], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

Pole placement by Ackermann's formula, `ctrl_ackermann`

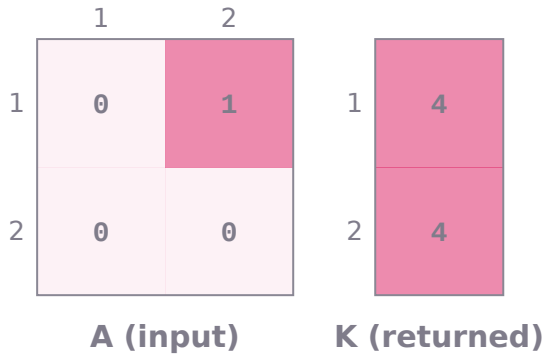
The state-feedback gain K that gives A - B K the desired characteristic polynomial alpha(s) (single input), computed exactly.

$$K = [0 \quad \cdots \quad 0 \quad 1]\mathcal{C}^{-1}\alpha(A)$$

input	meaning	condition
`A`	state matrix (n x n)	
`B`	input vector	
`alpha`	desired characteristic polynomial (n+1 coefficients, highest power first)	

output	meaning
`K`	state-feedback gain (row)

- **Method:** K = [0 ... 0 1] Wc^{-1} alpha(A) in exact fractions
- **Assumes:** (A, B) controllable
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); K (returned). Other inputs: B = [0, 1], alpha = [1, 4, 4].

```
control_systems_ai999x.solve('ctrl_ackermann', A=[[0, 1], [0, 0]], B=[0, 1], alpha=[1, 4, 4])
# -> K = [4.0, 4.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_ackermann', A=[[0, 1], [-2, -3]], B=[0, 1], alpha=[1, 6, 25])
# -> K = [23.0, 3.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ackermann pole placement: A = [[0, 1], [0, 0]], B = [0, 1], alpha = [1, 4, 4]	SOLVED	K = [4.0, 4.0]
Ackermann pole placement: A = [[0, 1], [-2, -3]], B = [0, 1], alpha = [1, 6, 25]	SOLVED	K = [23.0, 3.0]
Ackermann pole placement: B = [0, 1], alpha = [1, 4, 4]	NEEDS_INPUT	ctrl_ackermann needs A (state matrix (n x n)). Example: solve('ctrl_ackermann', A=[[0, 1], [0, 0]], B=[0, 1], alpha=[1,
Ackermann pole placement: A = [[0, 1], [0, 0]], B = [0, 1], alpha = [1, 4, 4], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Ackermann pole placement: A = [[-1, 0], [0, -2]], B = [1, 0], alpha = [1, 4, 4]	INVALID_INPUT	Pole placement by Ackermann's formula: (A, B) is not controllable: the poles cannot all be placed.

Observer (estimator) gain by pole placement, `ctrl_observer_gain`

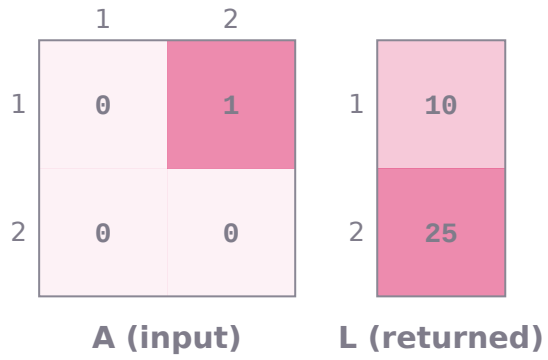
The observer gain L that gives A - L C the desired characteristic polynomial (single output), by duality with Ackermann's formula.

$$L = \alpha(A)\,\mathcal{O}^{-1}\begin{bmatrix}0\\ \vdots \\ 1\end{bmatrix}$$

input	meaning	condition
`A`	state matrix (n x n)	
`C`	output vector	
`alpha`	desired observer characteristic polynomial (n+1 coefficients)	

output	meaning
`L`	observer gain (column)

- **Method:** L^T = Ackermann(A^T, C^T, alpha)
- **Assumes:** (A, C) observable
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); L (returned). Other inputs: C = [1, 0], alpha = [1, 10, 25].

```
control_systems_ai999x.solve('ctrl_observer_gain', A=[[0, 1], [0, 0]], C=[1, 0], alpha=[1, 10, 25])
# -> L = [10.0, 25.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_observer_gain', A=[[0, 1], [-2, -3]], C=[1, 0], alpha=[1, 12, 36])
# -> L = [9.0, 7.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
observer gain: A = [[0, 1], [0, 0]], C = [1, 0], alpha = [1, 10, 25]	SOLVED	L = [10.0, 25.0]
observer gain: A = [[0, 1], [-2, -3]], C = [1, 0], alpha = [1, 12, 36]	SOLVED	L = [9.0, 7.0]
observer gain: C = [1, 0], alpha = [1, 10, 25]	NEEDS_INPUT	ctrl_observer_gain needs A (state matrix (n x n)). Example: solve('ctrl_observer_gain', A=[[0, 1], [0, 0]], C=[1, 0], al

question	outcome	result
observer gain: A = [[0, 1], [0, 0]], C = [1, 0], alpha = [1, 10, 25], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
observer gain: A = [[-1, 0], [0, -2]], C = [1, 0], alpha = [1, 4, 4]	INVALID_INPUT	Observer (estimator) gain by pole placement: (A, B) is not controllable: the poles cannot all be placed.

Deadbeat state feedback (discrete time), `ctrl_deadbeat_gain`

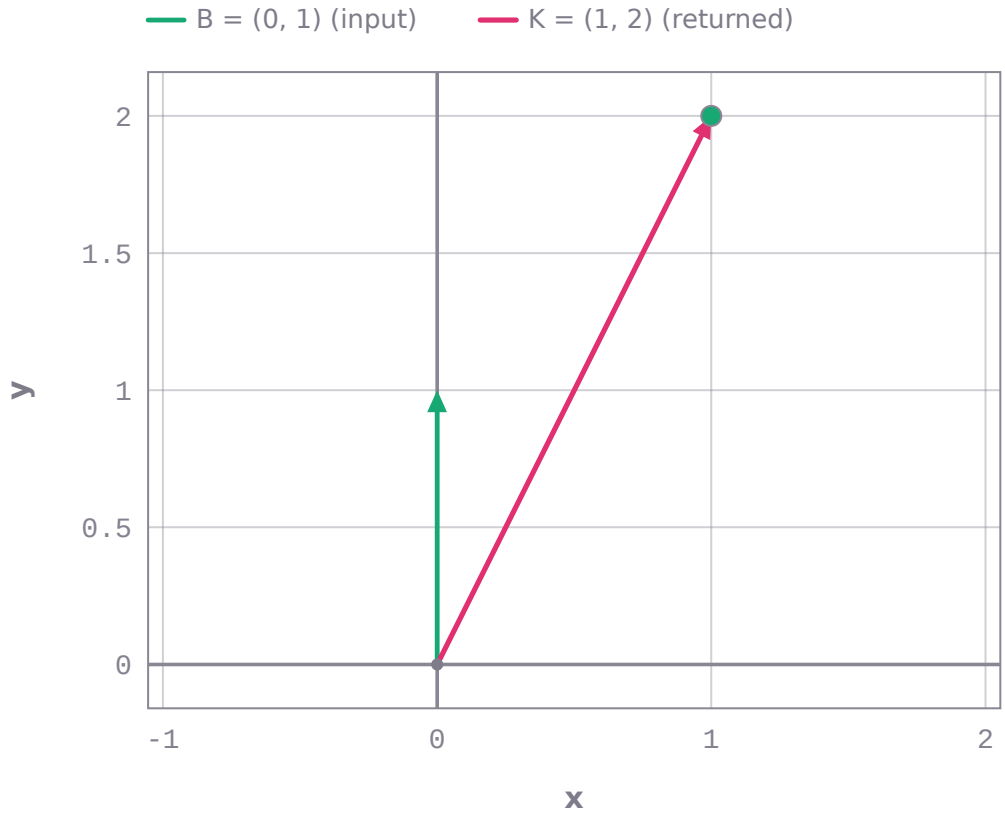
The gain K that places every eigenvalue of A - B K at $z = 0$, so the discrete closed loop reaches the origin in at most n steps.

$$K = [0 \cdots 1]\, \mathcal{C}^{-1} A^n$$

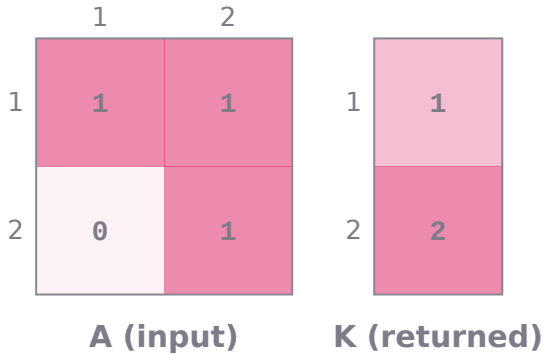
input	meaning	condition
A	discrete state matrix (n x n)	
B	input vector	

output	meaning
K	deadbeat state-feedback gain

- **Method:** Ackermann's formula with alpha(z) = z^n (exact)
- **Assumes:** (A, B) controllable
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input B = (0, 1); returned by control-systems-ai999x: K = (1, 2). Other inputs: A = [[1, 1], [0, 1]]. Marked: the tip of each returned vector.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); K (returned). Other inputs: B = [0, 1].

```
control_systems_ai999x.solve('ctrl_deadbeat_gain', A=[[1, 1], [0, 1]], B=[0, 1])
# -> K = [1.0, 2.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_deadbeat_gain', A=[[0, 1], [-0.5, 1.5]], B=[0, 1])
# -> K = [-0.5, 1.5]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
deadbeat gain: A = [[1, 1], [0, 1]], B = [0, 1]	SOLVED	K = [1.0, 2.0]

question	outcome	result
deadbeat gain: A = [[0, 1], [-0.5, 1.5]], B = [0, 1]	SOLVED	K = [-0.5, 1.5]
deadbeat gain: B = [0, 1]	NEEDS_INPUT	ctrl_deadbeat_gain needs A (discrete state matrix (n x n)). Example: solve('ctrl_deadbeat_gain', A=[[1, 1], [0, 1]], B=[
deadbeat gain: A = [[1, 1], [0, 1]], B = [0, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
deadbeat gain: A = [[1, 0], [0, 2]], B = [1, 0]	INVALID_INPUT	Deadbeat state feedback (discrete time): (A, B) is not controllable: the poles cannot all be placed.

State-transition matrix e^{At}, `ctrl_state_transition`

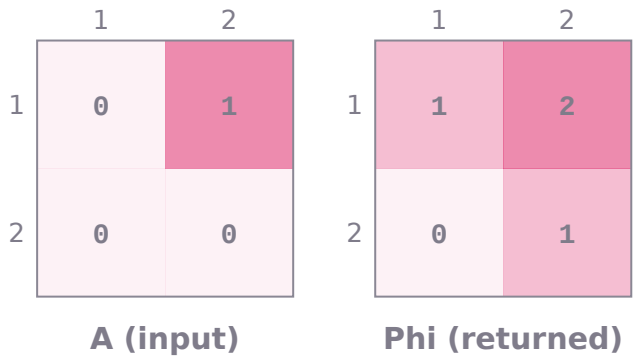
The state-transition matrix Phi(t) = e^{At} that maps x(0) to x(t) for x' = A x.

$$\Phi(t) = e^{At} = \sum_k \frac{(At)^k}{k!}$$

input	meaning	condition
`A`	state matrix (n x n)	
`t`	time in seconds	

output	meaning
`Phi`	state-transition matrix

- **Method:** scaling and squaring with a Taylor series
- **Assumes:** |A| $t \leq 40$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); Phi (returned). Other inputs: t = 2.0.

```
control_systems_ai999x.solve('ctrl_state_transition', A=[[0, 1], [0, 0]], t=2)
# -> Phi = [[1.0, 2.0], [0.0, 1.0]]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_state_transition', A=[[-1, 0], [0, -2]], t=1)
# -> Phi = [[0.367879441171, 0.0], [0.0, 0.135335283237]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
state transition matrix: A = [[0, 1], [0, 0]], t = 2	SOLVED	Phi = [[1.0, 2.0], [0.0, 1.0]]
state transition matrix: A = [[-1, 0], [0, -2]], t = 1	SOLVED	Phi = [[0.367879441171, 0.0], [0.0, 0.135335283237]]
state transition matrix: t = 2	NEEDS_INPUT	ctrl_state_transition needs A (state matrix (n x n)). Example: solve('ctrl_state_transition', A=[[0, 1], [0, 0]], t=2).
state transition matrix: A = [[0, 1], [0, 0]], t = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
state transition matrix: A = [[100, 0], [0, 1]], t = 10	INVALID_INPUT	State-transition matrix e^{At}: |A| t is too large for an accurate matrix exponential here (limit 40).

State response to an initial state and a constant input, `ctrl_ss_response`

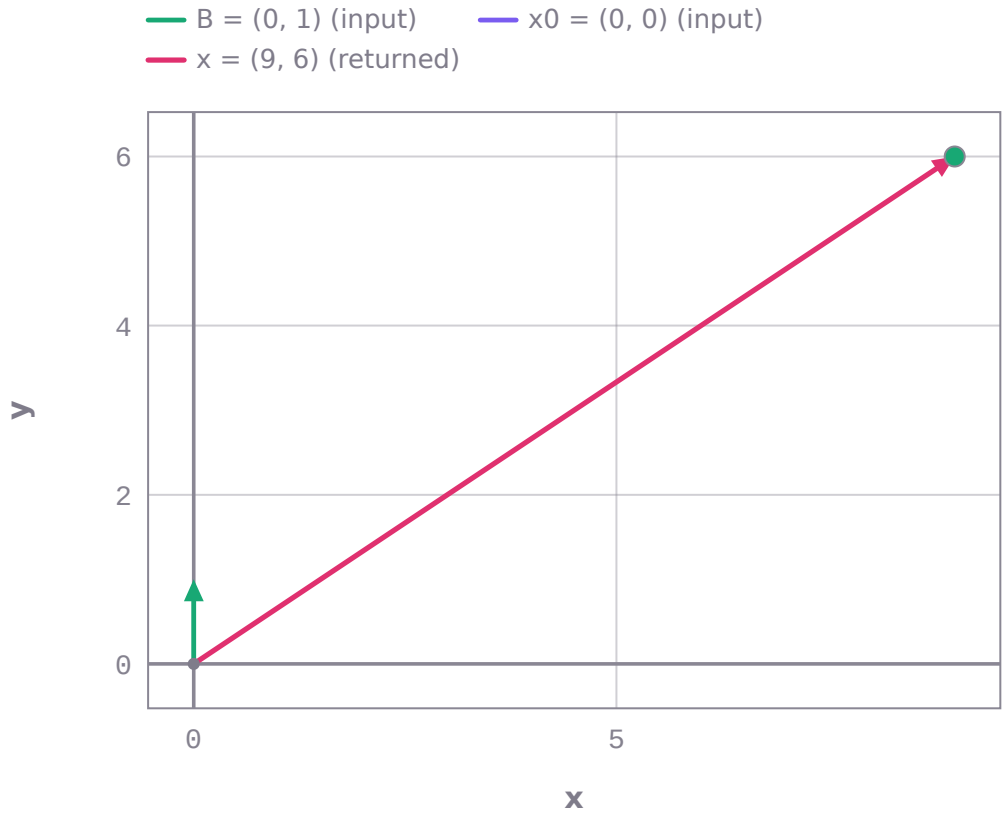
x(t) = e^{At} x(0) + (integral of e^{A tau} d tau) B u for x' = A x + B u with a constant input u.

$$x(t) = e^{At}x_0 + \int_0^t e^{A\tau}d\tau\, Bu$$

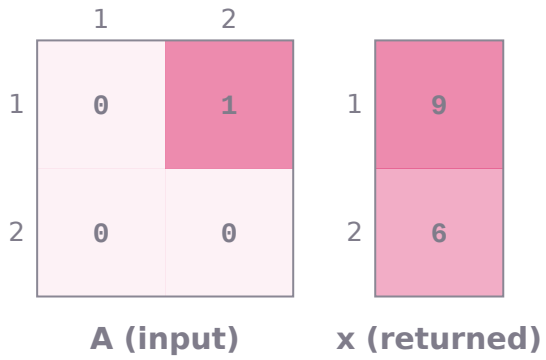
input	meaning	condition
`A`	state matrix (n x n)	
`B`	input vector	
`x0`	initial state	
`u`	constant input	
`t`	time in seconds	

output	meaning
`x`	state at time t

- **Method:** exponential of the augmented matrix [[A, B u], [0, 0]] t
- **Assumes:** |A| $t \le 40$; $t \ge 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input B = (0, 1), x0 = (0, 0); returned by control-systems-ai999x: x = (9, 6). Other inputs: A = [[0, 1], [0, 0]], u = 2.0, t = 3.0. Marked: the tip of each returned vector.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); x (returned). Other inputs: B = [0, 1], x0 = [0, 0], u = 2.0, t = 3.0.

```
control_systems_ai999x.solve('ctrl_ss_response', A=[[0, 1], [0, 0]], B=[0, 1], x0=[0, 0],
u=2, t=3)
# -> x = [9.0, 6.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_ss_response', A=[[-1]], B=[1], x0=[2], u=1, t=1)
# -> x = [1.36787944117]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
state response: A = [[0, 1], [0, 0]], B = [0, 1], x0 = [0, 0], u = 2, t = 3	SOLVED	x = [9.0, 6.0]
state response: A = [[-1]], B = [1], x0 = [2], u = 1, t = 1	SOLVED	x = [1.36787944117]
state response: B = [0, 1], x0 = [0, 0], u = 2, t = 3	NEEDS_INPUT	ctrl_ss_response needs A (state matrix (n x n)). Example: solve('ctrl_ss_response', A=[[0, 1], [0, 0]], B=[0, 1], x0=[0,
state response: A = [[0, 1], [0, 0]], B = [0, 1], x0 = [0, 0], u = 2, t = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
state response: A = [[0, 1], [0, 0]], B = [0, 1], x0 = [0, 0], u = 2, t = -3	INVALID_INPUT	State response to an initial state and a constant input: t must be zero or positive.

Zero-order-hold discretization of a transfer function, `ctrl_c2d_zoh`

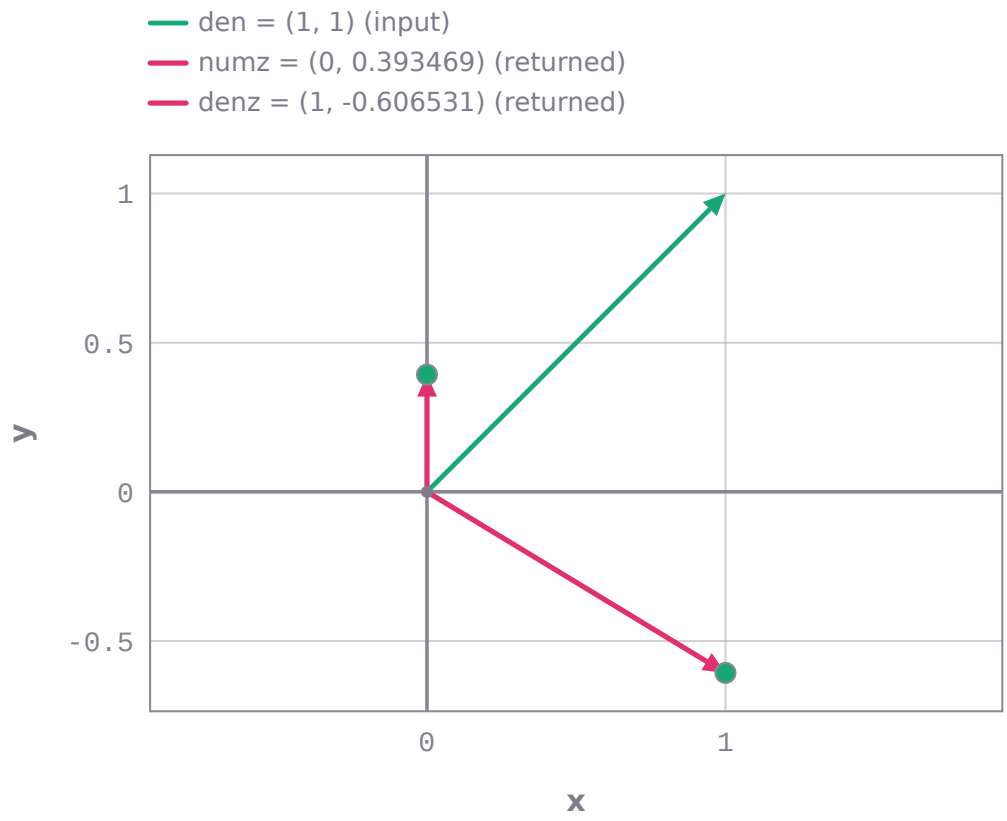
The step-invariant (ZOH) discrete equivalent G(z) = $(1 - z$^{-1}) Z{G(s)/s} of a proper G(s) sampled every T seconds.

$$A_d = e^{AT},\ B_d = \int_0^T e^{A\tau} d\tau\, B$$

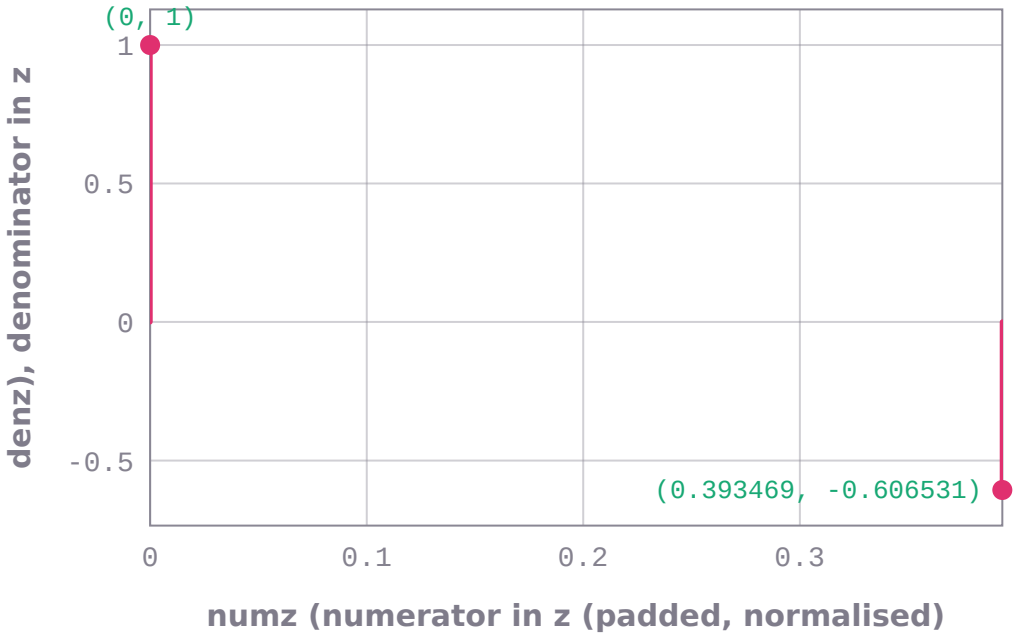
input	meaning	condition
`num`	numerator in s	
`den`	denominator in s	
`T`	sampling period in seconds	

output	meaning
`numz`	numerator in z (padded, normalised)
`denz`	denominator in z (leading coefficient 1)

- **Method:** exponential of [[A, B], [0, 0]] T in state space, then Faddeev-LeVerrier
- **Assumes:** proper G(s); $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input den = (1, 1); returned by control-systems-ai999x: numz = (0, 0.393469), denz = (1, -0.606531). Other inputs: num = [1], T = 0.5. Marked: the tip of each returned vector.



denominator in z (leading coefficient 1) (denz) at each numerator in z (padded, normalised) (numz), both returned by control-systems-ai999x for the worked example (num = [1], den = [1, 1], T = 0.5): 0 → 1; 0.393469 → -0.606531.

```
control_systems_ai999x.solve('ctrl_c2d_zoh', num=[1], den=[1, 1], T=0.5)
# -> numz = [0.0, 0.393469340287], denz = [1.0, -0.606530659713]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_c2d_zoh', num=[1], den=[1, 0, 0], T=1)
# -> numz = [0.0, 0.5, 0.5], denz = [1.0, -2.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ZOH discretization: num = [1], den = [1, 1], T = 0.5	SOLVED	numz = [0.0, 0.393469340287], denz = [1.0, -0.606530659713]
ZOH discretization: num = [1], den = [1, 0, 0], T = 1	SOLVED	numz = [0.0, 0.5, 0.5], denz = [1.0, -2.0, 1.0]
ZOH discretization: den = [1, 1], T = 0.5	NEEDS_INPUT	ctrl_c2d_zoh needs num (numerator in s). Example: solve('ctrl_c2d_zoh', num=[1], den=[1, 1], T=0.5).
ZOH discretization: num = [1], den = [1, 1], T = 0.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
ZOH discretization: num = [1], den = [1, 1], T = 0	INVALID_INPUT	Zero-order-hold discretization of a transfer function: T must be positive.

First-order-hold (triangle-hold) discretization, `ctrl_c2d_foh`

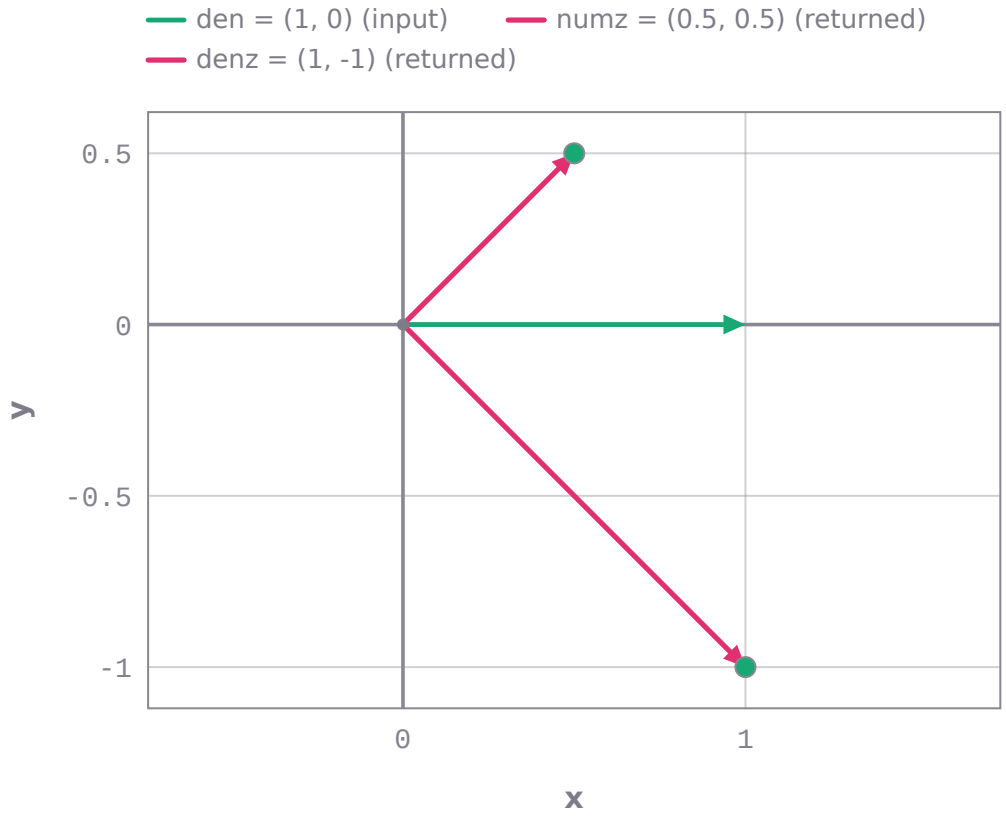
The ramp-invariant (triangle-hold, non-causal first-order hold) discrete equivalent of a proper G(s) sampled every T seconds.

$$B_d = \Gamma_1 - \Gamma_2 + \Phi\Gamma_2, \quad D_d = D + C\Gamma_2$$

input	meaning	condition
`num`	numerator in s	
`den`	denominator in s	
`T`	sampling period in seconds	

output	meaning
`numz`	numerator in z (padded, normalised)
`denz`	denominator in z (leading coefficient 1)

- **Method:** exponential of the augmented matrix [[A T, B T, 0], [0, 0, 1], [0, 0, 0]]
- **Assumes:** proper G(s); $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input den = (1, 0); returned by control-systems-ai999x: numz = (0.5, 0.5), denz = (1, -1). Other inputs: num = [1], T = 1.0. Marked: the tip of each returned vector.

```
control_systems_ai999x.solve('ctrl_c2d_foh', num=[1], den=[1, 0], T=1)
# -> numz = [0.5, 0.5], denz = [1.0, -1.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_c2d_foh', num=[1], den=[1, 1], T=1)
# -> numz = [0.367879441171, 0.264241117657], denz = [1.0, -0.367879441171]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
FOH discretization: num = [1], den = [1, 0], T = 1	SOLVED	numz = [0.5, 0.5], denz = [1.0, -1.0]
FOH discretization: num = [1], den = [1, 1], T = 1	SOLVED	numz = [0.367879441171, 0.264241117657], denz = [1.0, -0.367879441171]
FOH discretization: den = [1, 0], T = 1	NEEDS_INPUT	ctrl_c2d_foh needs num (numerator in s). Example: solve('ctrl_c2d_foh', num=[1], den=[1, 0], T=1).
FOH discretization: num = [1], den = [1, 0], T = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
FOH discretization: num = [1], den = [1, 1], T = -1	INVALID_INPUT	First-order-hold (triangle-hold) discretization: T must be positive.

Tustin discretization with frequency prewarping, `ctrl_tustin_prewarp`

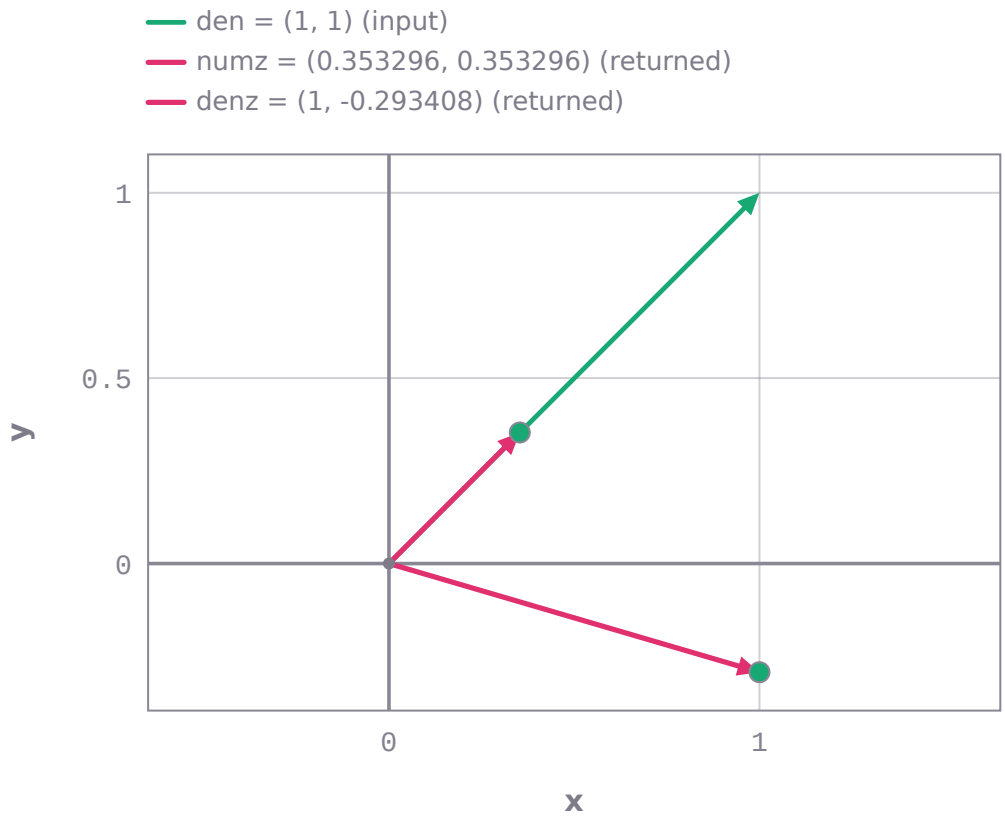
The bilinear transform s = (w/tan(w T/2)) $(z-1)/(z+1)$, which matches the continuous frequency response exactly at the prewarping frequency w.

$$s \leftarrow \frac{\omega}{\tan(\omega T/2)} \frac{z-1}{z+1}$$

input	meaning	condition
`num`	numerator in s	
`den`	denominator in s	
`T`	sampling period in seconds	
`w`	prewarping frequency in rad/s	

output	meaning
`numz`	numerator in z (padded, normalised)
`denz`	denominator in z (leading coefficient 1)

- **Method:** polynomial substitution with the prewarped constant
- **Assumes:** proper G(s); $0 < w\,T < pi$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input den = (1, 1); returned by control-systems-ai999x: numz = (0.353296, 0.353296), denz = (1, -0.293408). Other inputs: num = [1], T = 1.0, w = 1.0. Marked: the tip of each returned vector.

```
control_systems_ai999x.solve('ctrl_tustin_prewarp', num=[1], den=[1, 1], T=1, w=1)
# -> numz = [0.353296003487, 0.353296003487], denz = [1.0, -0.293407993026]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_tustin_prewarp', num=[1], den=[1, 0], T=0.5, w=2)
# -> numz = [0.273151244922, 0.273151244922], denz = [1.0, -1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
prewarped Tustin discretization: num = [1], den = [1, 1], T = 1, w = 1	SOLVED	numz = [0.353296003487, 0.353296003487], denz = [1.0, -0.293407993026]
prewarped Tustin discretization: num = [1], den = [1, 0], T = 0.5, w = 2	SOLVED	numz = [0.273151244922, 0.273151244922], denz = [1.0, -1.0]
prewarped Tustin discretization: den = [1, 1], T = 1, w = 1	NEEDS_INPUT	ctrl_tustin_prewarp needs num (numerator in s). Example: solve('ctrl_tustin_prewarp', num=[1], den=[1, 1], T=1, w=1).
prewarped Tustin discretization: num = [1], den = [1, 1], T = 1, w = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
prewarped Tustin discretization: num = [1], den = [1, 1], T = 1, w = 4	INVALID_INPUT	Tustin discretization with frequency prewarping: the prewarping frequency must be below the Nyquist frequency (w T < pi)

Jury (Schur-Cohn) stability test for a discrete polynomial, `ctrl_jury_test`

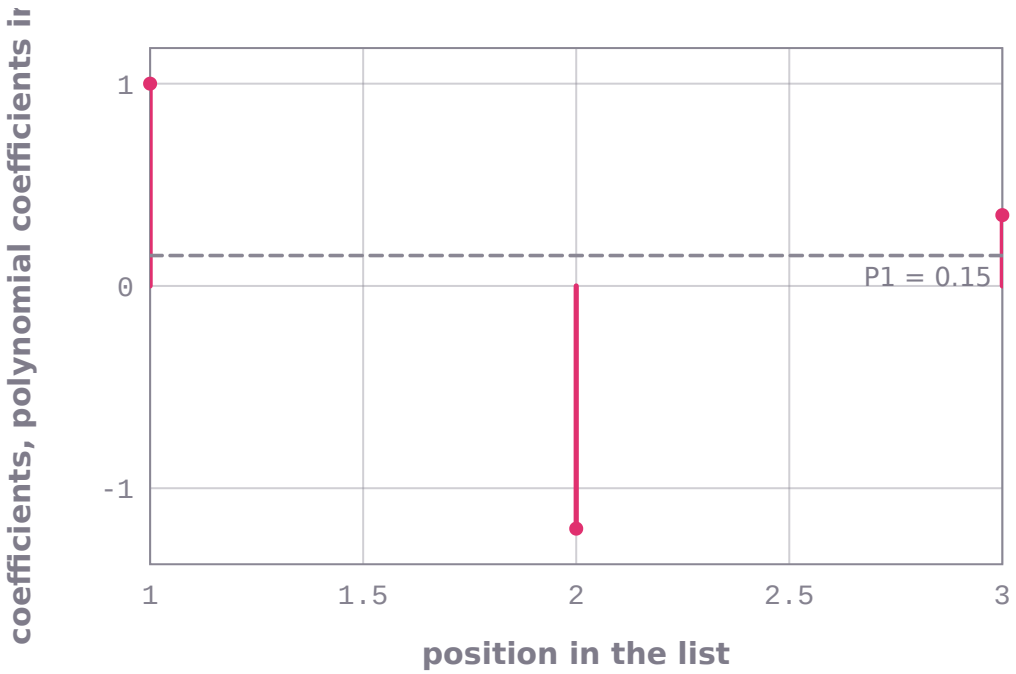
Whether every root of a polynomial in z lies inside the unit circle (Jury/Schur-Cohn table in exact arithmetic), with the necessary-condition values P(1) and $(-1)^n$ P(-1).

$$|a_n| < a_0,\ P(1) > 0,\ (-1)^n P(-1) > 0$$

input	meaning	condition
`coefficients`	polynomial coefficients in z, highest power first	

output	meaning
`stable`	true when every root is inside the unit circle
`P1`	P(1) (must be > 0)
`Pm1`	$(-1)^n$ P(-1) (must be > 0)

- **Method:** Schur-Cohn reduction q <- $q - (a_n/a_0)$ q_reversed in exact fractions
- **Assumes:** the table is not singular
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The input coefficients (polynomial coefficients in z, highest power first) as given in the worked example, one value per position (3 values); marked from what control-systems-ai999x returned: P1 = 0.15. Also returned: Pm1 = 2.55.

```
control_systems_ai999x.solve('ctrl_jury_test', coefficients=[1, -1.2, 0.35])
# -> stable = True, P1 = 0.15, Pm1 = 2.55   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_jury_test', coefficients=[1, -2, 1.5])
# -> stable = False, P1 = 0.5, Pm1 = 4.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Jury test: coefficients = [1, -1.2, 0.35]	SOLVED	stable = True, P1 = 0.15, Pm1 = 2.55
Jury test: coefficients = [1, -2, 1.5]	SOLVED	stable = False, P1 = 0.5, Pm1 = 4.5
Jury test:	NEEDS_INPUT	ctrl_jury_test needs coefficients (polynomial coefficients in z, highest power first). Example: solve('ctrl_jury_test',
Jury test: coefficients = [1, -1.2, 0.35], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Jury test: coefficients = [1, 0, 1]	INVALID_INPUT	Jury (Schur-Cohn) stability test for a discrete polynomial: the Jury table is singular (|a_n| = |a_0| at a step): a root

Step response of a transfer function at a given time, `ctrl_step_response`

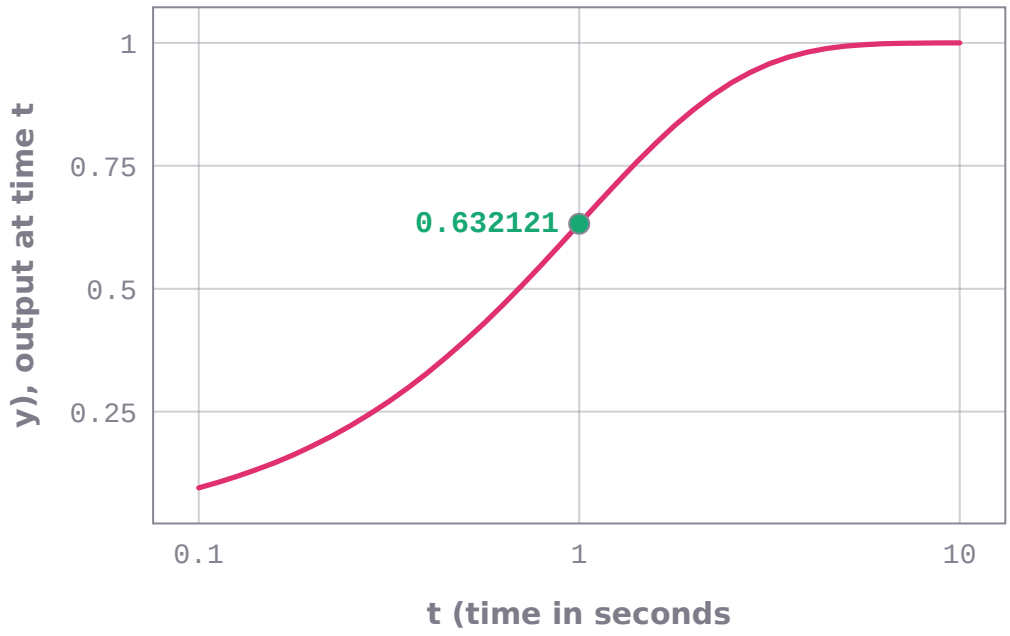
y(t) of any proper G(s) after a step of size A, computed exactly through the state-space matrix exponential.

$$y(t) = C \int_0^t e^{A\tau} d\tau \, B \, A_u + D A_u$$

input	meaning	condition
`num`	numerator	
`den`	denominator	
`t`	time in seconds	
`A`	step size (default 1)	

output	meaning
`y`	output at time t

- **Method:** controllable canonical form; exponential of [[A, B], [0, 0]] t
- **Assumes:** proper G(s); $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y (output at time t) against t (time in seconds) from 0.1 to 10 on a logarithmic axis. Other inputs as in the worked example: num = [1], den = [1, 1]. 41 points computed by control-systems-ai999x; marked: the worked example, t = 1, y = 0.632121.

```
control_systems_ai999x.solve('ctrl_step_response', num=[1], den=[1, 1], t=1)
# -> y = 0.632121   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_step_response', num=[4], den=[1, 2, 4], t=1.0)
# -> y = 0.849426   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
step response at time: num = [1], den = [1, 1], t = 1	SOLVED	y = 0.632121
step response at time: num = [4], den = [1, 2, 4], t = 1	SOLVED	y = 0.849426

question	outcome	result
step response at time: den = [1, 1], t = 1	NEEDS_INPUT	ctrl_step_response needs num (numerator). Example: solve('ctrl_step_response', num=[1], den=[1, 1], t=1).
step response at time: num = [1], den = [1, 1], t = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
step response at time: num = [1], den = [1, 1], t = -1	INVALID_INPUT	Step response of a transfer function at a given time: t must be zero or positive.

Exact settling time of a second-order step response, `ctrl_true_settling_time`

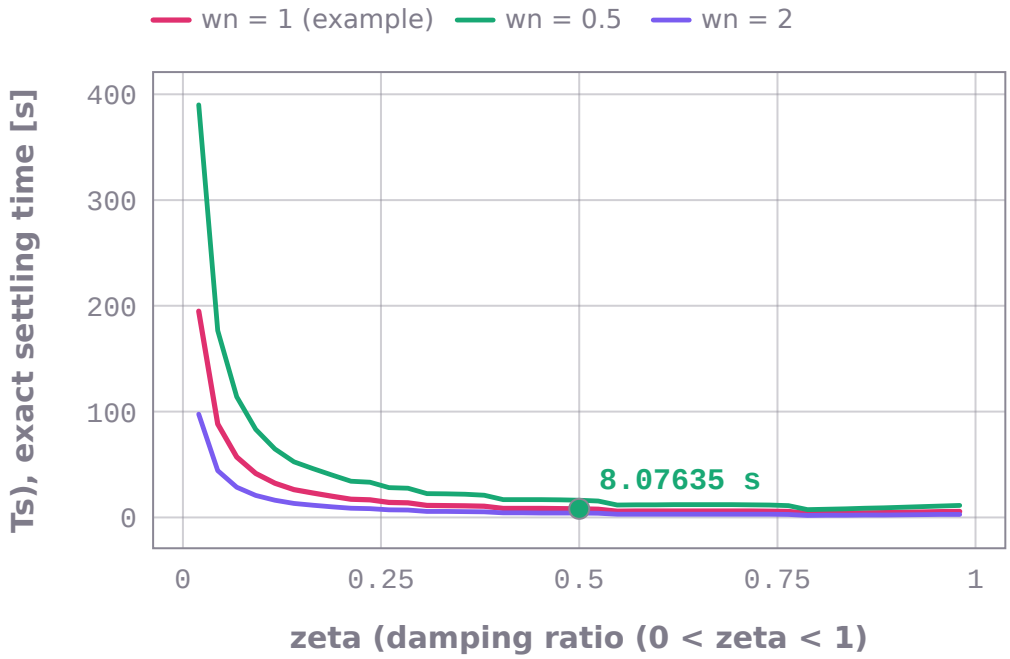
The last time at which the unit step response of wn^2/(s^2 + 2 zeta wn s + wn^2) leaves the band $|y - 1|$ = band (not the envelope estimate), and which extremum precedes it.

$$T_s = \max\{t : |y(t) - 1| = \text{band}\}$$

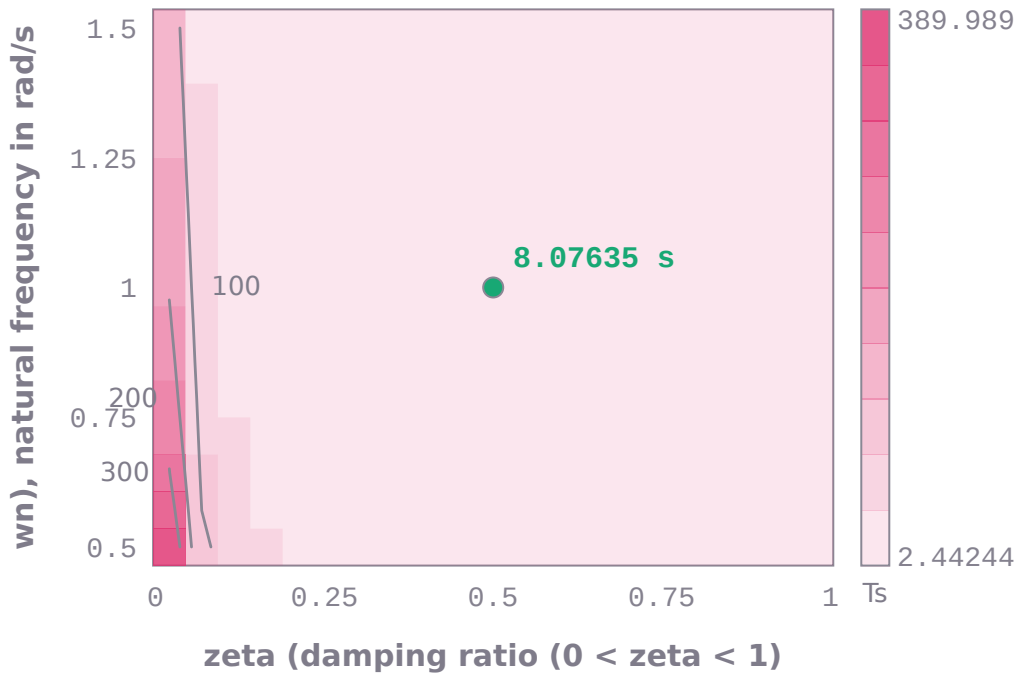
input	meaning	condition
`zeta`	damping ratio $(0 < \zeta < 1)$	
`wn`	natural frequency in rad/s	
`band`	settling band (default 2%)	

output	meaning
`Ts`	exact settling time
`last_extremum`	index k of the last extremum outside the band (0 = before the first peak)

- **Method:** extrema at k pi/wd; the last band crossing lies between the last extremum outside the band and the next zero of $y - 1$ (bisection)
- **Assumes:** $0 < \zeta < 1$; 0 < band < 50 %
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ts (exact settling time) against zeta (damping ratio (0 < zeta < 1)) from 0.02 to 0.98. One curve for each value of wn (natural frequency in rad/s): 1, 0.5, 2. 41 points per curve computed by control-systems-ai999x; marked: the worked example, zeta = 0.5, Ts = 8.07635 s.



Ts (exact settling time) over zeta (damping ratio (0 < zeta < 1)) from 0.02 to 0.98 and wn (natural frequency in rad/s) from 0.5 to 1.5: each of the 21 × 15 cells is one run of control-systems-ai999x, shaded by its value (light 2.44244 s to dark 389.989 s); iso-lines at Ts = 100, 200, 300 s. Marked: the worked example, zeta = 0.5, wn = 1, Ts = 8.07635 s.

```
control_systems_ai999x.solve('ctrl_true_settling_time', zeta=0.5, wn=1)
# -> Ts = 8.07635 s, last_extremum = 2   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_true_settling_time', zeta=0.9, wn=1, band=0.02)
# -> Ts = 4.6996 s, last_extremum = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
exact settling time: zeta = 0.5, wn = 1	SOLVED	Ts = 8.07635 s, last_extremum = 2
exact settling time: zeta = 0.9, wn = 1, band = 0.02	SOLVED	Ts = 4.6996 s, last_extremum = 0
exact settling time: wn = 1	NEEDS_INPUT	ctrl_true_settling_time needs zeta (damping ratio (0 < zeta < 1)). Example: solve('ctrl_true_settling_time', zeta=0.5, w
exact settling time: zeta = 0.5, wn = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
exact settling time: zeta = 1.5, wn = 1	INVALID_INPUT	Exact settling time of a second-order step response: this capability is for an underdamped system: 0 < zeta < 1.

Ziegler-Nichols tuning from a plant model, `ctrl_zn_from_plant`

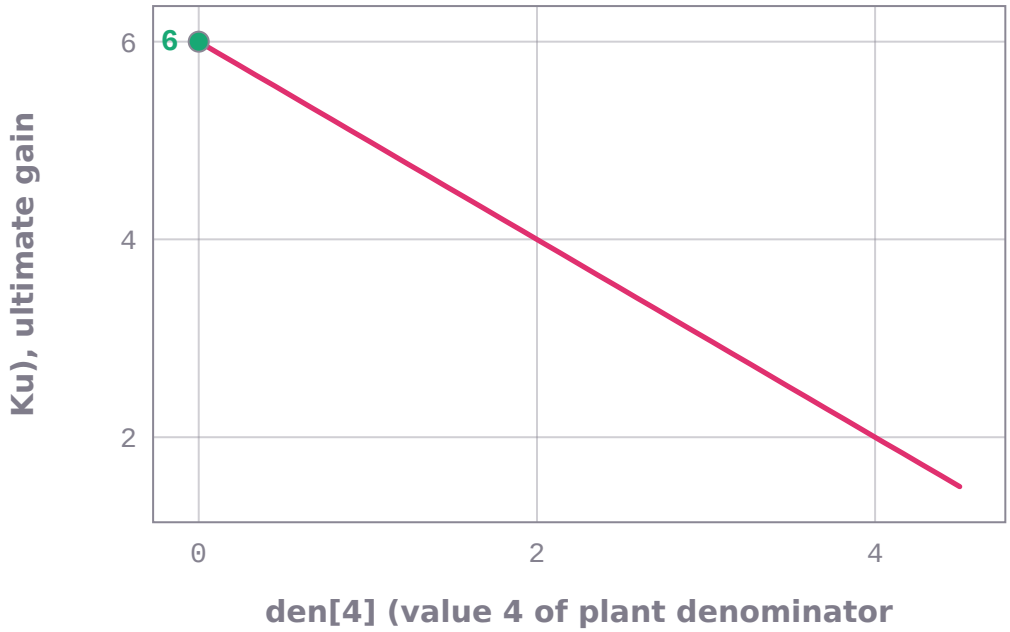
The ultimate gain $Ku = 1$/|G(j w180)| and period $Pu = 2\,pi/w180$ of a plant model, then the Ziegler-Nichols P, PI or PID settings.

$$K_u = \frac{1}{|G(j\omega_{180})|}, \quad P_u = \frac{2\pi}{\omega_{180}}$$

input	meaning	condition
`num`	plant numerator	
`den`	plant denominator	
`controller`	controller type	

output	meaning
`Kp`	proportional gain
`Ti`	integral time (0 when there is no integral action)
`Td`	derivative time
`Ki`	integral gain Kp/Ti
`Kd`	derivative gain Kp Td
`Ku`	ultimate gain
`Pu`	ultimate period

- **Method:** phase crossover from Im(N(jw) conj D(jw)) = 0; Ziegler-Nichols table
- **Assumes:** exactly one phase crossover
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ku (ultimate gain) as value 4 of den (plant denominator) moves from 0 to 4.5, the other values as given ([1, 3, 2, 0]); 41 runs of control-systems-ai999x. Other inputs: num = [1], controller = PID. Marked: the worked example, den[4] = 0, Ku = 6.

```
control_systems_ai999x.solve('ctrl_zn_from_plant', num=[1], den=[1, 3, 2, 0],
controller='PID')
# -> Kp = 3.6, Ti = 2.22144 s, Td = 0.55536 s, Ki = 1.62057, Kd = 1.9993, Ku = 6, Pu =
4.44288 s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_zn_from_plant', num=[1], den=[1, 6, 11, 6],
controller='P')
# -> Kp = 30, Ti = 0 s, Td = 0 s, Ki = 0, Kd = 0, Ku = 60, Pu = 1.89445 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ziegler-Nichols from plant: num = [1], den = [1, 3, 2, 0], controller = PID	SOLVED	Kp = 3.6, Ti = 2.22144 s, Td = 0.55536 s, Ki = 1.62057, Kd = 1.9993, Ku = 6, Pu = 4.44288 s
Ziegler-Nichols from plant: num = [1], den = [1, 6, 11, 6], controller = P	SOLVED	Kp = 30, Ti = 0 s, Td = 0 s, Ki = 0, Kd = 0, Ku = 60, Pu = 1.89445 s
Ziegler-Nichols from plant: den = [1, 3, 2, 0], controller = PID	NEEDS_INPUT	ctrl_zn_from_plant needs num (plant numerator). Example: solve('ctrl_zn_from_plant', num=[1], den=[1, 3, 2, 0], controll
Ziegler-Nichols from plant: num = [1], den = [1, 3, 2, 0], controller = PID, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Ziegler-Nichols from plant: num = [1], den = [1, 1], controller = PID	INVALID_INPUT	Ziegler-Nichols tuning from a plant model: the phase never reaches -180 degrees: there is no ultimate gain.

Lead compensator design for a required phase lead, `ctrl_lead_design`

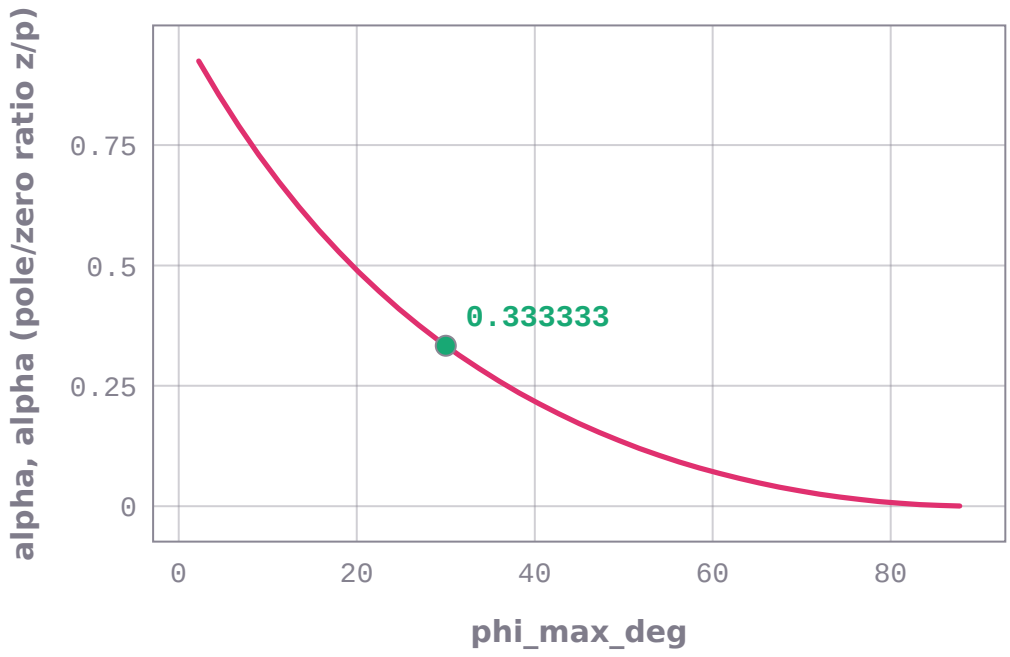
alpha = (1 - sin ϕ)/(1 + sin phi) and T = 1/(w_m sqrt(alpha)) so that C(s) = (T s + 1)/(α T s + 1) adds the phase lead phi at w_m; its zero, pole and gain at w_m.

$$\alpha = \frac{1 - \sin\phi_m}{1 + \sin\phi_m}, \quad T = \frac{1}{\omega_m \sqrt{\alpha}}$$

input	meaning	condition
`phi_max_deg`	required maximum phase lead in degrees (0 to 90)	
`w_m`	frequency of the maximum lead in rad/s	

output	meaning
`alpha`	alpha (pole/zero ratio z/p)
`T`	time constant T
`zero`	zero corner $1/T$ (rad/s)
`pole`	pole corner 1/(alpha T) (rad/s)
`gain_db`	|C(j w_m)| in $dB = -10$ log10 alpha

- **Method:** sin phi_max = (1 - alpha)/(1 + alpha), w_m = 1/(T sqrt(alpha))
- **Assumes:** $0 < \phi < 90$ degrees; $w_m > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



alpha (alpha (pole/zero ratio z/p)) against phi_max_deg (required maximum phase lead in degrees (0 to 90)) from 0 to 90. Other inputs as in the worked example: w_m = 1.0. 39 points computed by control-systems-ai999x; marked: the worked example, phi_max_deg = 30, alpha = 0.333333.

```
control_systems_ai999x.solve('ctrl_lead_design', phi_max_deg=30, w_m=1)
# -> alpha = 0.333333, T = 1.73205 s, zero = 0.57735, pole = 1.73205, gain_db = 4.77121
(SOLVED)
```

```
control_systems_ai999x.solve('ctrl_lead_design', phi_max_deg=53.13010235415598, w_m=3)
# -> alpha = 0.111111, T = 1 s, zero = 1, pole = 9, gain_db = 9.54243   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
lead compensator design: phi_max_deg = 30, w_m = 1	SOLVED	alpha = 0.333333, T = 1.73205 s, zero = 0.57735, pole = 1.73205, gain_db = 4.77121
lead compensator design: phi_max_deg = 53.13010235415598, w_m = 3	SOLVED	alpha = 0.111111, T = 1 s, zero = 1, pole = 9, gain_db = 9.54243
lead compensator design: w_m = 1	NEEDS_INPUT	ctrl_lead_design needs phi_max_deg (required maximum phase lead in degrees (0 to 90)). Example: solve('ctrl_lead_design'
lead compensator design: phi_max_deg = 30, w_m = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
lead compensator design: phi_max_deg = 95, w_m = 1	INVALID_INPUT	Lead compensator design for a required phase lead: the phase lead must lie between 0 and 90 degrees.

Lag compensator placement, `ctrl_lag_design`

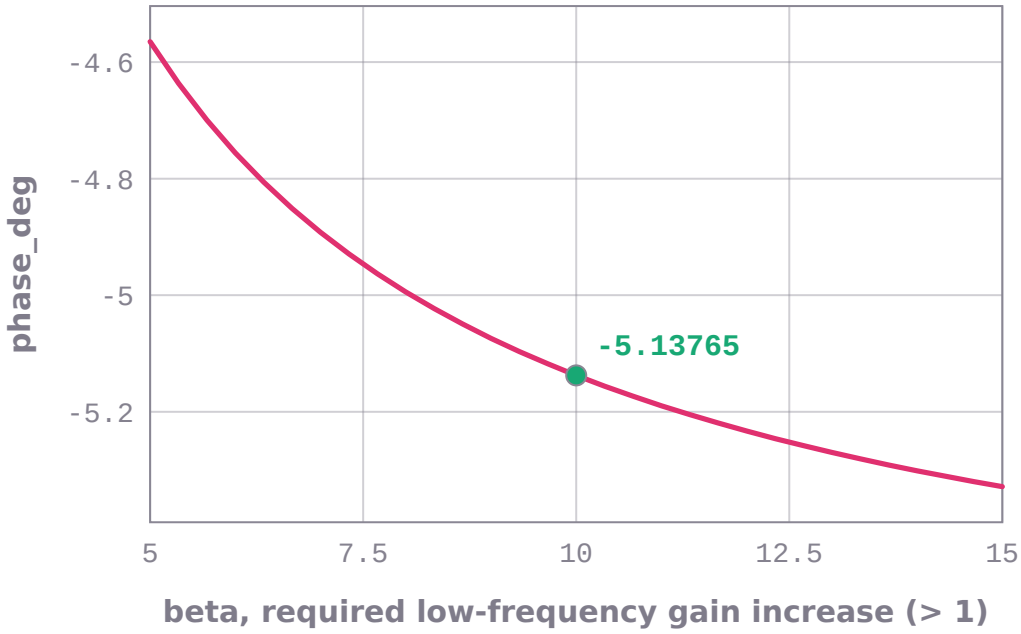
C(s) = (s + z)/(s + p) with z = w_c/ratio (default a decade below the crossover) and $p = z/\beta$: zero, pole, DC gain beta, and the phase lag and gain it adds at w_c.

$$C(s) = \frac{s + z}{s + z/\beta}, \quad z = \omega_c/10$$

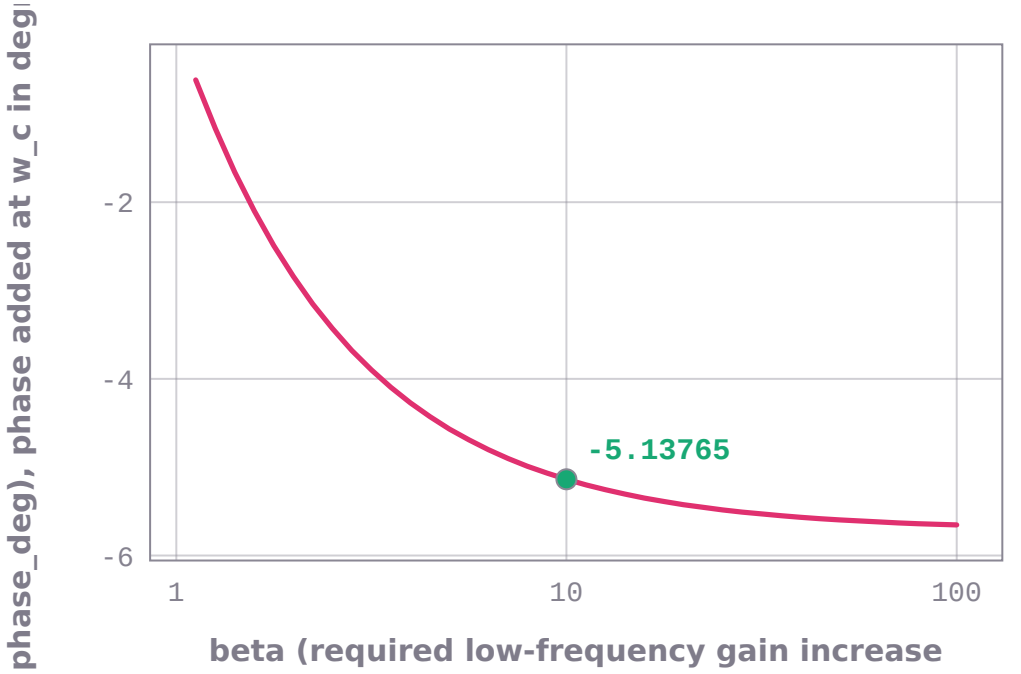
input	meaning	condition
`beta`	required low-frequency gain increase (> 1)	
`w_c`	gain-crossover frequency in rad/s	
`ratio`	w_c / zero (default 10)	

output	meaning
`zero`	zero (rad/s)
`pole`	pole (rad/s)
`dc_gain`	DC gain $z/p = \beta$
`phase_deg`	phase added at w_c in degrees (negative)
`magnitude_db`	gain added at w_c in dB

- **Method:** zero a decade below crossover, pole beta times lower
- **Assumes:** $\beta > 1$; ratio >= 2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



phase_deg (phase added at w_c in degrees (negative)) against beta (required low-frequency gain increase (> 1)), with w_c = 1. 31 points computed by control-systems-ai999x; marked: beta = 10, phase_deg = -5.13765.



phase_deg (phase added at w_c in degrees (negative)) against beta (required low-frequency gain increase (> 1)) from 1 to 100 on a logarithmic axis. Other inputs as in the worked example: w_c = 1.0. 40 points computed by control-systems-ai999x; marked: the worked example, beta = 10, phase_deg = -5.13765.

```
control_systems_ai999x.solve('ctrl_lag_design', beta=10, w_c=1)
# -> zero = 0.1, pole = 0.01, dc_gain = 10, phase_deg = -5.13765, magnitude_db = 0.0427795
(SOLVED)
```

```
control_systems_ai999x.solve('ctrl_lag_design', beta=5, w_c=2, ratio=4)
# -> zero = 0.5, pole = 0.1, dc_gain = 5, phase_deg = -11.1738, magnitude_db = 0.252446
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
lag compensator design: beta = 10, w_c = 1	SOLVED	zero = 0.1, pole = 0.01, dc_gain = 10, phase_deg = -5.13765, magnitude_db = 0.0427795
lag compensator design: beta = 5, w_c = 2, ratio = 4	SOLVED	zero = 0.5, pole = 0.1, dc_gain = 5, phase_deg = -11.1738, magnitude_db = 0.252446
lag compensator design: w_c = 1	NEEDS_INPUT	ctrl_lag_design needs beta (required low-frequency gain increase (> 1)). Example: solve('ctrl_lag_design', beta=10, w_c=
lag compensator design: beta = 10, w_c = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
lag compensator design: beta = 0.5, w_c = 1	INVALID_INPUT	Lag compensator placement: beta must exceed 1 (the factor by which the low-frequency gain is raised).

Pade approximation of a time delay, `ctrl_pade`

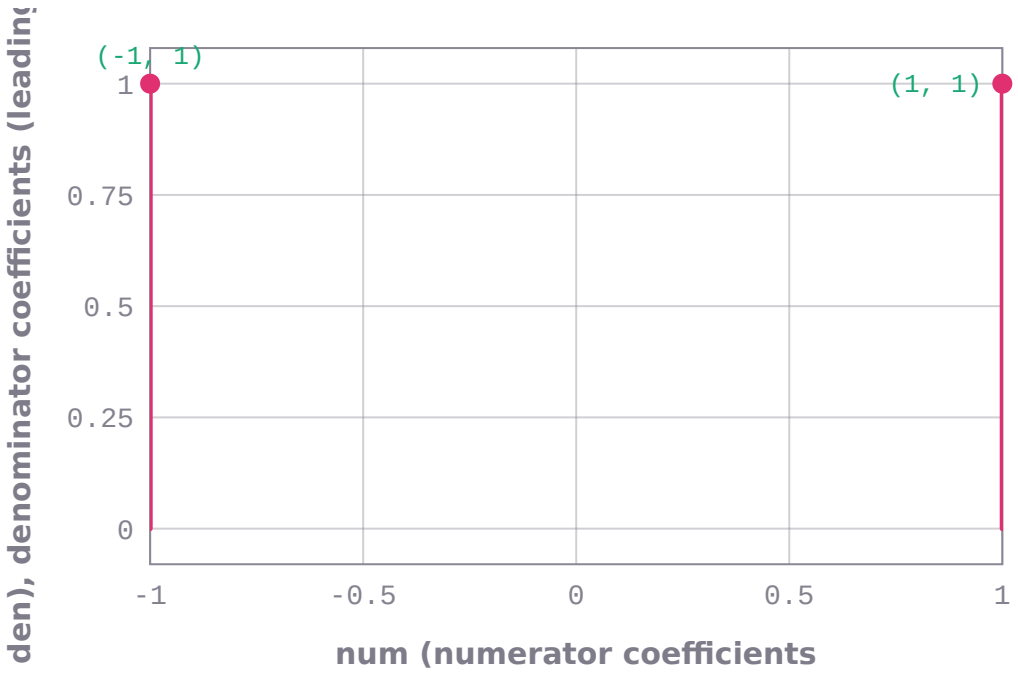
The (n, n) Pade approximation of e^{-theta s} as a rational transfer function (denominator's leading coefficient 1).

$$e^{-\theta s} \approx \frac{\sum_k c_k (-\theta s)^k}{\sum_k c_k (\theta s)^k}$$

input	meaning	condition
`theta`	time delay in seconds	
`n`	approximation order (1 to 6)	

output	meaning
`num`	numerator coefficients
`den`	denominator coefficients (leading 1)

- **Method:** Pade coefficients $c_k = (2n - k)!\, n!\, /\, ((2n)!\, k!\, (n-k)!)$
- **Assumes:** $\theta > 0$; $1 \le n \le 6$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



denominator coefficients (leading 1) (den) at each numerator coefficients (num), both returned by control-systems-ai999x for the worked example (theta = 2.0, n = 1): -1 → 1; 1 → 1.

```
control_systems_ai999x.solve('ctrl_pade', theta=2, n=1)
# -> num = [-1.0, 1.0], den = [1.0, 1.0]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_pade', theta=1, n=2)
# -> num = [1.0, -6.0, 12.0], den = [1.0, 6.0, 12.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pade approximation: theta = 2, n = 1	SOLVED	num = [-1.0, 1.0], den = [1.0, 1.0]
Pade approximation: theta = 1, n = 2	SOLVED	num = [1.0, -6.0, 12.0], den = [1.0, 6.0, 12.0]
Pade approximation: n = 1	NEEDS_INPUT	ctrl_pade needs theta (time delay in seconds). Example: solve('ctrl_pade', theta=2, n=1).
Pade approximation: theta = 2, n = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Pade approximation: theta = 1, n = 9	INVALID_INPUT	n: must be at most 6, got 9.

LQR for a first-order system, `ctrl_scalar_lqr`

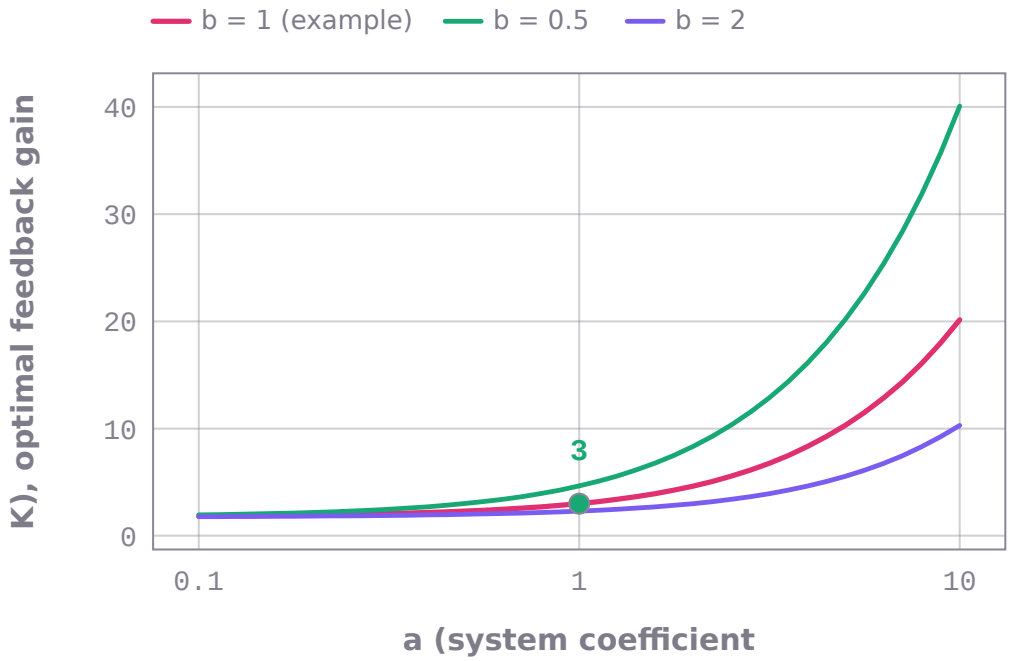
For x' = a x + b u minimising the integral of q $x^2 + r\,u^2$: the Riccati solution P, the optimal gain $K = b$ P/r and the closed-loop pole -sqrt($a^2 + b^2$ q/r).

$$2aP - \frac{b^2 P^2}{r} + q = 0, \quad K = \frac{bP}{r}$$

input	meaning	condition
`a`	system coefficient	
`b`	input coefficient	
`q`	state weight (> 0)	
`r`	input weight (> 0)	

output	meaning
`P`	Riccati solution
`K`	optimal feedback gain
`pole`	closed-loop pole a - b K

- **Method:** positive root of 2 a $P - b^2\,P^2/r + q = 0$
- **Assumes:** $b \neq 0$; q, $r > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



K (optimal feedback gain) against a (system coefficient) from 0.1 to 10 on a logarithmic axis. One curve for each value of b (input coefficient): 1, 0.5, 2. Other inputs as in the worked example: q = 3.0, r = 1.0. 41 points per curve computed by control-systems-ai999x; marked: the worked example, a = 1, K = 3.

```
control_systems_ai999x.solve('ctrl_scalar_lqr', a=1, b=1, q=3, r=1)
# -> P = 3, K = 3, pole = -2   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_scalar_lqr', a=0, b=2, q=4, r=1)
# -> P = 1, K = 2, pole = -4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
scalar LQR: a = 1, b = 1, q = 3, r = 1	SOLVED	P = 3, K = 3, pole = -2

question	outcome	result
scalar LQR: a = 0, b = 2, q = 4, r = 1	SOLVED	P = 1, K = 2, pole = -4
scalar LQR: b = 1, q = 3, r = 1	NEEDS_INPUT	ctrl_scalar_lqr needs a (system coefficient). Example: solve('ctrl_scalar_lqr', a=1, b=1, q=3, r=1).
scalar LQR: a = 1, b = 1, q = 3, r = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
scalar LQR: a = 1, b = 0, q = 3, r = 1	INVALID_INPUT	LQR for a first-order system: b must not be zero.

PI gains placing the closed-loop poles of a first-order plant, `ctrl_pi_pole_placement`

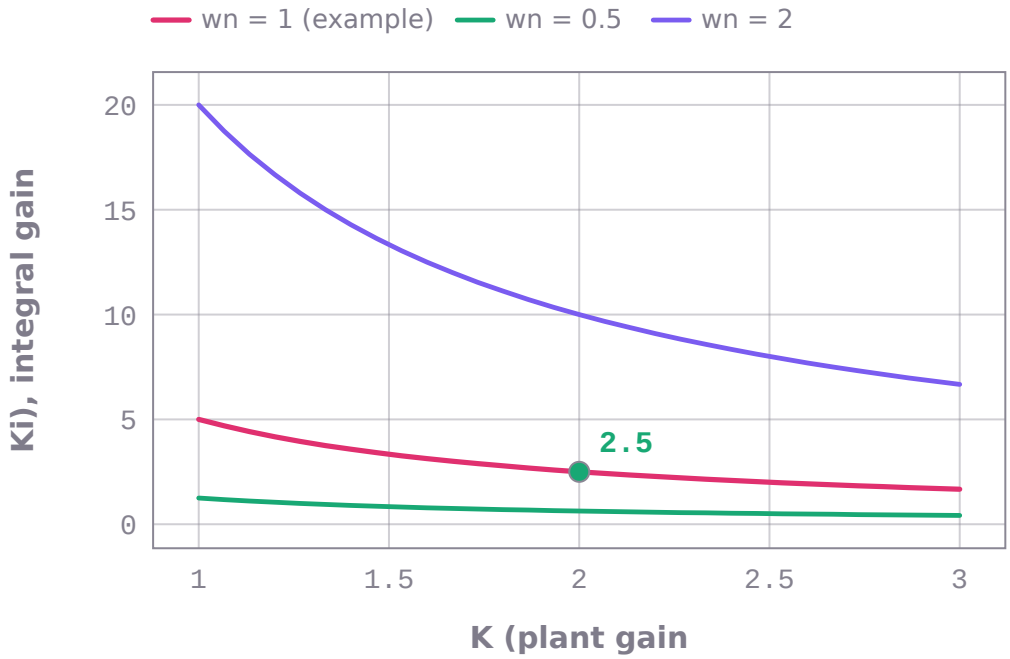
Kp = (2 zeta wn tau - 1)/K and $Ki = wn^2$ tau/K, so that K/(tau s + 1) under PI control with unity feedback has the characteristic polynomial $s^2 + 2$ zeta wn $s + wn^2$.

$$K_p = \frac{2\zeta\omega_n\tau - 1}{K}, \quad K_i = \frac{\omega_n^2\tau}{K}$$

input	meaning	condition
`K`	plant gain	
`tau`	plant time constant in seconds	
`zeta`	desired damping ratio	
`wn`	desired natural frequency in rad/s	

output	meaning
`Kp`	proportional gain
`Ki`	integral gain

- **Method:** match tau s^2 + (1 + K Kp) s + K Ki to tau (s^2 + 2 zeta wn s + wn^2)
- **Assumes:** tau, zeta, $wn > 0$; $K \neq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ki (integral gain) against K (plant gain) from 1 to 3. One curve for each value of wn (desired natural frequency in rad/s): 1, 0.5, 2. Other inputs as in the worked example: tau = 5.0, zeta = 0.7. 31 points per curve computed by control-systems-ai999x; marked: the worked example, K = 2, Ki = 2.5.

```
control_systems_ai999x.solve('ctrl_pi_pole_placement', K=2, tau=5, zeta=0.7, wn=1)
# -> Kp = 3, Ki = 2.5   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_pi_pole_placement', K=1, tau=1, zeta=1, wn=2)
# -> Kp = 3, Ki = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PI pole placement: K = 2, tau = 5, zeta = 0.7, wn = 1	SOLVED	Kp = 3, Ki = 2.5
PI pole placement: K = 1, tau = 1, zeta = 1, wn = 2	SOLVED	Kp = 3, Ki = 4
PI pole placement: tau = 5, zeta = 0.7, wn = 1	NEEDS_INPUT	ctrl_pi_pole_placement needs K (plant gain). Example: solve('ctrl_pi_pole_placement', K=2, tau=5, zeta=0.7, wn=1).
PI pole placement: K = 2, tau = 5, zeta = 0.7, wn = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
PI pole placement: K = 2, tau = -5, zeta = 0.7, wn = 1	INVALID_INPUT	PI gains placing the closed-loop poles of a first-order plant: tau must be positive.

Steady-state gain of a scalar Kalman filter, `ctrl_kalman_scalar_steady`

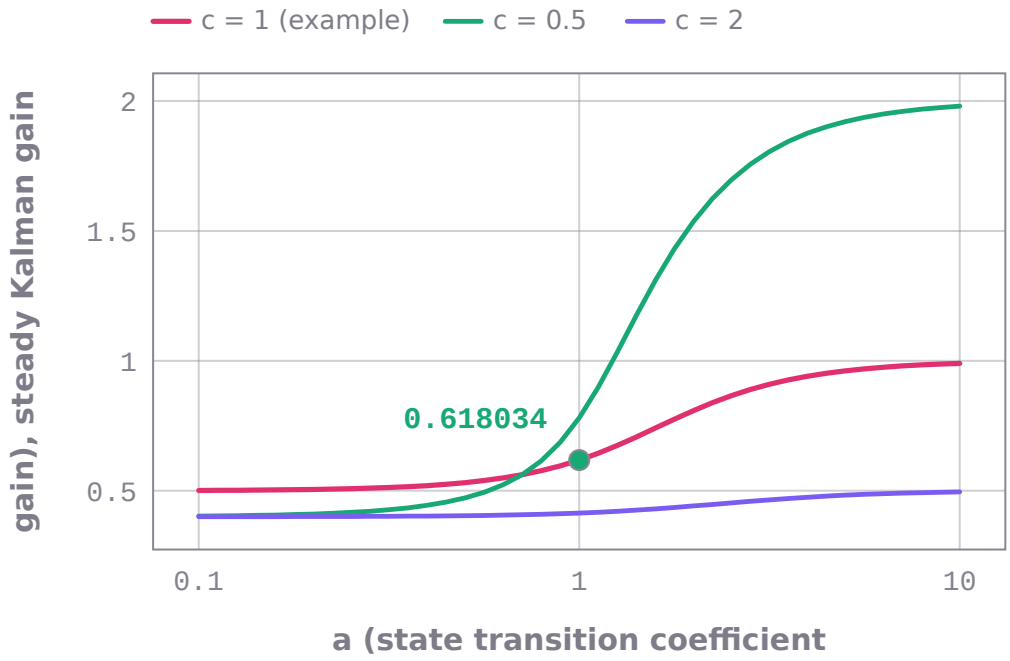
For $x_k = a$ x_{$k - 1$} + w (variance q) and $z_k = c$ x_k + v (variance r): the steady prior variance P (discrete Riccati equation), the steady Kalman gain and the posterior variance.

$$P = a^2 P - \frac{a^2 c^2 P^2}{c^2 P + r} + q, \quad K = \frac{P c}{c^2 P + r}$$

input	meaning	condition
`a`	state transition coefficient	
`c`	measurement coefficient	
`q`	process noise variance	
`r`	measurement noise variance	

output	meaning
`P_prior`	steady prior error variance
`gain`	steady Kalman gain
`P_post`	steady posterior error variance

- **Method:** positive root of c^2 P^2 + (r - a^2 r - q c^2) P - q $r = 0$
- **Assumes:** $c \neq 0$; q, $r > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



gain (steady Kalman gain) against a (state transition coefficient) from 0.1 to 10 on a logarithmic axis. One curve for each value of c (measurement coefficient): 1, 0.5, 2. Other inputs as in the worked example: q = 1.0, r = 1.0. 41 points per curve computed by control-systems-ai999x; marked: the worked example, a = 1, gain = 0.618034.

```
control_systems_ai999x.solve('ctrl_kalman_scalar_steady', a=1, c=1, q=1, r=1)
# -> P_prior = 1.61803, gain = 0.618034, P_post = 0.618034   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_kalman_scalar_steady', a=0, c=1, q=2, r=2)
# -> P_prior = 2, gain = 0.5, P_post = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
steady-state Kalman gain: a = 1, c = 1, q = 1, r = 1	SOLVED	P_prior = 1.61803, gain = 0.618034, P_post = 0.618034

question	outcome	result
steady-state Kalman gain: a = 0, c = 1, q = 2, r = 2	SOLVED	P_prior = 2, gain = 0.5, P_post = 1
steady-state Kalman gain: c = 1, q = 1, r = 1	NEEDS_INPUT	ctrl_kalman_scalar_steady needs a (state transition coefficient). Example: solve('ctrl_kalman_scalar_steady', a=1, c=1,
steady-state Kalman gain: a = 1, c = 1, q = 1, r = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
steady-state Kalman gain: a = 1, c = 0, q = 1, r = 1	INVALID_INPUT	Steady-state gain of a scalar Kalman filter: c must not be zero.

First-order-plus-dead-time fit by Smith's two-point method, `ctrl_fopdt_smith`

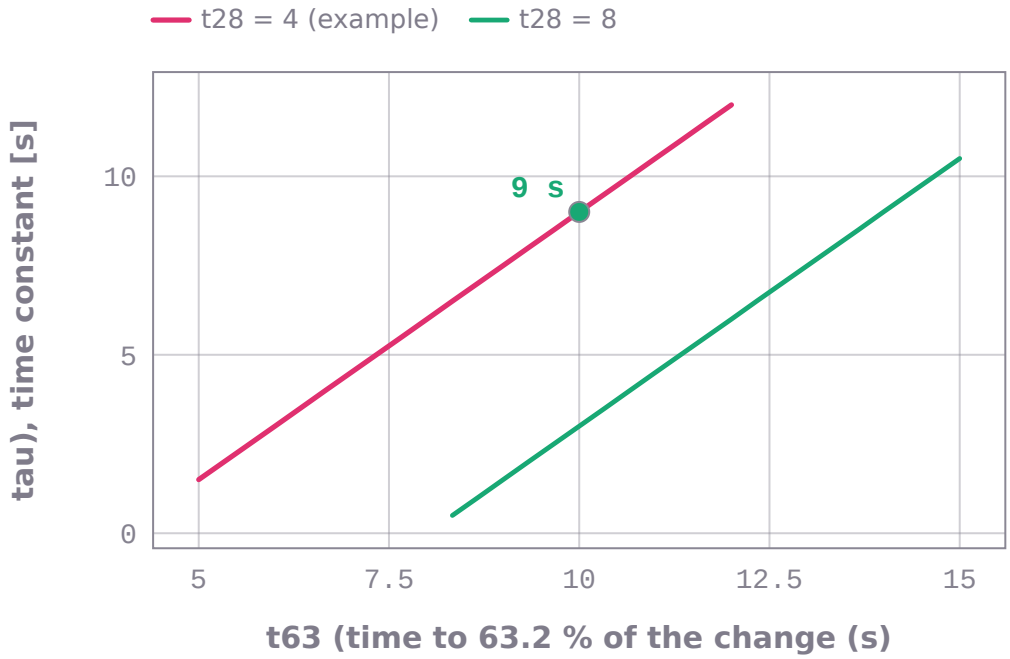
$K = dy/du$, $\tau = 1.5\,(t63 - t28)$ and $\theta = t63 - \tau$ from the times at which a step response reaches 28.3 % and 63.2 % of its change.

$$\tau = 1.5(t_{63} - t_{28}), \quad \theta = t_{63} - \tau$$

input	meaning	condition
`t28`	time to 28.3 % of the change (s)	
`t63`	time to 63.2 % of the change (s)	
`dy`	total output change	
`du`	input step size	

output	meaning
`K`	process gain
`tau`	time constant
`theta`	dead time

- **Method:** Smith's method
- **Assumes:** $0 < t28 < t63$; $\theta \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



tau (time constant) against t63 (time to 63.2 % of the change (s)) from 5 to 15. One curve for each value of t28 (time to 28.3 % of the change (s)): 4, 8. Other inputs as in the worked example: dy = 6.0, du = 2.0. 22 points per curve computed by control-systems-ai999x; marked: the worked example, t63 = 10, tau = 9 s.

```
control_systems_ai999x.solve('ctrl_fopdt_smith', t28=4, t63=10, dy=6, du=2)
# -> K = 3, tau = 9 s, theta = 1 s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_fopdt_smith', t28=2, t63=5, dy=1, du=1)
# -> K = 1, tau = 4.5 s, theta = 0.5 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Smith FOPDT fit: t28 = 4, t63 = 10, dy = 6, du = 2	SOLVED	K = 3, tau = 9 s, theta = 1 s
Smith FOPDT fit: t28 = 2, t63 = 5, dy = 1, du = 1	SOLVED	K = 1, tau = 4.5 s, theta = 0.5 s
Smith FOPDT fit: t63 = 10, dy = 6, du = 2	NEEDS_INPUT	ctrl_fopdt_smith needs t28 (time to 28.3 % of the change (s)). Example: solve('ctrl_fopdt_smith', t28=4, t63=10, dy=6, d
Smith FOPDT fit: t28 = 4, t63 = 10, dy = 6, du = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Smith FOPDT fit: t28 = 1, t63 = 10, dy = 6, du = 2	INVALID_INPUT	First-order-plus-dead-time fit by Smith's two-point method: the times give a negative dead time: the response is not fir

Continuous-time LQR gain (Riccati equation), `ctrl_lqr`

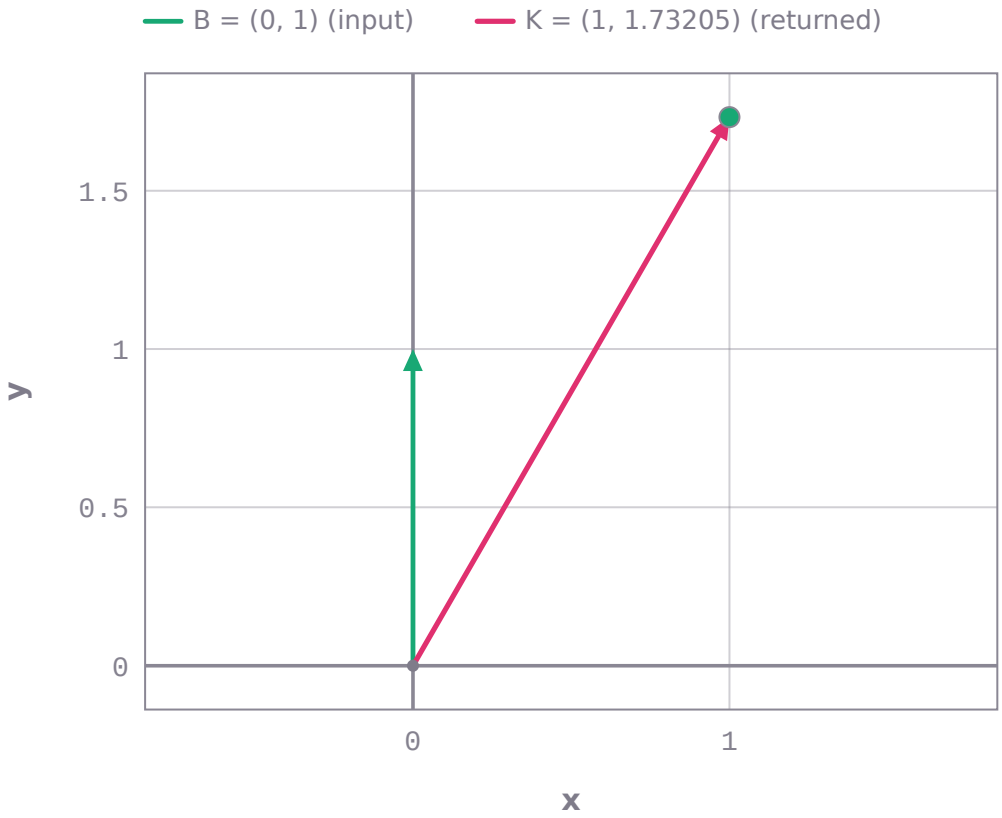
The optimal state feedback $u = -K$ x minimising the integral of x^T Q x + R u^2 for x' = A x + B u: the Riccati solution P, $K = R$^{-1} B^T P and the closed-loop poles.

$$A^T P + P A - P B R^{-1} B^T P + Q = 0, \quad K = R^{-1} B^T P$$

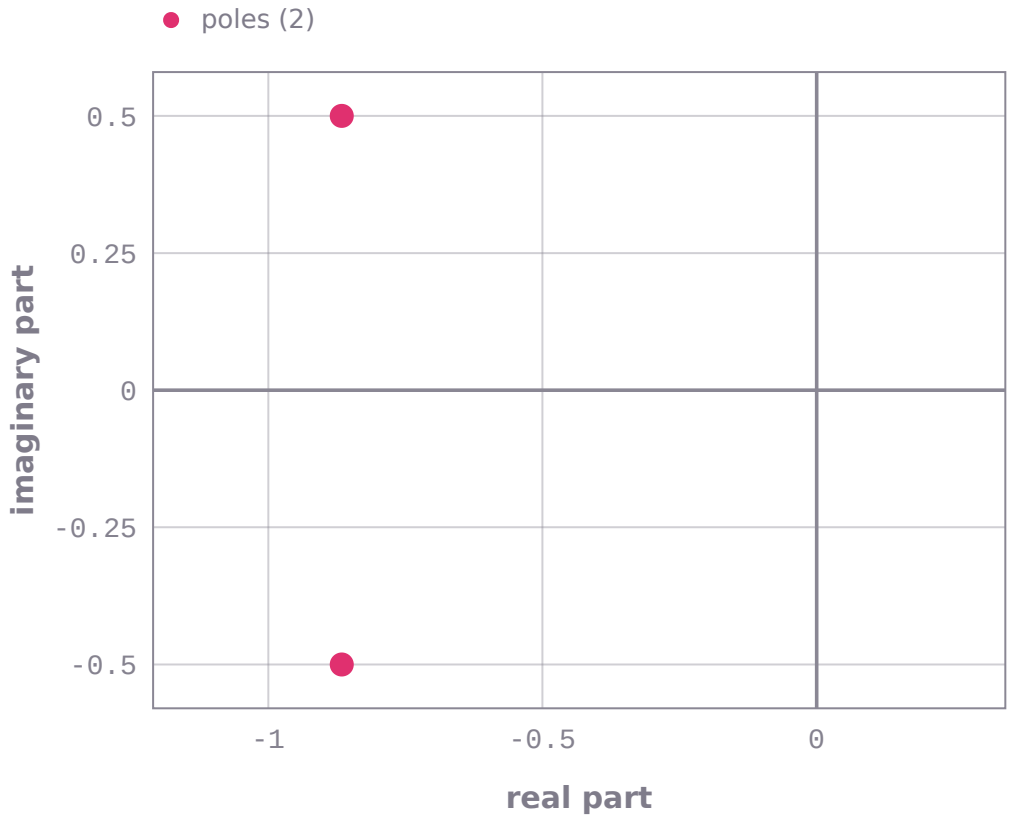
input	meaning	condition
`A`	state matrix (n x n)	
`B`	input vector	
`Q`	state weight (symmetric positive definite)	
`R`	input weight (> 0)	

output	meaning
`K`	optimal gain
`P`	Riccati solution
`poles`	closed-loop poles

- **Method:** Newton-Kleinman iteration (a Lyapunov equation per step) from a stabilising Ackermann gain
- **Assumes:** (A, B) controllable; Q symmetric positive definite; $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input B = (0, 1); returned by control-systems-ai999x: K = (1, 1.73205). Other inputs: A = [[0, 1], [0, 0]], Q = [[1, 0], [0, 1]], R = 1.0. Marked: the tip of each returned vector.



closed-loop poles (poles) returned by control-systems-ai999x for the worked example (A = [[0, 1], [0, 0]], B = [0, 1], Q = [[1, 0], [0, 1]], R = 1.0), drawn in the complex plane: -0.866025 − 0.5i, -0.866025 + 0.5i.

```
control_systems_ai999x.solve('ctrl_lqr', A=[[0, 1], [0, 0]], B=[0, 1], Q=[[1, 0], [0, 1]], R=1)
# -> K = [1.0, 1.73205080757], P = [[1.73205080757, 1.0], [1.0, 1.73205080757]], poles = [(-0.866025403784-0.5j), (-0.866025403784+0.5j)]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_lqr', A=[[0, 1], [-1, 0]], B=[0, 1], Q=[[1, 0], [0, 0.5]], R=2)
# -> K = [0.224744871392, 0.836355033932], P = [[2.04864307694, 0.449489742783], [0.449489742783, 1.67271006786]], poles = [(-0.418177516966-1.0246328297j), (-0.418177516966+1.0246328297j)]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
LQR gain: A = [[0, 1], [0, 0]], B = [0, 1], Q = [[1, 0], [0, 1]], R = 1	SOLVED	K = [1.0, 1.73205080757], P = [[1.73205080757, 1.0], [1.0, 1.73205080757]], poles = [(-0.866025403784-0.5j), (-0.866025403784+0.5j)]
LQR gain: A = [[0, 1], [-1, 0]], B = [0, 1], Q = [[1, 0], [0, 0.5]], R = 2	SOLVED	K = [0.224744871392, 0.836355033932], P = [[2.04864307694, 0.449489742783], [0.449489742783, 1.67271006786]], poles = [(-0.418177516966-1.0246328297j), (-0.418177516966+1.0246328297j)]
LQR gain: B = [0, 1], Q = [[1, 0], [0, 1]], R = 1	NEEDS_INPUT	ctrl_lqr needs A (state matrix (n x n)). Example: solve('ctrl_lqr', A=[[0, 1], [0, 0]], B=[0, 1], Q=[[1, 0], [0, 1]], R=
LQR gain: A = [[0, 1], [0, 0]], B = [0, 1], Q =	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(

question	outcome	result
[[1, 0], [0, 1]], R = 1, zz = 1		
LQR gain: A = [[0, 1], [0, 0]], B = [0, 1], Q = [[1, 2], [0, 1]], R = 1	INVALID_INPUT	Continuous-time LQR gain (Riccati equation): Q must be symmetric.

Discrete-time LQR gain (discrete Riccati equation), `ctrl_dlqr`

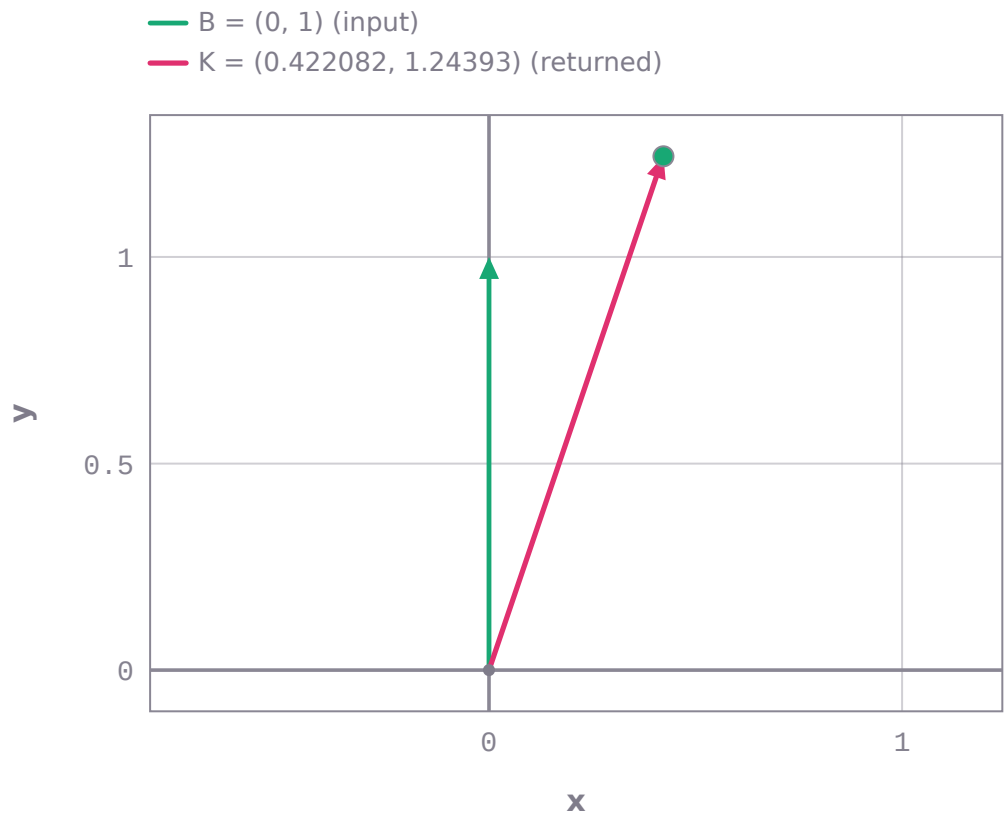
The optimal state feedback $u_k = -K$ x_k minimising the sum of x^T Q x + R u^2 for x_{$k+1$} = A x_k + B u_k: P from the discrete Riccati equation, K = (R + B^T P B)^{-1} B^T P A and the closed-loop poles.

$$P = A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A + Q$$

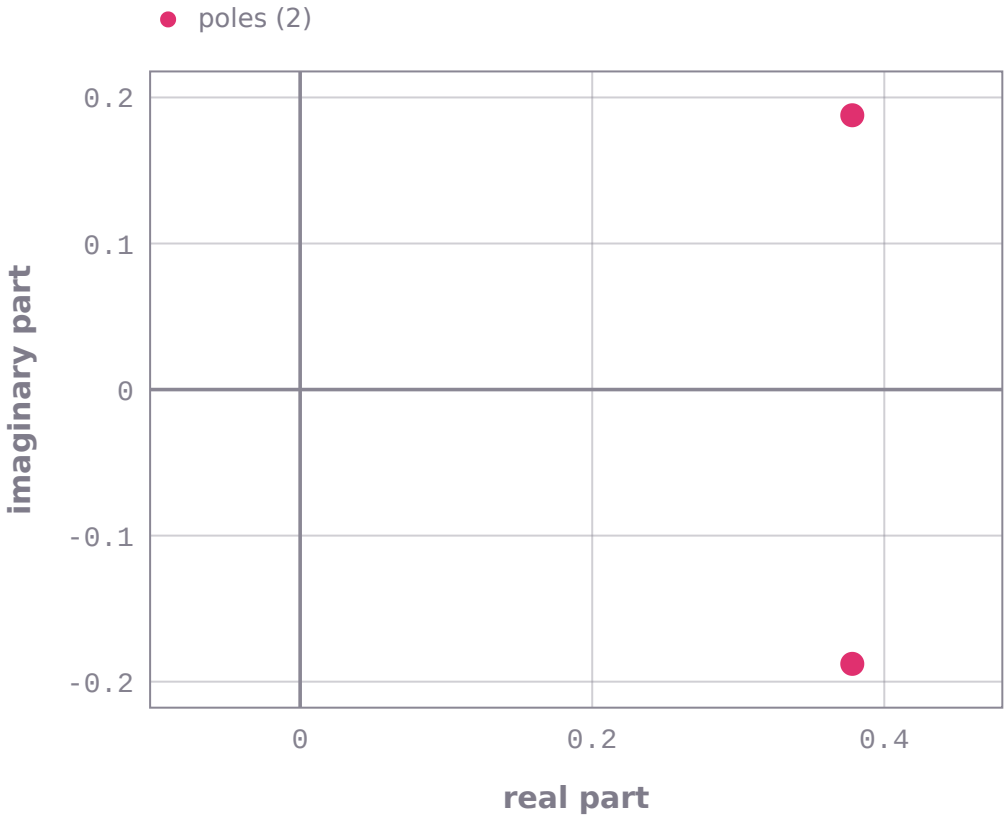
input	meaning	condition
`A`	discrete state matrix (n x n)	
`B`	input vector	
`Q`	state weight (symmetric positive definite)	
`R`	input weight (> 0)	

output	meaning
`K`	optimal gain
`P`	Riccati solution
`poles`	closed-loop poles

- **Method:** Hewer's iteration (a discrete Lyapunov equation per step) from the deadbeat gain
- **Assumes:** (A, B) controllable; Q symmetric positive definite; $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input B = (0, 1); returned by control-systems-ai999x: K = (0.422082, 1.24393). Other inputs: A = [[1, 1], [0, 1]], Q = [[1, 0], [0, 1]], R = 1.0. Marked: the tip of each returned vector.



closed-loop poles (poles) returned by control-systems-ai999x for the worked example (A = [[1, 1], [0, 1]], B = [0, 1], Q = [[1, 0], [0, 1]], R = 1.0), drawn in the complex plane: 0.378036 − 0.18773i, 0.378036 + 0.18773i.

```
control_systems_ai999x.solve('ctrl_dlqr', A=[[1, 1], [0, 1]], B=[0, 1], Q=[[1, 0], [0, 1]], R=1)
# -> K = [0.422082440385, 1.2439288539], P = [[2.94712296671, 2.36920540709], [2.36920540709, 4.613134261]], poles = [(0.378035573048-0.187730370457j), (0.378035573048+0.187730370457j)]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_dlqr', A=[[0.5, 0], [0, 0.8]], B=[1, 1], Q=[[1, 0], [0, 1]], R=1)
# -> K = [0.142441813061, 0.33693723649], P = [[1.24129941786, -0.272125618133], [-0.272125618133, 1.7049507782]], poles = [0.184973494743, 0.635647455706]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
discrete LQR gain: A = [[1, 1], [0, 1]], B = [0, 1], Q = [[1, 0], [0, 1]], R = 1	SOLVED	K = [0.422082440385, 1.2439288539], P = [[2.94712296671, 2.36920540709], [2.36920540709, 4.613134261]], poles = [(0.378035573048-0.187730370457j), (0.378035573048+0.187730370457j)]
discrete LQR gain: A = [[0.5, 0], [0, 0.8]], B = [1, 1], Q = [[1, 0], [0, 1]], R = 1	SOLVED	K = [0.142441813061, 0.33693723649], P = [[1.24129941786, -0.272125618133], [-0.272125618133, 1.7049507782]], poles = [0.184973494743, 0.635647455706]
discrete LQR gain: B = [0, 1], Q = [[1, 0], [0, 1]], R = 1	NEEDS_INPUT	ctrl_dlqr needs A (discrete state matrix (n x n)). Example: solve('ctrl_dlqr', A=[[1, 1], [0, 1]], B=[0, 1], Q=[[1, 0],
discrete LQR gain: A = [[1, 1], [0, 1]], B = [0, 1], Q = [[1, 0], [0, 1]], R = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
discrete LQR gain: A = [[1, 0], [0, 2]], B = [1, 0], Q = [[1, 0], [0, 1]], R = 1	INVALID_INPUT	Discrete-time LQR gain (discrete Riccati equation): (A, B) must be controllable.

Steady-state Kalman-Bucy filter gain, `ctrl_kalman_bucy`

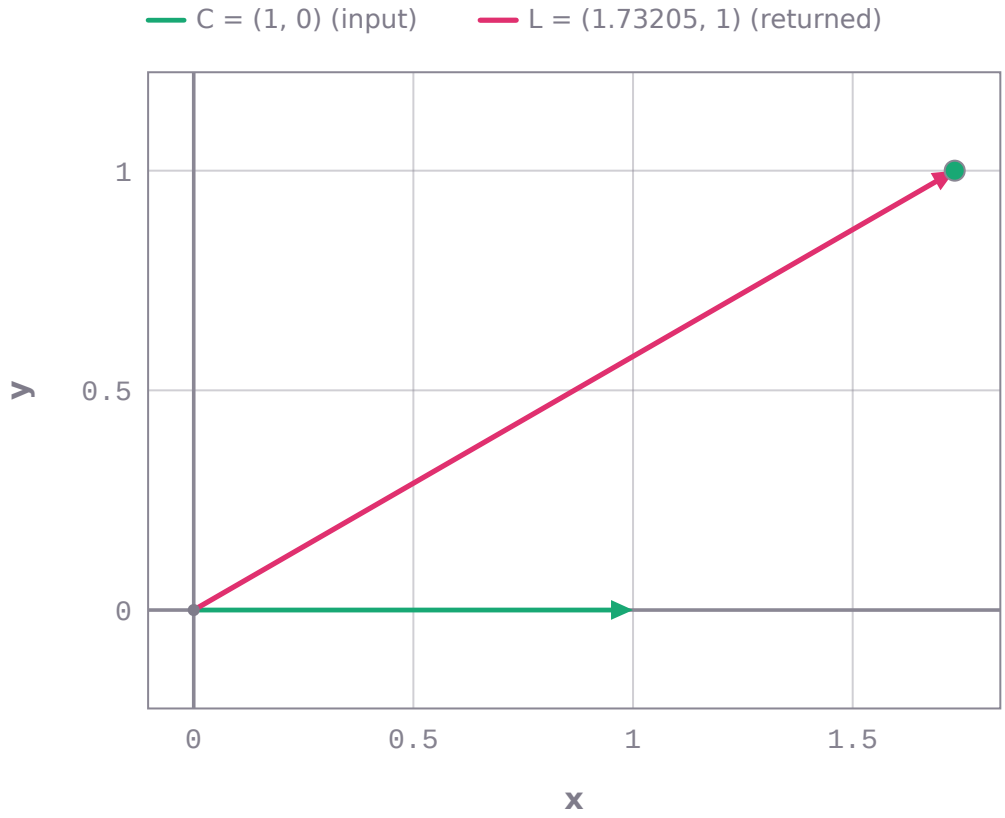
For x' = A x + w (covariance W), $y = C$ x + v (variance V): the steady error covariance P from the filter Riccati equation A P + P $A^T - P\,C^T$ V^{-1} C $P + W = 0$, the gain $L = P\,C^T/V$ and the estimator poles (eigenvalues of A - L C).

$$AP + PA^T - PC^TV^{-1}CP + W = 0, \quad L = PC^TV^{-1}$$

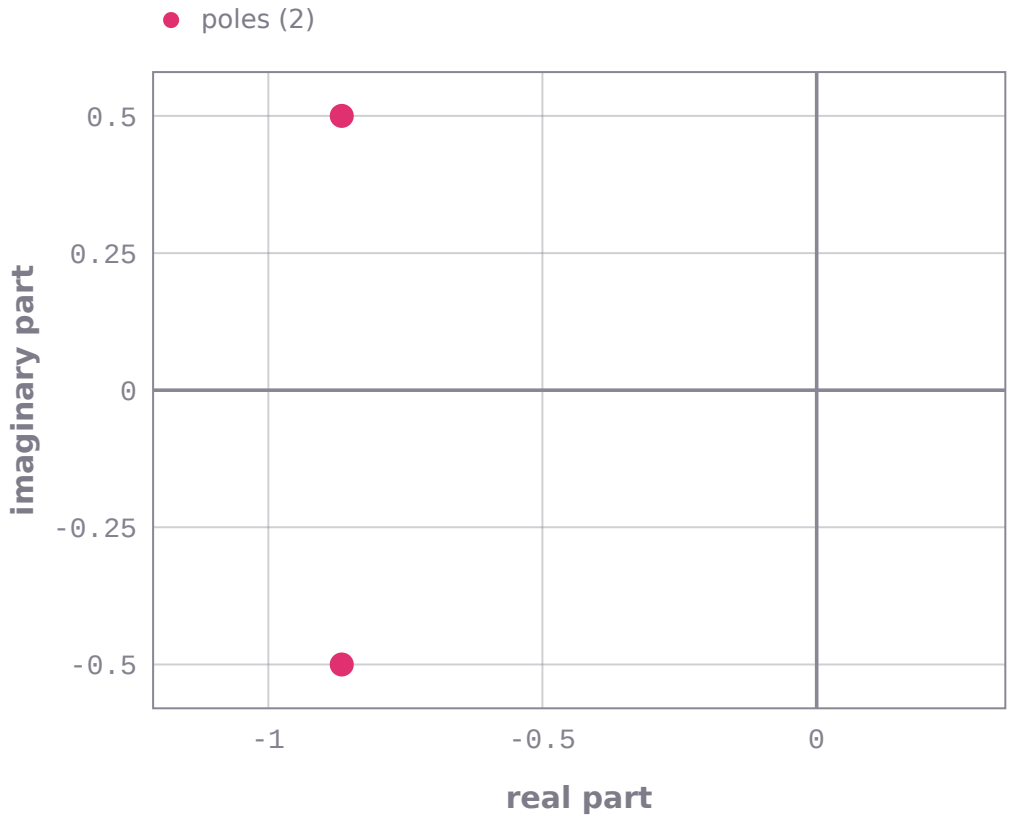
input	meaning	condition
`A`	state matrix (n x n)	
`C`	output vector	
`W`	process noise covariance (symmetric positive definite)	
`V`	measurement noise variance (> 0)	

output	meaning
`L`	Kalman gain (column)
`P`	steady error covariance
`poles`	estimator poles

- **Method:** the dual LQR problem (A^T, C^T, W, V) by Newton-Kleinman iteration
- **Assumes:** (A, C) observable; W positive definite; $V > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The plane vectors of the worked example as arrows from the origin: input C = (1, 0); returned by control-systems-ai999x: L = (1.73205, 1). Other inputs: A = [[0, 1], [0, 0]], W = [[1, 0], [0, 1]], V = 1.0. Marked: the tip of each returned vector.



estimator poles (poles) returned by control-systems-ai999x for the worked example (A = [[0, 1], [0, 0]], C = [1, 0], W = [[1, 0], [0, 1]], V = 1.0), drawn in the complex plane: -0.866025 − 0.5i, -0.866025 + 0.5i.

```
control_systems_ai999x.solve('ctrl_kalman_bucy', A=[[0, 1], [0, 0]], C=[1, 0], W=[[1, 0], [0, 1]], V=1)
# -> L = [1.73205080757, 1.0], P = [[1.73205080757, 1.0], [1.0, 1.73205080757]], poles = [(-0.866025403784-0.5j), (-0.866025403784+0.5j)]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_kalman_bucy', A=[[-1]], C=[1], W=[[3]], V=1)
# -> L = [1.0], P = [[1.0]], poles = [-2.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Kalman-Bucy gain: A = [[0, 1], [0, 0]], C = [1, 0], W = [[1, 0], [0, 1]], V = 1	SOLVED	L = [1.73205080757, 1.0], P = [[1.73205080757, 1.0], [1.0, 1.73205080757]], poles = [(-0.866025403784-0.5j), (-0.866025403784+0.5j)]
Kalman-Bucy gain: A = [[-1]], C = [1], W = [[3]], V = 1	SOLVED	L = [1.0], P = [[1.0]], poles = [-2.0]
Kalman-Bucy gain: C = [1, 0], W = [[1, 0], [0, 1]], V = 1	NEEDS_INPUT	ctrl_kalman_bucy needs A (state matrix (n x n)). Example: solve('ctrl_kalman_bucy', A=[[0, 1], [0, 0]], C=[1, 0], W=[[1,
Kalman-Bucy gain: A = [[0, 1], [0, 0]], C = [1, 0], W = [[1, 0], [0, 1]], V = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Kalman-Bucy gain: A = [[0, 1], [0, 0]], C = [0, 1], W = [[1, 0], [0, 1]], V = 1	INVALID_INPUT	Steady-state Kalman-Bucy filter gain: (A, C) must be observable.

Steady-state discrete Kalman filter gain, `ctrl_discrete_kalman_gain`

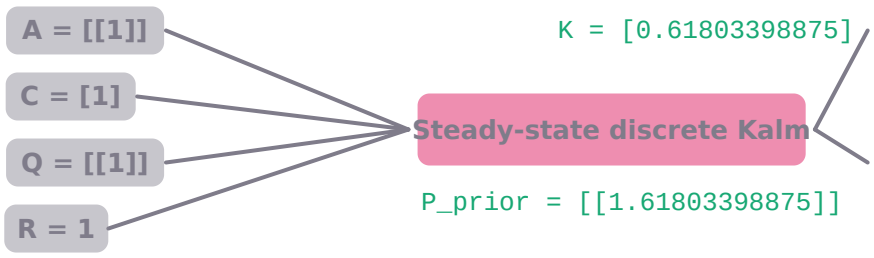
For x_{$k+1$} = A x_k + w (covariance Q), $y_k = C$ x_k + v (variance R): the steady prior covariance P from the discrete filter Riccati equation and the measurement-update gain $K = P\,C^T$ (C P C^T + R)^{-1}.

$$P = APA^T - APC^T(CPC^T + R)^{-1}CPA^T + Q, \quad K = PC^T(CPC^T + R)^{-1}$$

input	meaning	condition
`A`	discrete state matrix (n x n)	
`C`	output vector	
`Q`	process noise covariance (symmetric positive definite)	
`R`	measurement noise variance (> 0)	

output	meaning
`K`	steady Kalman gain
`P_prior`	steady prior error covariance

- **Method:** the dual discrete Riccati equation by Hewer's iteration
- **Assumes:** (A, C) observable; Q positive definite; $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Steady-state discrete Kalman filter gain as inputs and results: stated A = [[1]], C = [1], Q = [[1]], R = 1; returned by control-systems-ai999x: K = [0.61803398875], P_prior = [[1.61803398875]].

```
control_systems_ai999x.solve('ctrl_discrete_kalman_gain', A=[[1]], C=[1], Q=[[1]], R=1)
# -> K = [0.61803398875], P_prior = [[1.61803398875]]   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_discrete_kalman_gain', A=[[1, 1], [0, 1]], C=[1, 0], Q=
[[1, 0], [0, 1]], R=1)
# -> K = [0.821846413518, 0.422082440385], P_prior = [[4.613134261, 2.36920540709],
[2.36920540709, 2.94712296671]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
discrete Kalman gain: A = [[1]], C = [1], Q = [[1]], R = 1	SOLVED	K = [0.61803398875], P_prior = [[1.61803398875]]
discrete Kalman gain: A = [[1, 1], [0, 1]], C = [1, 0], Q = [[1, 0], [0, 1]], R = 1	SOLVED	K = [0.821846413518, 0.422082440385], P_prior = [[4.613134261, 2.36920540709], [2.36920540709, 2.94712296671]]
discrete Kalman gain: C = [1], Q = [[1]], R = 1	NEEDS_INPUT	ctrl_discrete_kalman_gain needs A (discrete state matrix (n x n)). Example: solve('ctrl_discrete_kalman_gain', A=[[1]],
discrete Kalman gain: A = [[1]], C = [1], Q = [[1]], R = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
discrete Kalman gain: A = [[1, 0], [0, 1]], C = [1, 0], Q = [[1, 0], [0, 1]], R = 1	INVALID_INPUT	Steady-state discrete Kalman filter gain: (A, C) must be observable.

Continuous Lyapunov equation, `ctrl_lyapunov`

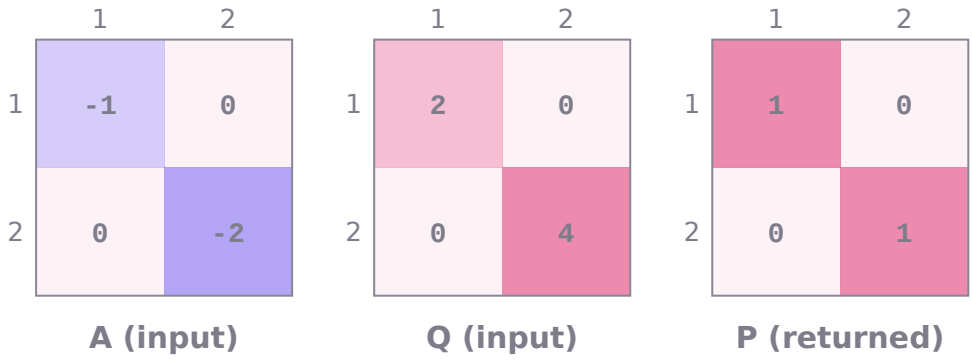
The solution P of A^T P + P $A + Q = 0$ for a stable A (exact), and whether P is positive definite (with $Q > 0$ that certifies stability).

$$A^T P + P A + Q = 0$$

input	meaning	condition
`A`	state matrix (stable, n x n)	
`Q`	symmetric matrix (n x n)	

output	meaning
`P`	solution P
`positive_definite`	true when P is positive definite

- **Method:** Kronecker linear system in exact fractions
- **Assumes:** A stable; Q symmetric
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); Q (input); P (returned).

```
control_systems_ai999x.solve('ctrl_lyapunov', A=[[-1, 0], [0, -2]], Q=[[2, 0], [0, 4]])
# -> P = [[1.0, 0.0], [0.0, 1.0]], positive_definite = True   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_lyapunov', A=[[0, 1], [-2, -3]], Q=[[1, 0], [0, 1]])
# -> P = [[1.25, 0.25], [0.25, 0.25]], positive_definite = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lyapunov equation: A = [[-1, 0], [0, -2]], Q = [[2, 0], [0, 4]]	SOLVED	P = [[1.0, 0.0], [0.0, 1.0]], positive_definite = True
Lyapunov equation: A = [[0, 1], [-2, -3]], Q = [[1, 0], [0, 1]]	SOLVED	P = [[1.25, 0.25], [0.25, 0.25]], positive_definite = True
Lyapunov equation: Q = [[2, 0], [0, 4]]	NEEDS_INPUT	ctrl_lyapunov needs A (state matrix (stable, n x n)). Example: solve('ctrl_lyapunov', A=[[-1, 0], [0, -2]], Q=[[2, 0], [
Lyapunov equation: A = [[-1, 0], [0, -2]], Q = [[2, 0], [0, 4]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Lyapunov equation: A = [[1, 0], [0, -2]], Q = [[1, 0], [0, 1]]	INVALID_INPUT	Continuous Lyapunov equation: A must be stable (every eigenvalue in the left half plane).
Solve the Lyapunov equation A^T P + P A + Q = 0 for P with A = [-1 0; 0 -2] and Q = [2 0; 0 4].	OUT_OF_SCOPE	Out of scope: write the documented form 'Lyapunov equation: A = ..., Q = ...'.

Discrete Lyapunov equation, `ctrl_discrete_lyapunov`

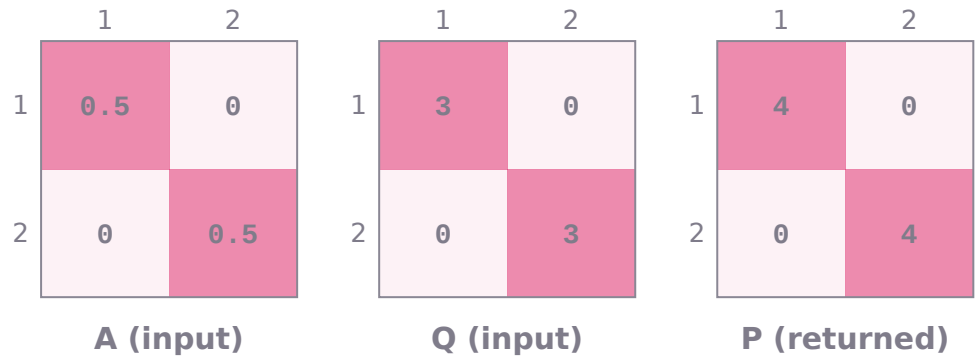
The solution P of A^T P $A - P + Q = 0$ for a Schur-stable A (exact), and whether P is positive definite.

$$A^T P A - P + Q = 0$$

input	meaning	condition
`A`	discrete state matrix (eigenvalues inside the unit circle)	
`Q`	symmetric matrix (n x n)	

output	meaning
`P`	solution P
`positive_definite`	true when P is positive definite

- **Method:** Kronecker linear system in exact fractions
- **Assumes:** A Schur stable; Q symmetric
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); Q (input); P (returned).

```
control_systems_ai999x.solve('ctrl_discrete_lyapunov', A=[[0.5, 0], [0, 0.5]], Q=[[3, 0], [0, 3]])
# -> P = [[4.0, 0.0], [0.0, 4.0]], positive_definite = True   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_discrete_lyapunov', A=[[0, 1], [0, 0]], Q=[[1, 0], [0, 1]])
# -> P = [[1.0, 0.0], [0.0, 2.0]], positive_definite = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
discrete Lyapunov equation: A = [[0.5, 0], [0, 0.5]], Q = [[3, 0], [0, 3]]	SOLVED	P = [[4.0, 0.0], [0.0, 4.0]], positive_definite = True
discrete Lyapunov equation: A = [[0, 1], [0, 0]], Q = [[1, 0], [0, 1]]	SOLVED	P = [[1.0, 0.0], [0.0, 2.0]], positive_definite = True
discrete Lyapunov equation: Q = [[3, 0], [0, 3]]	NEEDS_INPUT	ctrl_discrete_lyapunov needs A (discrete state matrix (eigenvalues inside the unit circle)). Example: solve('ctrl_discre
discrete Lyapunov equation: A = [[0.5, 0], [0, 0.5]], Q = [[3, 0], [0, 3]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
discrete Lyapunov equation: A = [[2, 0], [0, 0.5]], Q = [[1, 0], [0, 1]]	INVALID_INPUT	Discrete Lyapunov equation: A must be stable (every eigenvalue inside the unit circle).
Solve the discrete Lyapunov equation A^T P A - P + Q = 0 for P with A = [0.5 0; 0 0.5] and Q = [3 0; 0 3].	OUT_OF_SCOPE	Out of scope: write the documented form 'discrete Lyapunov equation: A = ..., Q = ...'.

Controllability Gramian, `ctrl_ctrb_gramian`

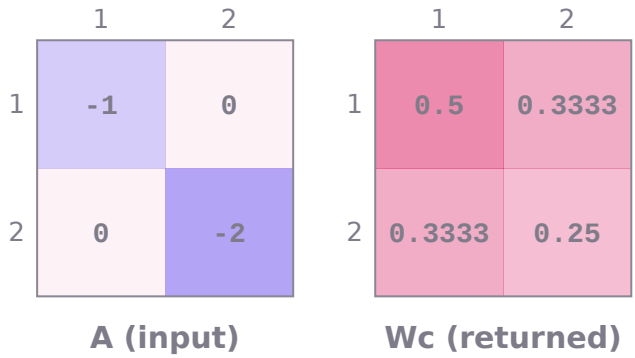
The infinite-horizon controllability Gramian Wc solving A Wc + Wc $A^T + B\,B^T = 0$ for a stable A (exact), its determinant and whether (A, B) is controllable (Wc nonsingular).

$$AW_c + W_cA^T + BB^T = 0$$

input	meaning	condition
`A`	state matrix (stable, n x n)	
`B`	input vector	

output	meaning
`Wc`	controllability Gramian
`controllable`	true when Wc is nonsingular

- **Method:** Lyapunov equation in exact fractions
- **Assumes:** A stable
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); Wc (returned). Other inputs: B = [1, 1].

```
control_systems_ai999x.solve('ctrl_ctrb_gramian', A=[[-1, 0], [0, -2]], B=[1, 1])
# -> Wc = [[0.5, 0.333333333333], [0.333333333333, 0.25]], controllable = True   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_ctrb_gramian', A=[[-1, 0], [0, -2]], B=[1, 0])
# -> Wc = [[0.5, 0.0], [0.0, 0.0]], controllable = False   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
controllability Gramian: A = [[-1, 0], [0, -2]], B = [1, 1]	SOLVED	Wc = [[0.5, 0.333333333333], [0.333333333333, 0.25]], controllable = True
controllability Gramian: A = [[-1, 0], [0, -2]], B = [1, 0]	SOLVED	Wc = [[0.5, 0.0], [0.0, 0.0]], controllable = False
controllability Gramian: B = [1, 1]	NEEDS_INPUT	ctrl_ctrb_gramian needs A (state matrix (stable, n x n)). Example: solve('ctrl_ctrb_gramian', A=[[-1, 0], [0, -2]], B=[1
controllability Gramian: A = [[-1, 0], [0, -2]], B = [1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
controllability Gramian: A = [[1, 0], [0, -2]], B = [1, 1]	INVALID_INPUT	Controllability Gramian: A must be stable for the infinite-horizon Gramian.

Observability Gramian, `ctrl_obsv_gramian`

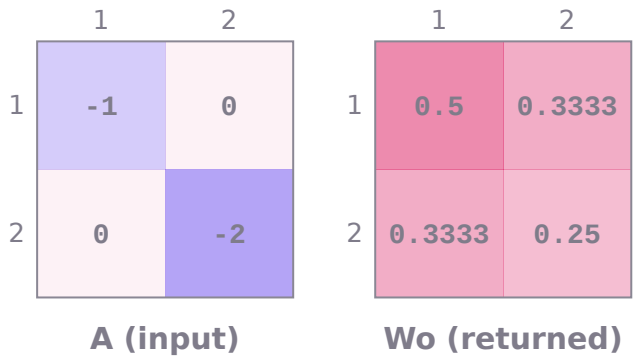
The infinite-horizon observability Gramian Wo solving A^T Wo + Wo $A + C^T C = 0$ for a stable A (exact), its determinant and whether (A, C) is observable.

$$A^T W_o + W_o A + C^T C = 0$$

input	meaning	condition
`A`	state matrix (stable, n x n)	
`C`	output vector	

output	meaning
`Wo`	observability Gramian
`observable`	true when Wo is nonsingular

- **Method:** Lyapunov equation in exact fractions
- **Assumes:** A stable
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): A (input); Wo (returned). Other inputs: C = [1, 1].

```
control_systems_ai999x.solve('ctrl_obsv_gramian', A=[[-1, 0], [0, -2]], C=[1, 1])
# -> Wo = [[0.5, 0.333333333333], [0.333333333333, 0.25]], observable = True   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_obsv_gramian', A=[[0, 1], [-2, -3]], C=[1, 0])
# -> Wo = [[0.916666666667, 0.25], [0.25, 0.0833333333333]], observable = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
observability Gramian: A = [[-1, 0], [0, -2]], C = [1, 1]	SOLVED	Wo = [[0.5, 0.333333333333], [0.333333333333, 0.25]], observable = True
observability Gramian: A = [[0, 1], [-2, -3]], C = [1, 0]	SOLVED	Wo = [[0.916666666667, 0.25], [0.25, 0.0833333333333]], observable = True

question	outcome	result
observability Gramian: C = [1, 1]	NEEDS_INPUT	ctrl_obsv_gramian needs A (state matrix (stable, n x n)). Example: solve('ctrl_obsv_gramian', A=[[-1, 0], [0, -2]], C=[1
observability Gramian: A = [[-1, 0], [0, -2]], C = [1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
observability Gramian: A = [[1, 0], [0, -2]], C = [1, 1]	INVALID_INPUT	Observability Gramian: A must be stable for the infinite-horizon Gramian.

H2 norm of a stable transfer function, `ctrl_h2_norm`

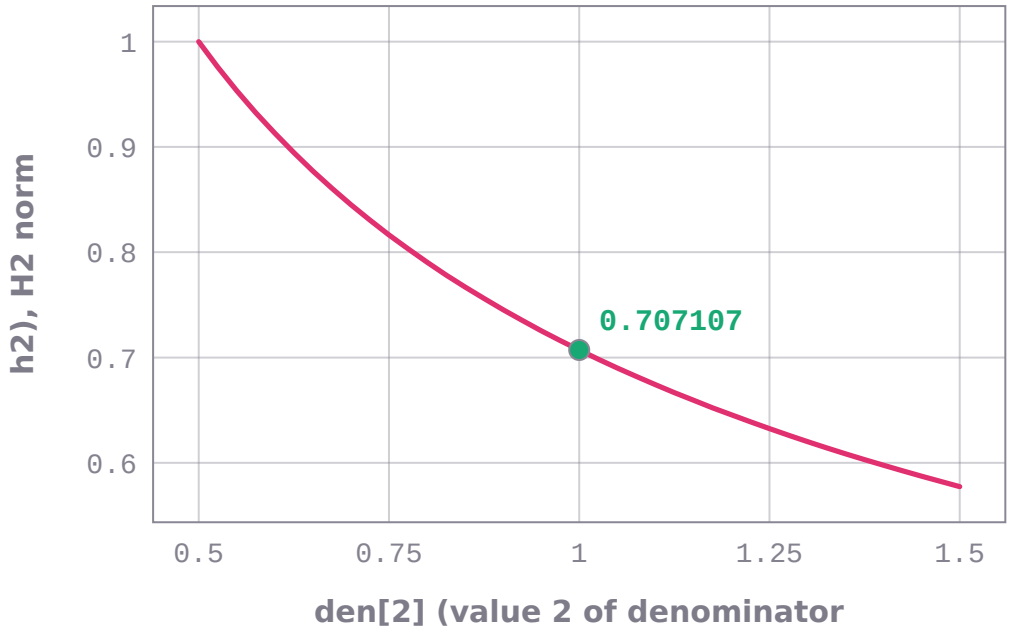
||G||$_2$ = $sqrt$((1/(2 pi)) integral of |G(jw)|^2 dw) of a stable, strictly proper G(s), computed exactly as B^T Wo B from the observability Gramian.

$$\|G\|_2^2 = \frac{1}{2\pi}\int_{-\infty}^{\infty} |G(j\omega)|^2 d\omega = B^T W_o B$$

input	meaning	condition
`num`	numerator	
`den`	denominator	

output	meaning
`h2`	H2 norm
`h2_squared`	squared H2 norm

- **Method:** Lyapunov equation of the controllable canonical form in exact fractions
- **Assumes:** stable; strictly proper
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



h2 (H2 norm) as value 2 of den (denominator) moves from 0.5 to 1.5, the other values as given ([1, 1]); 41 runs of control-systems-ai999x. Other inputs: num = [1]. Marked: the worked example, den[2] = 1, h2 = 0.707107.

```
control_systems_ai999x.solve('ctrl_h2_norm', num=[1], den=[1, 1])
# -> h2 = 0.707107, h2_squared = 0.5   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_h2_norm', num=[1], den=[1, 2, 1])
# -> h2 = 0.5, h2_squared = 0.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
H2 norm: num = [1], den = [1, 1]	SOLVED	h2 = 0.707107, h2_squared = 0.5
H2 norm: num = [1], den = [1, 2, 1]	SOLVED	h2 = 0.5, h2_squared = 0.25
H2 norm: den = [1, 1]	NEEDS_INPUT	ctrl_h2_norm needs num (numerator). Example: solve('ctrl_h2_norm', num=[1], den=[1, 1]).
H2 norm: num = [1], den = [1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
H2 norm: num = [1], den = [1, -1]	INVALID_INPUT	H2 norm of a stable transfer function: the system must be stable.
H2 norm: num = [1, 1], den = [1, 2]	INVALID_INPUT	H2 norm of a stable transfer function: the transfer function must be strictly proper (numerator degree below the denomin

H-infinity norm (peak gain) of a stable transfer function, `ctrl_hinf_norm`

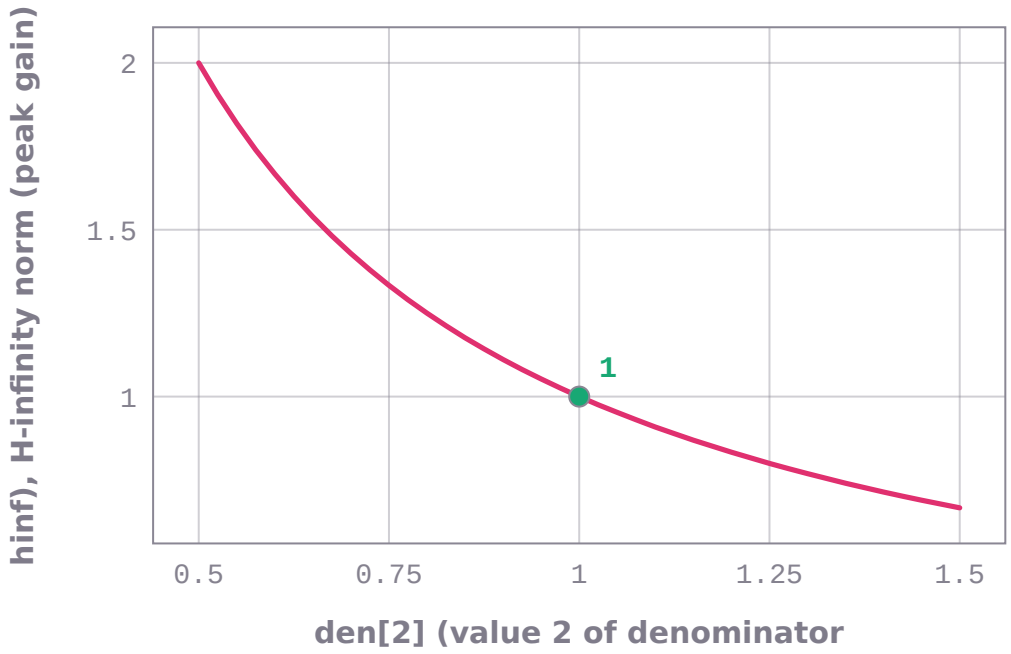
The peak gain sup_w |G(jw)| of a stable proper G(s), in ratio and dB, and the frequency where it occurs (0 when the peak is the DC gain).

$$\|G\|_\infty = \sup_\omega |G(j\omega)|$$

input	meaning	condition
`num`	numerator	
`den`	denominator	

output	meaning
`hinf`	H-infinity norm (peak gain)
`hinf_db`	peak gain in dB
`w_peak`	frequency of the peak

- **Method:** stationary points of |N(jw)|^2/|D(jw)|^2 (positive real roots of f' g - f g') and $w = 0$
- **Assumes:** stable; proper; peak at a finite frequency
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



hinf (H-infinity norm (peak gain)) as value 2 of den (denominator) moves from 0.5 to 1.5, the other values as given ([1, 1]); 41 runs of control-systems-ai999x. Other inputs: num = [1]. Marked: the worked example, den[2] = 1, hinf = 1.

```
control_systems_ai999x.solve('ctrl_hinf_norm', num=[1], den=[1, 1])
# -> hinf = 1, hinf_db = 0, w_peak = 0 rad/s   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_hinf_norm', num=[1], den=[1, 0.2, 1])
# -> hinf = 5.02519, hinf_db = 14.023, w_peak = 0.989949 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
H-infinity norm: num = [1], den = [1, 1]	SOLVED	hinf = 1, hinf_db = 0, w_peak = 0 rad/s
H-infinity norm: num = [1], den = [1, 0.2, 1]	SOLVED	hinf = 5.02519, hinf_db = 14.023, w_peak = 0.989949 rad/s

question	outcome	result
H-infinity norm: den = [1, 1]	NEEDS_INPUT	ctrl_hinf_norm needs num (numerator). Example: solve('ctrl_hinf_norm', num=[1], den=[1, 1]).
H-infinity norm: num = [1], den = [1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
H-infinity norm: num = [1], den = [1, -1]	INVALID_INPUT	H-infinity norm (peak gain) of a stable transfer function: the system must be stable.

Nyquist criterion: encirclements of -1, `ctrl_nyquist`

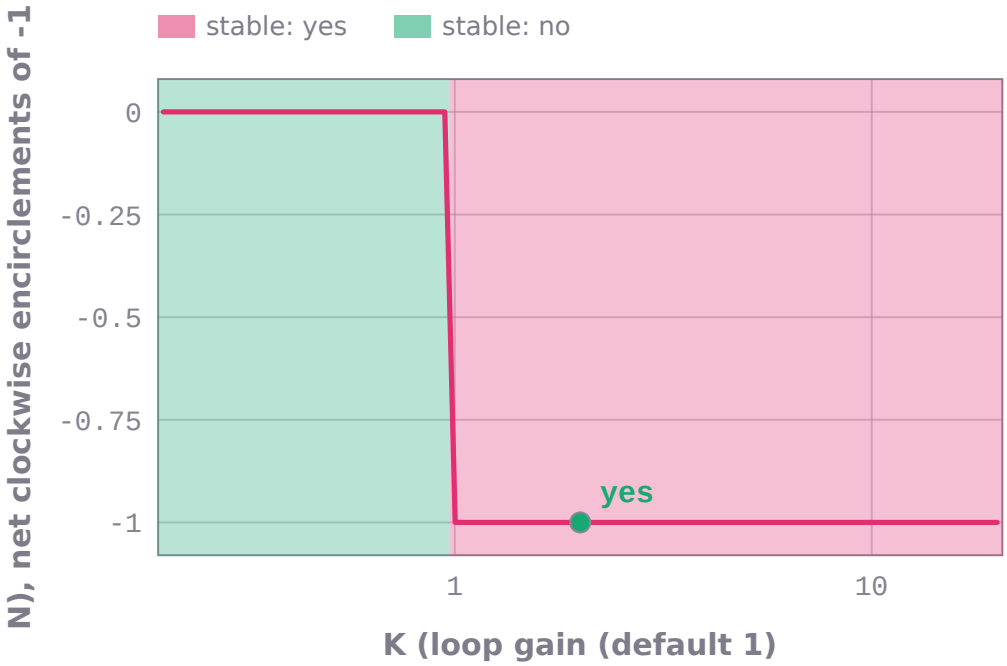
For the open loop K G(s) under unity negative feedback: the open-loop right-half-plane poles P, the closed-loop right-half-plane poles Z and the net clockwise encirclements $N = Z - P$ of the point -1 by the Nyquist curve.

$$Z = N + P$$

input	meaning	condition
`num`	numerator of G(s)	
`den`	denominator of G(s)	
`K`	loop gain (default 1)	

output	meaning
`P`	open-loop poles in the right half plane
`Z`	closed-loop poles in the right half plane
`N`	net clockwise encirclements $of -1$
`stable`	true when $Z = 0$

- **Method:** $Z = N + P$ from the poles of G and of $1 + K$ G
- **Assumes:** no open-loop or closed-loop pole on the imaginary axis
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



true when Z = 0 (stable) along K (loop gain (default 1)) from 0.2 to 20 (logarithmic axis): 81 runs of control-systems-ai999x, each band coloured by the verdict returned (yes, no); it changes near K = 0.973935; the line is N (net clockwise encirclements of -1). Other inputs as in the worked example: num = [1], den = [1, -1]. Marked: the worked example, K = 2 → yes.



N (net clockwise encirclements of -1) against K (loop gain (default 1)) from 0.2 to 20 on a logarithmic axis. Other inputs as in the worked example: num = [1], den = [1, -1]. 41 points computed by control-systems-ai999x; marked: the worked example, K = 2, N = -1.

```
control_systems_ai999x.solve('ctrl_nyquist', num=[1], den=[1, -1], K=2)
# -> P = 1, Z = 0, N = -1, stable = True   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_nyquist', num=[1], den=[1, 6, 11, 6], K=100)
# -> P = 0, Z = 2, N = 2, stable = False   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nyquist encirclements: num = [1], den = [1, -1], K = 2	SOLVED	P = 1, Z = 0, N = -1, stable = True
Nyquist encirclements: num = [1], den = [1, 6, 11, 6], K = 100	SOLVED	P = 0, Z = 2, N = 2, stable = False
Nyquist encirclements: den = [1, -1], K = 2	NEEDS_INPUT	ctrl_nyquist needs num (numerator of G(s)). Example: solve('ctrl_nyquist', num=[1], den=[1, -1], K=2).
Nyquist encirclements: num = [1], den = [1, -1], K = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
Nyquist encirclements: num = [1], den = [1, 0, 1]	INVALID_INPUT	Nyquist criterion: encirclements of -1: the open loop has a pole on the imaginary axis (the contour needs an indentation

Peak sensitivity Ms and modulus margin, `ctrl_sensitivity_peak`

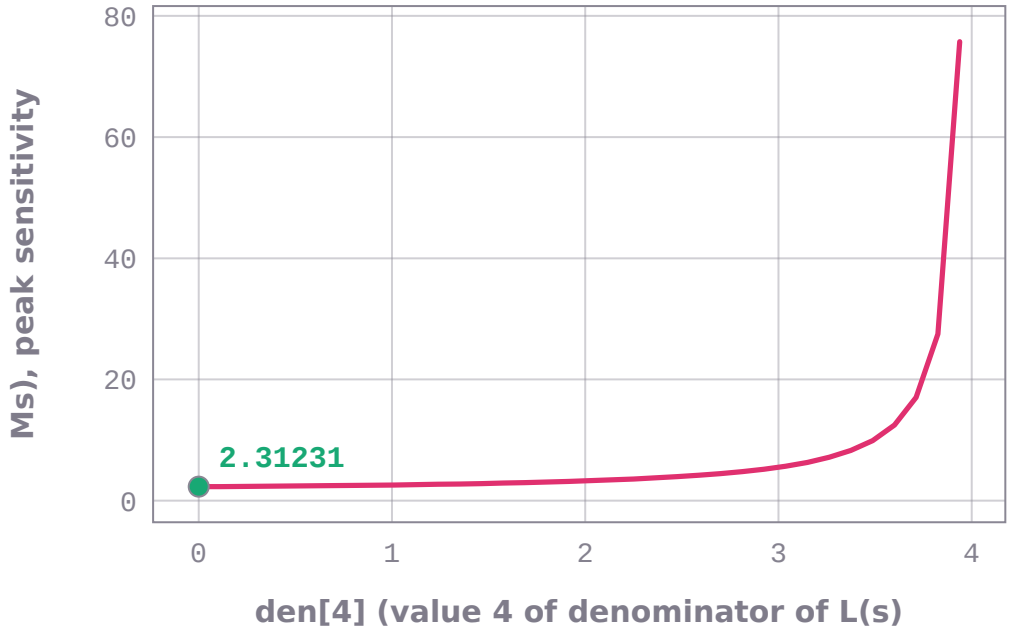
Ms = max |1/(1 + L(jw))| of a stable unity-feedback loop, its frequency, and the modulus (stability) margin $1/Ms$: the smallest distance from the Nyquist curve $to -1.$

$$M_s = \max_{\omega} \frac{1}{|1 + L(j\omega)|}$$

input	meaning	condition
`num`	numerator of the open loop L(s)	
`den`	denominator of L(s)	

output	meaning
`Ms`	peak sensitivity
`Ms_db`	peak sensitivity in dB
`w_ms`	frequency of the peak
`sm`	modulus margin $1/Ms$

- **Method:** stationary points of |D(jw)|^2/|D(jw) + N(jw)|^2 (positive real roots of f' g - f g')
- **Assumes:** closed loop stable; strictly proper L(s)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ms (peak sensitivity) as value 4 of den (denominator of L(s)) moves from 0 to 4.5, the other values as given ([1, 3, 2, 0]); 36 runs of control-systems-ai999x. Other inputs: num = [2]. Marked: the worked example, den[4] = 0, Ms = 2.31231.

```
control_systems_ai999x.solve('ctrl_sensitivity_peak', num=[2], den=[1, 3, 2, 0])
# -> Ms = 2.31231, Ms_db = 7.28094, w_ms = 0.92529 rad/s, sm = 0.432467   (SOLVED)
```

```
control_systems_ai999x.solve('ctrl_sensitivity_peak', num=[1], den=[1, 1, 0])
# -> Ms = 1.46789, Ms_db = 3.33387, w_ms = 1.16877 rad/s, sm = 0.68125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
peak sensitivity: num = [2], den = [1, 3, 2, 0]	SOLVED	Ms = 2.31231, Ms_db = 7.28094, w_ms = 0.92529 rad/s, sm = 0.432467
peak sensitivity: num = [1], den = [1, 1, 0]	SOLVED	Ms = 1.46789, Ms_db = 3.33387, w_ms = 1.16877 rad/s, sm = 0.68125
peak sensitivity: den = [1, 3, 2, 0]	NEEDS_INPUT	ctrl_sensitivity_peak needs num (numerator of the open loop L(s)). Example: solve('ctrl_sensitivity_peak', num=[2], den=
peak sensitivity: num = [2], den = [1, 3, 2, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: control_systems_ai999x.templates(
peak sensitivity: num = [10], den = [1, 3, 2, 0]	INVALID_INPUT	Peak sensitivity Ms and modulus margin: the closed loop must be stable.